Proiectare Logică

I CB: Prof. dr. ing. Mariana Mocanu

- mariana.mocanu@cs.pub.ro

I CA & CD: Conf. dr. ing. Costin Chiru

- costin.chiru@cs.pub.ro

I CC: Prof. dr. ing. Anca Morar

- anca.morar@cs.pub.ro

Curs 1: Introducere

Viața de student

- Cantitate copleșitoare de informație → apucați-vă de treabă din timp
- Legăturile curs-laborator-seminar nu sunt mereu evidente
- Activitate în timpul cursurilor/laboratoarelor:
  - Puneți întrebări (atât vouă cât și profesorilor)!
  - Participați la toate orele! Facultatea nu e „facultativă”!
  - Disciplinele sunt legate între ele → nu merge să învățați doar ce vă place
- Informatica din liceu → infomațiile au fost predate la nivelul la care erați atunci → învațați altceva/altfel/în plus/ mai riguros dpdv științific

Viața de student (2)

- Responsabilitate → nu trage nimeni de voi ca în liceu
- Nu vă „lăsați pe o ureche"! Sunteți excepționali, dar așa au fost și cei care nu termină facultatea (~30%)
- Secretariat:
  - Un/o secretar/ă la un an de studenți (anul 1 ≈ 700 studenți)
  - Programul nu e ca să vă împiedice pe voi, ci să le dea și lor posibilitatea să își facă treaba!

Viața de student (3)

- Structura anului universitar:
  - 2 semestre cu materii diferite, finalizate cu examen sau verificare pe parcurs
  - Fiecare disciplină are asociate puncte de credit (PC)
  - În urma procesului de evaluare obțineți o notă (1 .. 10) sau un calificativ (admis / respins)
- Discipline:
  - Obligatorii
  - Opționale
  - Facultative

Viața de student (4)

- Discipline
  - fundamentale (Mate, Fizică, Elth., ...)
  - de domeniu (Calculatoare și Tehnologia informației)
  - de specialitate (ex.: Grafică pe calculator, …)
  - complementare (ex.: Limba străină)
- Sistemul de credite
  - Exprimare matematică a volumului de muncă de la o materie (1 p credit ≈ 26 ore de muncă);
  - 30 PC / semestru; 60 PC / an
  - PC se obțin doar pentru disciplinele promovate;
  - Condiția de promovare
    - din anul 1 în anul 2 = minim 30 PC
    - din anul 2 în anul 3 = minim 90 PC, etc.
    - PC obținute la discipline facultative nu se iau în calcul la condiția de promovare

Viața de student (5)

- Repetare materie:
  - Costă
  - Nu va fi mai ușor să treceți examenul la anul decât a fost anul ăsta!
- Examene în restanță:
  - Nu sunt mai ușoare examenele în restanță decât au fost în sesiunea normală (la PL din contră!)
  - Vară, mare, etc. → stați să învățați?
- Tentativele de fraudă:
  - Pot duce la exmatriculare!

Viața de student (6)

- Contul de student:
  - Permite urmărirea situației școlare la sfârșitul sesiunii → obligația voastră să verificați notele respective și să anunțați dacă apar probleme.
  - Aveți acces la materialele puse la dispoziție de fiecare cadru didactic (http://acs.curs.pub.ro)
  - Aveți acces la știri și discuții → verificați frecvent mailul de la facultate!
  - Enroll la disciplinele la care participați (dacă nu ați fost înscriși automat) → obligația voastră (contactați profesorul de curs dacă apar probleme)
  - Furnizare feedback la sfârșitul cursului!
  - Platforme de lucru: Moodle și Teams
- Utilizați toate posibilitățile de contact cu cadrele didactice, oferite de UPB
- Învățați ritmic
- Luați notițe și puneți întrebări celor autorizați (cadre didactice; secretariat)

Obiectivele cursului

- Curs
  - Se învață concepte/metode și nu tehnologii ca la celelalte materii
  - Reprezintă baza pentru o serie de alte cursuri pe care le veți face de-a lungul facultății (Calculatoare Numerice, Electronică Digitală, Proiectare cu Microprocesoare, VLSi, etc.)
- Laborator
  - Aplicarea concretă a conceptelor și metodelor prin rezolvarea unor probleme specifice și realizarea unor proiecte individuale și în echipă

Evaluare

- **Activitatea pe parcurs: 50 puncte**
  - Activitate de laborator – 10 puncte
  - Un test - 15 puncte
  - Proiect individual 10 puncte
  - Proiect realizat în echipă 15 puncte

  Condiția de intrare în examen (promovarea laboratorului): minim 25 puncte acumulate în activitatea pe parcurs
- **Examen final: 50 puncte**

  Condiția de promovare a examenului: 25 puncte

Introducere

- Proiectarea Logică = Analiza și Sinteza Dispozitivelor Numerice
- Dispozitiv = ansamblu de componente care îndeplinesc o anumită funcție logică
- Analiză = determinarea funcționării circuitelor
- Sinteză = realizarea unui circuit/dispozitiv pe baza unor reguli folosind niște componente

Dispozitive de calcul

- Mecanice: ABAC, Riglă de calcul
- Electrice:
  - Analogice
  - Digitale (Numerice)

- Primul calculator: Charles Babbage – calculator mecanic
  - Bazat pe avansul științei în Teoria Numerelor din matematică: orice număr poate fi reprezentat în orice bază de numerație, deci și în baza 2 folosind 0 și 1

Legătura dintre Teoria Numerelor și Calculatoare

- Inițial, legătura 0 = fals, 1 = adevărat
- Ulterior, legătura logic / fizic (memorare):
  - Mecanic: hârtie perforată (lumină + detectare)
  - Electromecanic: electromagnet (releu) = cuplează/decuplează un circuit și e acționat în mod electric (→ asociere închis-deschis cu valorile 0-1)



    - Dezavantaje: contactul mecanic-electric este imperfect → erori, volum, fiabilitate, viteză de prelucrare
  - Electric/Electronic: lămpi (diode, triode) → semiconductori → integrare pe scară mică, medie, mare, VLSI

Baze de numerație

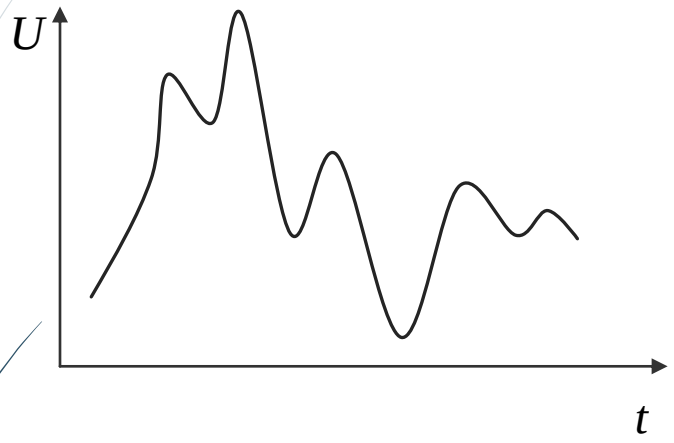
- Caracterizate de mulțimea simbolurilor folosite și de regulile și operațiile disponibile:
  - Baza 10: {0, 1 … 9}
  - Baza 8: {0, 1 … 7}
  - Baza 16: {0, 1 … 9, A, B, C, D, E, F}
- Operații: $9_{10} + 1_{10} = 10_{10}$
- Conversii: $A = A_H = 10_{10} = 1010_2$
- Bitul = unitatea de informație în binar (0, 1)

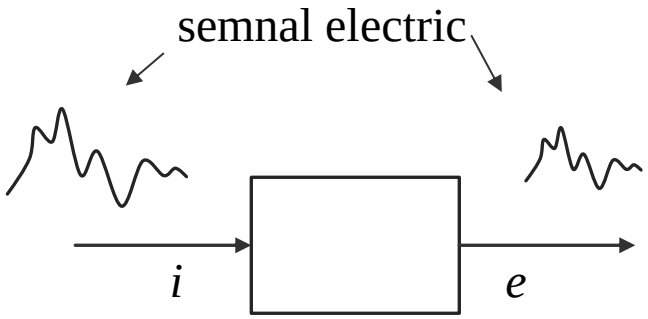
Cuvântul

- Ce valoare are: 101101_2?
  - De ce nu 55 ? $\underline{101}101_2$
- Nu se specifică câți biți se prelucrează odată

  (= lungimea cuvântului)
- Cuvântul = ansamblul de biți care sunt procesați odată
- 1 byte = 8 biți = 1 octet = 1 cuvânt (0 → 255)
- Acum calculatoarele funcționează pe 32/64 biți

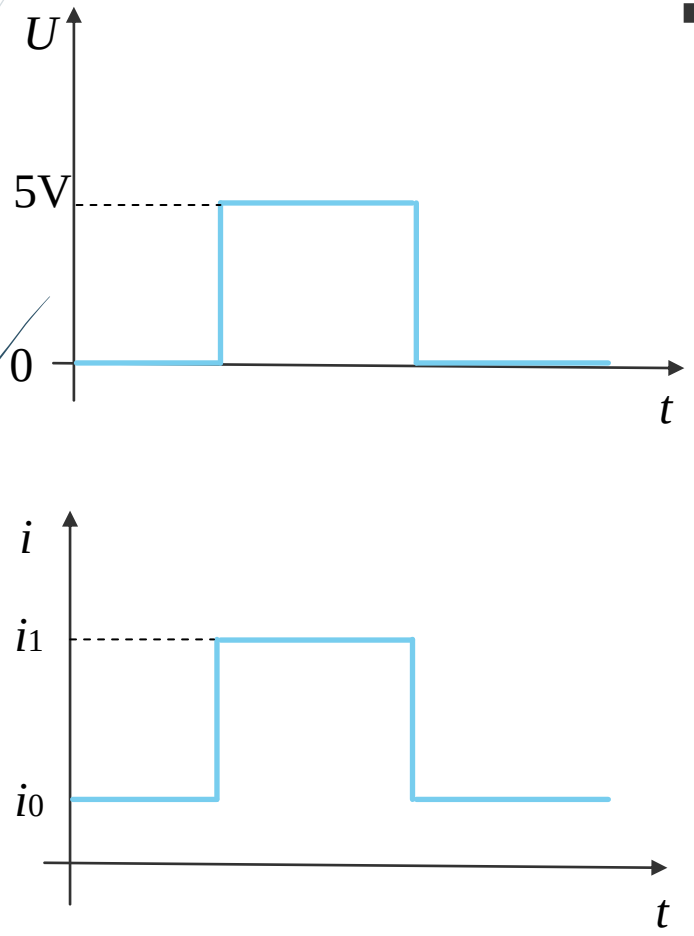
Semnale electrice. Dispozitive electrice





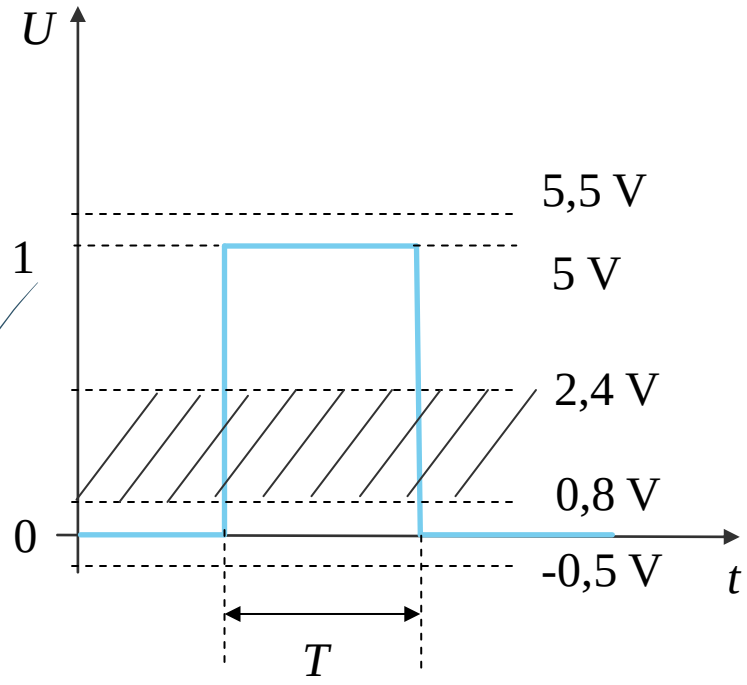
- Valoarea variației tensiunii unui semnal electric în timp
- Semnal analogic – variază continuu într-un interval (poate lua orice valoare)
- Dispozitiv electric: intrări/ieșiri de tip semnal electric
  - Funcționarea se reprezintă prin desenarea variației semnalului electric (fie in **tensiune**, fie în intensitate)

Semnale digitale - Impuls



- Impuls ideal:
  - trecere instantanee de la 0 la 1
  - Fix tensiunile nominale de 0 sau 5V
  - Convenții:
    - Logica pozitivă:
      - 0 V → „0” logic – F;
      - 5 V → „1” logic – A
    - Logica negativă: (0 → „1” logic și 5V → „0” logic)

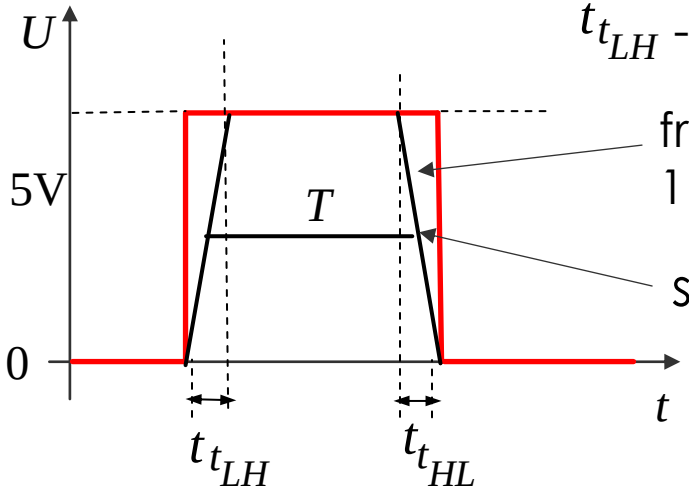
Caracteristicile impulsului digital (1)



- Tensiunea:
  - Nivel 0: -0.5V → 0.8V
  - Nivel 1: 2.4V → 5.5V
  - Între intervale, funcționarea e imprevizibilă (aleatoare)
    - De ce apare? Ex: priza: prea mulți consumatori → scade tensiunea →
    - la fel și aici → se pot cupla numai un număr limitat de circuite pentru că altfel se pierde semnalul

Caracteristicile impulsului digital (2)

- Durata impulsului: se calculează la 0.5 U
  - Importantă pentru că dacă e prea scurtă, atunci semnalul e posibil să nu fie detectat
- Fronturile impulsurilor:
  - Crescător: durata între 0.1 U și 0.9 U
  - Descrescător : durata între 0.9 U și 0.1 U



$t_{t_{LH}}$ -> timp de tranziție de la low la high

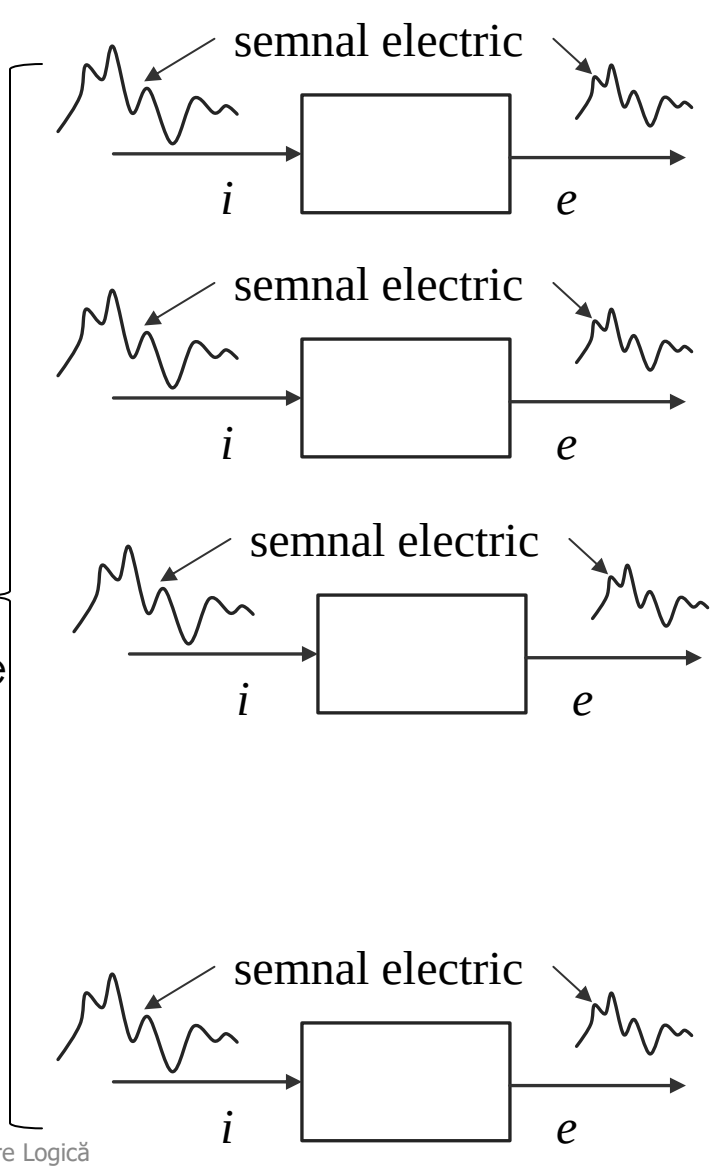
front descrescător (trecere de la 1 la 0)

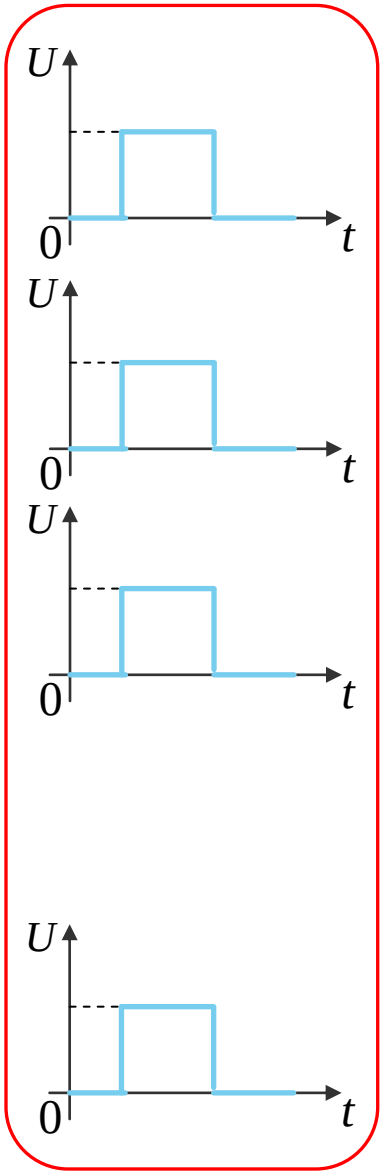
se măsoară la 0.5 din tensiune

$t_{t_{LH}} \neq t_{t_{HL}}$

Caracteristicile impulsului digital (3)

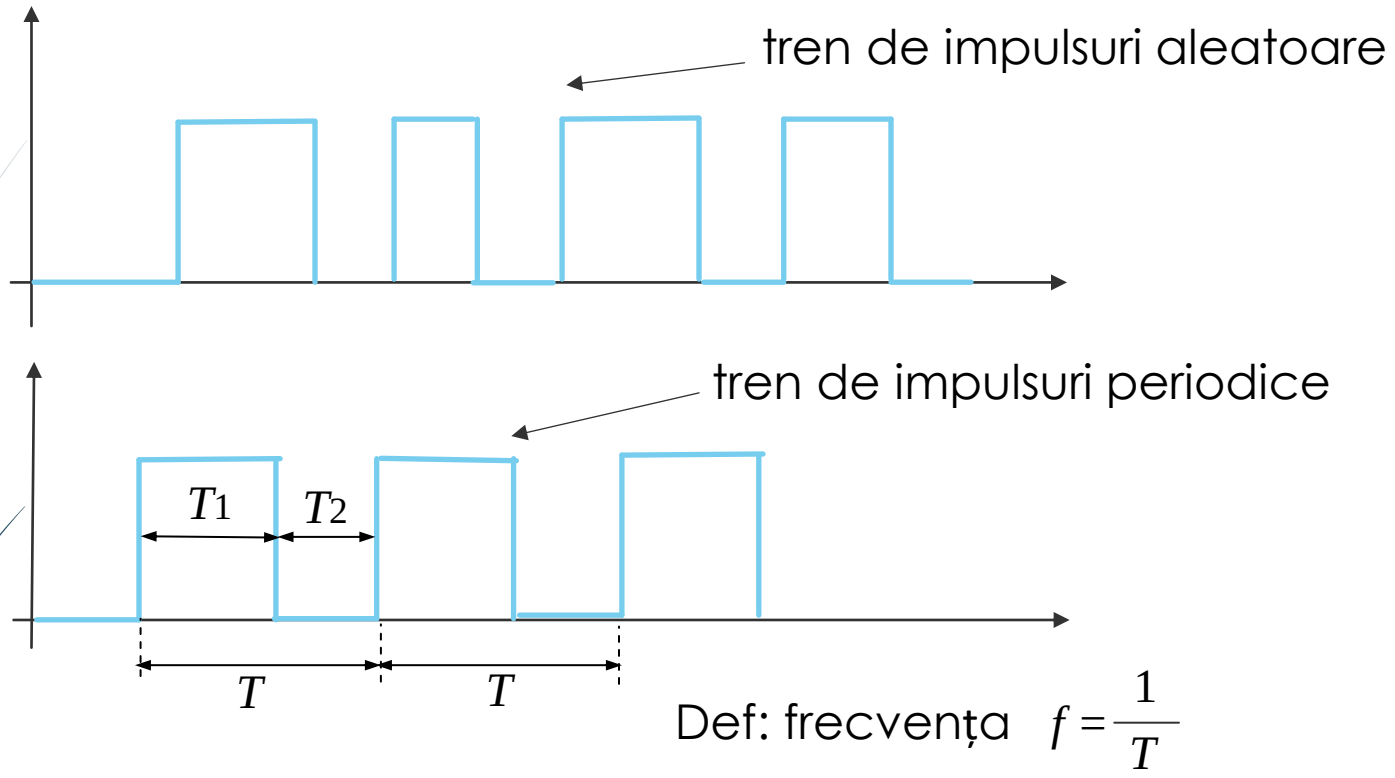
8 → 64 dispozitive electronice





= Cuvântul

Caracteristicile impulsului digital (4)



- Frecvența impulsurilor
  - Ex: la procesor: f mare = gândește mai repede, adică perioada în care se prelucrează un cuvânt (*T*) este mai mică
- Semnalul periodic nu conține informație → este semnal de sincronizare (stabilește niște repere de timp comun) și se numește purtătoare

Caracteristicile impulsului digital (5)

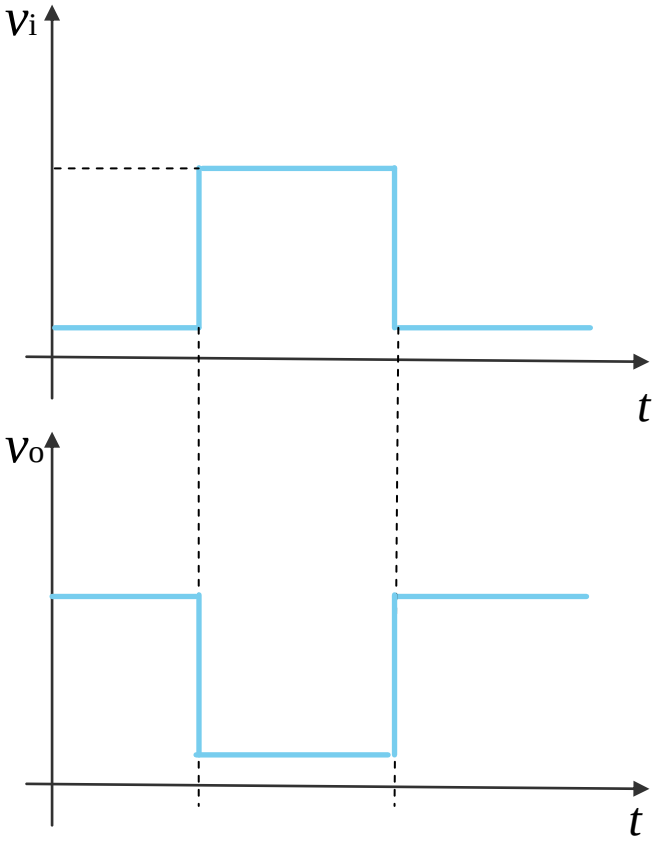
- Factor de umplere: $f_u = T_1/T_2$ sau T_1/T



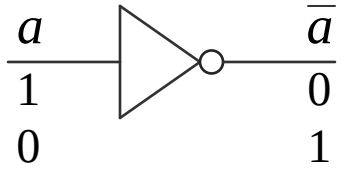
- Timpul de propagare: Δt_p - intervalul de timp între momentul în care mărimea de intrare comută între cele două niveluri logice și momentul în care mărimea de ieșire trece de la valoarea logică inițială la cea finală

Caracteristicile impulsului digital (6)



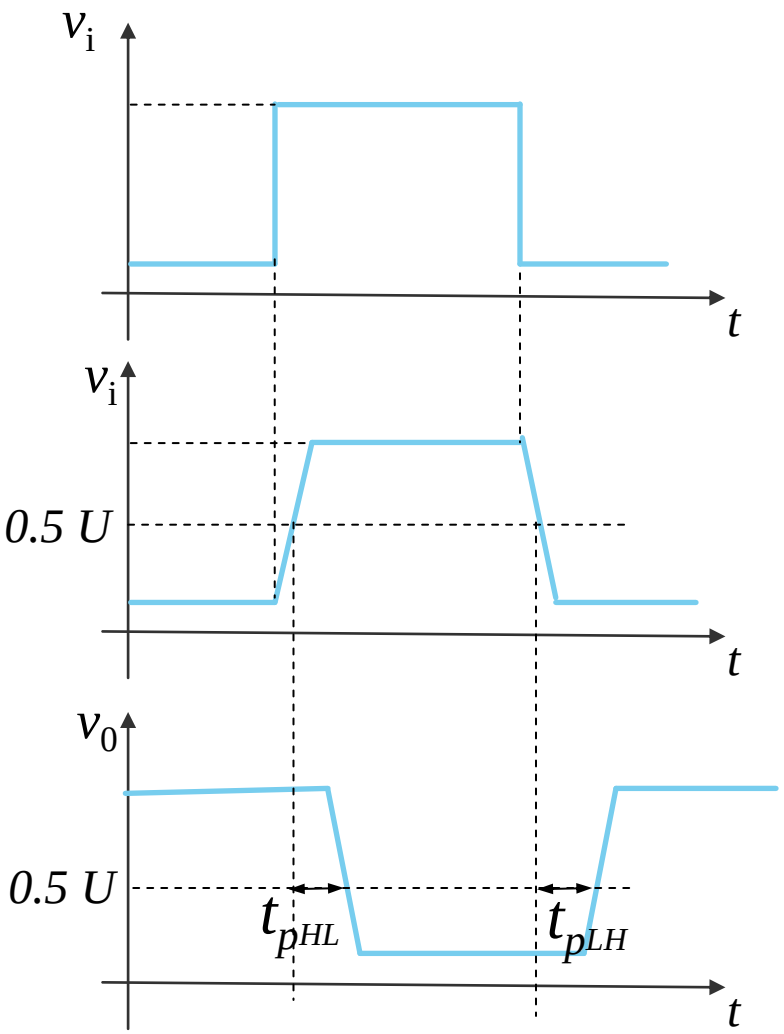


- Inversorul – inversează valoarea logică



- Ideal:
  - $t_{t_{LH}} = t_{t_{HL}} = 0$
  - $\Delta t_p = 0$

Caracteristicile impulsului digital (7)



Cazul real

Caracteristicile impulsului digital (8)

- În curs vom lucra în general cu impulsuri ideale, dar la un moment dat vom discuta și implicațiile nedorite ale circuitelor reale (hazard, cursă critică, etc.)
- Parametrii prezentați depind de lotul de circuite construite
  - *Circuite de viteză*: timp de tranziție mic, plajă de tensiuni mare
  - *Circuite de putere*: plajă de tensiuni mică, timp de tranziție mare



Proiectare Logică

I CB: Prof. dr. ing. Mariana Mocanu

- mariana.mocanu@cs.pub.ro

I CA & CD: Conf. dr. ing. Costin Chiru

- costin.chiru@cs.pub.ro

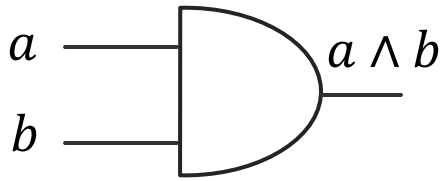
I CC: Prof. dr. ing. Anca Morar

- anca.morar@cs.pub.ro

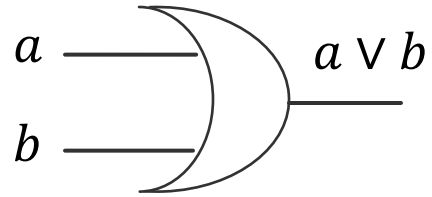
Curs 2: Algebra booleană. Constante booleene, variabile booleene și funcții booleene. Seturi complete de operatori

Fundamentele matematice ale proiectării logice – Algebra Booleană

- O algebră booleană se definește ca un triplet $(B, \wedge, \vee)$, unde B este o mulțime de valori binare, iar $\wedge$ (și), $\vee$ (sau) sunt operatori binari pe B astfel încât să fie adevărate următoarele proprietăți:
  - **P_1**: Pe B, $\wedge$ și $\vee$ sunt comutative
    - $\forall\, a, b \in B$: $a \wedge b = b \wedge a$ și $a \vee b = b \vee a$



AND — Circuit "ȘI" ($\wedge$,·)



OR — Circuit "SAU" ($\vee$, +)

Fundamentele matematice ale proiectării logice – Algebra Booleană

- **P_2**: În B există elemente distincte de identitate 1 și 0 relative la ∧, respectiv ∨
  - $\forall a \in B: a \wedge 1 = a$ *și* $a \vee 0 = a$
- **P_3**: ∧ și ∨ sunt distributive unul față de altul
  - $\forall a, b, c \in B: a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$
  - $\forall a, b, c \in B: a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
- **P_4**: Pentru oricare $a \in B$ există $\bar{a}$, astfel încât
  - $a \wedge \bar{a} = 0$ *și*
  - $a \vee \bar{a} = 1$

- **P_1-P_4 sunt considerate AXIOMELE algebrei booleene**

Algebra Booleană

- Algebra booleană nu e decât o particularizare peste B relativ la proprietățile definite
- Unele concepte sunt cunoscute de la programare:
  - Tipuri de date: numerice, logice, text
  - Operatori: numerici, logici, relaționali
- Diferențe/asemănări între operatori logici și relaționali: cei relaționali operează pe date numerice și întorc rezultate logice

Teoremele algebrei booleene (1)

- **T_1**: Elementul $\bar{a}$ asociat cu un element $a \in B$ prin proprietatea P_4 este unic.
  - Demonstratie: prin reducere la absurd (demonstrația matematică)
    - Presupunem că $\exists\, x, y \in B$ a.î.
      - $a \wedge x = 0, a \vee x = 1$
      - $a \wedge y = 0, a \vee y = 1$
    - Utilizând P_2 ⇨ $x = y$
  - Altă demonstrație: metoda verificării comportamentului → nu e metodă matematică, dar e ușor de adaptat la PL (prin tabela de adevăr)

În curs sunt foarte puține lucruri care trebuie memorate, restul se pot deduce una din alta! → Nu e important numărul teoremei, ci să știți să o aplicați!

Teoremele algebrei booleene (2)

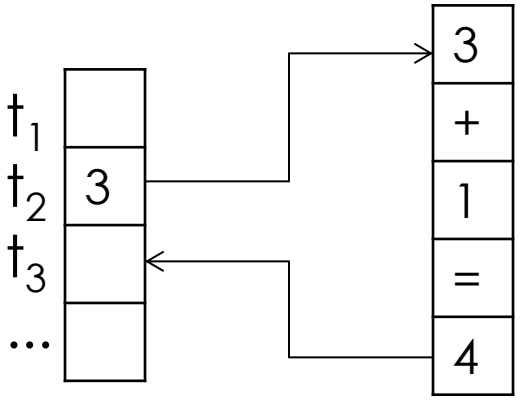
- **T_2**: Fie $a \in B$; atunci $a \vee a = a$ *și* $a \wedge a = a$
- **T_3**: Fie $a \in B$; atunci $a \vee 1 = 1$ *și* $a \wedge 0 = 0$
- **T_4**: Legea de absorbție:
  - $a \vee (a \wedge b) = a$
  - $a \wedge (a \vee b) = a$
- **T_5**: ∨ şi ∧ sunt asociative:
  - $a \wedge (b \wedge c) = (a \wedge b) \wedge c$
  - $a \vee (b \vee c) = (a \vee b) \vee c$
- **T_6**: Principiul dublei negări: $\forall a \in B, \bar{\bar{a}} = a$

Teoremele algebrei booleene (3)

- **T_7**: $\bar{0} = 1 \; \text{și} \; \bar{1} = 0$
- **T_8**: Regulile lui de Morgan:
  - $\overline{a \wedge b} = \bar{a} \vee \bar{b}$
  - $\overline{a \vee b} = \bar{a} \wedge \bar{b}$
- Generalizare de Morgan: Fie $x_1, x_2, \ldots, x_n \in B$
  - $\overline{x_1 \wedge x_2 \wedge \cdots \wedge x_n} = \overline{x_1} \vee \overline{x_2} \vee \cdots \vee \overline{x_n}$
  - $\overline{x_1 \vee x_2 \vee \cdots \vee x_n} = \overline{x_1} \wedge \overline{x_2} \wedge \cdots \wedge \overline{x_n}$

Convenție

- $\bar{} = !,\ \vee = +\ iar\ \wedge = \cdot$
- Observație: sunt operatori logici nu matematici
  - Ex: $1 + 1 = 1$
- Unde ați mai întâlnit diferențe față de matematică?
  - Incrementare: $x = x + 1$
  - De ce e corect? Are conotație temporală



La ce sunt utile teoremele?

- 2 aspecte:
  - Matematic: expresii, funcții, reducerea (**minimizarea**) acestora
  - Practic: asociem fiecărei funcții un anumit circuit → punând laolaltă diverse dispozitive în diferite feluri putem genera diverse funcționalități, dar e important câte astfel de dispozitive se folosesc, mai ales având în vedere T_2,T_4 și T_6

Constante, variabile, funcții booleene

- Într-o algebră booleană, 0 și 1 sunt singurele constante.
- O variabilă logică este o mărime logică ce ia o valoare ce se poate schimba.
- O expresie logică este o succesiune de variabile/constante unite prin operatori, respectând regulile de evaluare ale operatorilor
  - $a, b \epsilon B$, $ab+$ nu e expresie, $a+b$ e expresie
- O funcție booleană de n variabile $x_1, x_2, \dots, x_n$, notată $f(x_1, x_2, \dots, x_n)$ se definește ca o aplicație a lui B^n $(B \times B \times \cdots \times B) \rightarrow B$. Variabilele $x_1, x_2, \dots, x_n$ sunt independente
- O funcție booleană = rezultatul oricărei combinații finite de constante și variabile realizate cu ajutorul operatorilor !, ·, +

Forme de reprezentare a funcțiilor booleene

- 1. Algebrică: se definește funcția pe baza unei expresii:
  - Teoremele învățate ne ajută la reprezentarea algebrică $f(x_1, x_2, \dots, x_n) = expresie$
  - Ex: $f(a, b) = a + b$
  - Expresiile pot fi combinate → forme relativ complicate
- 2. Tabel de adevăr
  - (-) Foarte multe combinații dacă funcția depinde de multe variabile
  - (+) Se cunoaște rezultatul pentru fiecare combinație în parte
- 3. Grafică – diagrame Karnaugh
- 4. Zecimală – asociază fiecărui monom o valoare zecimală și permite automatizarea minimizării

Reguli de precedență a operatorilor

- Evaluarea unei expresii se face respectând anumite reguli (reguli de precedență a operatorilor):
  - 1. se evaluează, mai întâi, toate subexpresiile conținute în paranteze, evaluarea pornind de la zona cea mai imbricată (folosind regulile de mai jos) : $a \cdot (!\, b + (a \cdot c))$
  - 2. se aplică toți operatorii NEGAȚIE (NOT, !, $\bar{}$): $a + b + c \cdot !\, b + !\, c$
  - 3. Se aplică toți operatorii ȘI ($\wedge$,$\cdot$): $a + b \cdot c$
  - 4. Se aplică toți operatorii SAU (v, +)
  - 5. Evaluarea se face de la stânga la dreapta dacă avem același operator peste tot

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Redă într-o formă ordonată funcționalitatea unei funcții pentru fiecare combinație de intrare posibilă (2^N combinații)

x_1	x_2	…	x_n	f
0	0	…	0	$f(0, 0, \dots 0)$
0	0	…	1	$f(0, 0, \dots 1)$
1	1	…	1	$f(1, 1, \dots 1)$

- Pe fiecare linie există o singură valoare logică

- De exemplu, $f_1(a) = \bar{a}$

a	$\bar{a}$
0	1
1	0

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:
  - $f_2 = a \wedge b = a \cdot b$ (AND)
  - $f_3 = a \vee b = a + b$ (OR)

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

- De unde știm că aceste valori sunt așa și nu altfel?
  - Din $P_{1\text{-}4}$ și din teoreme

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:
- $f(a,b,c) = \bar{a}b\bar{c} + ab\bar{c} + \bar{a}c + bc$

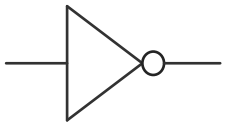
a	*b*	*c*	*f*

- Cum facem?
  - Scriem toate combinațiile de valori ordonate
    - Atenție la simetrie!
  - Evaluăm fiecare combinație în parte

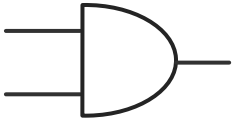
Dispozitive

- Există diverse dispozitive (elemente de comutație) ce generează rezultatele funcțiilor f_1 - f_3

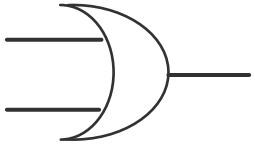
$f_1(a) = \bar{a}$



$f_2 = a \cdot b$ (AND)



$f_3 = a + b$ (OR)



- Pe unde vedeți ∘, înseamnă că ieșirea e negată!

a	$\bar{a}$
0	1
1	0

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Teoremele algebrei booleene (4)

- Există operatori fundamentali și operatori derivați (care se exprimă cu ajutorul celor fundamentali):
- Se definește operatorul relațional „→" a.î. pentru fiecare pereche de elemente $a, b \in B$, $a \to b$ este echivalent cu $a \cdot \bar{b} = 0$.
- **T_9**: Relația „→" este reflexivă: $a \to a, \ \forall a \in B$
- **T_{10}**: Relația „→" este tranzitivă: dacă $a \to b$ și $b \to c$, atunci $a \to c$
- **T_{11}**: Dacă $x \to y$ şi $x \to z$, atunci $x \to y \cdot z$
- **T_{12}**: Dacă $x \to y$, atunci $x \to y + z$, $\forall z \in B$
- **T_{13}**: Dacă $\bar{x} \to \bar{y}$, atunci $y \to x$
- **T_{14}**: Dacă $x \to y$ şi $y \to x$, atunci $x = y$

Monom

- Un monom de n variabile $x_1, x_2, \ldots, x_n$ este o expresie de forma ${x_1}^{e_1} \cdot {x_2}^{e_2} \cdot \ldots \cdot {x_n}^{e_n}$, unde

$${x_i}^{e_i} = \begin{cases} x_i, & dacă \quad e_i = 1 \\ !\, x_i, & dacă \quad e_i = 0 \end{cases}$$

- Ex: având variabilele $a,b,c,$ se pot crea $2^3 = 8$ monoame diferite: $\bar{a}\bar{b}\bar{c}$, $\bar{a}\bar{b}c$, $\bar{a}b\bar{c}$, $\bar{a}bc$, $a\bar{b}\bar{c}$, $a\bar{b}c$, $ab\bar{c}$ și abc

Forma Disjunctivă

- Reprezentarea funcției sub formă de monoame conduce la forma disjunctivă a funcției
- Ex: $f_6 = ab + a\bar{b}$ - funcție disjunctivă ce depinde de 2 variabile (a, b)
- Dar $f_6 = \overset{P_3}{a} \cdot \overset{P_4}{(b + \bar{b})} = a \overset{P_2}{\cdot 1} = a$
- Câte circuite sunt necesare pentru implementare:
  - a) în forma inițială? 1 NOT, 2 ȘI, 1 SAU
  - b) în forma finală? Nimic
- Iată de ce sunt utile teoremele și proprietățile!

Forma Disjunctivă Normală (FDN)

- $f_7(a,b,c) = ab + a\bar{b}c$→ uneori ne ajută ca funcțiile să fie într-o formă omogenă → FDN
- FDN cuprinde monoame ce depind de toate variabilele de care depinde funcția
- Monoamele ce depind de toate variabilele de care depinde funcția (adică sunt complete) se numesc mintermi.
- $ab = ab \cdot 1 = ab\,(c + \bar{c}) = abc + ab\bar{c}$
- $f_7(a,b,c) = abc + ab\bar{c} + a\bar{b}c$ FDN

Forma Disjunctivă Normală (FDN)

- De ce am face asta? (de ce să complicăm funcțiile)?
  - La unele circuite e posibil să fie mai ușor așa
  - Prin FDN se trece foarte ușor de la tabel de adevăr la forma algebrică și invers

a	b	c	$f_7 = ab + a\bar{b}c$
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

- $f_7(a,b,c) = abc + ab\bar{c} + a\bar{b}c$
- În forma inițială e mai dificil să evaluăm decât în FDN, pentru că $f = 1$ dacă cel puțin unul din mintermeni e 1 → $x + 1 = 1$
- Se poate și invers: se dă tabelul de adevăr și se cere forma algebrică a funcției → FDN cu termenii care au 1 în tabel

Forma Disjunctivă Normală Completă (FDNC)

- Def: Suma tuturor mintermilor ne dă FDNC.
- Ex: $f_8(a,b) = \bar{a}\,\bar{b} + \bar{a}\,b + a\,\bar{b} + ab = 1$
- **T_{15}**: Orice funcție reprezentată ca FDNC are valoarea 1.
  - Dem: recursiv, pentru modul general SAU particular, prin factor comun SAU prin tabel de adevăr – peste tot e 1, deci valoarea e 1
- Două funcții se consideră echivalente dacă iau aceleași valori pentru fiecare combinație de valori ale variabilelor de intrare
- **T_{16}**: Fiecare funcție dintr-o algebră booleană este echivalentă cu o FDN (oricât de complicată e o expresie, ea poate fi adusă la FDN)

Forma Conjunctivă Normală (FCN)

- Suma logică a tuturor variabilelor în forma lor directă sau negată se numește maxterm.
- Ex: $a + b + \bar{c}$
- O funcție este reprezentată în FCN dacă este reprezentată ca produs logic al maxtermilor săi.
- Ex: $f_9(a, b, c) = (a + b)(a + c)$ nu e FCN
  $f_{10}(a, b) = (a + b)(a + \bar{b})$ e FCN
- FDN: sumă de mintermi
- FCN: produs de maxtermi

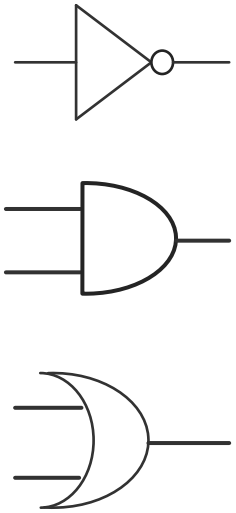
FDN ⇿ FCN

- Pentru a trece dintr-o formă în alta se folosesc regulile lui de Morgan
- Ex: $f_{10}(a,b) = (a+b)\left(a+\bar{b}\right) = \overline{\overline{(a+b)(a+\bar{b})}} = \overline{\bar{a}\bar{b} + \bar{a}b}$

- Se mai poate apela și la desfacerea parantezelor:
- Ex: $f_{10}(a,b) = (a+b)\left(a+\bar{b}\right) = a + a\bar{b} + ab + b\bar{b} = a + a \cdot 1 = a$

Set complet de operatori

- Introducerea de operatori nu se face oricum, ci trebuie să respecte niște reguli.
- Un set de operatori se consideră complet din punct de vedere funcțional, dacă orice funcție booleană poate fi reprezentată prin intermediul acestor operatori.
- Ex: (!, ·, +) = set complet fundamental de operatori.



Seturi complete de operatori

- (!, +) = set complet de operatori: de Morgan

$$a \cdot b = \overline{\bar{a} + \bar{b}}$$

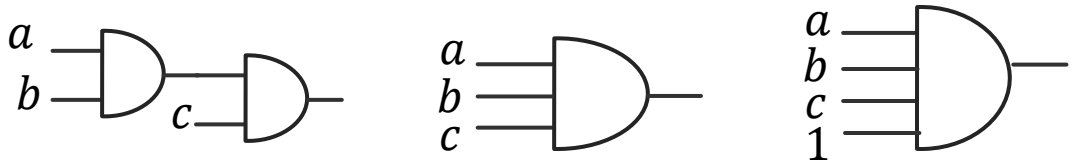
- (!, ·) = set complet de operatori: de Morgan

$$a + b = \overline{\bar{a} \cdot \bar{b}}$$

- Orice funcție poate fi implementată folosind exclusiv



- Observație: porțile pot avea și mai multe intrări, nu fix 2. În practică pot avea 2,3,4 sau 8 intrări.
- Ex: $a \cdot b \cdot c$ cu ȘI cu 2, 3, 4 intrări: $abc = (ab)c$



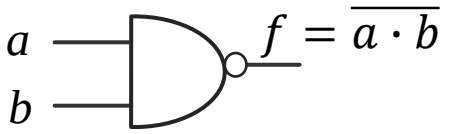
Seturi complete de operatori

- Operator nou: NAND – „ȘI-NU" (↑): $a \uparrow b = !\,(a \cdot b)$
- Operator nou: NOR – „SAU-NU" (↓): $a \downarrow b = !\,(a + b)$
- $a \cdot b = !\,(a \uparrow b)$ → $\cdot$ se poate exprima prin (!, ↑)
- Dar (!, $\cdot$) era set complet de operatori → (!, ↑) e set complet de operatori
- Similar, (!, ↓) e set complet de operatori pentru că (!, +) e set complet de operatori, iar $a + b = !\,(a \downarrow b)$
- $a \uparrow a = !\,(a \cdot a) = !\,a$ → ! se poate exprima prin ↑ și cum (!, ↑) e set complet de operatori → (↑) e set complet de operatori
- Similar, $a \downarrow a = !\,(a + a) = !\,a$ → (↓) e set complet de operatori

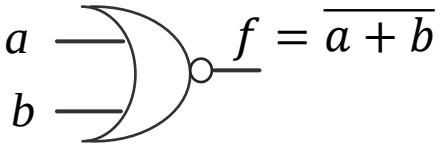
Seturi complete de operatori

- Datorită faptului că NOR și NAND reprezintă seturi complete de operatori, există elemente de comutație pentru ele:

a	b	$a \uparrow b$
0	0	1
0	1	1
1	0	1
1	1	0



a	b	$a \downarrow b$
0	0	1
0	1	0
1	0	0
1	1	0



Seturi complete de operatori

- Investigăm „ →” : $a \to b$ => $a \cdot !\, b = 0$ →
- $a \to b = !\,(a \cdot !\, b)$ → $!\,(a \to b) = a \cdot !\, b$
- Notăm $!\, b = B$ → $a \cdot B = !\,(a \to !\, B)$ → · se poate exprima prin ! Si →
- Cum (!, ·) = set complet de operatori → (!, →) = set complet de operatori
- Concluzie:
  - (!, ·), (↑), (!, +), (↓), (!, →) = seturi complete de operatori
  - (!, ·, +), (!, ↑), (!, ↓) = seturi supracomplete de operatori



Proiectare Logică

I CB: Prof. dr. ing. Mariana Mocanu

- mariana.mocanu@cs.pub.ro

I CA & CD: Conf. dr. ing. Costin Chiru

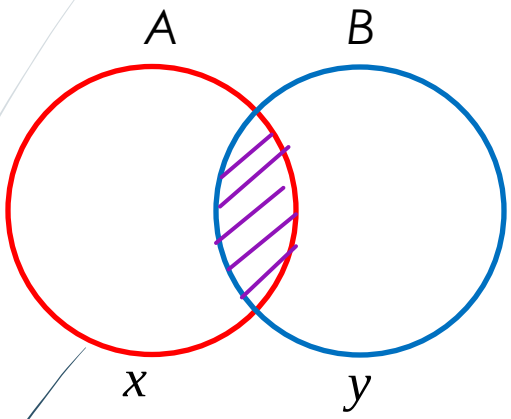
- costin.chiru@cs.pub.ro

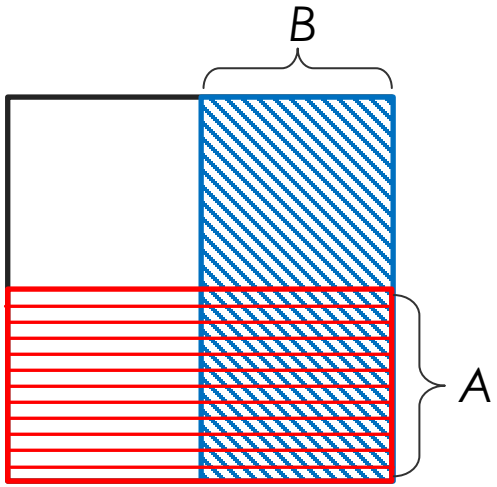
I CC: Prof. dr. ing. Anca Morar

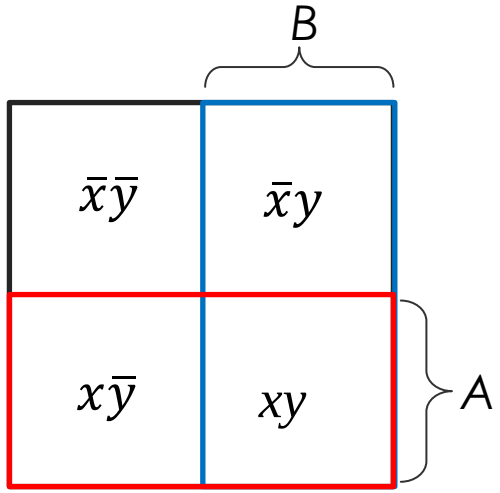
- anca.morar@cs.pub.ro

Curs 3: Clase de circuite digitale.
Reprezentarea grafică a funcțiilor booleene.
Reprezentarea algebrică a funcțiilor booleene.

Reprezentarea grafică – diagrame Venn & diagrame Karnaugh







x \ y	0	1
0	$\bar{x}\bar{y}$	$\bar{x}y$
1	$x\bar{y}$	xy

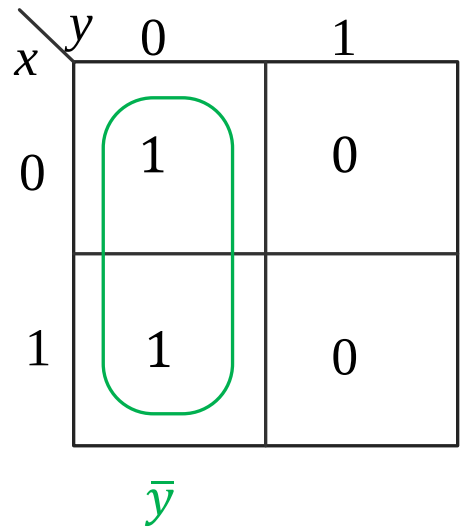
$$f(x,y) = c_1\bar{x}\bar{y} + c_2\bar{x}y + c_3x\bar{y} + c_4xy$$

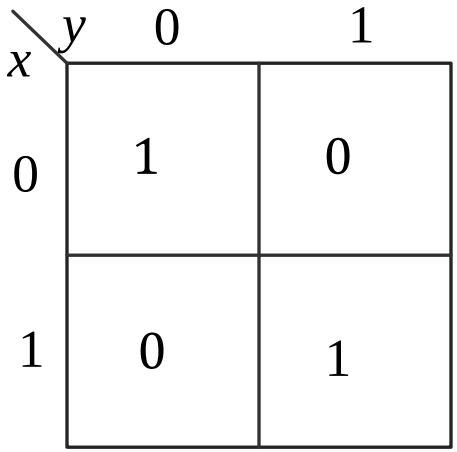
$c_k, k = \overline{1,4}$ sunt constante booleene

Diagrame Karnaugh – 2 variabile

- Utilizate pentru minimizarea funcțiilor
- Exemple:

$$f(x,y) = \bar{x}\bar{y} + x\bar{y} = \bar{y}$$





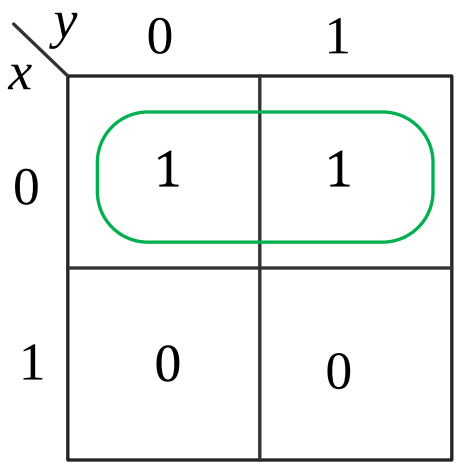
$$f(x,y) = \bar{x}\bar{y} + xy$$

Diagrame Karnaugh

- Grupând căsuțe adiacente putem obține un monom care depinde de mai puține variabile decât monoamele inițiale.
- Se pot grupa 2, 4, 8, 16 căsuțe adiacente care să formeze un dreptunghi sau un pătrat (căsuțele unite trebuie să fie ori pe aceeași linie, ori pe aceeași coloană, ori să creeze pătrate).
- Un termen poate să apară în oricâte grupări, condiția fiind să apară în minim 1.

Diagrame Karnaugh – exemplu

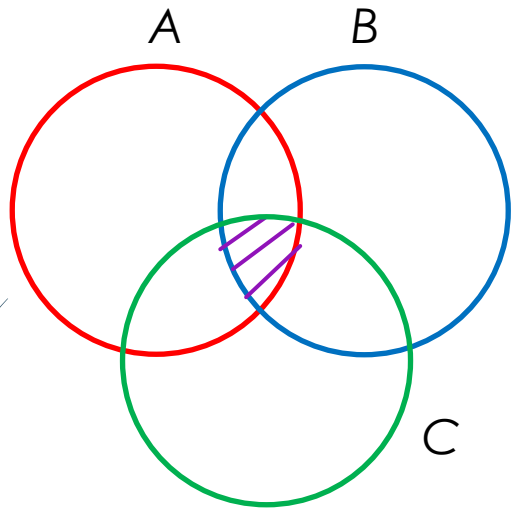
- Întotdeauna va dispărea variabila care își modifică valoarea



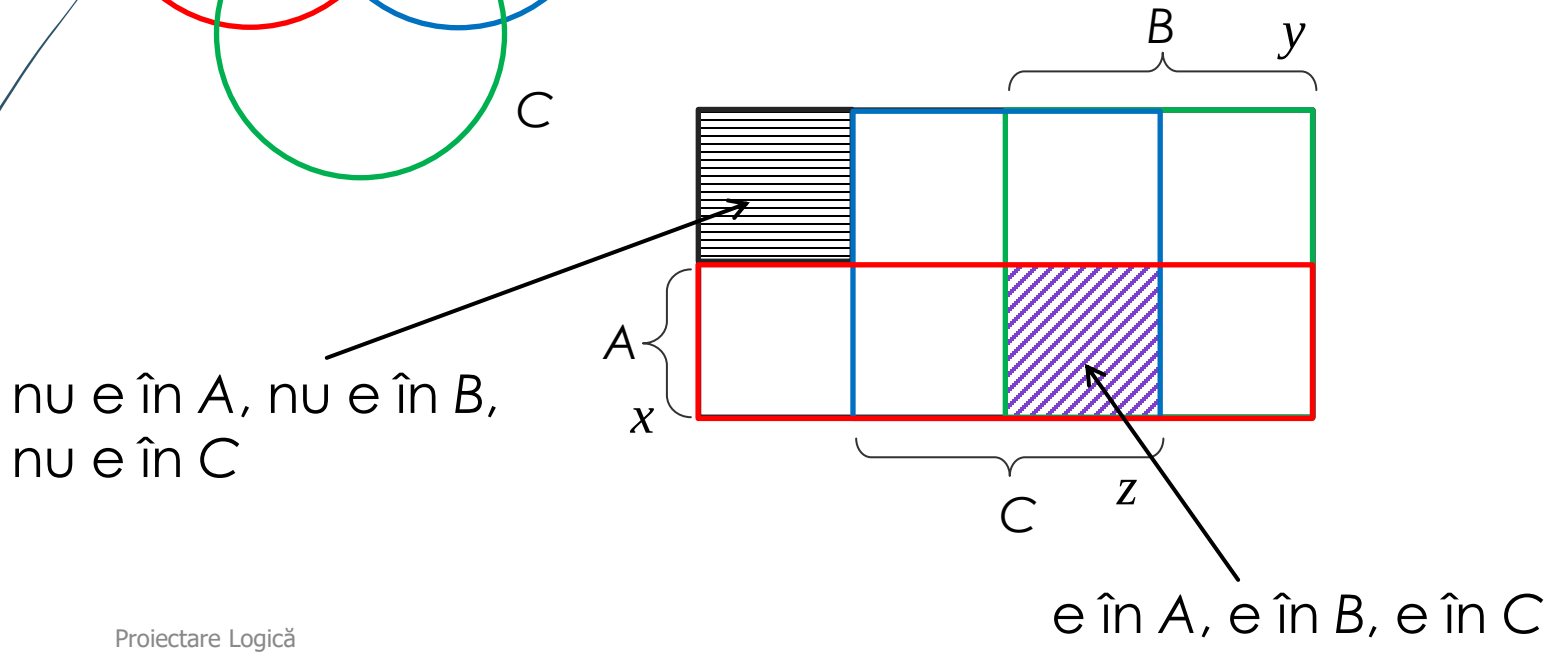
$$\bar{x}\bar{y} + \bar{x}y$$

Diagrame Karnaugh – 3 variabile

- Pentru funcții de 3 variabile:

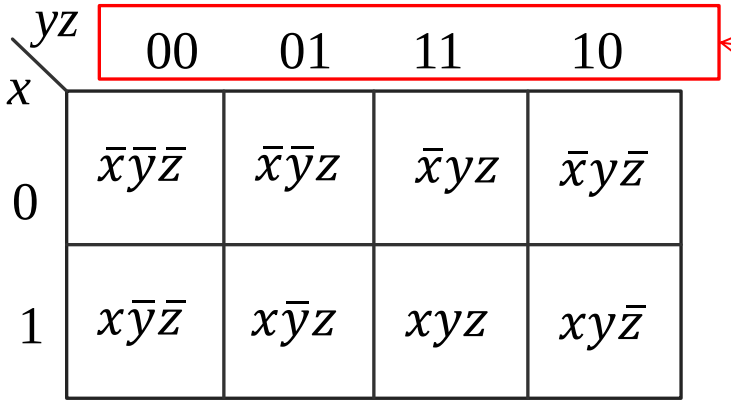


8 zone diferite (2^3)



Diagrame Karnaugh – cod Gray

- Pentru funcții de 3 variabile:



Cod GRAY

- Observații!
  - Fiecare căsuță corespunde unei combinații de n variabile
  - Codurile sunt adiacente (diferă printr-un singur bit) pentru toate căsuțele
  - Fiecare linie reprezintă o axă de simetrie (desparte căsuțe cu coduri adiacente – ce diferă printr-un singur bit)

Diagrame Karnaugh – Algoritm de minimizare (1)

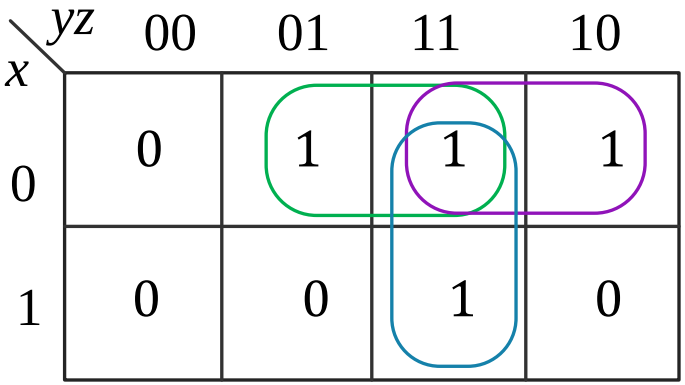
- Pentru fiecare produs din funcția de minimizat se trece 1 în căsuța corespunzătoare din diagramă;
- Se formează suprafețe de minimizare pătrate sau dreptunghiulare alcătuite din 1, 2, 4, 8 sau 16 pătrățele adiacente ce conțin unități;
  - **Observație**: Căsuțele de pe prima linie sunt adiacente cu cele de pe ultima linie, iar cele de pe prima coloană sunt adiacente cu cele de pe ultima coloană.
  - **Proprietăți ale suprafețelor de minimizare**:
    - Trebuie să aibă aria cât mai mare;
    - Trebuie să fie în număr cât mai redus;
    - O căsuță ce conține 1 trebuie să fie conținută în cel puțin o suprafață.

Diagrame Karnaugh – Algoritm de minimizare (2)

- Se formează produse pe baza suprafețelor de minimizare astfel:
  - Dacă în interiorul unei suprafețe de minimizare o variabilă își schimbă valoarea, atunci variabila respectivă nu mai apare în produs;
  - Dacă o variabilă își păstrează valoarea 0, atunci ea apare în produs în forma negată;
  - Dacă o variabilă își păstrează valoarea 1, atunci ea apare în produs în forma directă;
- Expresia minimizată este dată de suma produselor obținute la pasul anterior.

Diagrame Karnaugh – exemple

x \ yz	00	01	11	10
0	0	1	1	1
1	0	0	1	0



$$f = \bar{x}\bar{y}z + \bar{x}yz + \bar{x}y\bar{z} + xyz = \bar{x}z + \bar{x}y + yz$$

Diagrame Karnaugh - exemple

$$f = \bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + \bar{x}yz + xyz$$

x	y	z	f
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

z \ xy	00	01	11	10
0	1	1	0	0
1	0	1	1	0

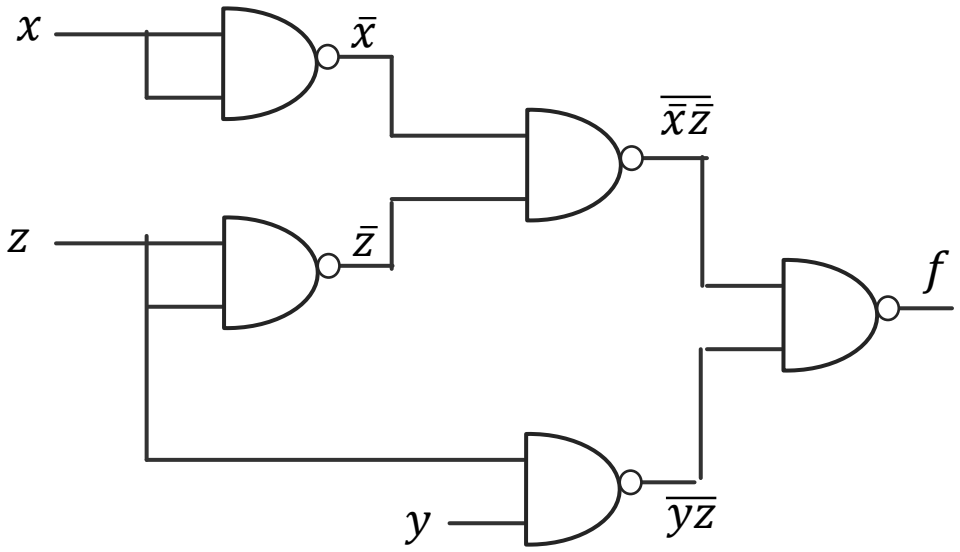
z \ xy	00	01	11	10
0	1	1	0	0
1	0	1	1	0

$$f = \bar{x}\bar{z} + yz = \overline{\overline{\bar{x}\bar{z}} \cdot \overline{yz}}$$

Diagrame Karnaugh – exemple

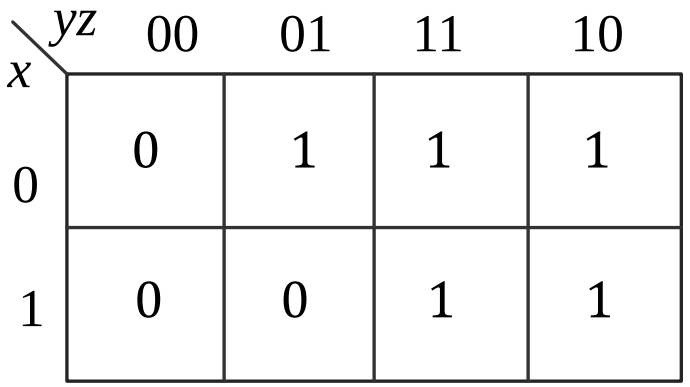
$$f = \bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + \bar{x}yz + xyz$$

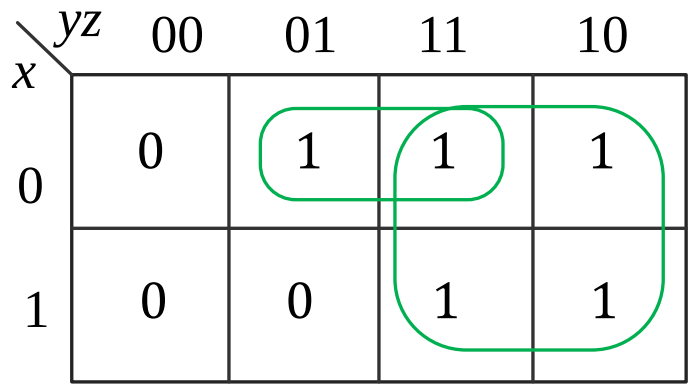
$$f = \bar{x}\bar{z} + yz = \overline{\overline{\bar{x}\bar{z}} \cdot \overline{yz}}$$



Diagrame Karnaugh – exemple

x	y	z	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

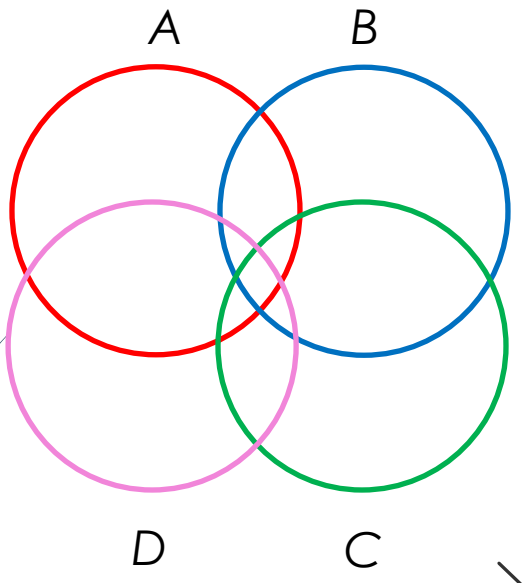


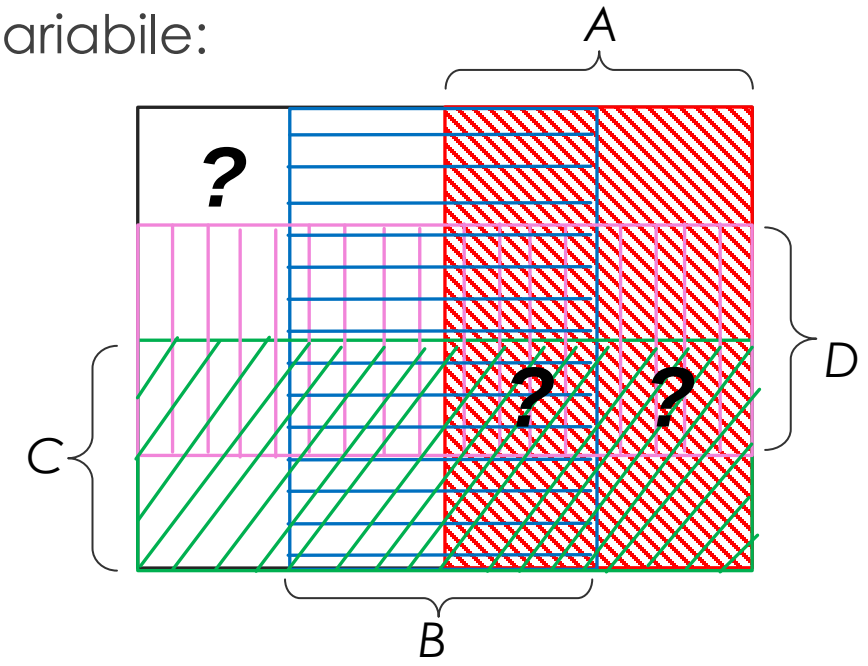


$$f = y + \bar{x}z$$

Diagrame Karnaugh – 4 variabile

- Pentru funcții de 4 variabile:

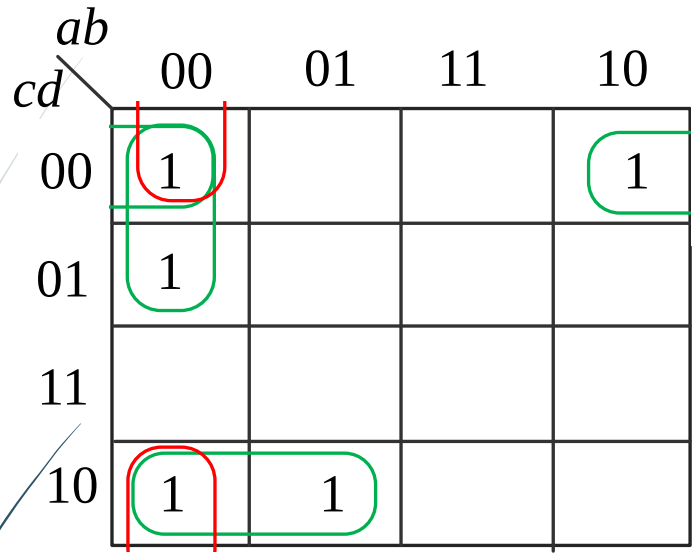




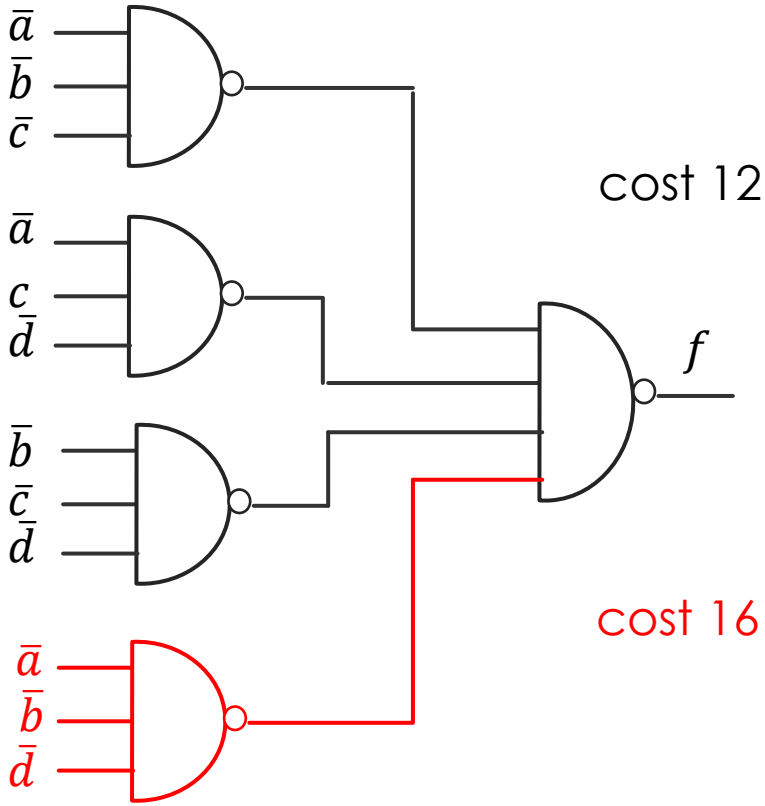
ab / cd	00	01	11	10
00	$\bar{a}\bar{b}\bar{c}\bar{d}$			
01				
11			$abcd$	$a\bar{b}cd$
10				

Ordinea de scriere a variabilelor: Cod Gray

Diagrame Karnaugh – exemple

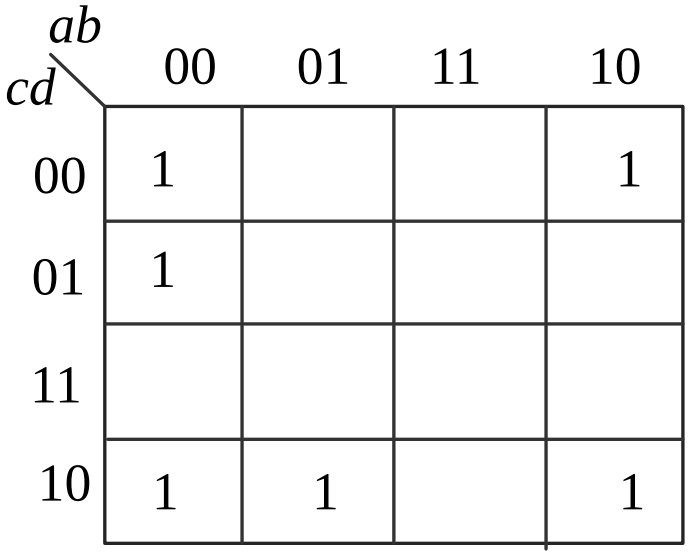


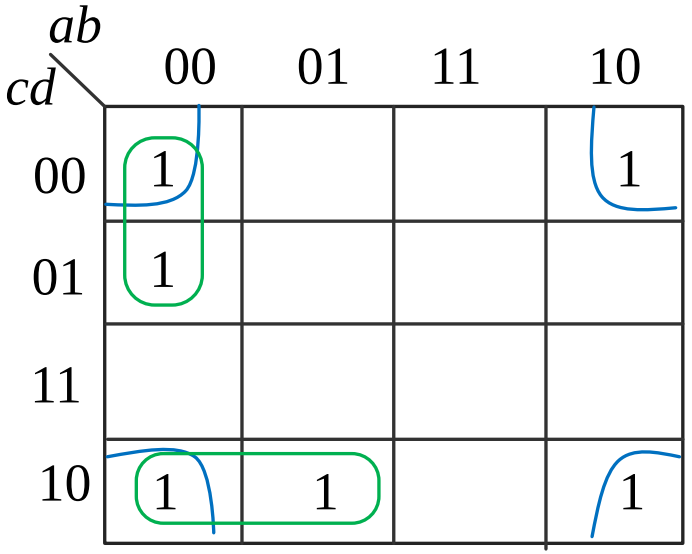
$$f = \bar{a}\bar{b}\bar{c} + \bar{a}c\bar{d} + \bar{b}\bar{c}\bar{d} + \bar{a}\bar{b}\bar{d}$$



- Costul unui circuit este dat de numărul de intrări al acestuia

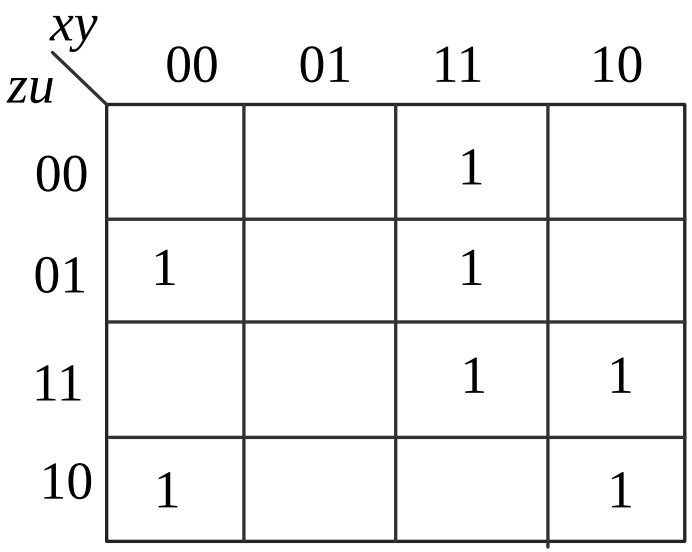
Diagrame Karnaugh – exemple

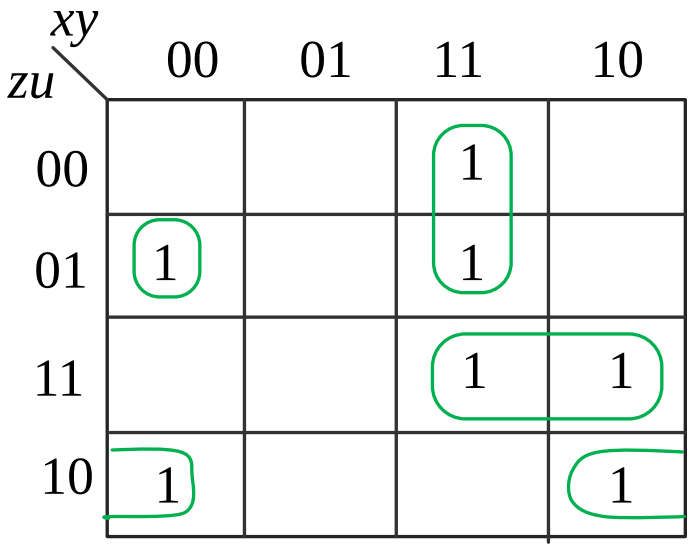




$$f = \bar{a}\bar{b}\bar{c} + \bar{a}c\bar{d} + \bar{b}\bar{d}$$

Diagrame Karnaugh – exemple

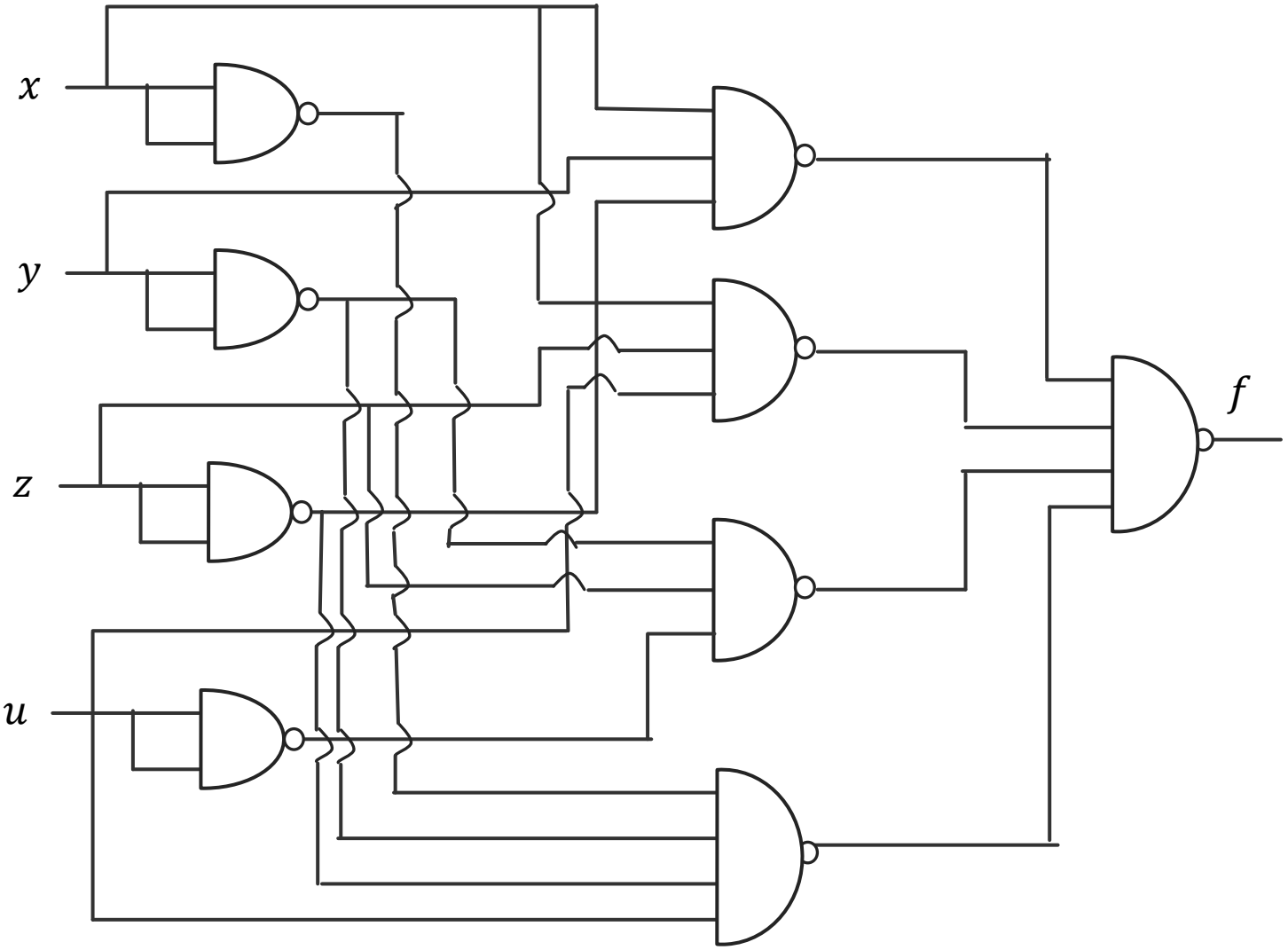




$$f(x,y,z,u) = xy\bar{z} + xzu + \bar{y}z\bar{u} + \bar{x}\bar{y}\bar{z}u$$

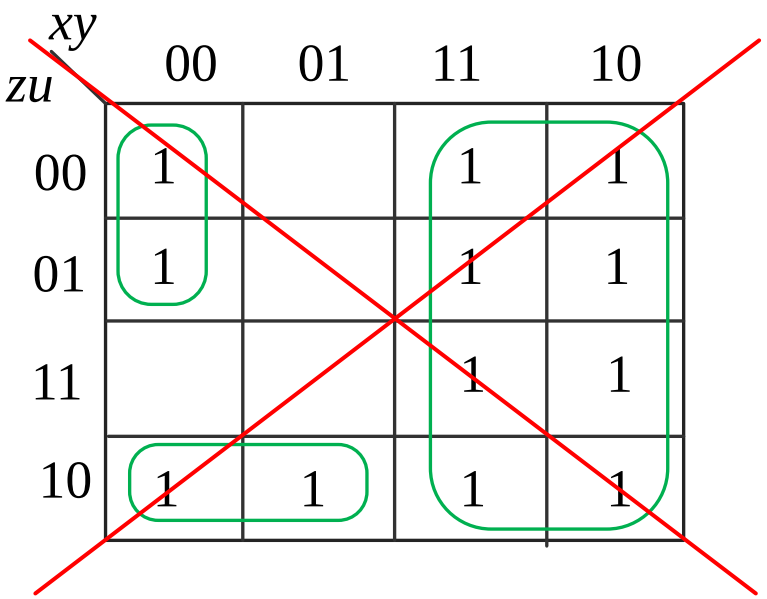
$$= \overline{\overline{xy\bar{z}} \cdot \overline{xzu} \cdot \overline{\bar{y}z\bar{u}} \cdot \overline{\bar{x}\bar{y}\bar{z}u}}$$

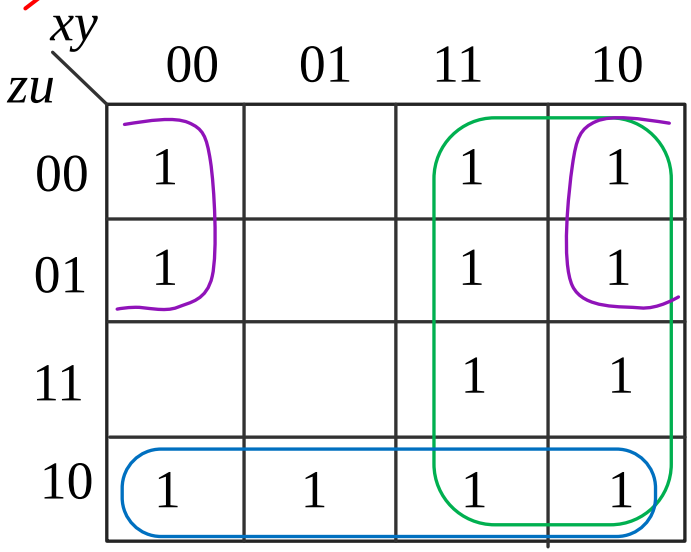
Diagrame Karnaugh – exemple



Diagrame Karnaugh – exemple

zu \ xy	00	01	11	10
00	1		1	1
01	1		1	1
11			1	1
10	1	1	1	1

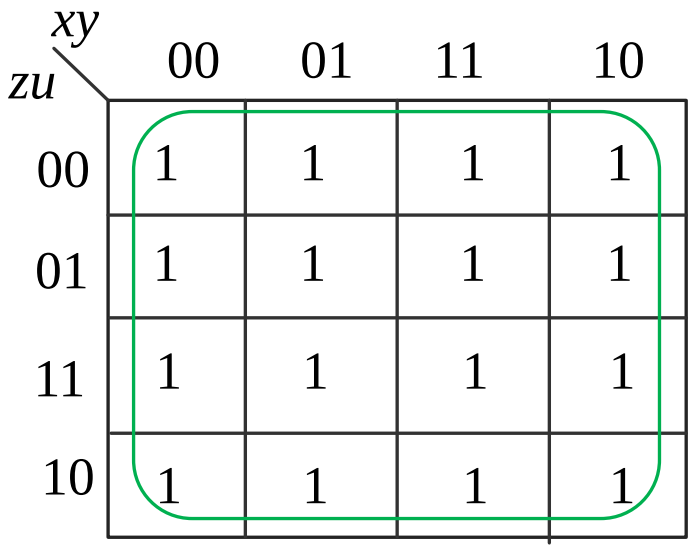




- $f = x + \bar{y}\bar{z} + z\bar{u}$

Diagrame Karnaugh – exemple

zu \ xy	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1



- $f = 1$
- Caz particular: orice funcție în FDNC are valoarea 1 (conform T16)

Diagrame Karnaugh – Funcții incomplet specificate

- Sunt funcții la care nu știm/nu ne interesează valorile de ieșire pentru anumite combinații de intrare
- Există situații când anumite combinații de intrare nu se pot folosi
- Aplicație: Să se proiecteze un circuit ce acceptă ca variabile de intrare codul binar corespunzător fiecărei luni în parte și generează la ieșire valoarea 1 pentru lunile ce au 30 zile
- Plecăm de la tabela de adevăr

Lunile anului (1)

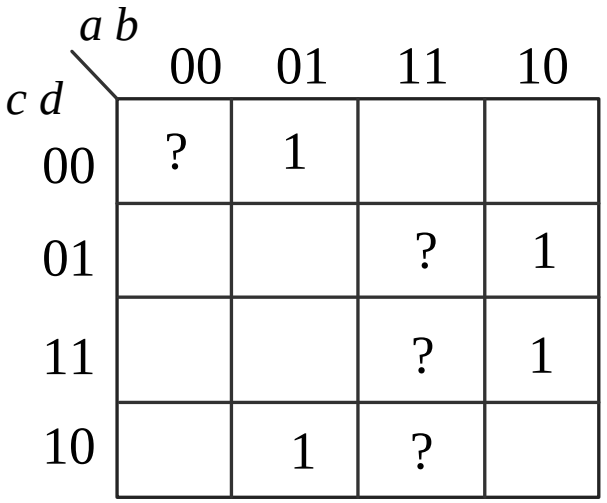
luna	*a*	*b*	*c*	*D*	*f(a,b,c,d)*
-	0	0	0	0	?
Ianuarie	0	0	0	1	0
Februarie	0	0	1	0	0
Martie	0	0	1	1	0
Aprilie	0	1	0	0	1
Mai	0	1	0	1	0
Iunie	0	1	1	0	1
Iulie	0	1	1	1	0
August	1	0	0	0	0
Septembrie	1	0	0	1	1
Octombrie	1	0	1	0	0
Noiembrie	1	0	1	1	1
Decembrie	1	1	0	0	0
-	1	1	0	1	?
-	1	1	1	0	?
-	1	1	1	1	?

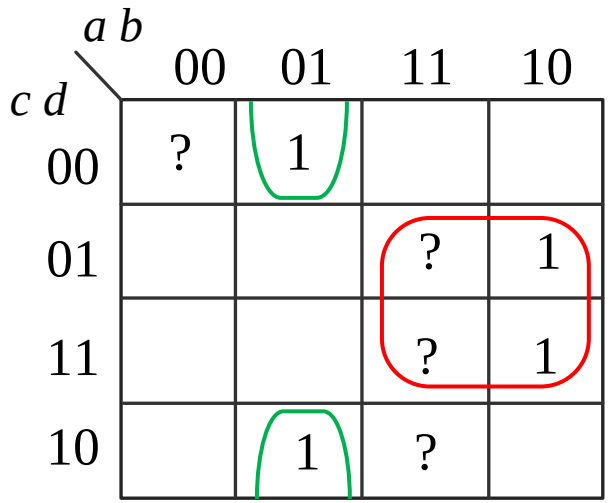
Lunile anului (2)

- Combinațiile marcate cu ? apar pentru că nu există luni cu codul respectiv → funcția e incomplet specificată (are valori redundante)
- Pe viitor, în loc de ? vom folosi fie * fie –
- Acum am folosit ? Pentru că nu știm ce dorim să punem acolo (0 sau 1) → cu aceste poziții facem ce ne convine mai tare astfel încât să obținem minimizarea optimă (se numesc poziții redundante)
- Facem diagrama Karnaugh

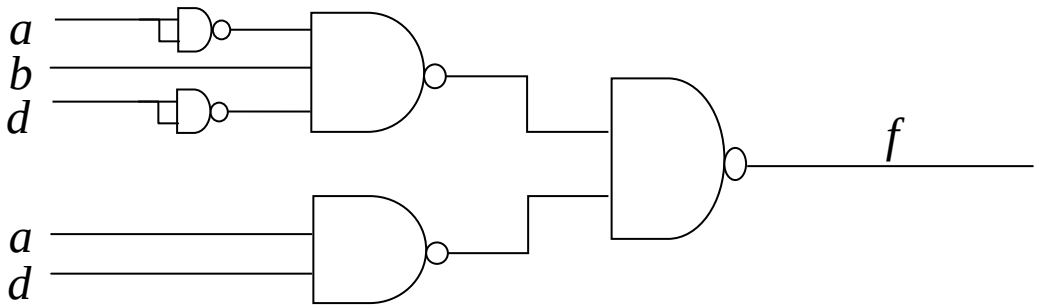
Lunile anului (3)

a	b	c	d	f
0	0	0	0	?
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	?
1	1	1	0	?
1	1	1	1	?





$$f = ad + \bar{a}b\bar{d}$$



Lunile anului (4)

- Observație:dacă în definirea problemei nu se specifica modul de codificare al lunilor, puteam să fac în felul următor:

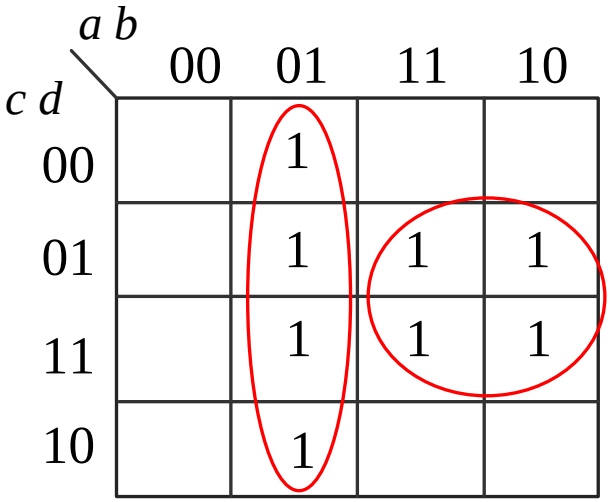
c d \ *a b*	00	01	11	10
00	ian	feb	mar	mai
01	iul	aug	oct	dec
11	?	?	apr	iun
10	?	?	sept	nov

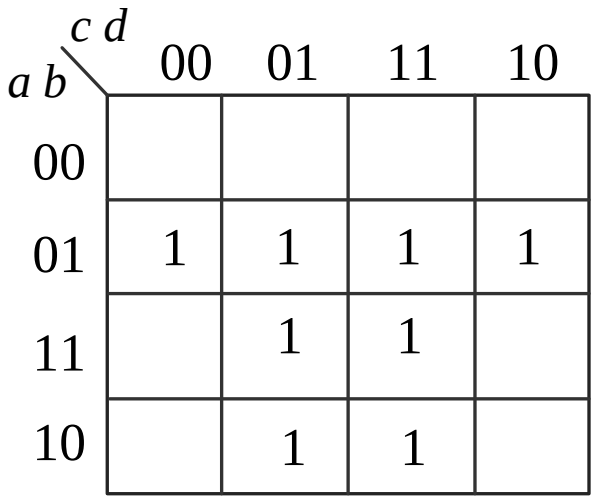
- $f = c$ → circuit mult mai simplu!
- Oricând aveți posibilitatea să alegeți codificarea, trebuie să încercați să o găsiți pe aceea care este cea mai convenabilă (vă conduce la minimizarea optimă)

Diagrame Karnaugh – exemplu

Minimizați $f(a,b,c,d) = a\bar{b}d + \bar{a}b\bar{d} + bcd + b\bar{c}d$

De câte căsuțe este nevoie pentru a reprezenta $a\bar{b}d$? De ce?



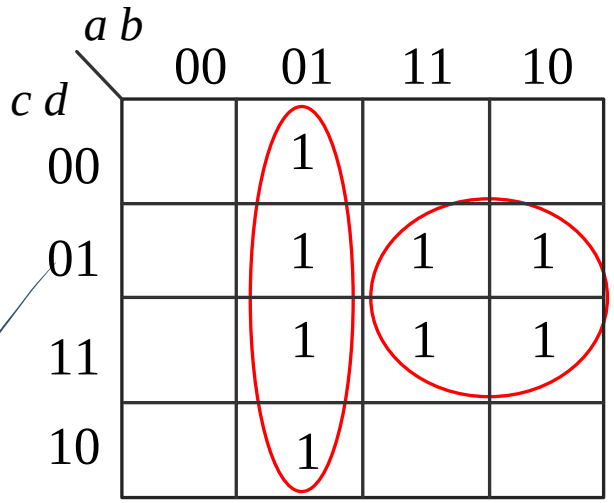


$$f = \bar{a}b + ad$$

Atenție la examen/lucrări: este posibil să vi se impună o anumită ordine a variabilelor! → Cele două diagrame diferă, dar de fapt este aceeași funcție.

Diagrame Karnaugh cu variabile externe (1)

Reprezentați funcția f pe o diagramă cu 3 variabile: a, b, c



a \ $b\,c$	00	01	11	10
0	$\bar{a}\bar{b}\bar{c}$	$\bar{a}\bar{b}c$	$\bar{a}bc$	$\bar{a}b\bar{c}$
1	$a\bar{b}\bar{c}$	$a\bar{b}c$	abc	$ab\bar{c}$

$$f = \bar{a}b + ad$$

$$f = \sum a_i^j b_k^l c_m^n = c_1\bar{a}\bar{b}\bar{c} + c_2\bar{a}\bar{b}c + c_3\bar{a}bc + c_4\bar{a}b\bar{c}$$

$$+c_5 a\bar{b}\bar{c} + c_6 a\bar{b}c + c_7 abc + c_8 ab\bar{c}$$

Diagrame Karnaugh cu variabile externe (2)

- Odată scrisă funcția în FDN (considerând doar variabilele a,b,c) nu ne rămâne decât să identificăm constantele pentru a le trece în diagramă
- $f(a,b,c,d) = a\bar{b}d + \bar{a}b\bar{d} + bcd + b\bar{c}d$
- Pentru a putea lucra mai ușor, trecem mai întâi funcția în FDN considerând toate cele 4 variabile:

$$f(a,b,c,d) = a\bar{b}cd + a\bar{b}\bar{c}d + \bar{a}bc\bar{d} + \bar{a}b\bar{c}\bar{d} + abcd + \bar{a}bcd + ab\bar{c}d + \bar{a}b\bar{c}d$$

Diagrame Karnaugh cu variabile externe (3)

- $f(a,b,c,d) = a\bar{b}cd + a\bar{b}\bar{c}d + \bar{a}bc\bar{d} + \bar{a}b\bar{c}\bar{d} + abcd + \bar{a}bcd + ab\bar{c}d + \bar{a}b\bar{c}d$
- Revenind la forma de 3 variabile:

- $f(a,b,c,d) = 0 \cdot \bar{a}\bar{b}\bar{c} + 0 \cdot \bar{a}\bar{b}c + \left(d + \bar{d}\right) \cdot \bar{a}bc + \left(d + \bar{d}\right) \cdot \bar{a}b\bar{c} + d \cdot a\bar{b}\bar{c} + d \cdot a\bar{b}c + d \cdot abc + d \cdot ab\bar{c}$

a \ $b\,c$	00	01	11	10
0	0	0	1	1
1	d	d	d	d

Diagrame Karnaugh cu variabile externe (4)

- Orice funcție de un număr de variabile se poate reprezenta printr-o altă diagramă cu un număr mai mic sau mai mare de variabile (dar nu are niciun sens să reprezentăm prin mai multe variabile)
- Ex: 2 variabile:

$$f(a,b,c,d) = a\bar{b}cd + a\bar{b}\bar{c}d + \bar{a}bc\bar{d} + \bar{a}b\bar{c}\bar{d} + abcd + \bar{a}bcd + ab\bar{c}d + \bar{a}b\bar{c}d$$

a \ b	0	1
0	$\bar{a}\bar{b}$	$\bar{a}b$
1	$a\bar{b}$	ab

a \ b	0	1
0	$0_{\bar{a}\bar{b}}$	$1_{\bar{a}b}$
1	$d_{a\bar{b}}$	d_{ab}

a \ c	0	1
0	$\bar{a}\bar{c}$	$\bar{a}c$
1	$a\bar{c}$	ac

a \ c	0	1
0	$b_{\bar{a}\bar{c}}$	$b_{\bar{a}c}$
1	$d_{a\bar{c}}$	d_{ac}

Minimizarea diagramelor Karnaugh cu variabile externe

- 1. Toate variabilele externe se consideră = 0 și se minimizează diagrama conform regulilor cunoscute
- 2. Pe rând, fiecare variabilă externă se face = 1, iar unitățile existente în diagramă se consideră redundante (se iau în considerare sau nu în funcție de necesități - similar cu ? de la funcții incomplet specificate). Diagrama se minimizează și apoi rezultatul se înmulțește logic cu variabila externă considerată
- 3. Rezultatul final este dat de suma produselor obținute în pașii anteriori
- Această metodă de minimizare este posibilă prin aplicarea superpoziției (suprapunem rezultatele).

Minimizarea diagramelor Karnaugh cu variabile externe

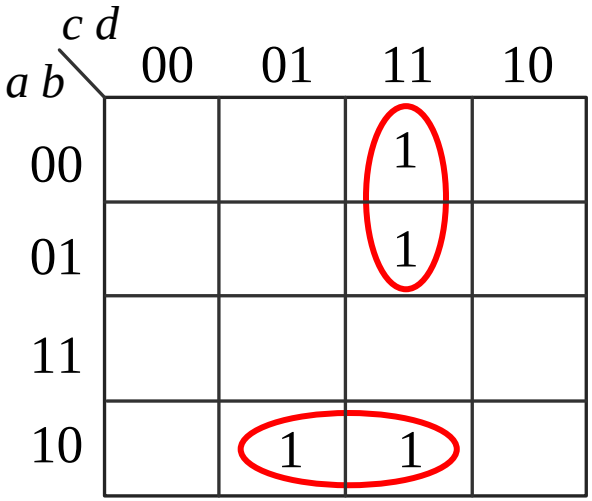
- În cazul în care avem de minimizat o diagramă ce conține atât o variabilă cât și negatul acesteia, acestea se consideră ca fiind două variabile diferite independente (fără nicio legătură una cu alta).
- În cazul în care avem variabile externe în cadrul funcțiilor incomplet specificate, pozițiile nespecificate pot fi considerate diferit în cei doi pași (pot fi într-un pas 0 și în altul 1, de exemplu).

Minimizarea diagramelor Karnaugh cu variabile externe
Exemplu (1)

- Minimizați funcția de mai jos

$a\,b$ \ $c\,d$	00	01	11	10
00		y	1	
01			1	
11			x	$\bar{x}$
10		1	1	

Pasul 1



$$f' = a\bar{b}d + \bar{a}cd$$

Minimizarea diagramelor Karnaugh cu variabile externe
Exemplu (2)

- Minimizați funcția de mai jos

Pasul 2 – se face de 3 ori pentru că avem 3 variabile diferite (x, y și $\bar{x}$)

$a\ b$ \ $c\ d$	00	01	11	10
00		y	1	
01			1	
11			x	$\bar{x}$
10		1	1	

$a\ b$ \ $c\ d$	00	01	11	10
00			?	
01			?	
11			1	
10		?	?	

Avem posibilitatea să folosim dintre unități doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

2.1: $x = 1 \rightarrow cdx$

Minimizarea diagramelor Karnaugh cu variabile externe Exemplu (3)

- Minimizați funcția de mai jos

Pasul 2 – se face de 3 ori pentru că avem 3 variabile diferite (x, y și $\bar{x}$)

$a\ b$ \ $c\ d$	00	01	11	10
00		y	1	
01			1	
11			x	$\bar{x}$
10		1	1	

$a\ b$ \ $c\ d$	00	01	11	10
00		1	?	
01			?	
11				
10		?	?	

Avem posibilitatea să folosim dintre unități doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

2.2: $y = 1 \rightarrow \bar{b}dy$

Minimizarea diagramelor Karnaugh cu variabile externe
Exemplu (4)

- Minimizați funcția de mai jos

$a\,b$ \ $c\,d$	00	01	11	10
00		y	1	
01			1	
11			x	$\bar{x}$
10		1	1	

Pasul 2 – se face de 3 ori pentru că avem 3 variabile diferite (x, y și $\bar{x}$)

$a\,b$ \ $c\,d$	00	01	11	10
00			?	
01			?	
11				1
10		?	?	

Avem posibilitatea să folosim dintre unități doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

2.3: $\bar{x} = 1 \rightarrow abcd\bar{x}$

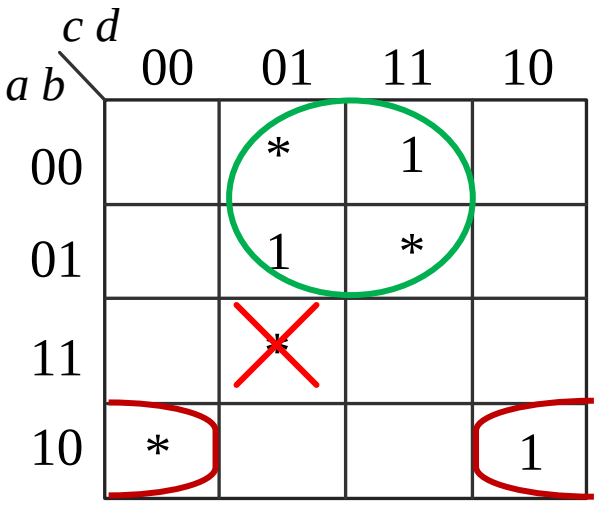
$$f = a\bar{b}d + \bar{a}cd + cdx + \bar{b}dy + abcd\bar{x}$$

Minimizarea diagramelor Karnaugh cu variabile externe
Alt exemplu (1)

- Minimizați funcția de mai jos

$a\,b$ \ $c\,d$	00	01	11	10
00		*	1	x
01	x	1	*	y
11		*	x	y
10	*	x	y	1

Pasul 1



Avem posibilitatea să folosim dintre steluțe doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

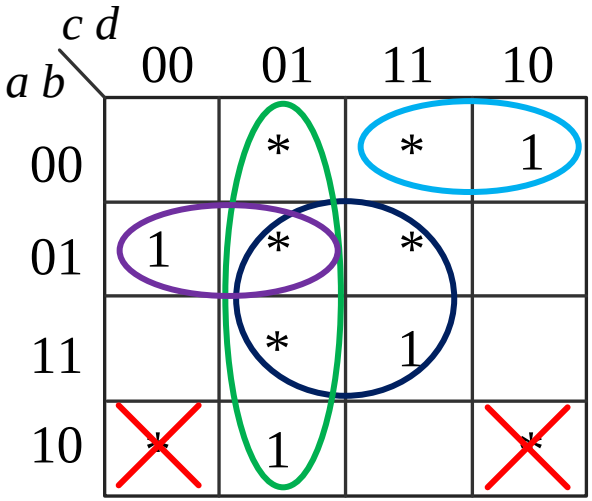
$$f' = \bar{a}d + a\bar{b}\bar{d}$$

Minimizarea diagramelor Karnaugh cu variabile externe – Alt exemplu (2)

- Minimizați funcția de mai jos

Pasul 2 – se face de 2 ori pentru că avem 2 variabile diferite (x și y)

$a\,b$ \ $c\,d$	00	01	11	10
00		*	1	x
01	x	1	*	y
11		*	x	y
10	*	x	y	1



Avem posibilitatea să folosim dintre unități / steluțe doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

2.1: $x = 1 \rightarrow$

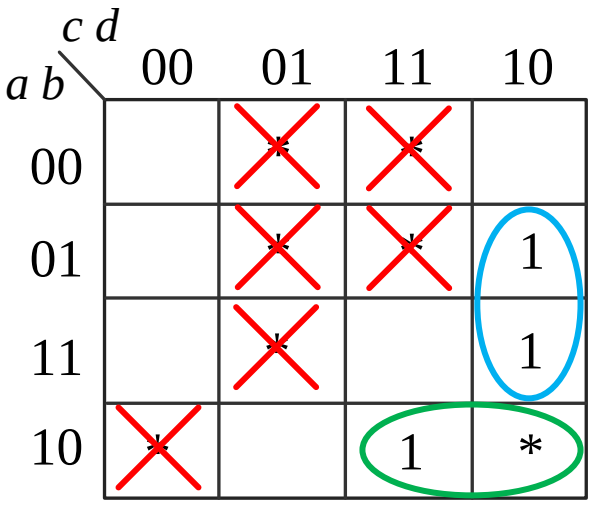
$$\bar{c}dx + bdx + \bar{a}\bar{b}cx + \bar{a}b\bar{c}x$$

Minimizarea diagramelor Karnaugh cu variabile externe – Alt exemplu (2)

- Minimizați funcția de mai jos

Pasul 2 – se face de 2 ori pentru că avem 2 variabile diferite (x și y)

$a\,b$ \ $c\,d$	00	01	11	10
00		*	1	x
01	x	1	*	y
11		*	x	y
10	*	x	y	1



Avem posibilitatea să folosim dintre unități / steluțe doar pe acelea care ne sunt utile și să le eliminăm pe celelalte.

2.2: $y = 1 \rightarrow a\bar{b}cy + bc\bar{d}y$

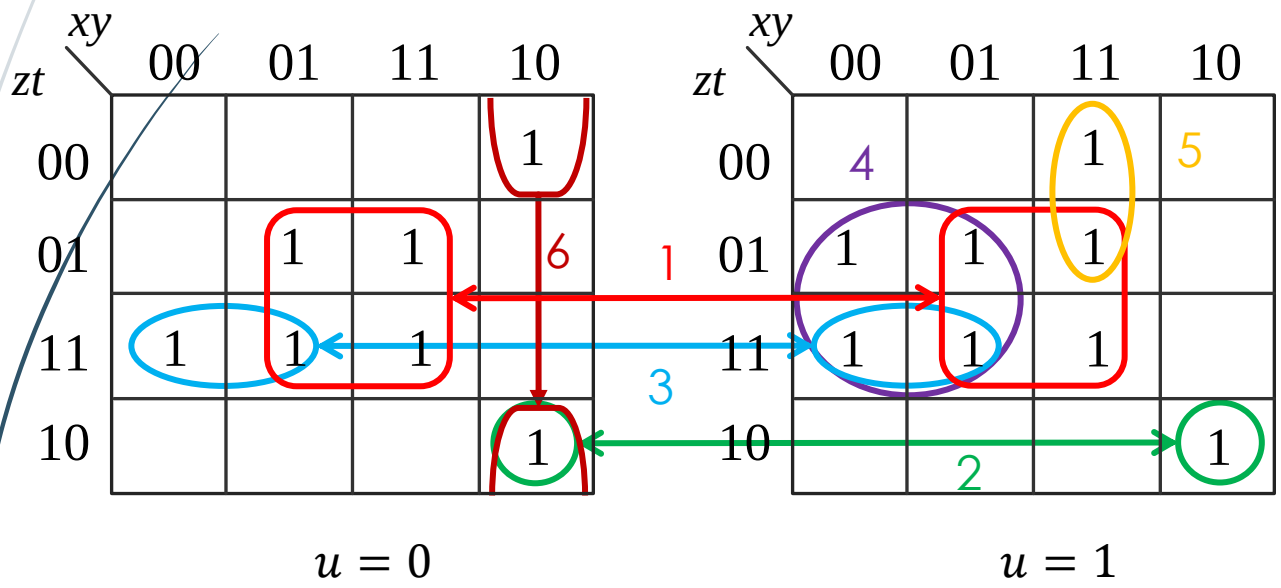
$$f = \bar{a}d + a\bar{b}\bar{d} + \bar{c}dx + bdx + \bar{a}\bar{b}cx + \bar{a}b\bar{c}x + a\bar{b}cy + bc\bar{d}y$$

Diagrame Karnaugh pentru 5 variabile

- Se recomandă folosirea a 2 diagrame Karnaugh de câte 4 variabile: una când a 5-a variabilă este 0 și alta când este 1
- În diagramă, fiecare linie era o axă de simetrie
- Acum, fiecare plan este o axă de simetrie (fiecare căsuță dintr-o diagramă diferă de căsuța corespunzătoare din cealaltă diagramă printr-un singur bit)

Diagrame Karnaugh pentru 5 variabile - exemplu

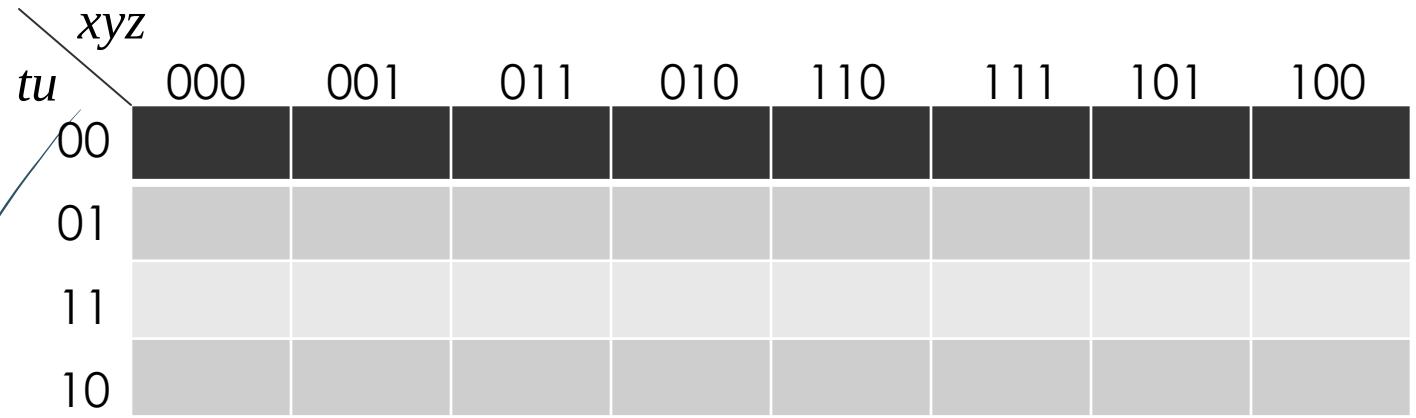
$$f(x,y,z,t,u) =$$
$$\bar{x}\bar{y}zt\bar{u} + \bar{x}y\bar{z}t\bar{u} + xy\bar{z}t\bar{u} + xyzt\bar{u} + x\bar{y}\bar{z}\bar{t}\bar{u} + x\bar{y}z\bar{t}\bar{u} + \bar{x}yzt\bar{u} +$$
$$\bar{x}\bar{y}\bar{z}tu + \bar{x}\bar{y}ztu + \bar{x}y\bar{z}tu + \bar{x}yztu + xy\bar{z}\bar{t}u + xy\bar{z}tu + xyztu + x\bar{y}z\bar{t}u$$



1. $yt\bar{u} + ytu = yt$
2. $x\bar{y}z\bar{t}$
3. $\bar{x}zt$
4. $\bar{x}tu$
5. $xy\bar{z}u$
6. $x\bar{y}\bar{t}\bar{u}$

Diagrame Karnaugh pentru 5 variabile - alternativă

- O singură diagramă în care să folosim codul Gray pe 3 biți:



- Nu veți avea așa ceva la examen / lucrări /proiect.

Diagrame Karnaugh pentru 5 variabile – cu variabile externe

$$f(x,y,z,t,u) = \bar{x}\bar{y}zt\bar{u} + \bar{x}y\bar{z}t\bar{u} + xy\bar{z}t\bar{u} + xyzt\bar{u} + x\bar{y}\bar{z}\bar{t}\bar{u} + x\bar{y}z\bar{t}\bar{u} + \bar{x}yzt\bar{u} + \bar{x}\bar{y}\bar{z}tu + \bar{x}\bar{y}ztu + \bar{x}y\bar{z}tu + \bar{x}yztu + xy\bar{z}\bar{t}u + xy\bar{z}tu + xyztu + x\bar{y}z\bar{t}u$$

zt \ xy	00	01	11	10
00			u	$\bar{u}$
01	u	1	1	
11	1	1	1	
10				1

x	y	z	t	f
0	0	0	0	
0	0	0	1	u
0	0	1	0	
0	0	1	1	1
0	1	0	0	
0	1	0	1	1
0	1	1	0	
0	1	1	1	1
1	0	0	0	$\bar{u}$
1	0	0	1	
1	0	1	0	1
1	0	1	1	
1	1	0	0	u
1	1	0	1	1
1	1	1	0	
1	1	1	1	1

Diagrame Karnaugh pentru 5 variabile – cu variabile externe

$$f(x,y,z,t,u) =$$
$$\bar{x}\bar{y}zt\bar{u} + \bar{x}y\bar{z}t\bar{u} + xy\bar{z}t\bar{u} + xyzt\bar{u}$$
$$+ x\bar{y}\bar{z}\bar{t}\bar{u} + x\bar{y}z\bar{t}\bar{u} + \bar{x}yzt\bar{u} +$$
$$\bar{x}\bar{y}\bar{z}tu + \bar{x}\bar{y}ztu + \bar{x}y\bar{z}tu + \bar{x}yztu$$
$$+ xy\bar{z}\bar{t}u + xy\bar{z}tu + xyztu + x\bar{y}z\bar{t}u$$

zt \ xy	00	01	11	10
00			u	$\bar{u}$
01	u	1	1	
11	1	1	1	
10				1

Pasul 1 – minimizare pt 1

zt \ xy	00	01	11	10
00				
01		1	1	
11	1	1	1	
10				1

$$yt + \bar{x}zt + x\bar{y}z\bar{t}$$

Diagrame Karnaugh pentru 5 variabile – cu variabile externe

$$f(x,y,z,t,u) =$$
$$\bar{x}\bar{y}zt\bar{u} + \bar{x}y\bar{z}t\bar{u} + xy\bar{z}t\bar{u} + xyzt\bar{u}$$
$$+ x\bar{y}\bar{z}\bar{t}\bar{u} + x\bar{y}z\bar{t}\bar{u} + \bar{x}yzt\bar{u} +$$
$$\bar{x}\bar{y}\bar{z}tu + \bar{x}\bar{y}ztu + \bar{x}y\bar{z}tu + \bar{x}yztu$$
$$+ xy\bar{z}\bar{t}u + xy\bar{z}tu + xyztu + x\bar{y}z\bar{t}u$$

zt \ xy	00	01	11	10
00			u	$\bar{u}$
01	u	1	1	
11	1	1	1	
10				1

Pasul 2.1 – minimizare pt u

zt \ xy	00	01	11	10
00			1	
01	1	*	*	
11	*	*	*	
10				*

$$\bar{x}tu + xy\bar{z}u$$

Diagrame Karnaugh pentru 5 variabile – cu variabile externe

$$f(x,y,z,t,u) =$$
$$\bar{x}\bar{y}zt\bar{u} + \bar{x}y\bar{z}t\bar{u} + xy\bar{z}t\bar{u} + xyzt\bar{u}$$
$$+ x\bar{y}\bar{z}\bar{t}\bar{u} + x\bar{y}z\bar{t}\bar{u} + \bar{x}yzt\bar{u} +$$
$$\bar{x}\bar{y}\bar{z}tu + \bar{x}\bar{y}ztu + \bar{x}y\bar{z}tu + \bar{x}yztu$$
$$+ xy\bar{z}\bar{t}u + xy\bar{z}tu + xyztu + x\bar{y}z\bar{t}u$$

zt \ xy	00	01	11	10
00			u	$\bar{u}$
01	u	1	1	
11	1	1	1	
10				1

Pasul 2.2 – minimizare pt $\bar{u}$

zt \ xy	00	01	11	10
00				1
01		*	*	
11	*	*	*	
10				*

$x\bar{y}\bar{t}\bar{u}$

Reprezentarea zecimală a funcțiilor

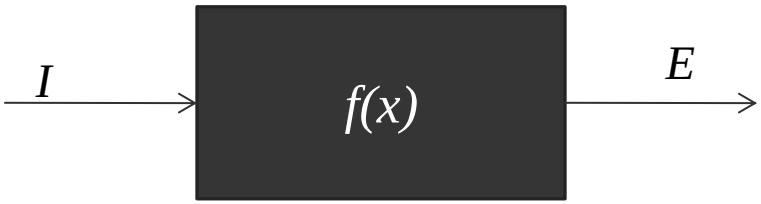
Termen	x	y	z	f
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	0
7	1	1	1	1

$$f(x,y,z) = \bar{x}\bar{y}z + \bar{x}y\bar{z} + xyz$$

$$f = \sum\{1,2,7\}$$

Clase de circuite digitale

- 2 categorii mari de circuite:
  - **Combinaționale** – porți, MUX, decodificatoare, etc. – modificările intrărilor se reflectă „instantaneu” la ieșire
  - **Secvențiale** – circuite a căror ieșire depinde și de momentul de timp (istoria circuitului), nu numai de intrări → chiar dacă nu se modifică intrările, valoarea ieșirii lui f de la momentul t poate fi diferită de valoarea ieșirii lui f de la momentul $t + 1$



Reprezentarea standard a circuitelor

Circuite combinaționale: $f(a, b)$

Circuite secvențiale: $f(a, b, t)$



Proiectare Logică

- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- **I CC: Anca Morar** anca.morar@cs.pub.ro
- I CA & CD: Costin Chiru costin.chiru@cs.pub.ro

Curs 4. Circuite combinationale MSI (multiplexoare, decodificatoare, demultiplexoare, comparatoare, sumatoare)

Circuite combinaționale

- Porți
- Multiplexoare / demultiplexoare
- Codificatoare / decodificatoare
- Comparatoare
- Convertoare
- Sumatoare



$$Q^t = f(I_1{}^t, ..., I_n{}^t)$$

Porți (Gates)

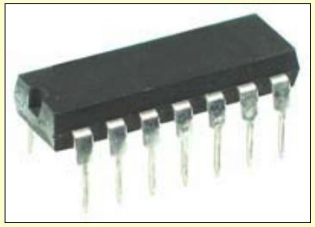
- Inversor
- Repetor
- Poarta ȘI *(AND)*
- Poarta SAU *(OR)*
- Poarta ȘI-NU *(NAND)*
- Poarta SAU-NU *(NOR)*
- Poarta SAU-EXCLUSIV *(XOR)*
- Poarta SAU-EXCLUSIV-NEGAT *(XNOR)*

- 2, 3, 4, 8 intrări
- Timp de propagare
- Timp de tranziție
- Fan-out
- Intrările nefolosite nu se lasă în aer, se conectează între ele sau se leagă la 1 logic

Poarta NAND cu 2 intrări

7400 - 7400 Quad 2-Input NAND Gate Datasheet

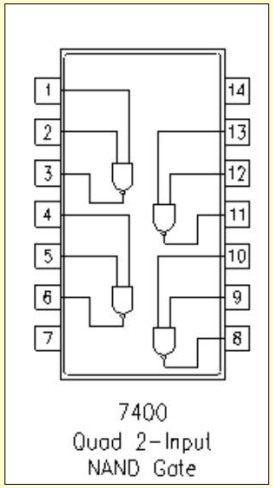
Photograph



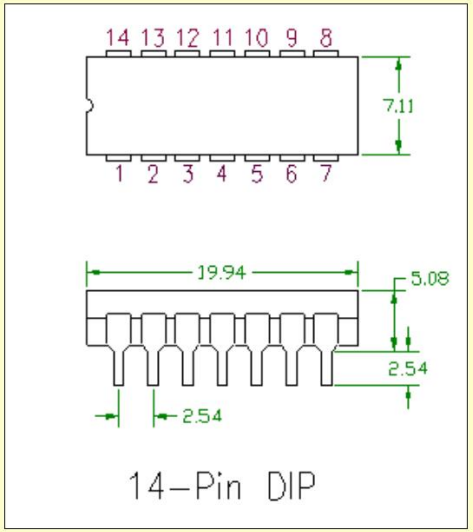
Features

- Four Independent 2-Input NAND Gates
- Outputs Directly Interface to CMOS, NMOS and TTL
- Large Operating Voltage Range
- Wide Operating Conditions
- Not Recommended for New Designs Use 74LS00

Pin Layout



Dimensional Drawing



SNx400, SNx4LS00, and SNx4S00 Quadruple 2-Input Positive-NAND Gates

1 Features

- Package Options Include:
  - Plastic Small-Outline (D, NS, PS)
  - Shrink Small-Outline (DB)
  - Ceramic Flat (W)
  - Ceramic Chip Carriers (FK)
  - Standard Plastic (N)
  - Ceramic (J)
- Also Available as Dual 2-Input Positive-NAND Gate in Small-Outline (PS) Package
- Inputs Are TTL Compliant; V_{IH} = 2 V and V_{IL} = 0.8 V
- Inputs Can Accept 3.3-V or 2.5-V Logic Inputs
- SN5400, SN54LS00, and SN54S00 are Characterized For Operation Over the Full Military Temperature Range of –55°C to 125°C

2 Applications

- AV Receivers
- Portable Audio Docks
- Blu-Ray Players
- Home Theater
- MP3 Players or Recorders
- Personal Digital Assistants (PDAs)

3 Description

The SNx4xx00 devices contain four independent, 2-input NAND gates. The devices perform the Boolean function $Y = \overline{A \cdot B}$ or $Y = \overline{A} + \overline{B}$ in positive logic.

Device Information(1)

PART NUMBER	PACKAGE	BODY SIZE (NOM)
SN74LS00DB	SSOP (14)	6.20 mm × 5.30 mm
SN7400D, SN74LS00D, SN74S00D	SOIC (14)	8.65 mm × 3.91 mm
SN74LS00NSR	PDIP (14)	19.30 × 6.35 mm
SNJ5400J, SNJ54LS00J, SNJ54S00J	CDIP (14)	19.56 mm × 6.67 mm
SNJ5400W, SNJ54LS00W, SNJ54S00W	CFP (14)	9.21 mm × 5.97 mm
SN54LS00FK, SN54S00FK	LCCC (20)	8.89 mm × 8.89 mm
SN7400NS, SN74LS00NS, SN74S00NS	SO (14)	10.30 mm × 5.30 mm
SN7400PS, SN74LS00PS	SO (8)	6.20 mm × 5.30 mm

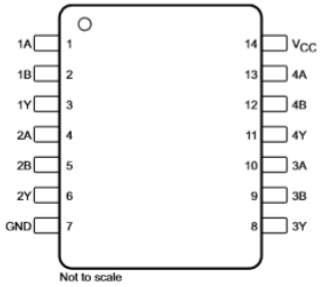
(1) For all available packages, see the orderable addendum at the end of the data sheet.

Logic Diagram, Each Gate (Positive Logic)



5 Pin Configuration and Functions

SN5400 J, SN54xx00 J and W, SN74x00 D, N, and NS
SN74LS00 D, DB, N, and NS Packages
14-Pin CDIP, CFP, SOIC, PDIP, SO, or SSOP
Top View



Pin Functions

PIN					I/O	DESCRIPTION
NAME	CDIP, CFP, SOIC, PDIP, SO, SSOP	SO (SN74xx00)	CFP (SN5400)	LCCC		
1A	1	1	1	2	I	Gate 1 input
1B	2	2	2	3	I	Gate 1 input
1Y	3	3	3	4	O	Gate 1 output
2A	4	6	6	6	I	Gate 2 input
2B	5	7	7	8	I	Gate 2 input
2Y	6	5	5	9	O	Gate 2 output
3A	10	—	9	13	I	Gate 3 input
3B	9	—	10	14	I	Gate 3 input

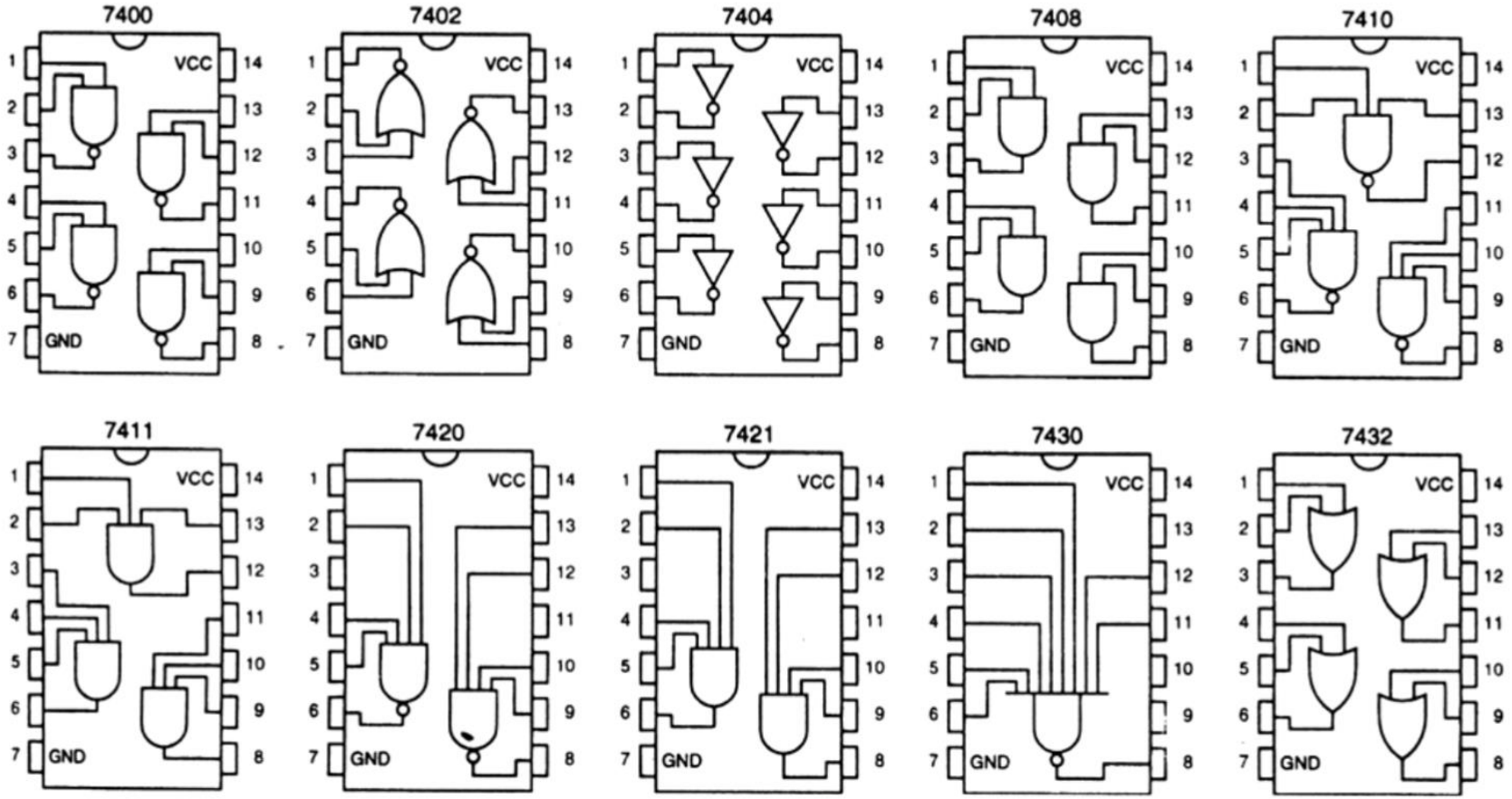
Copyright © 1983–2017, Texas Instruments Incorporated

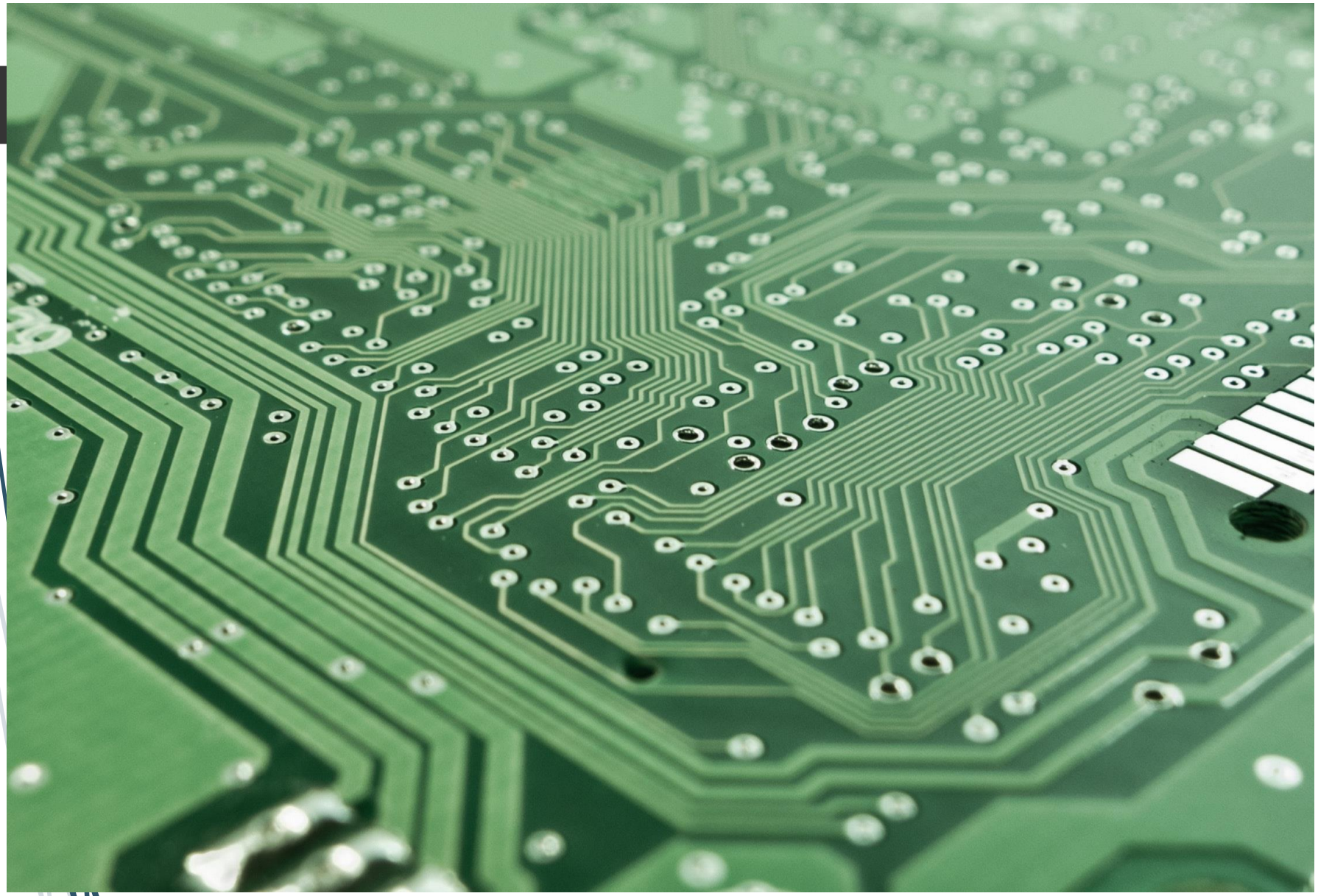
6.5 Electrical Characteristics: SNx400

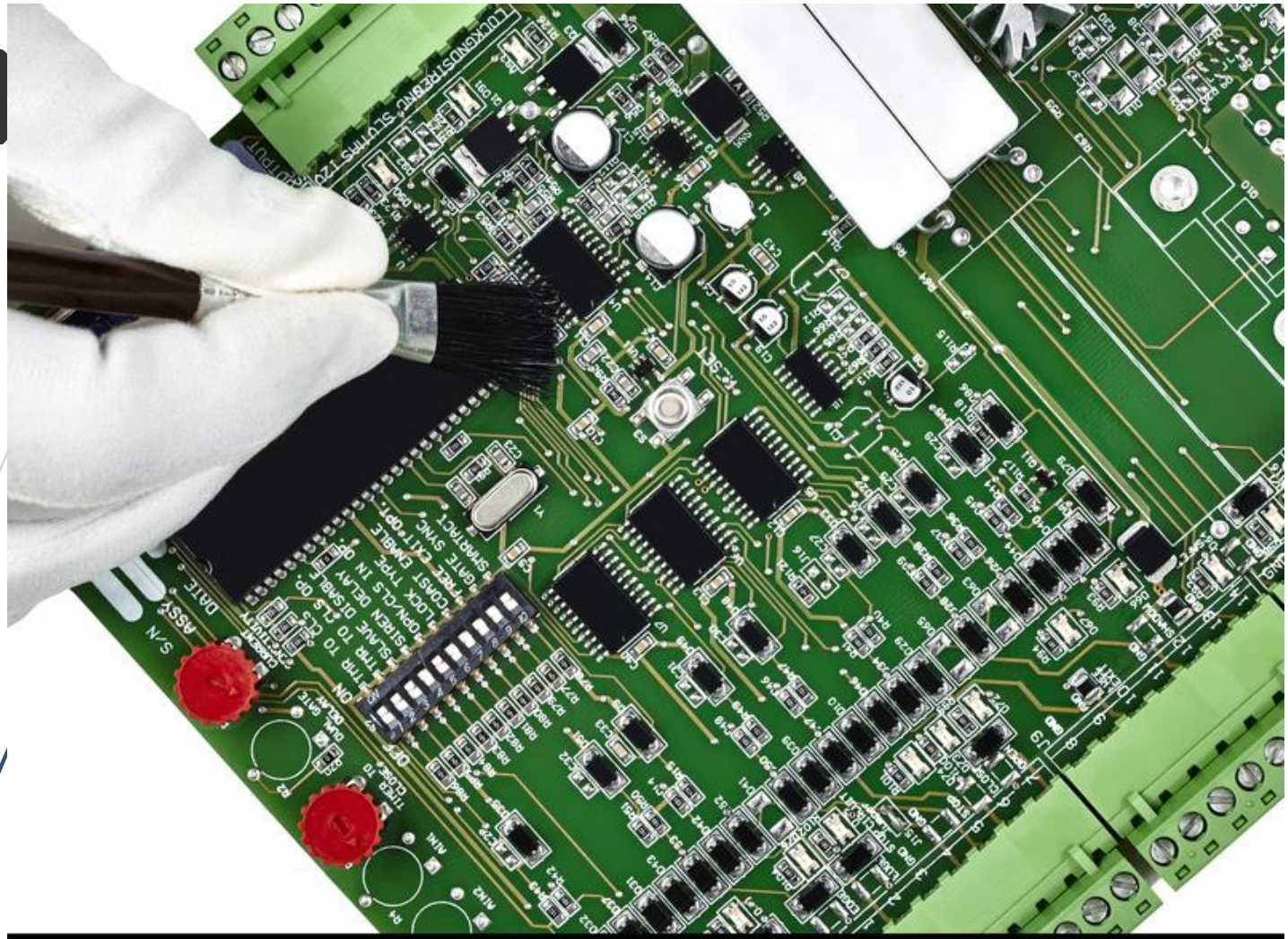
over operating free-air temperature range (unless otherwise noted)

PARAMETER	TEST CONDITIONS		MIN	TYP	MAX	UNIT
V_{IK}	V_{CC} = MIN and I_I = –12 mA				–1.5	V
V_{OH}	V_{CC} = MIN, V_{IL} = 0.8 V, and I_{OH} = –0.4 mA		2.4	3.4		V
V_{OL}	V_{CC} = MIN, V_{IH} = 2 V, and I_{OL} = 16 mA			0.2	0.4	V
I_I	V_{CC} = MAX and V_I = 5.5 V				1	mA
I_{IH}	V_{CC} = MAX and V_I = 2.4 V				40	µA
I_{IL}	V_{CC} = MAX and V_I = 0.4 V				–1.6	mA
I_{OS}	V_{CC} = MAX	SN5400	–20		–55	mA
		SN7400	–18		–55	
I_{CCH}	V_{CC} = MAX and V_I = 0 V			4	8	mA
I_{CCL}	V_{CC} = MAX and V_I = 4.5 V			12	22	mA

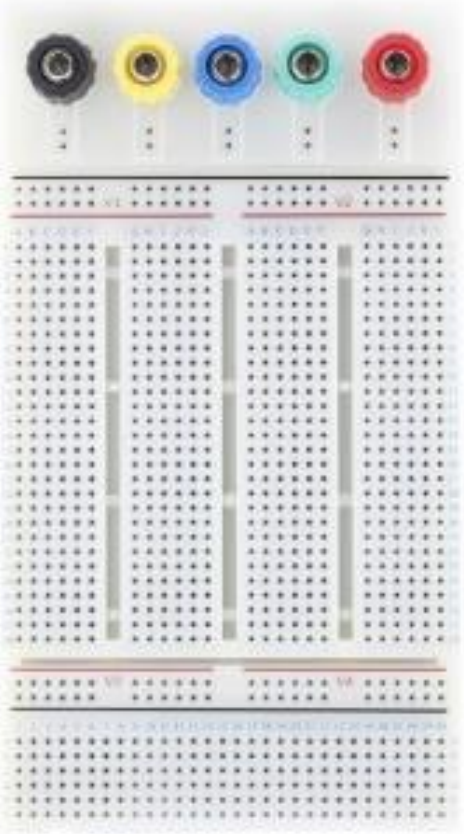
Capsule (chip-uri) cu porți logice

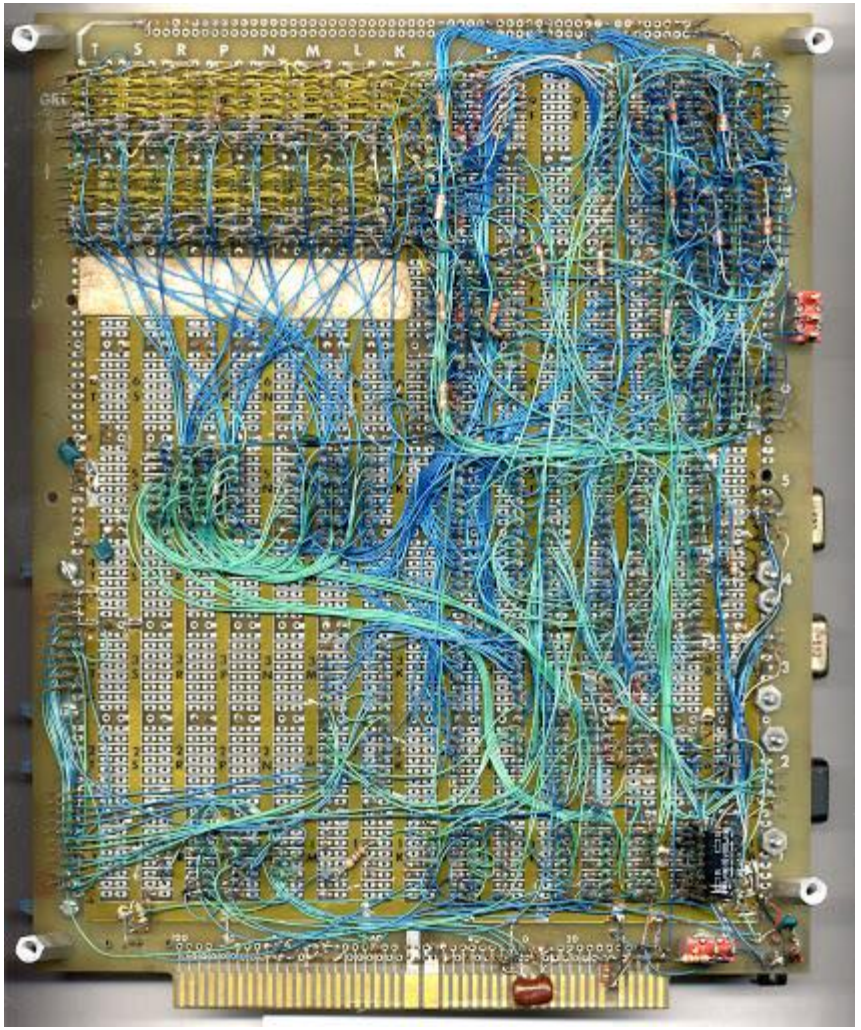






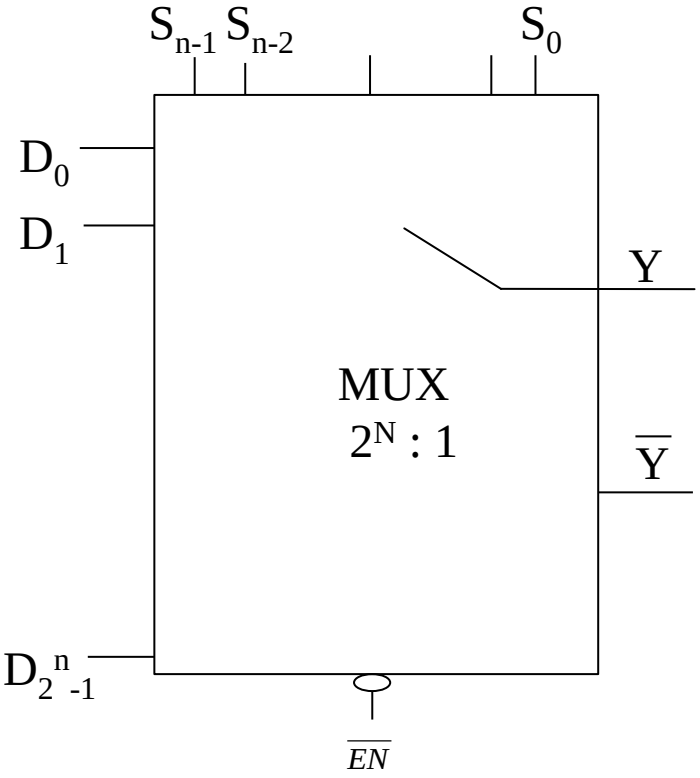
ADVANCEMENT & FUTURE OF PRINTED CIRCUIT BOARD





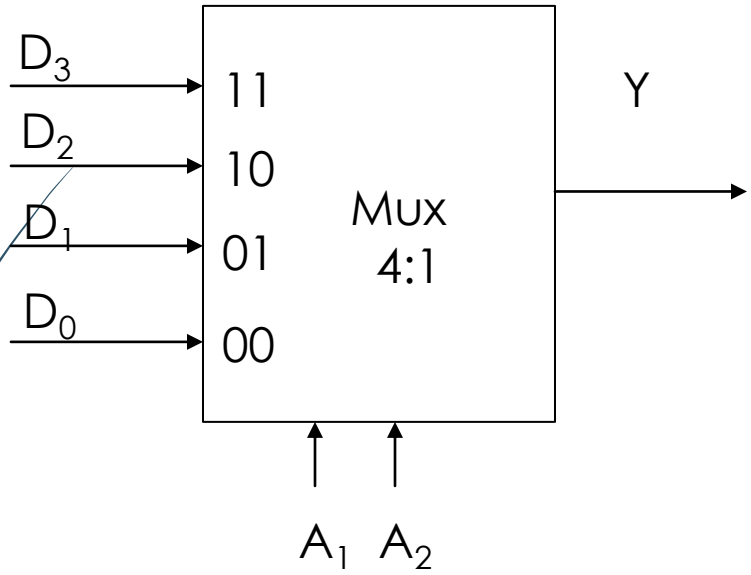
Multiplexoare

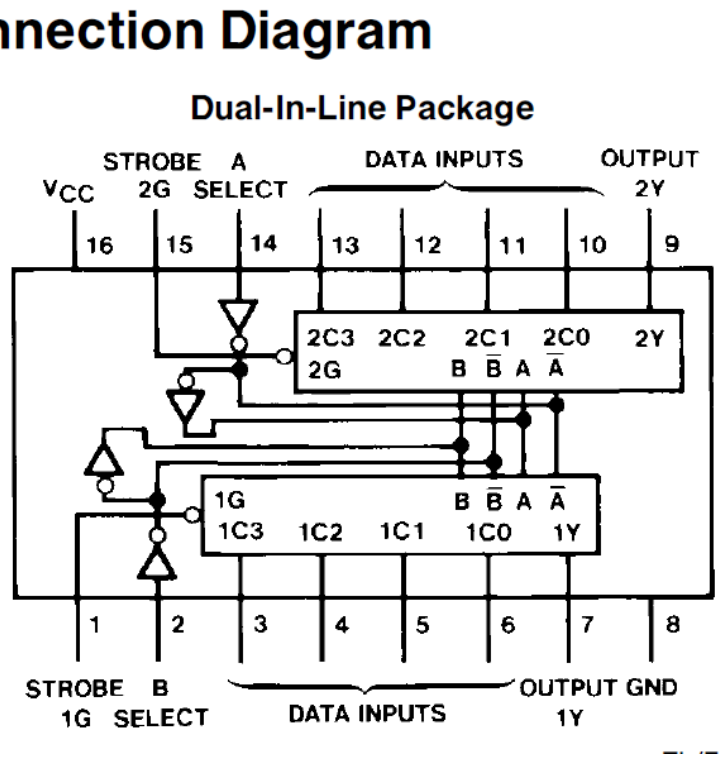
- Circuite combinaționale cu n intrări de selecție, 2^n intrări de date, o ieșire și eventual ieșirea negată, și o intrare de Enable.
- Circuite care selectează pe baza unui cod, una dintre intrări, a cărei valoare o trimite la ieșire
- Fiecare intrare este identificată printr-un cod
- Ieșirea $Y = D_k$, $k = (S_{n-1}S_{n-2}S_0)_{10}$
- Poate fi folosit pentru a genera funcții logice
- MUX 2/4/8/16:1



- **Observație:** Pentru a se implementa o funcție cu multiplexoare, acea funcție nu se mai minimizează in prealabil!

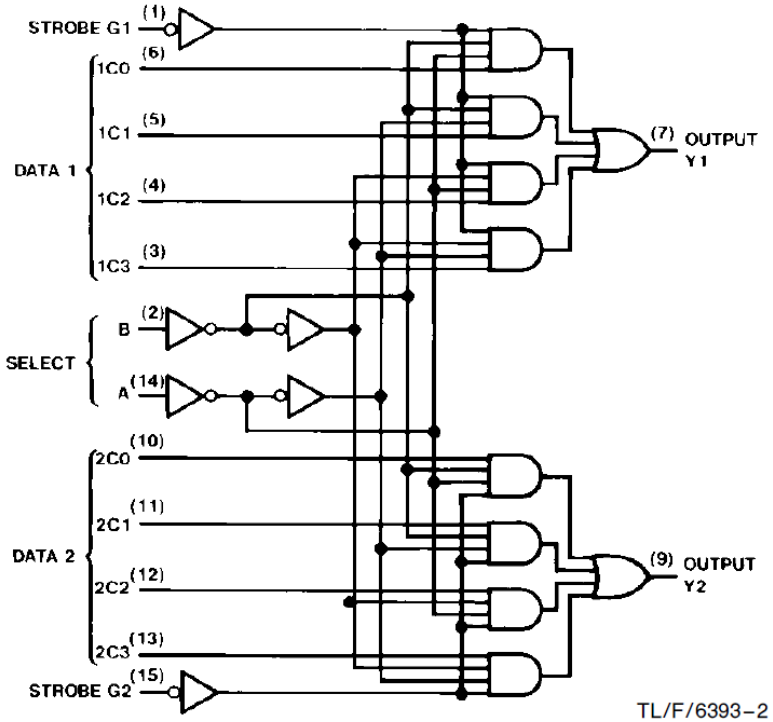
Multiplexorul 4:1





Multiplexorul 4:1

Logic Diagram



TL/F/6393–2

Function Table

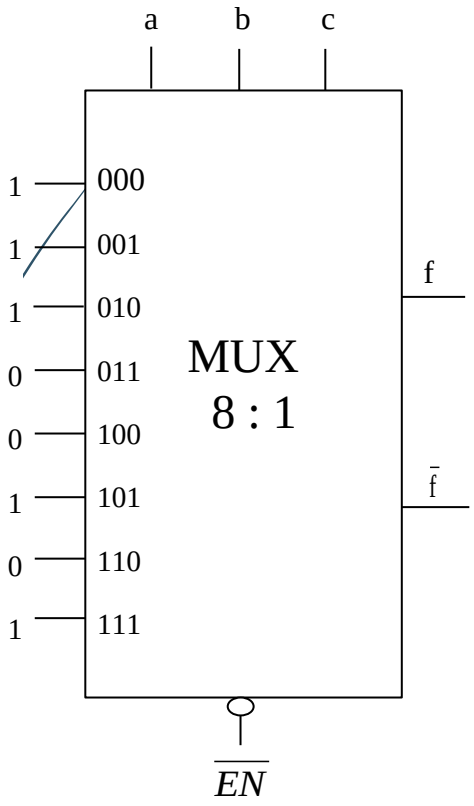
Select Inputs		Data Inputs				Strobe	Output
B	A	C0	C1	C2	C3	G	Y
X	X	X	X	X	X	H	L
L	L	L	X	X	X	L	L
L	L	H	X	X	X	L	H
L	H	X	L	X	X	L	L
L	H	X	H	X	X	L	H
H	L	X	X	L	X	L	L
H	L	X	X	H	X	L	H
H	H	X	X	X	L	L	L
H	H	X	X	X	H	L	H

Select inputs A and B are common to both sections.

H = High Level, L = Low Level, X = Don't Care

Multiplexoare - exemplu

- $f(a,b,c) = a\bar{b}c + \bar{a}\bar{b}c + abc + \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}$
- Să se implementeze funcția folosind un multiplexor 8:1



- Cum rezolvăm?
- Scriem forma generală a unei funcții ca sumă de monoame:
- $f(a,b,c) = c_0 \cdot \bar{a}\bar{b}\bar{c} + c_1 \cdot \bar{a}\bar{b}c + \cdots + c_7 \cdot abc$
- De ce am scris funcția așa?
- Ca să putem identifica coeficienții și să îi punem pe intrările MUX-ului

Multiplexoare – conectare în cascadă (1)

- $f(a,b,c) = a\bar{b}c + \bar{a}\bar{b}c + abc + \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}$
- Să se implementeze funcția folosind multiplexoare 2:1
- Cum se face?
- Se descompune un MUX mai mare în mai multe MUX-uri mai mici
- Numărul total de intrări în primul nivel al MUX-urilor MUX-urile legate în cascadă = numărul de intrări din MUX-ul mare = numărul de linii din tabelul de adevăr.
- Dimensiunea MUX-ului impune numărul de intrări în MUX, dar și numărul de variabile de selecție → MUX-uri mai mici = mai puține variabile de selecție

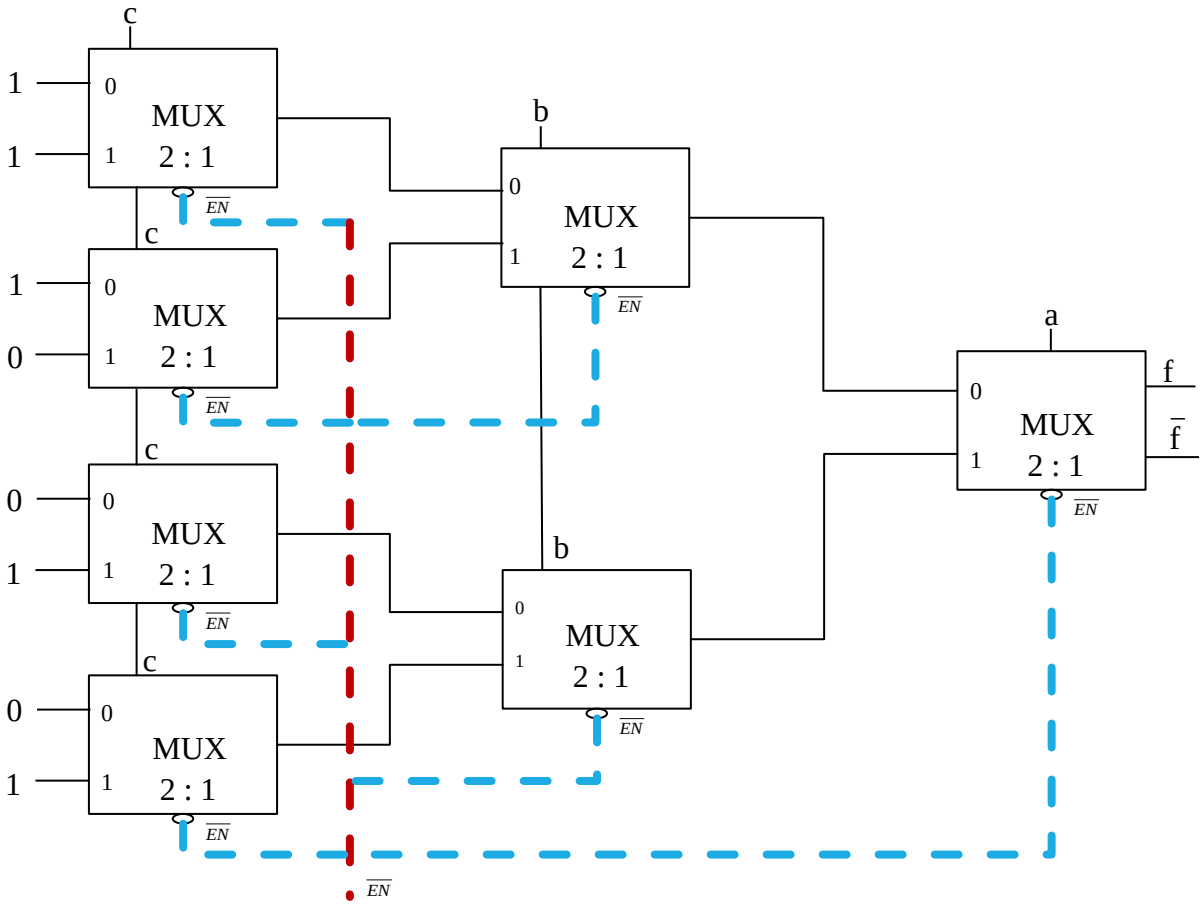
Multiplexoare – conectare în cascadă (2)

- Ex: o funcție ce depinde de n variabile de selecție poate fi implementată prin:
  - 1 MUX 2^n:1 sau
  - 2 MUX 2^{n-1}:1 (n-1 din variabilele de selecție) + 1 MUX 2:1 (ultima variabilă de selecție) care decide care dintre ieșirile celor două MUX-uri va fi folosită pentru a stabili valoarea finală a funcției sau
  - 4 MUX 2^{n-2}:1 , n >= 4 (n-2 din variabilele de selecție) + 1 MUX 4:1 (ultimele 2 variabile de selecție) sau
  - 4 MUX 2^{n-2}:1, n >= 4 (n-2 din variabilele de selecție) + 3 MUX 2:1 cuplate în cascadă (2 MUX 2:1 au ca intrări ieșirile din cele 4 MUX 2^{n-2}:1, urmând ca ultimul MUX 2:1 să aibă ca intrări ieșirile din cele 2 MUX 2:1) sau ...

Multiplexoare – conectare în cascadă (3)

- $f(a,b,c) = a\bar{b}c + \bar{a}\bar{b}c + abc + \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}$
- Câte MUX 2:1 sunt necesare?
- 4 + 2 + 1
- Cum se completează?
- Se face tabelul de adevăr și se identifică valorile de la fiecare combinație

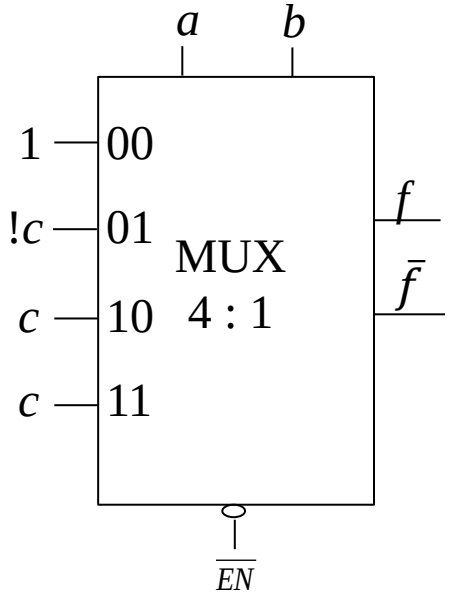
a	*b*	*c*	*f*
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



MUX – multiplexarea intrărilor (1)

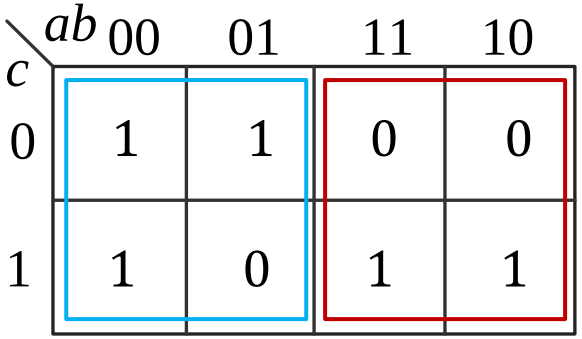
- Problemă: cu cât e mai mare un / sunt mai multe MUX, cu atât e mai scump →implementez aceeași funcție cu un singur MUX 4:1
- Se obține același rezultat dacă pun a, b sau a, c sau b, c pe condiții?
- $f(a,b,c) = a\bar{b}c + \bar{a}\bar{b}c + abc + \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}$
- $f(a,b) = c_0 \cdot \bar{a}\bar{b} + c_1 \cdot \bar{a}b + c_2 \cdot a\bar{b} + c_3 \cdot ab$
- Identific coeficienții și completez MUX

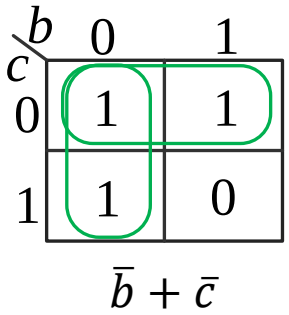
c \ ab	00	01	11	10
0	1	1	0	0
1	1	0	1	1



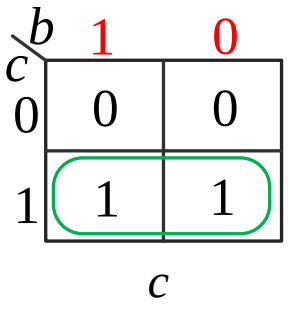
MUX – multiplexarea intrărilor (2)

- $f(a,b,c) = a\bar{b}c + \bar{a}\bar{b}c + abc + \bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c}$
- Pot trece și la MUX 2:1 în același fel, dar este mai complicat! → varianta mai simplă e să folosim diagramele Karnaugh

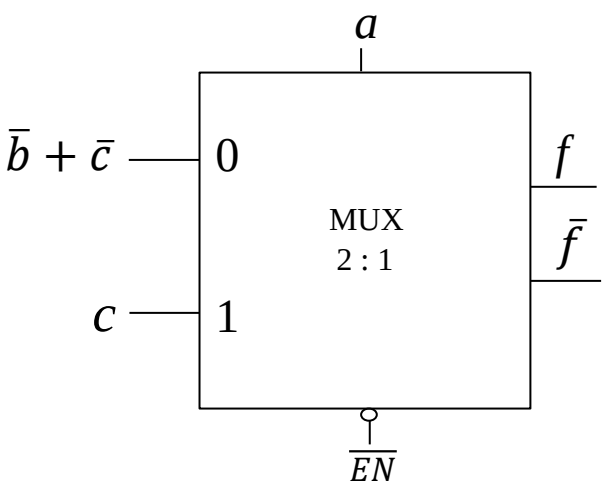




$\bar{b} + \bar{c}$



c



Terminologie

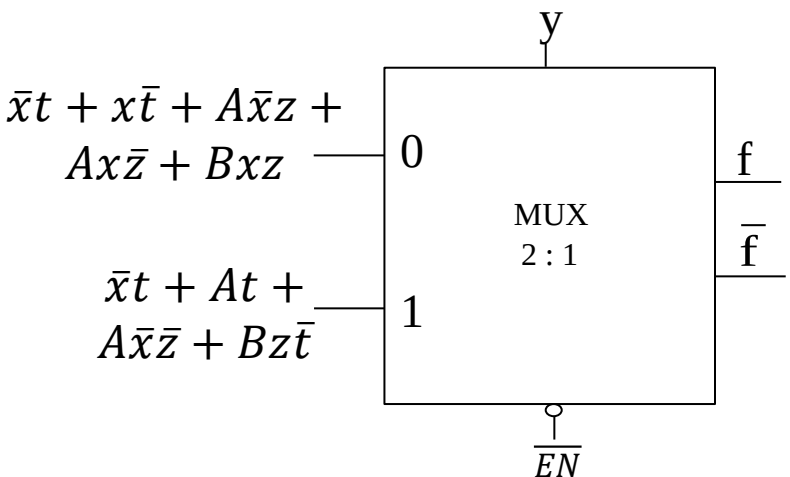
- Spațiul de adresare este dat de variabilele de selecție ale multiplexorului
  - în exemplul dat a fost $\{a, b, c\}; \{a, b\}; \{a\}$
- Spațiul de definiție al funcției este dat de variabilele de care depinde funcția
  - în exemplul dat $\{a, b, c\}$
- Dacă spațiul de adresare este mai mic decât spațiul de definire al funcției, diferența dintre cele două o constituie variabilele reziduu
  - în exemplul dat $\{c\}; \{b, c\};$

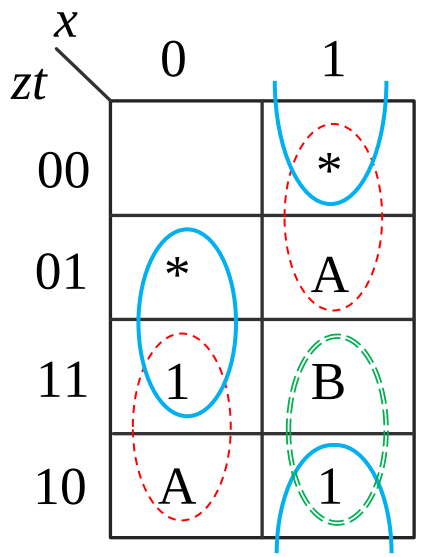
Multiplexarea intrărilor - exemplu

- Implementați funcția folosind un singur MUX 2:1, având y ca variabilă de selecție

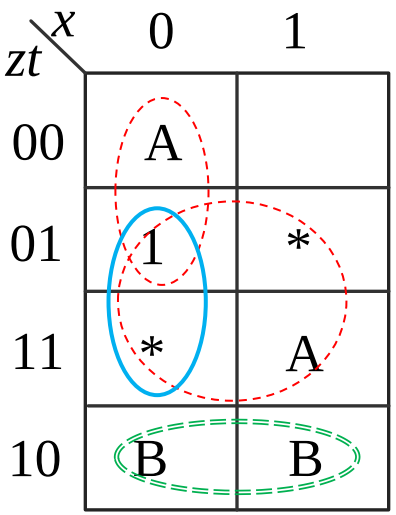
x	y	z	t	f
0	0	0	0	**0**
0	0	0	1	*
0	0	1	0	**A**
0	0	1	1	**1**
0	1	0	0	**A**
0	1	0	1	**1**
0	1	1	0	**B**
0	1	1	1	*
1	0	0	0	*
1	0	0	1	**A**
1	0	1	0	**1**
1	0	1	1	**B**
1	1	0	0	**0**
1	1	0	1	*
1	1	1	0	**B**
1	1	1	1	**A**

zt \ xy	00	01	11	10
00		A		*
01	*	1	*	A
11	1	*	A	B
10	A	B	B	1



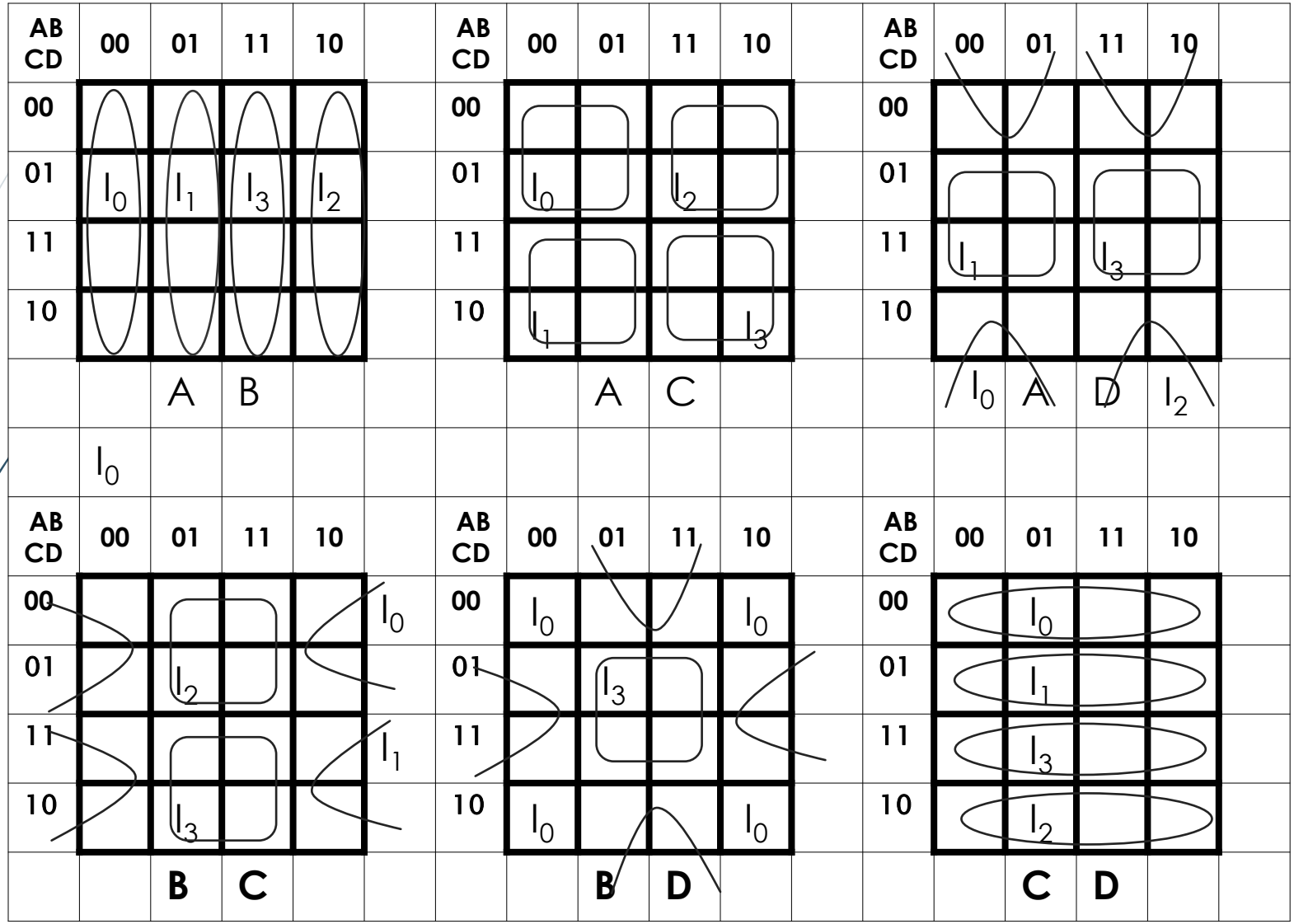


$\bar{x}t + x\bar{t} + A\bar{x}z + Ax\bar{z} + Bxz$



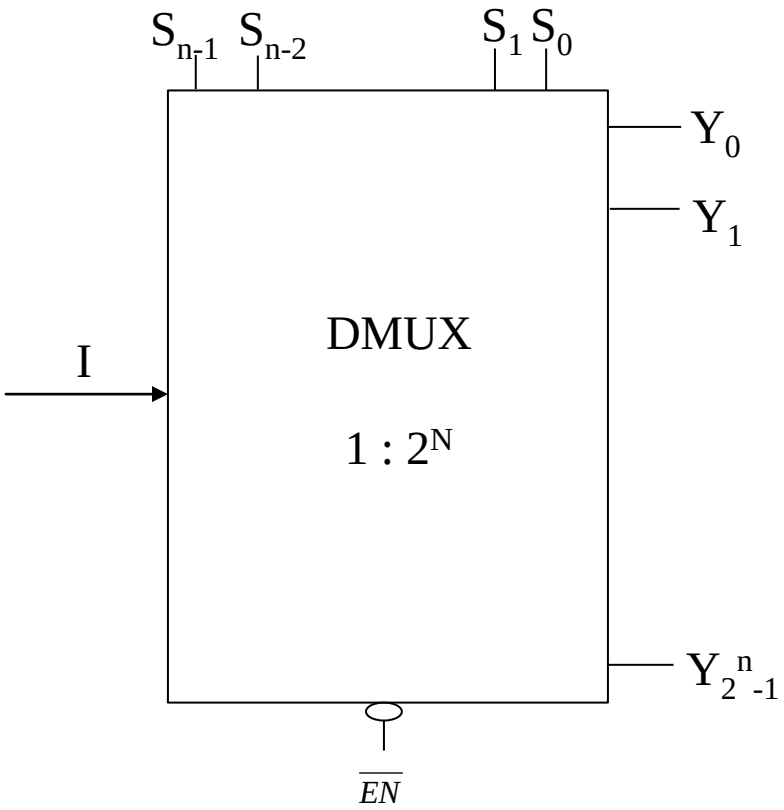
$\bar{x}t + At + A\bar{x}\bar{z} + Bz\bar{t}$

Spații de adresare și de selecție



Demultiplexorul

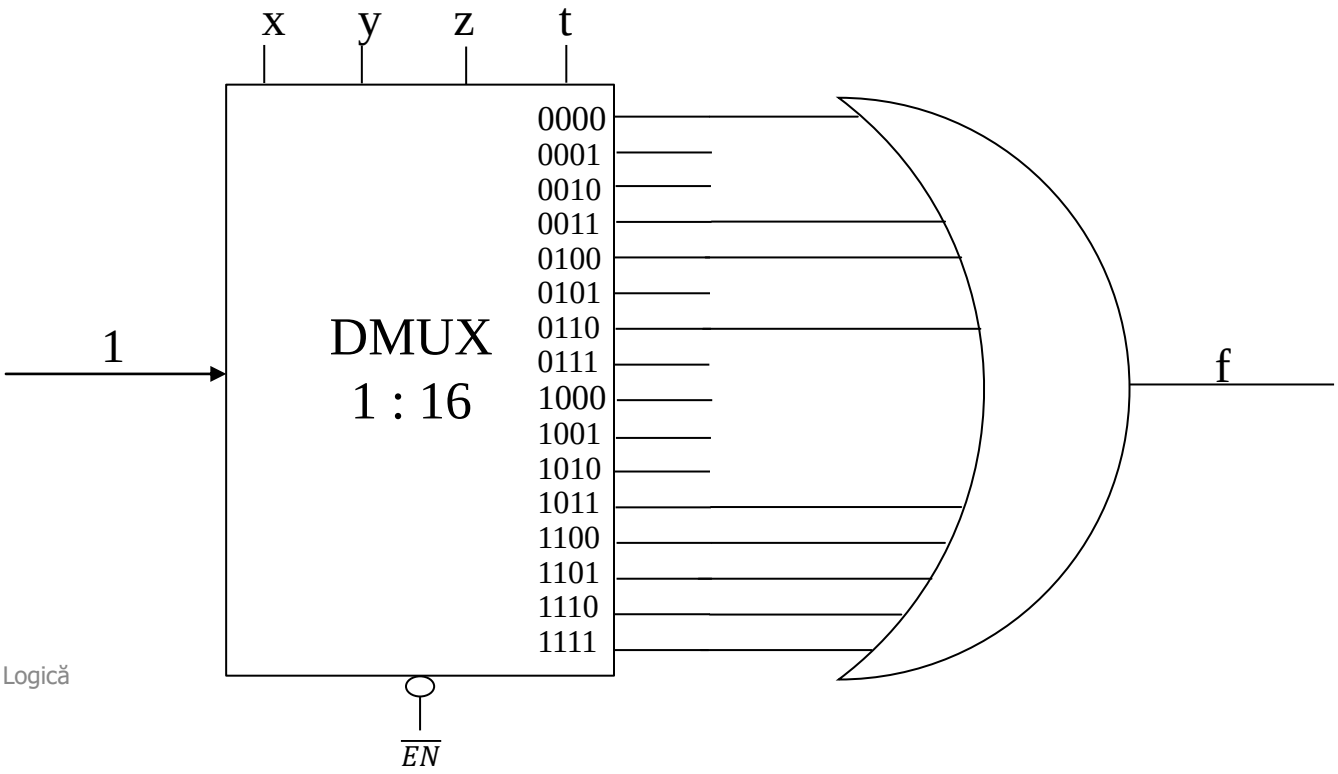
- Circuite complementare multiplexoarelor care permit transmiterea datelor de pe o intrare de date comună pe ieșirea selectată prin cod
- Un demultiplexor are o intrare de date (setată pe „1” logic), o intrare de Enable, n intrări de selecție si 2^n ieșiri.
- $Y_k = I, k = (S_{n-1}, \dots, S_0)_{10}$



Demultiplexoare - exemplu (1)

x	y	z	t	f
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

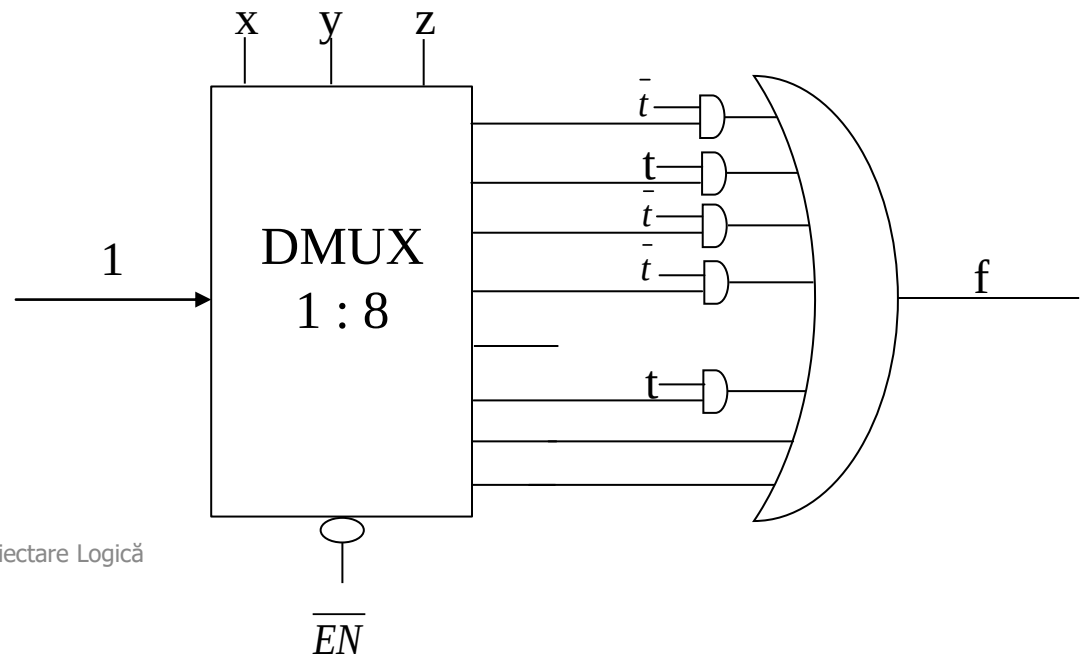
- Implementați funcția folosind DMUX 1:16
- Cum rezolvăm?
- Scriem funcția ca sumă de monoame
- De ce să scriem funcția așa?
- Pentru sumă există element de circuit – poarta OR!
- Identificăm pentru ce combinații de valori funcția trebuie să ia valoarea „1” și punem ieșirile corespunzătoare într-o poartă de tip OR



x	y	z	t	f
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

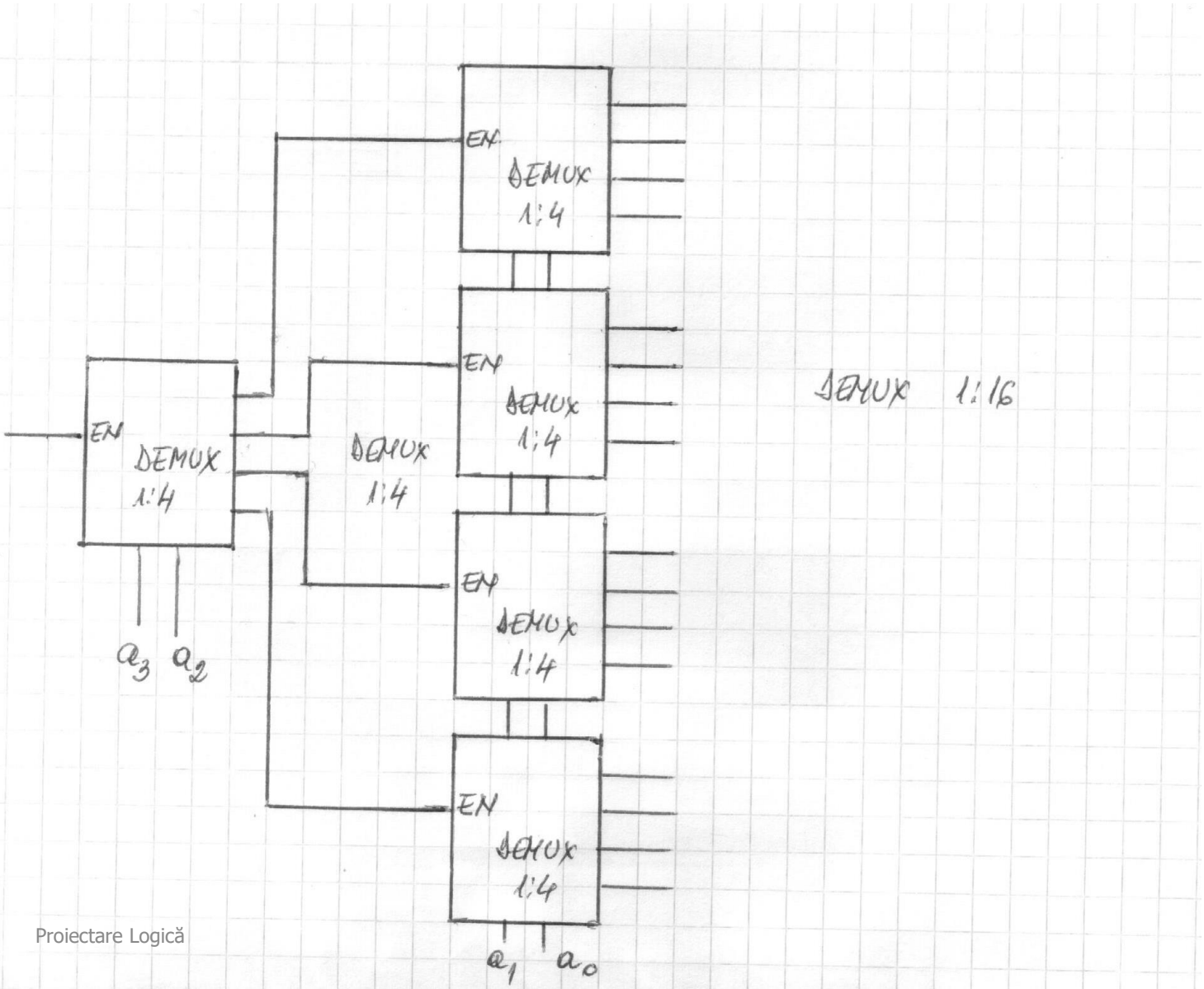
Demultiplexoare - exemplu (2)

- Implementați funcția folosind DMUX 1:8
- Cum rezolvăm?
- Trecem o variabilă în coloana rezultat și apoi rezolvăm ca în cazul anterior
- Pentru realizarea valorilor ce depind de variabile, se intercalează între demultiplexor și poarta OR porți AND în care se pune variabila respectivă



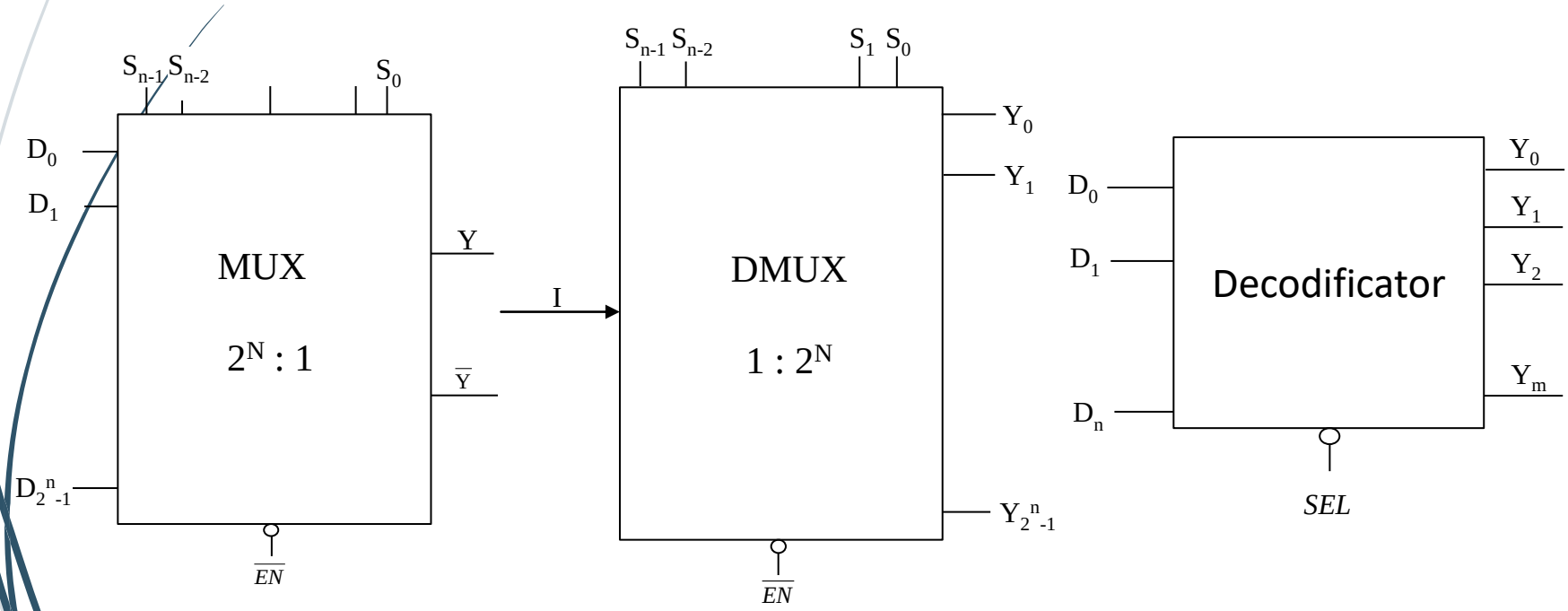
x	y	z	f
0	0	0	!t
0	0	1	t
0	1	0	!t
0	1	1	!t
1	0	0	0
1	0	1	t
1	1	0	1
1	1	1	1

Demultiplexoare – conectare în cascadă (3)



Codificatoare / Decodificatoare (1)

- Circuite combinaționale integrate pe scară medie
- Similare cu MUX/ DMUX
- Majoritatea lucrează în logică negativă (active pe 0)
  - În cazul unui decodificator semnalele de intrare aplicate căilor de adresare determină tranziția uneia din ieşiri în 0 logic.



Codificatoare / Decodificatoare (2)

- *Codificatoare*: Vrem să obținem la ieșire codul binar al cifrei introduse la intrare
- Ex: la tastaturi (telefon)
- *Decodificatoare*: la intrare avem un anumit cod și la ieșire generează semnal pe o singură ieșire, cea corespunzătoare codului de la intrare (de la o reprezentare compactă a informației se obține una și doar una dintre porți activă)
- Ex: instalația de luminițe pentru brad când vrem să aprindem un anumit bec (serie de becuri) și nu alta / toate.

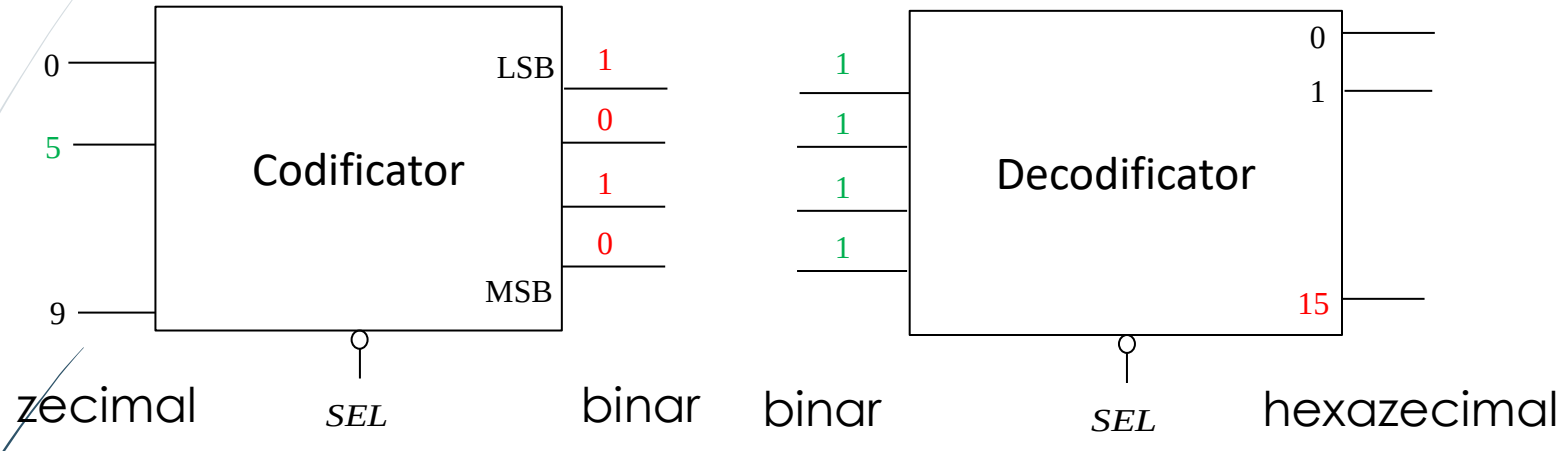
Codificatoare / Decodificatoare (3)

- Un **circuit codificator** atribuie o combinație de valori pentru ieșiri, unică, unui anumit semnal de intrare
- Doar o intrare are valoare 1, la un moment dat

A	B	C	D	C_1	C_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
Restul combinațiilor sunt nepermise				*	*

- Codificator 4:2
- Nu este obligatorie o relație între numărul de intrări și numărul de ieșiri, deși în general avem relația 2^n - n

Codificatoare / Decodificatoare (4)



- Există o legătură între numărul de intrări și numărul de ieșiri:
  - Mux: 2^{sel} = intrări, Decodificatoare: $2^{intrări}$ = ieșiri
- La codificatoare nu mai există intrări de selecție ci numai un semnal care îl lasă să funcționeze sau nu

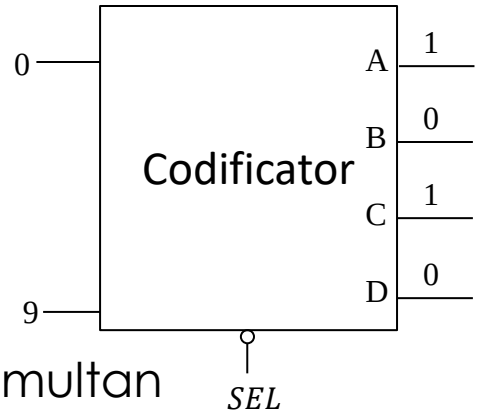
Codificatoare / Decodificatoare (5)

- Există codificatoare / decodificatoare binar – hexa, binar – zecimal (ultimele 6 ieșiri nu mai apar), binar -12
- În activitățile curente folosim baza 10 → avem nevoie de codificator zecimal-binar
  - Considerăm logica pozitivă (în cataloage sunt în logică negativă)

Câte linii în tabel? 2^{10}
Câte ne interesează 10

i_9	i_8	i_7	...	i_1	i_0	D	C	B	A
0	0	0	...	0	0	-	-	-	-
0	0	0	...	0	1	0	0	0	0
0	0	0	...	1	0	0	0	0	1
0	0	0	...	1	1	*	*	*	*

→invalidă

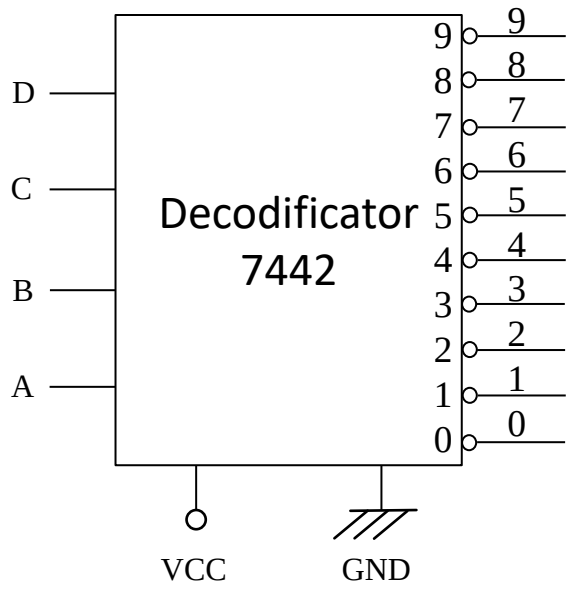


nu apăs 2 taste simultan (primesc semnal pe o singură intrare)

Decodificatorul 7442

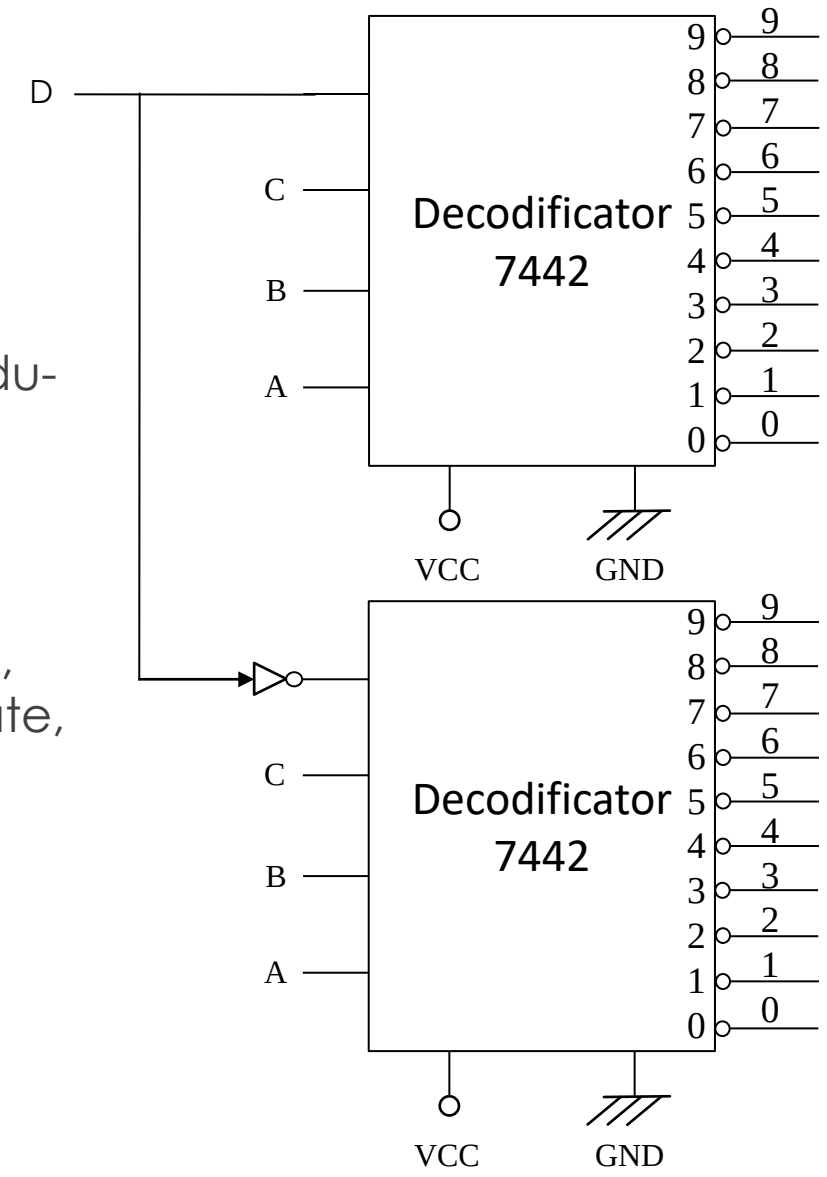
- Decodificator binar – zecimal, activ pe 0
- Nu are nevoie de EN, ci doar de VCC și GND
- Folosit pentru decodificarea spațiului stărilor în circuitele secvențiale și pentru generarea ieșirilor la circuite de tip Moore (le vom studia mai târziu)

D	C	B	A	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
.	.	.	.	.	.	.	.	.	.	.	.	.	.



Decodificatoare

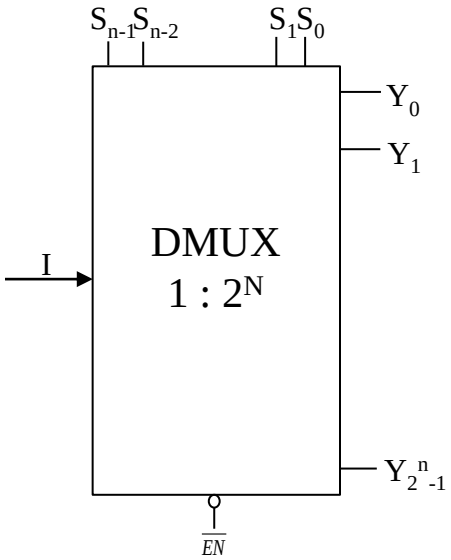
- Insistăm pe decodificator pentru că îl vom utiliza mai târziu la ieșirile circuitelor secvențiale, intrările realizându-se cu multiplexoare.
- Cum putem să scoatem mai multe ieșiri?
- Prin legarea în cascadă (A, B, C) se leagă la ambele aparate, iar D intră o dată direct și o dată negat

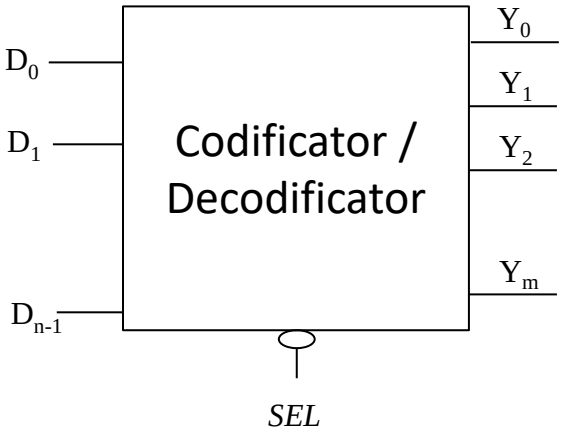


Decodificatoare / demultiplexoare (1)

- Cu excepția lui 7442 care nu are intrări de selecție, decodificatoarele au o intrare de selecție (dacă e activată, atunci în funcție de intrare se activează o ieșire).
- În cataloage nu veți găsi separat decodificatoarele și DMUX, ci le veți găsi în aceeași categorie decodificatoare / DMUX

Transformarea decodificatorului în DMUX se face prin aplicarea pe intrările de date a variabilelor de selecție și folosirea SEL pe post de semnal de date.





Decodificatoare / demultiplexoare (2)

- Dacă tot sunt similare, când le folosim pe unele, când pe altele?
- Se preferă utilizarea DMUX în cazul în care nu sunt prea multe funcții, deoarece chiar dacă folosirea DMUX necesită mai puține porți suplimentare, pentru fiecare funcție este necesar un DMUX.
- Utilizarea unui decodificator se preferă în cazul existenței multor funcții deoarece ieşirile unui singur decodificator pot fi folosite pentru generarea unui număr mare de funcții.



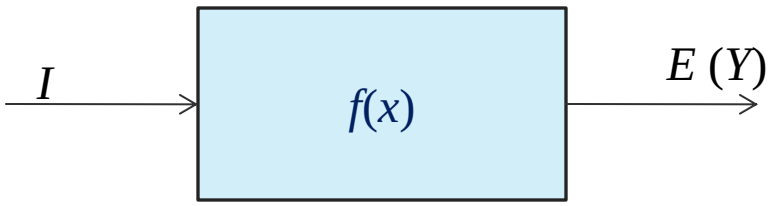
Proiectare Logică

- I CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- ICC: prof. dr. ing. Anca Morar
  - anca.morar@cs.pub.ro

Curs 6: Circuite secvențiale.

Circuite secvențiale (1)

- 2 categorii mari de circuite:
  - **Combinaționale** – porți, MUX, Decodificatoare, etc. : modificările intrărilor se reflectă „instantaneu” la ieșire
  - **Secvențiale** – circuite a căror ieșire depinde și de momentul de timp (istoria circuitului), nu numai de intrări → chiar dacă nu se modifică intrările, valoarea ieșirii lui f de la momentul t poate fi diferită de valoarea ieșirii lui f de la momentul $t + 1$



Reprezentarea standard a circuitelor

Circuite combinaționale: $f(a, b)$

Circuite secvențiale: $f(a, b, t)$

Circuite secvențiale (2)

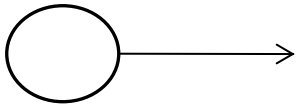
- Multitudine de astfel de circuite – ***circuite de comandă și control***
- **Exemplu 1: semaforul**
  - Primesc o comandă din X în X secunde să schimbe culoarea (aceeași comandă mereu)
  - Culorile sunt diferite de la un moment la altul, depinzând de valoarea pe care au avut-o anterior.
- **Exemplul 2: proiectul**
  - Pentru implementarea circuitelor secvențiale aveți nevoie și de circuite combinaționale (nu puteam începe cu circuite secventiale)

Analiza circuitelor secvențiale

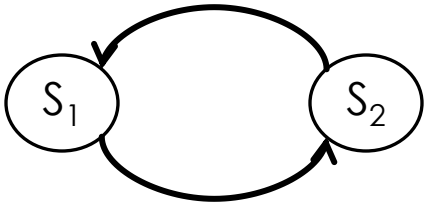
- Un circuit secvențial e caracterizat prin ***starea*** acestuia, exprimată printr-o combinație unică a unui set de valori ale ***variabilelor de stare***.
- **Exemplu: sala de curs**
  - Caracterizată prin X *uși,* Y *ferestre*, fiecare putând fi închise sau deschise.
  - *Starea curentă*: toate sunt închise
  - *Când ați intrat*: o ușă era deschisă
  - Pentru *aerisire*: se poate deschide un geam (sau mai multe)
  - *Ușile și ferestrele = variabilele de stare*, pot fi închise sau deschise și caracterizează starea sălii
  - Pot avea diverse combinații ale variabilelor de stare ($X + Y$ variabile) ce pot influența temperatura din sală (ieșirea circuitului)

Reprezentarea circuitelor secvențiale prin grafuri

- Trecerea dintr-o stare în alta se numește ***tranziție***.
- Convenție:
  - Starea are durată bine determinată
  - Tranziția este instantanee

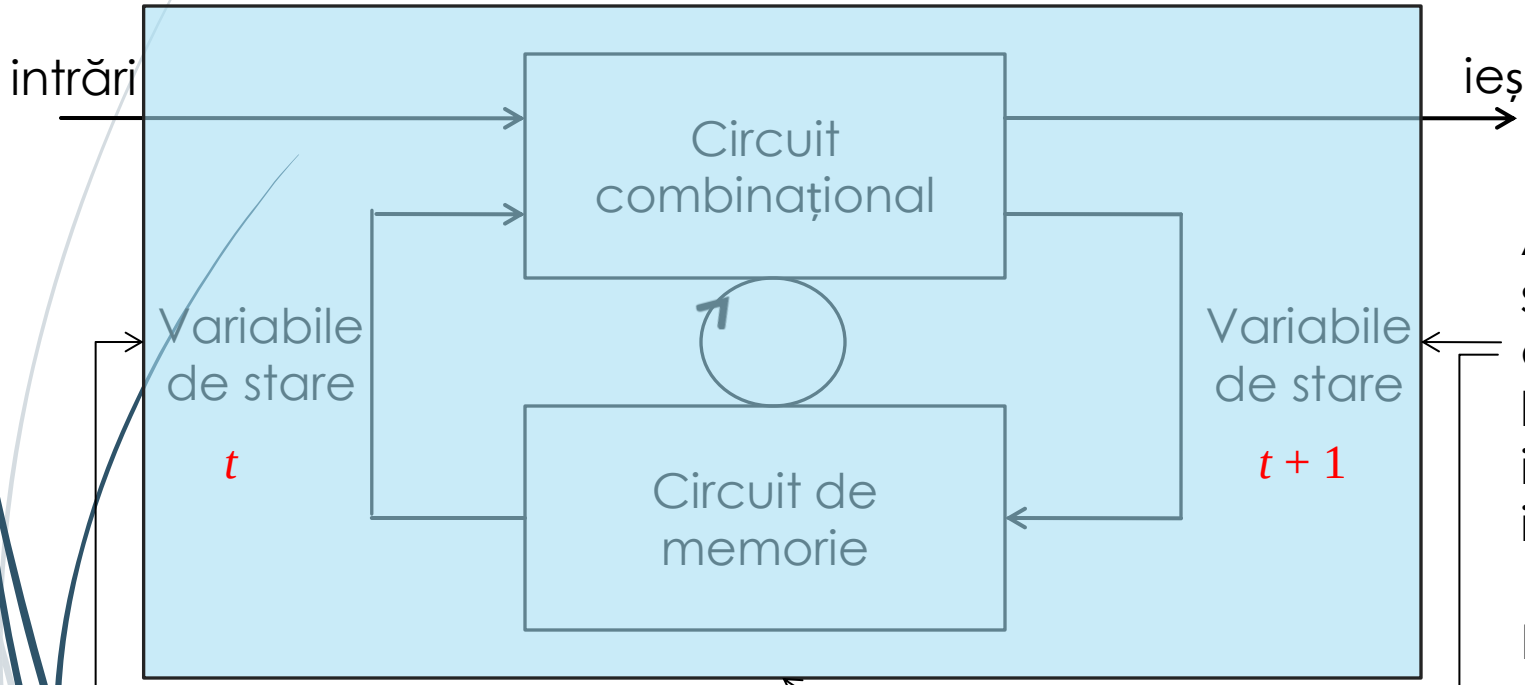


- Exemplu: reprezentarea unei uși
  - S_1 = ușa închisă
  - S_2 = ușa deschisă



Reprezentarea tipică a circuitelor secvențiale

- Circuitul de memorie păstrează informații referitoare la starea circuitului.



Avem voie să punem același lucru și la intrare și la ieșire?

Da, dacă adăugăm timpul $(t, t + 1)$

Forma simplificată a circuitului

Structura circuitelor secvențiale

- La orice circuit secvențial avem o buclă închisă (o parte din ieșire se întoarce la intrare) → ***reacție*** → circuitele secvențiale sunt ***circuite cu reacție***
- **Observație**: avem ieșiri și avem variabile de stare. De cele mai multe ori, variabilele de stare nu sunt accesibile din exterior, nu se pot măsura ci le calculăm indirect.
- Cum este la exemplul cu sala de curs?

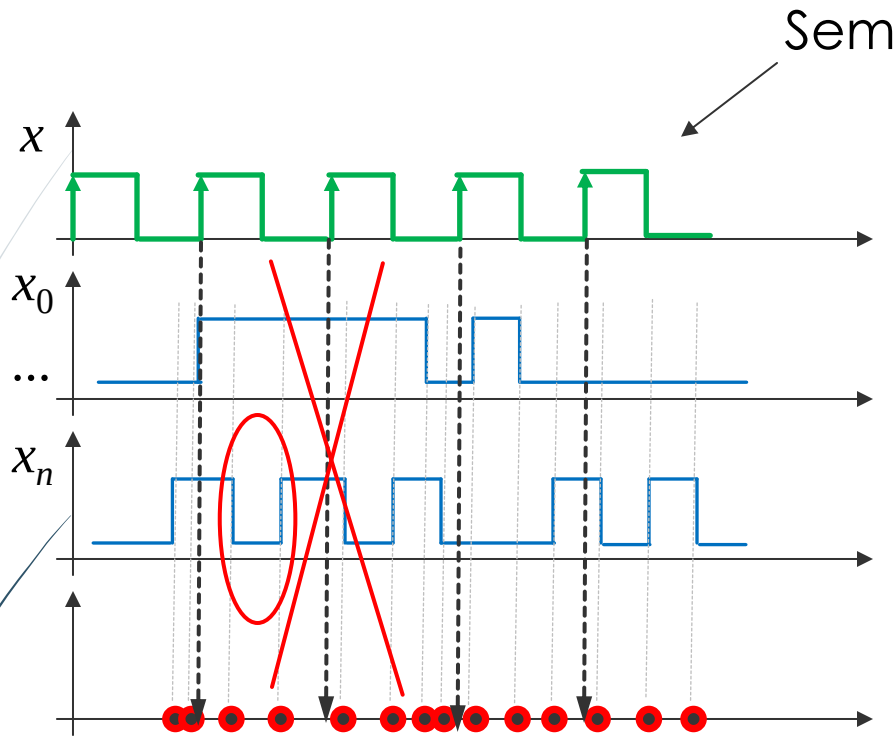
Clasificarea circuitelor secvențiale

- Circuitele secvențiale se clasifică în funcție de:
  - Modul de realizare a tranzițiilor de la o stare la alta
  - Modul de construcție a circuitului de memorare
  - Natura semnalelor aplicate la intrare
  - Natura semnalelor de ieșire generate de automatul secvențial

Clasificarea după modul de realizare a tranzițiilor de la o stare la alta

- ***Circuite asincrone***:
  - tranzițiile au loc la momente arbitrare de timp, ori de câte ori la intrarea circuitului sunt îndeplinite condițiile de trecere dintr-o stare în alta (la variația semnalelor de intrare)
- ***Circuite sincrone***:
  - tranzițiile au loc la momente bine stabilite de timp, și anume la intervale egale, date de semnalul de sincronizare

Diagrame de semnal



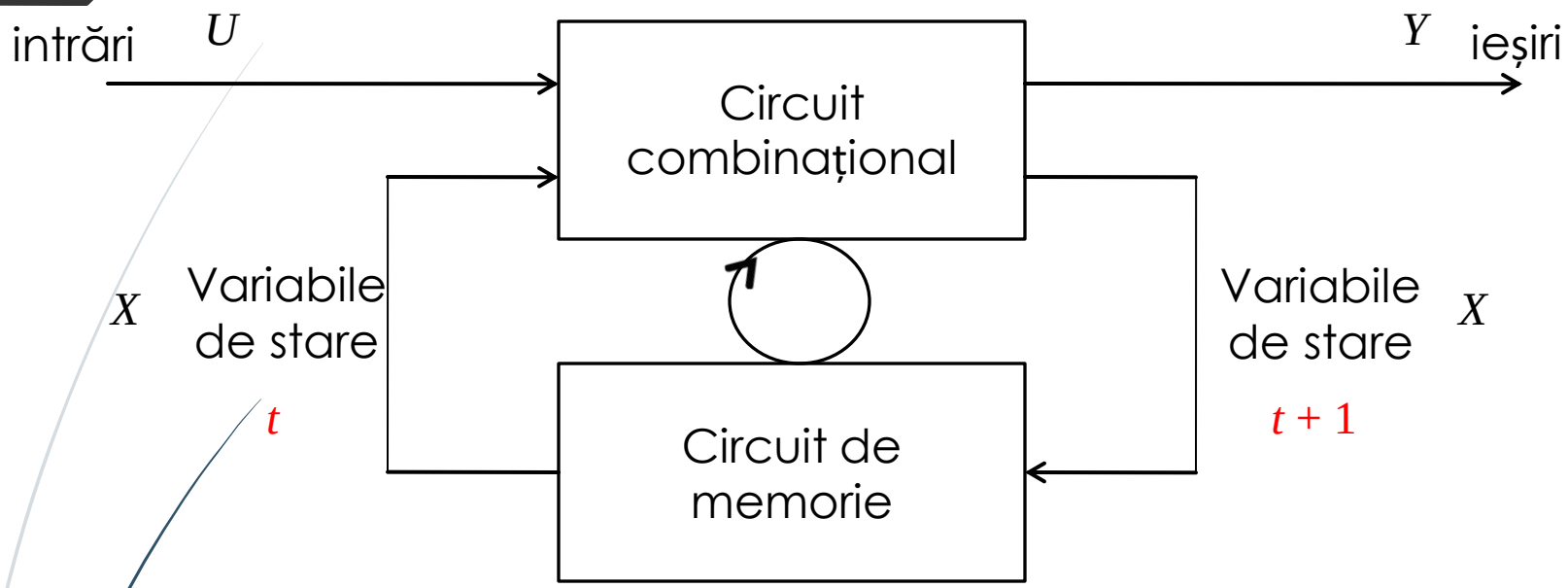
La circuitele asincrone nu există semnal de sincronizare și tranziția se face atunci când se modifică intrările

- La circuitele sincrone modificarea se poate face doar la momente bine stabilite de timp (frontul crescător sau descrescător al ceasului) → indiferent ce se întâmplă între 2 fronturi succesive, ieșirea nu se modifică → evaluarea nu se face decât la momentul stabilit de timp

Automat finit

- Se numește un *automat finit* un cvintuplu ordonat $A = (U, X, Y, f, g)$, unde
  - U = mulțimea finită a semnalelor de intrare
  - X = mulțimea finită a stărilor
  - Y = mulțimea finită a semnalelor de ieșire
  - $f: X \times U \rightarrow X$ este *funcția de tranziție* și precizează starea viitoare în care ajunge automatul în cazul aplicării unei intrări
  - $g: X \times U \rightarrow Y$ este *funcția de ieșire* și precizează ieșirea generată de automat în cazul aplicării unei intrari

Automat finit



- Circuitele secvențiale au o funcționare deterministă → tranziția de la o stare la alta este perfect determinată (dându-se o anumită stare și o anumită configurație de intrare se trece într-o stare bine definită)
- Tranziția se face pe baza unui algoritm → mașinile care se bazează pe astfel de algoritmi: ***ASM (algorithm state machine)***

Codificarea stărilor

- Cum putem descrie starea unui circuit?
- Ex cu sala simplificat: avem doar 1ușă și un geam

Ușa	Geam	Starea circuitului
Deschis	Deschis	Starea 1 = pregătire cursuri
Deschis	Închis	Starea 2 = pauză
Închis	Deschis	Starea 3 = curs
Închis	Închis	Starea 4 = vacanță

Ce sunt astea?

Variabilele de stare

Ce este ăsta?

Tabel de adevăr

- Starea = ceva global ce se aplică dispozitivului și depinde de valorile variabilelor de stare (valori din logica Booleană)

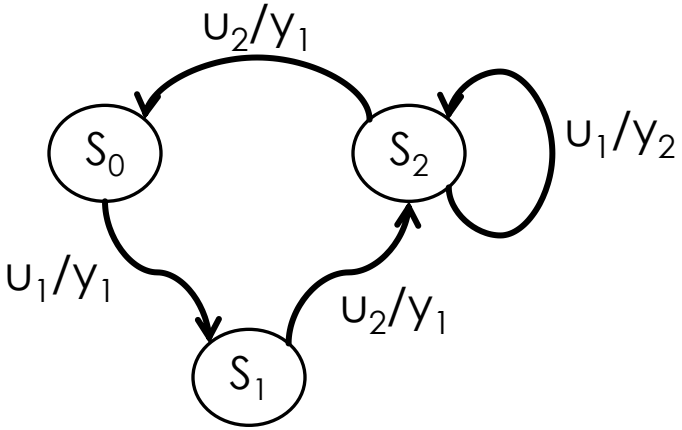
Codificarea stărilor

- Starea se poate descrie prin simboluri sau mnemonice dar pentru a putea lucra trebuie să ajungem la dependența stărilor de ansamblul valorilor variabilelor de stare
- **Nr de stări** = $2^{\text{Nr. de variabile de stare}}$
- Un circuit va avea atâtea stări câte se pot forma prin combinațiile valorilor variabilelor de stare
- **Codificarea stărilor** = numere binare date stărilor

Codificare	Starea circuitului
00	Starea 1 = pregătire cursuri
01	Starea 2 = pauză
10	Starea 3 = curs
11	Starea 4 = vacanță

Codificarea stărilor

- În funcționarea unui circuit nu este întotdeauna nevoie de toate stările.
- Exemplu:



- Am desenat stările, tranzițiile, condițiile pe baza cărora se fac tranzițiile, precum și ieșirile
- Câte variabile are automatul?
  - Nr de stări = $2^{\text{Nr. de variabile de stare}}$
  - Aici știm nr de stări → nr. variabilelor de stare

Codificarea stărilor

- Având acoperitor X_1X_2 → 4 combinații → avem mai multe posibilități de codificare

X_1	X_2	Stări		
0	0	S_0	*	S_0
0	1	S_1	S_0	*
1	0	S_2	S_1	S_1
1	1	*	S_2	S_2

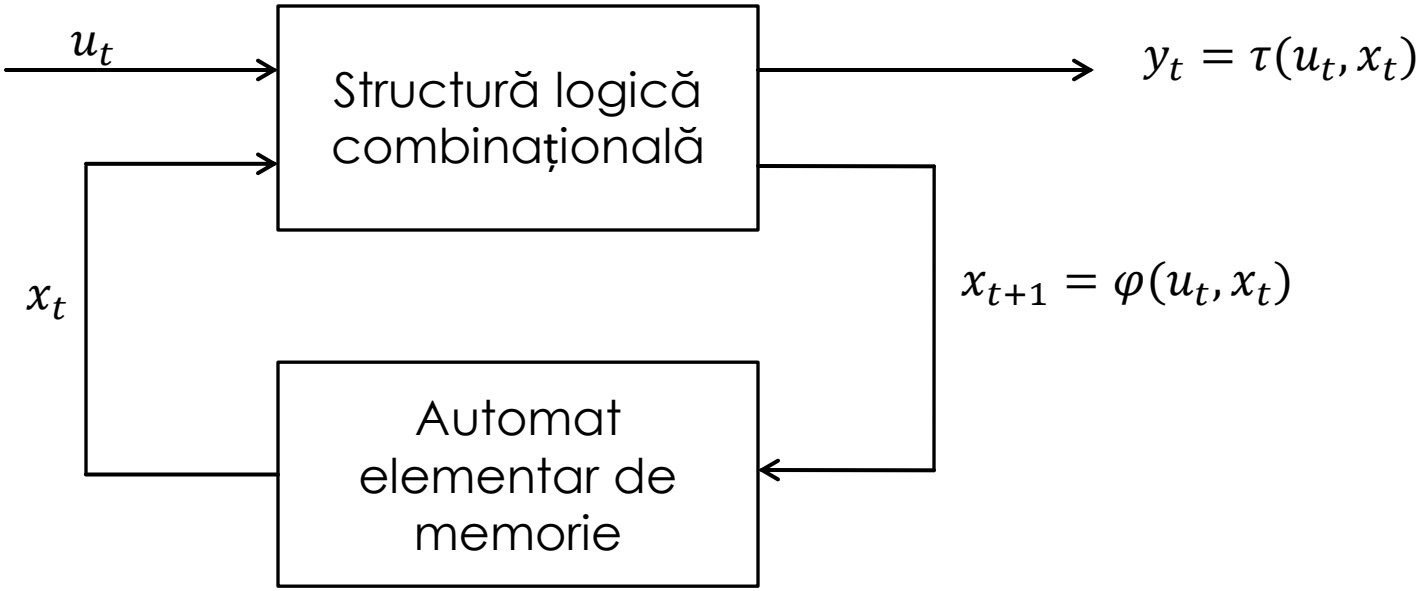
- Metoda de alegere a combinațiilor de valori pentru diferite stări se numește ***asignarea stărilor*** și se face după anumite reguli → joacă un rol extrem de important
- De obicei întâi știm stările și apoi alegem variabilele de stare și realizăm asignarea lor

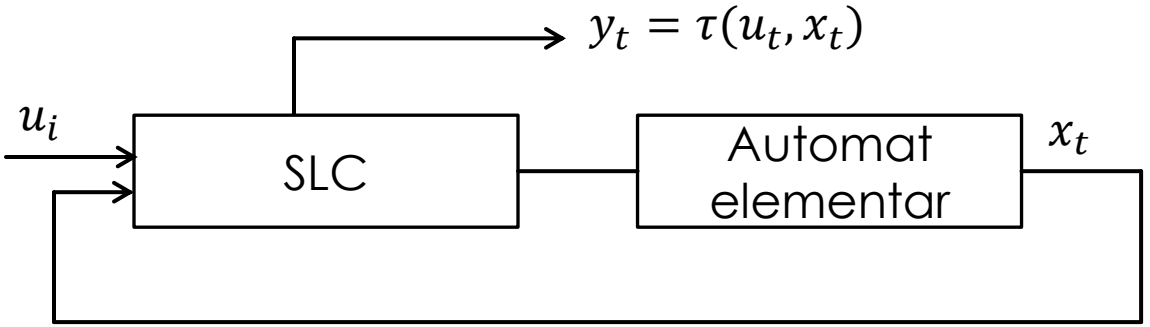
Clasificarea după natura semnalelor de ieșire

- ***Circuite Mealy***: La automatul de tip Mealy, evoluția tranzițiilor interne de stare cât și secvența ieșirilor sunt univoc determinate dacă se specifică starea inițială a automatului și secvența de impulsuri de intrare

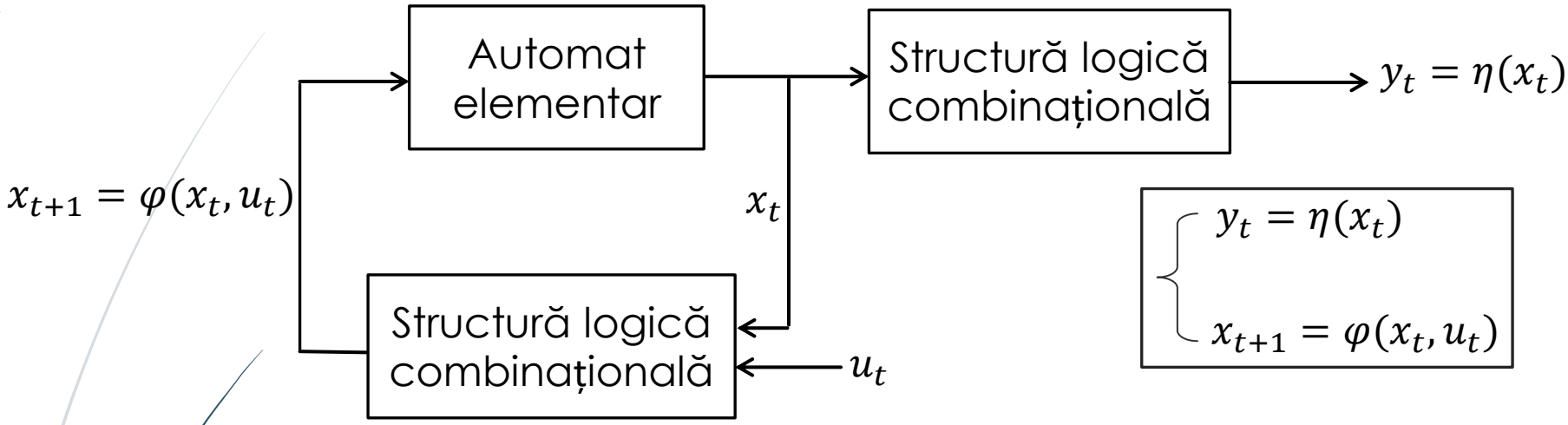
- ***Circuite Moore***: La automatul de tip Moore, ieșirile sunt obținute printr-un set de funcții ce depind exclusiv de stare. (Ieșirile depind doar de stări, stările depind de intrări și de stări).

Automatul de tip Mealy

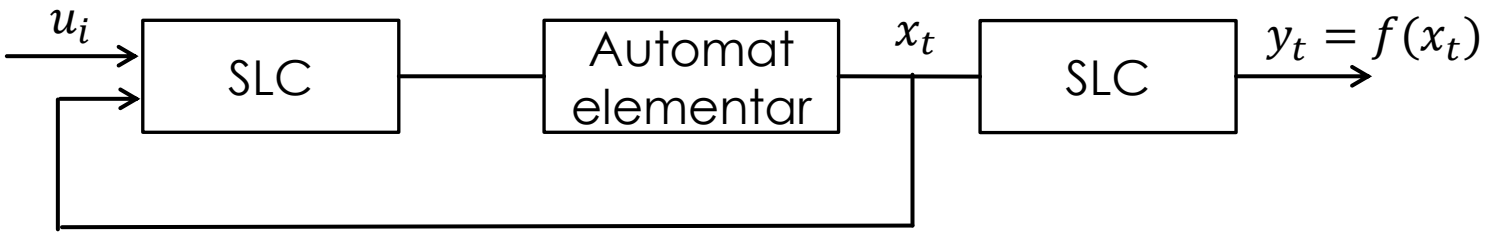




Automatul de tip Moore



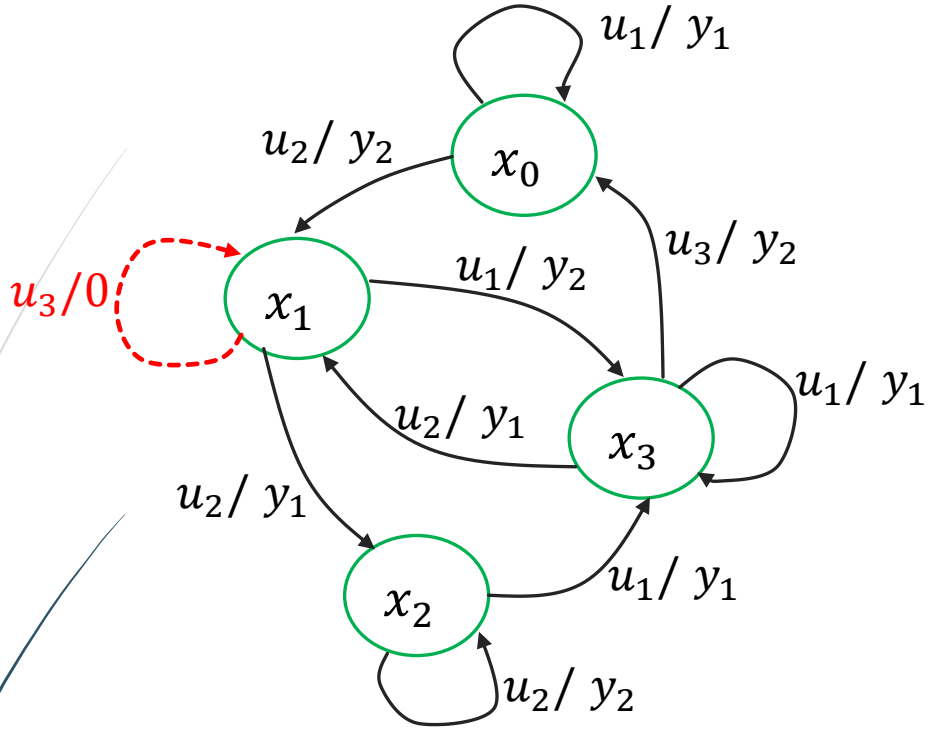
Foarte similar cu circuitul Mealy



Reprezentarea circuitelor secvențiale

- Circuitele secvențiale se pot reprezenta prin:
  - Grafuri
  - Forma algebrică (prin definirea funcțiilor)
  - Tabele
  - Organigrame

Automat de tip Mealy - Exemplu



- $U =?$
- $U = \{u_1,\ u_2, u\}$
- $X =?$
- $X = \{x_0, x_1, x_2, x_3\}$
- $Y =?$
- $Y\ = \{y_1, y_2\}$
- $f\colon X \times U \rightarrow X$
- $g\colon X \times U \rightarrow Y$

Reprezentarea prin graf

- Incomplet specificat – nu știm ce se întâmplă în x_1 dacă vine u_3
- Dacă e automat asincron → tranziția se face când apare semnalul →putem avea această reprezentare simplificată (nu e nevoie să scriu toate combinațiile ca să nu complic). Altfel, trebuie definit tot ($f(x_1, u_3)\ =\ x_1, g(x_1, u_3)\ =\ 0 \ldots$)

Automat de tip Mealy – Exemplu (2)

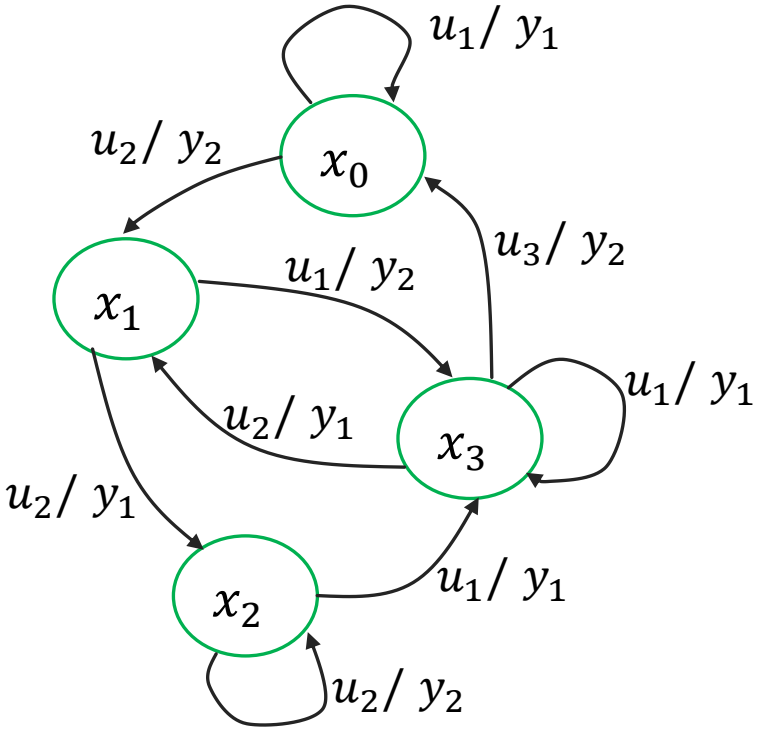
- **Reprezentarea algebrică:**
- $U = \{u_1, u_2, u_3\}$
- $X = \{x_0, x_1, x_2, x_3\}$
- $Y = \{y_1, y_2\}$
- $f(x_0, u_1) = x_0 \quad g(x_0, u_1) = y_1$
- $f(x_0, u_2) = x_1 \quad g(x_0, u_2) = y_2$
- $f(x_1, u_1) = x_3 \quad g(x_1, u_1) = y_2$
- $f(x_1, u_2) = x_2 \quad g(x_1, u_2) = y_1$
- …
- Numărul de funcții f este egal cu numărul de funcții g

Identificarea funcționării unui automat de tip Mealy

Intrări: u_2 u_1 u_2 u_2 u_2 u_1 u_1 u_3 …

Stări: x_0 x_1 x_3 x_1

Ieșiri: y_2 y_2 y_1 y_1



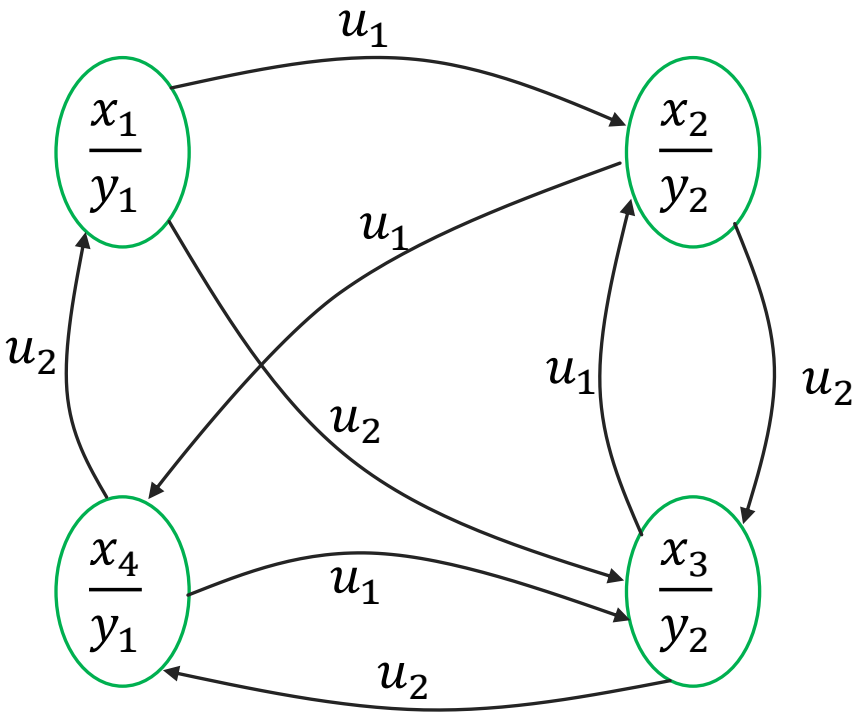
Automatul de tip Moore

$$\left.\begin{array}{l} f(x_1,u_1)=x_2 \\ f(x_1,u_2)=x_3 \end{array}\right\} g(x_1)=y_1$$

$$\left.\begin{array}{l} f(x_2,u_1)=x_4 \\ f(x_2,u_2)=x_3 \end{array}\right\} g(x_2)=y_2$$

$$\left.\begin{array}{l} f(x_3,u_1)=x_2 \\ f(x_3,u_2)=x_4 \end{array}\right\} g(x_3)=y_2$$

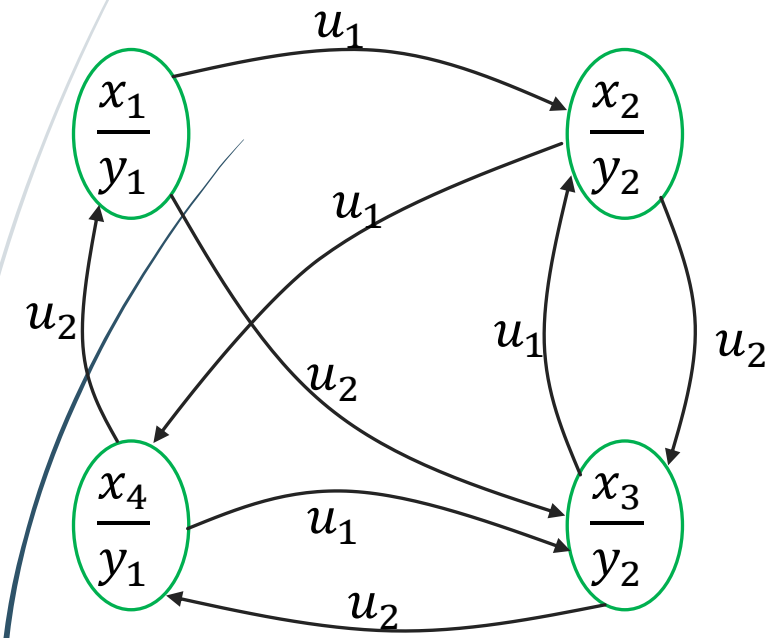
$$\left.\begin{array}{l} f(x_4,u_1)=x_3 \\ f(x_4,u_2)=x_1 \end{array}\right\} g(x_4)=y_1$$



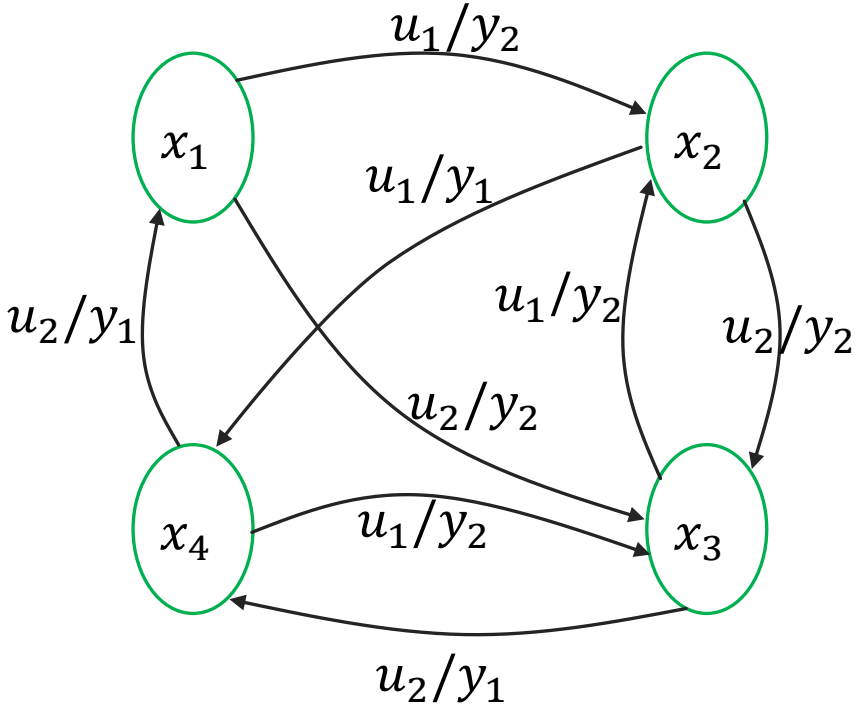
- Numărul de funcții f este diferit de numărul de funcții g
- Numărul de funcții g = numărul de stări

Conversia automatului Moore în automat Mealy

- Se asociază fiecărei tranziții din automatul Moore ieșirea corespunzătoare stării la care duce tranziția (copiem ieșirea pe tranziții)



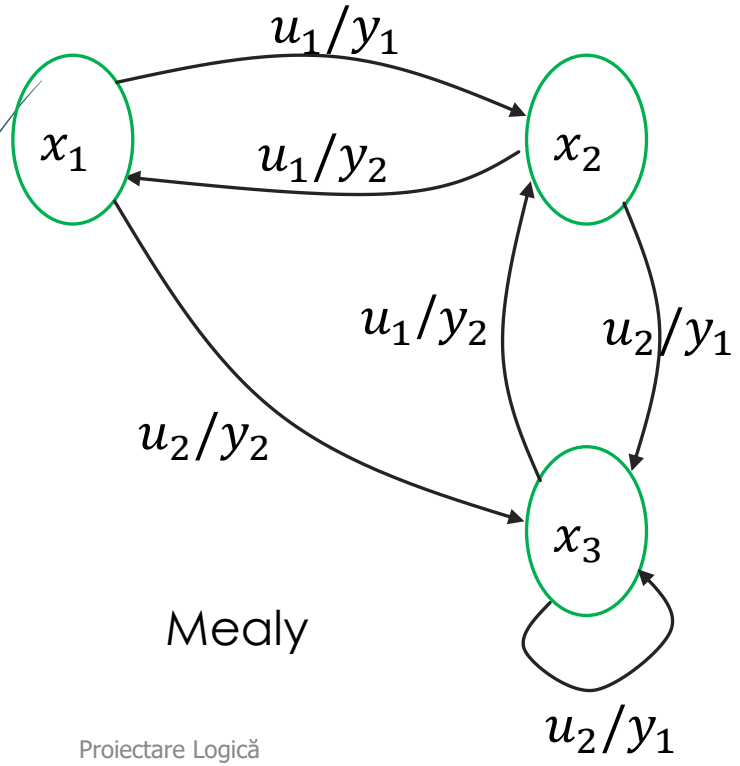
Moore



Mealy

Conversia automatului Mealy in automat Moore

- Se înlocuiește fiecare stare a automatului Mealy cu atâtea stări câte ieșiri diferite sunt asociate tranzițiilor care intră în acea stare.



$f(x_1, u_1) = x_2$ $\quad g(x_1, u_1) = y_1$

$f(x_1, u_2) = x_3$ $\quad g(x_1, u_2) = y_2$

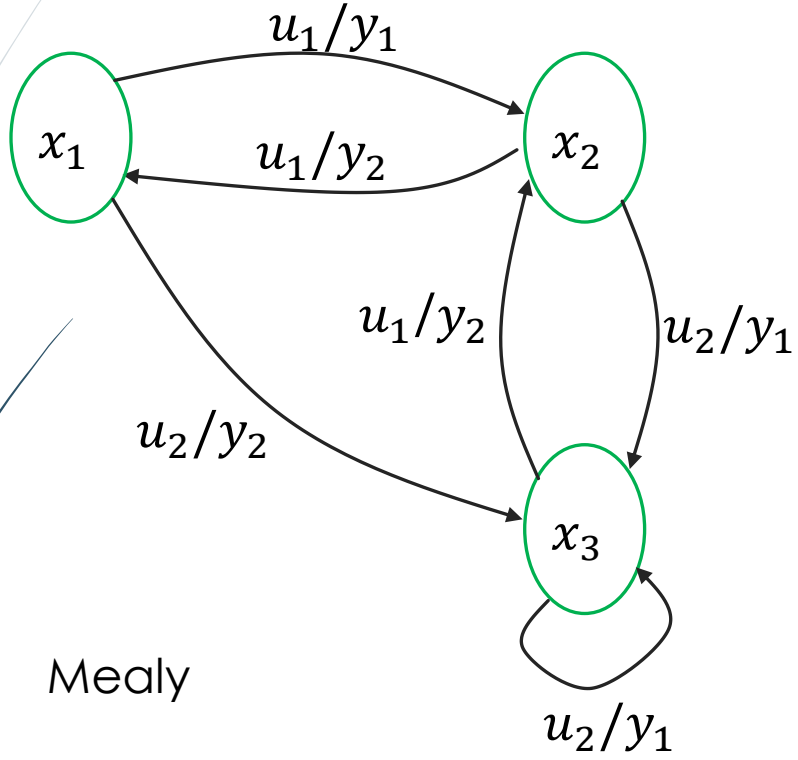
$f(x_2, u_1) = x_1$ $\quad g(x_2, u_1) = y_2$

$f(x_2, u_2) = x_3$ $\quad g(x_2, u_2) = y_1$

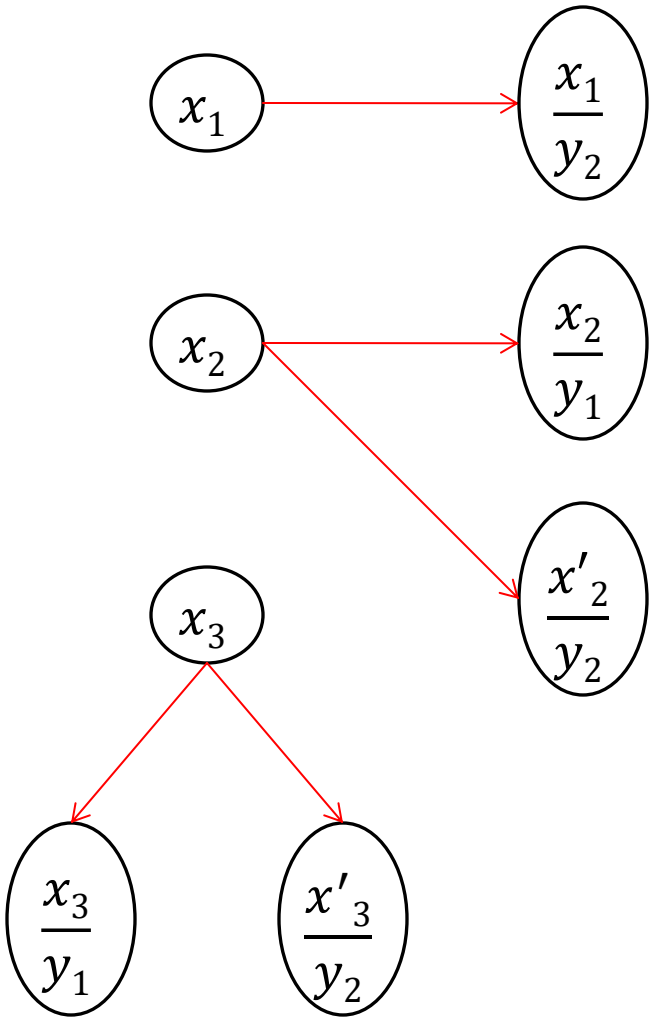
$f(x_3, u_1) = x_2$ $\quad g(x_3, u_1) = y_2$

$f(x_3, u_2) = x_3$ $\quad g(x_3, u_2) = y_1$

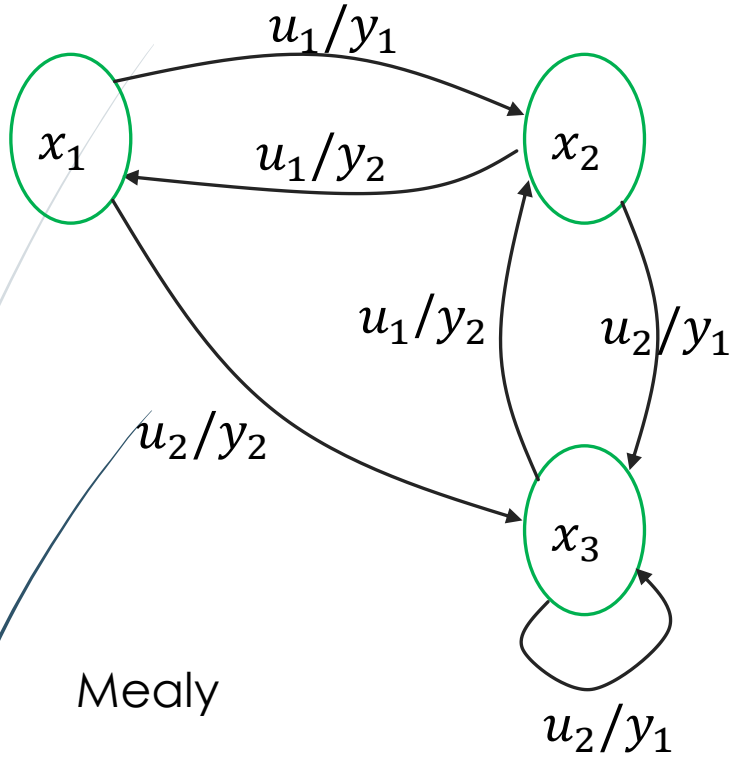
Conversia automatului Mealy in automat Moore



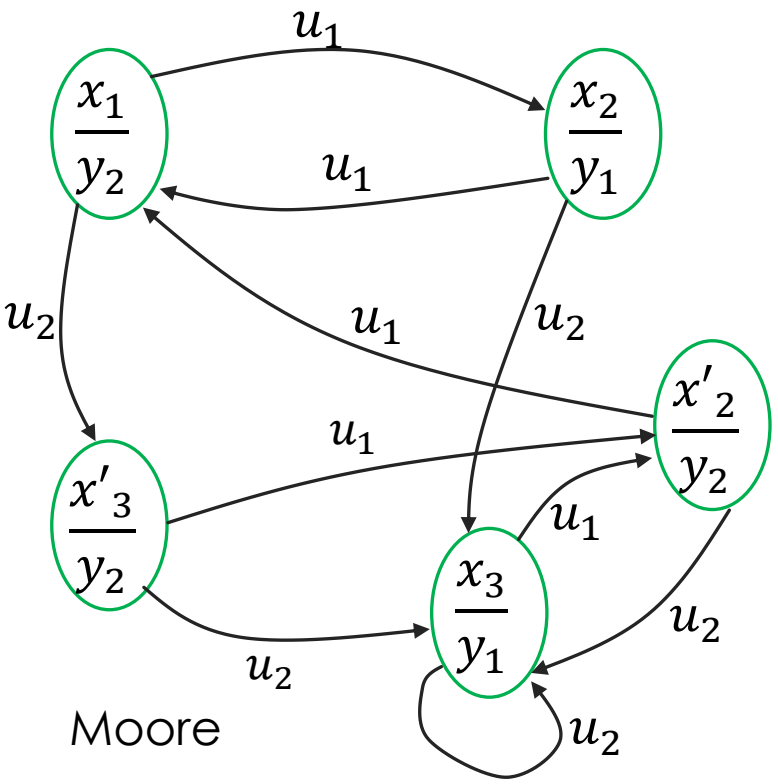
Mealy



Conversia automatului Mealy in automat Moore



Mealy



Moore

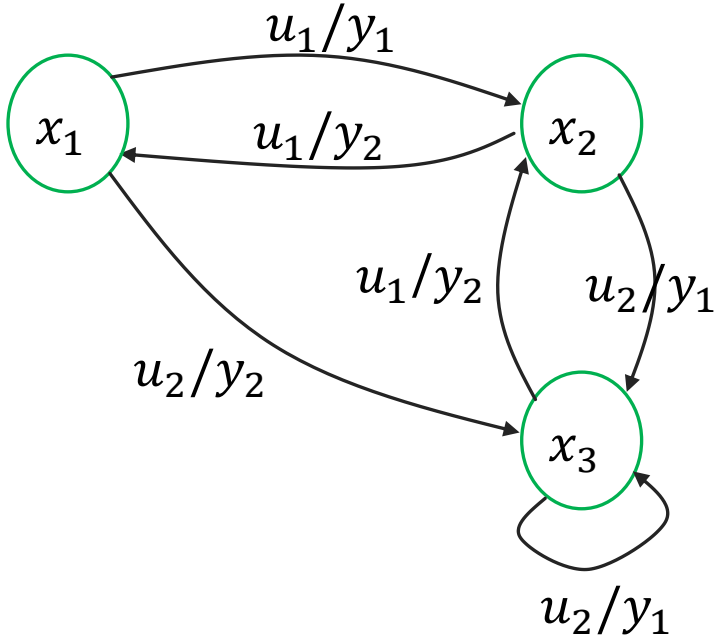
- Două automate secvențiale sunt echivalente dacă pentru aceeași secvență a intrărilor se obține aceeași secvență a ieșirilor și dacă starea internă a aparatelor evoluează din aceeași stare inițială.

Reprezentarea automatelor prin tabele

- Circuitele secvențiale se pot reprezenta prin:
  - Grafuri
  - Forma algebrică (prin definirea funcțiilor)
  - Tabele
  - Organigrame

sunt în starea x_1, pentru intrarea u_1, se trece în starea x_2 generând ieșirea y_1

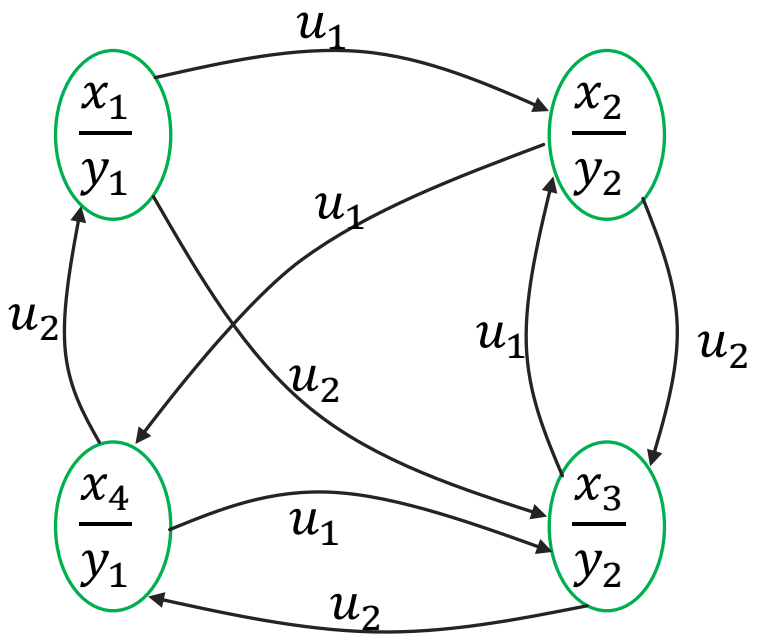
Stare curenta \ Semnal intrare	u_1	u_2
x_1	x_2/y_1	x_3/y_2
x_2	x_1/y_2	x_3/y_1
x_3	x_2/y_2	x_3/y_1



Reprezentarea automatelor prin tabele

- Circuitele secvențiale se pot reprezenta prin:
  - Grafuri
  - Forma algebrică (prin definirea funcțiilor)
  - Tabele
  - Organigrame

Stare curenta / Semnal intrare	u_1	u_2	$g(x_i)$
x_1	x_2	x_3	y_1
x_2	x_4	x_3	y_2
x_3	x_2	x_4	y_2
x_4	x_3	x_1	y_1

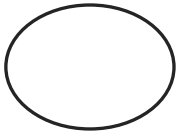


Reprezentarea automatelor prin organigrame

- Notații:
  - Ieșirile sunt reprezentate printr-un dreptunghi

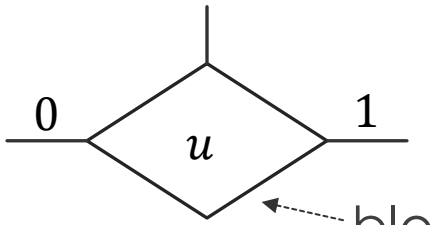


ieșirea



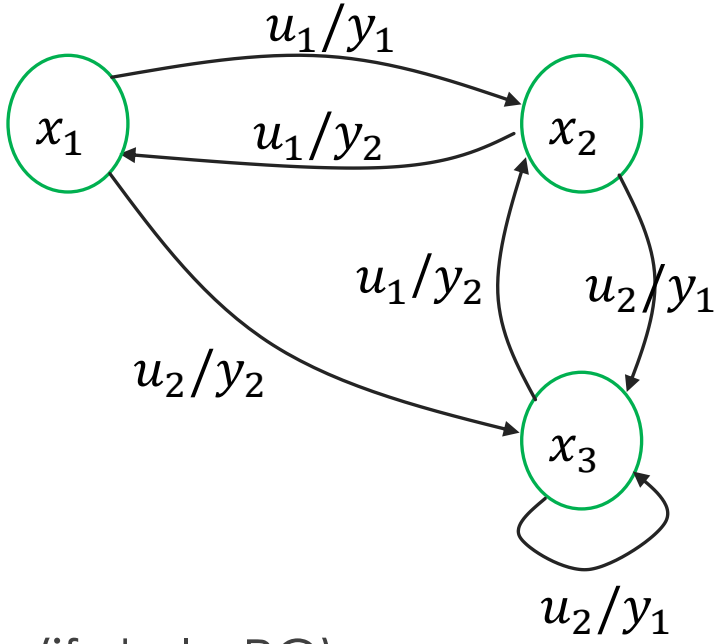
starea

tranziția



intrarea

bloc de decizie (if de la PC)

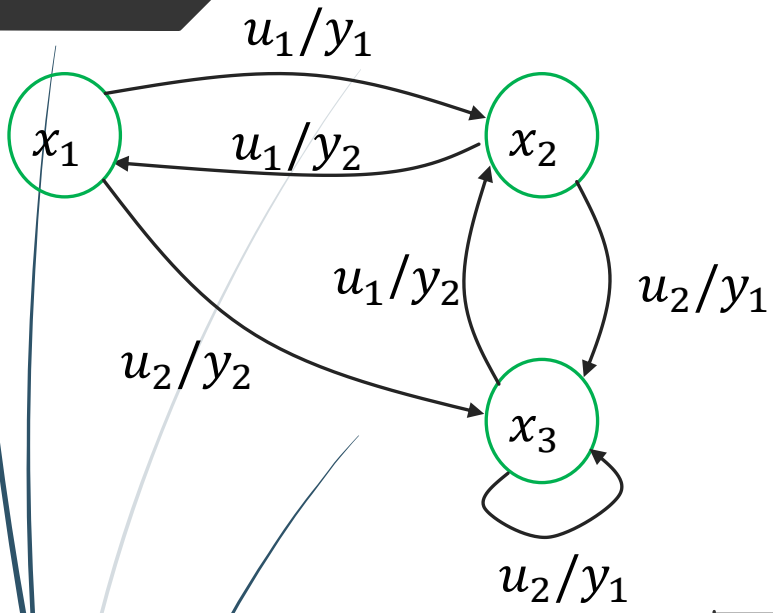


Aici decizia apare explicit. Ea apărea și în forma grafică / tabelară, dar în formă implicită. Ex: sunt în x_3. Dacă vine u_1, atunci mă duc în x_2. Dacă vine u_2, rămân în x_3.

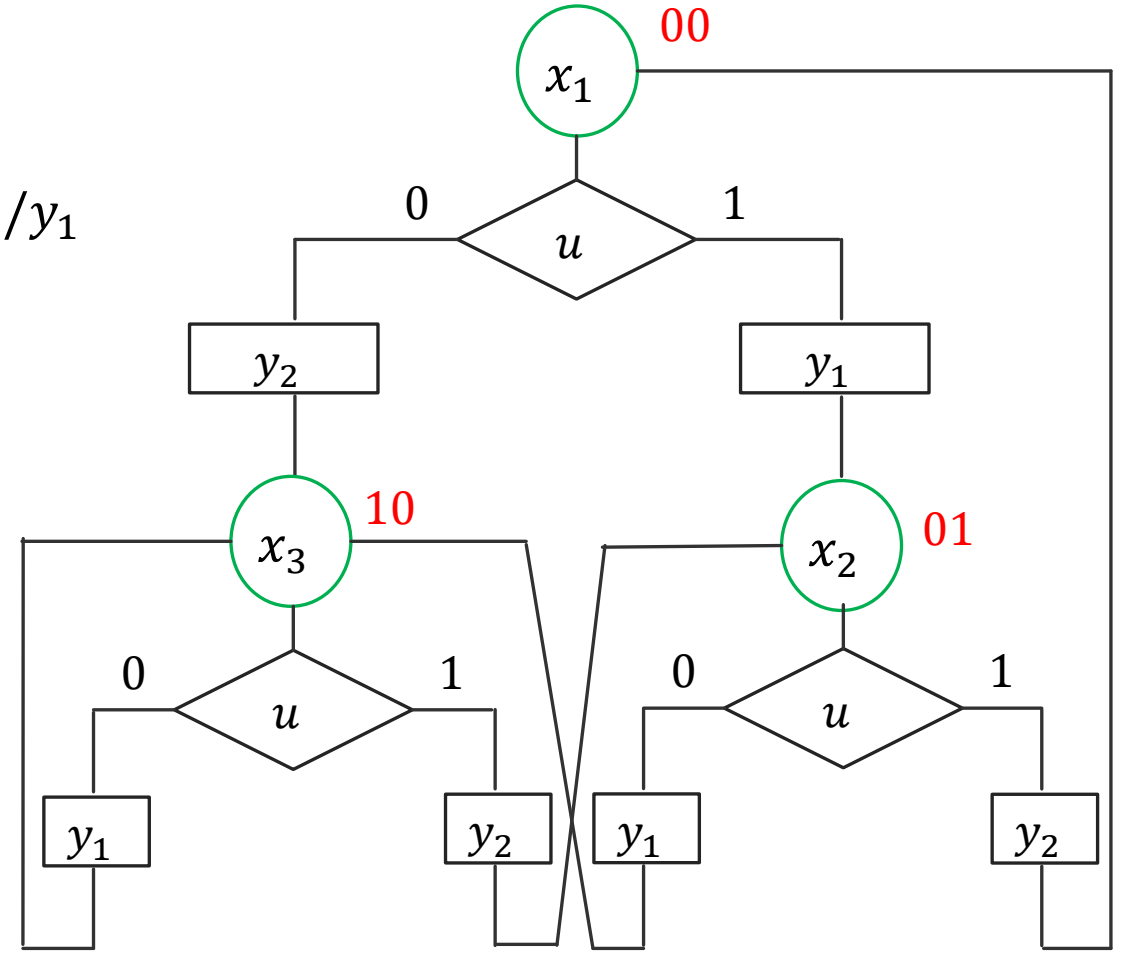
Reprezentarea automatelor prin organigrame

- **Convenție**: considerăm că avem un aparat cu o singură variabilă de intrare (u) și dacă $u = 1$ avem u_1, iar dacă $u = 0$ avem u_2
- u_1: $u = 1$ (u_1 ii corespunde lui $u = 1$)
- u_2: $u = 0$
- Câte variabile de stare avem?
- $2 < 3 < 2^2$
- Numele variabilelor vor fi q_2 și q_1

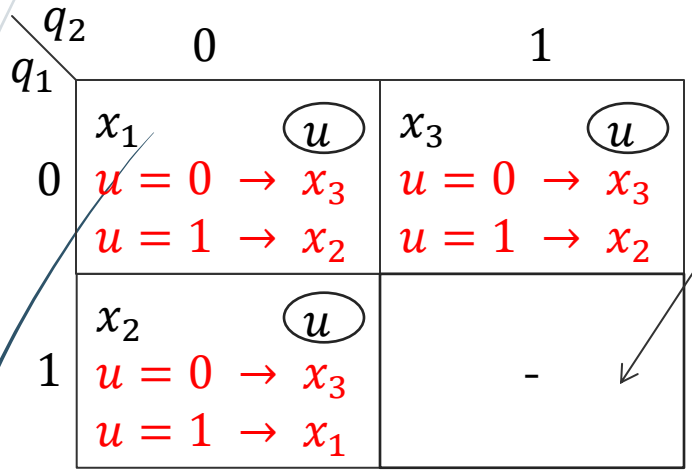
Reprezentarea automatelor prin organigrame



q_2, q_1 variabile de stare



Reprezentarea automatelor prin organigrame



- Aici nu se ajunge niciodată
- Trecerea de la o stare la alta este condiționată de o variabilă externă (u)



Proiectare Logică

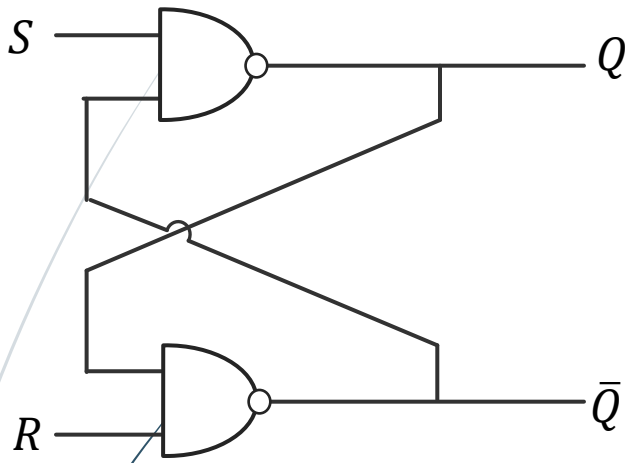
- I CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- **I CC: prof. dr. ing. Anca Morar**
  - anca.morar@cs.pub.ro

Curs 7: Circuite basculante bistabile.

Circuite basculante bistabile (CBB)

- Circuitele secvențiale care prezintă două stări stabile diferite se numesc circuite basculante bistabile (CBB). Ele sunt cele mai cunoscute elemente de memorare digitală.
- Circuitele basculante bistabile se diferențiază în principal în funcție de:
  - Numărul de intrări de comandă,
  - Frontul pe care au loc tranzițiile,
  - Modul de funcționare (sincron/asincron) şi
  - Funcția de stare realizată.

CBB fundamental – Latch RS



- Ce fel de circuit este?
- Secvențial
- De ce e secvențial? Avem feedback?
- Care e feedback-ul?
- Ieșirea depinde de momentul de timp
- Încercăm să îi facem tabela de adevăr
- Reprezentăm sub formă de diagramă Karnaugh

R	S	Q	!Q
0	0	1*	1*
0	1	0	1
1	0	1	0
1	1	Nu se modifică starea anterioară	

stare nepermisă

S \ R	0	1
0	-	1
1	0	Q^t

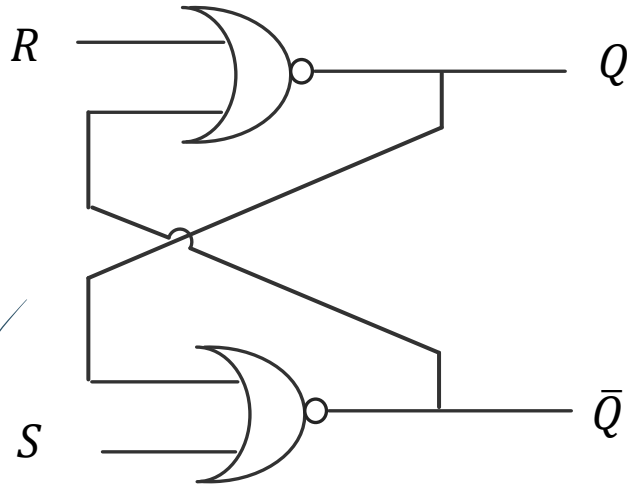
$$Q^{t+1} = RQ^t + R\bar{S}$$

CBB fundamental – Latch RS

- Observație: la circuitele combinaționale am învățat circuite cu ieșiri multiple, dar acolo ieșirile erau independente între ele. Aici, Q și $!Q$ sunt legate una de alta.
- Latch RS = circuit flip-flop = circuit secvențial fundamental asincron (ieșirea se modifică imediat ce se modifică intrările)
- Circuitul funcționează corect dacă nu există combinația $R = S = 0$ → Combinația aceasta este NEPERMISĂ → proiectantul circuitului trebuie să se asigure că cele 2 valori nu sunt simultan 0 pentru că altfel circuitul este neprotejat și are funcționare imprevizibilă (poate lua valori în mod aleator, poate să se blocheze într-o anumită stare etc.)
- Pentru $R = S = 1$, ieșirea depinde de starea anterioară

CBB fundamental – Latch RS

- Implementăm circuitul cu SAU-NU:



R	S	Q	$\bar{Q}$
0	0	Nu se modifică starea anterioară	
0	1	1	0
1	0	0	1
1	1	0*	0*

stare nepermisă

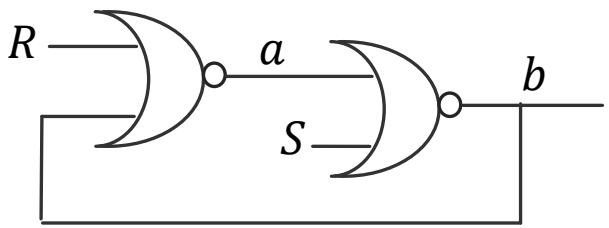
- Dualitatea dintre forma disjunctivă și cea conjunctivă

$$Q^{t+1} = !RS + !RQ^t$$

S \ R	0	1
0	Q^t	0
1	1	-

CBB fundamental – Latch RS

- De reținut: circuitele flip-flop sunt circuite asincrone (nu au intrare de ceas), sunt realizate folosind circuite combinaționale → sunt circuite secvențiale de ordin 1
- Care e valoarea lui Q ?
- Redesenăm circuitul:



$$a = \overline{R + b} \qquad a = \overline{R + \overline{S + a}}$$

$$b = \overline{S + a}$$

$$a = \bar{R} \cdot (S + a) = \bar{R}S + \bar{R}a$$

$$b = \overline{S + \overline{R + b}} = \bar{S} \cdot (R + b) = \bar{S}R + \bar{S}b$$

CBB fundamental – Latch RS

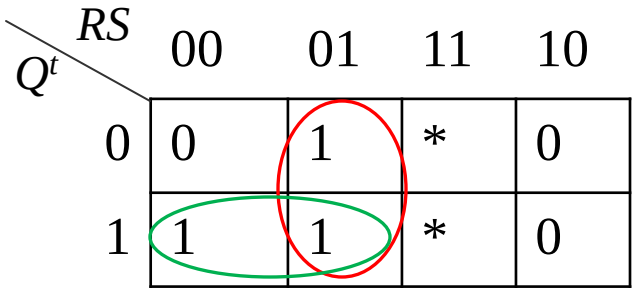
- Observație: sunt 2 momente de timp (t și $t+1$):
- Pe baza circuitului: $Q^{t+1} = !RS + !RQ^t$
- Cum Q^{t+1} depinde și de Q^t, facem tabelul de adevăr care să includă și Q^t

R	S	Q	$\overline{Q}$
0	0	Nu se modifică starea anterioară	
0	1	1	0
1	0	0	1
1	1	0*	0*

stare nepermisă

R	S	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	Nepermisă
1	1	1	Nepermisă

Latch RS – ecuația de stare următoare



Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

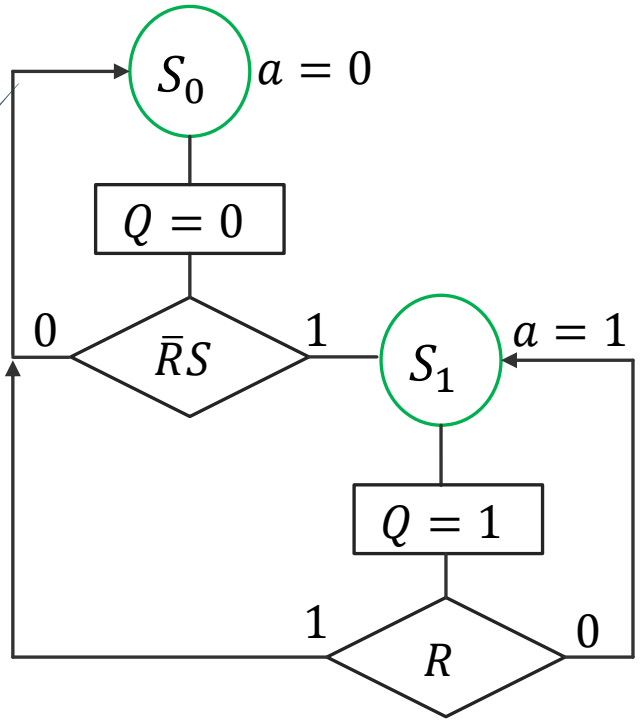
- $Q^{t+1} = !RS + !RQ^t$
- Ecuația de stare următoare → descrie funcționarea circuitului
- Câte stări are circuitul?
- 2
- Câte variabile de stare avem?
- 1 → Q este variabila de stare ce ia valoarea 0 sau 1

Organigrama circuitului - Latch RS

- Cum o construim? Fie pornind de la ecuația de stare, fie de la tabelul de adevăr, fie de la diagrama Karnaugh

$$Q^{t+1} = !RS + !RQ^t$$

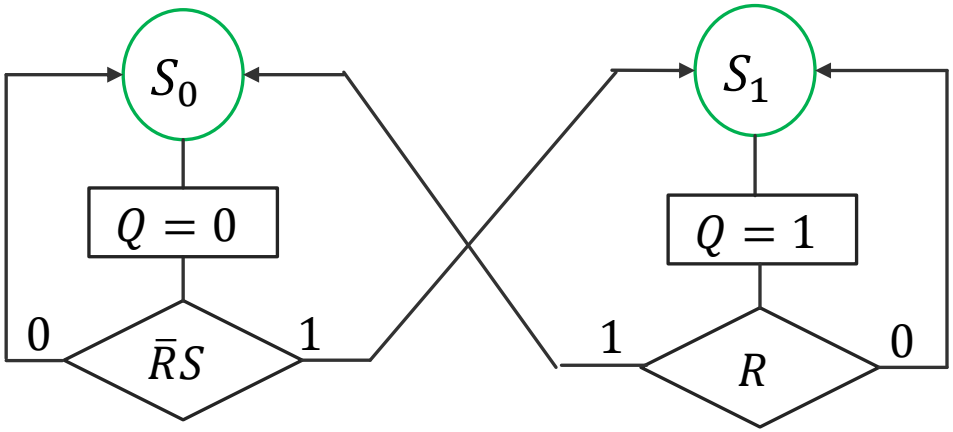
Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0



R	S	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	Nepermisă
1	1	1	Nepermisă

Organigrama circuitului - Latch RS

- Redesenăm:



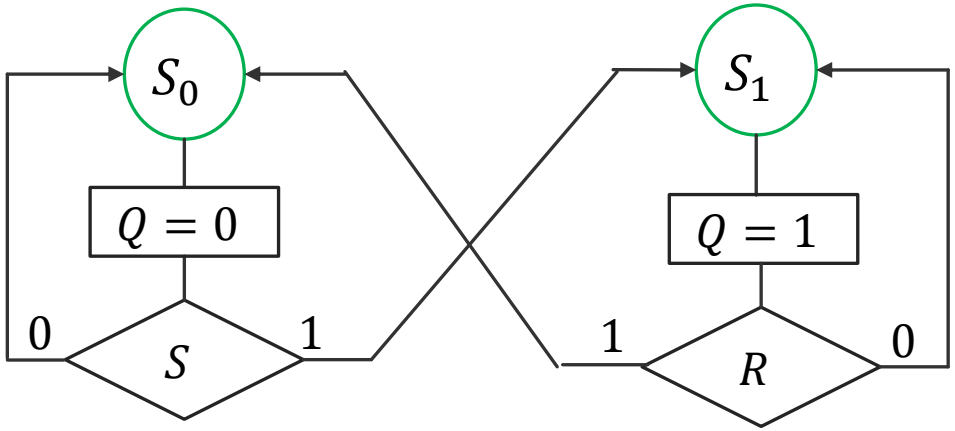
- Observație: Organigrama are o simetrie geometrică, dar prezintă asimetrie dpdv al controlului tranziției: S_0 e controlată de $!RS$ în timp ce S_1 e controlată de R

Organigrama circuitului - Latch RS

- Dacă se interzice simultaneitatea $R = S = 1$ astfel încât Q și $!Q$ să fie consistente.

Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

- $Q^{t+1} = S + !RQ^t$



Diagrame Karnaugh - Latch RS

- Am făcut diagrama Karnaugh în care R,S au fost variabile

S \ R	0	1
0	Q^t	1
1	0	-

Q^t \ RS	00	01	11	10
0	0	1	*	0
1	1	1	*	0

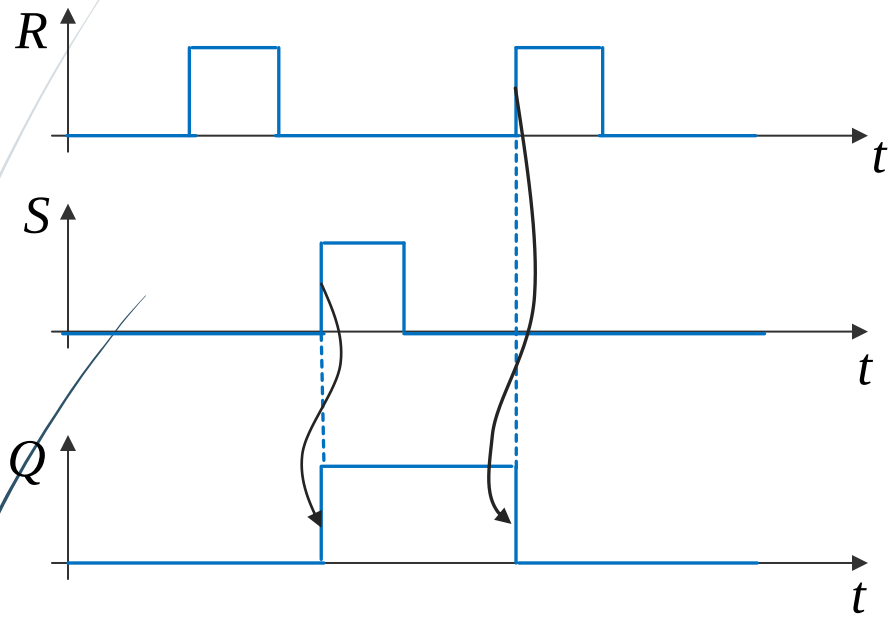
Mealy pentru că Q^{t+1} depinde și de stări și de intrări

- Apoi am adăugat Q
- Putem face și invers? În care să avem doar dependența Q^{t+1} de Q^t și RS să fie interioare
- $Q^{t+1} = S + !RQ^t = SQ^t + S!Q^t + !RQ^t$
- Care e de tip Mealy și care e de tip Moore și de ce?

Q^t	0	1
	S	$S + !R$

Moore pentru că ieșirea Q^{t+1} depinde doar de stări

Diagrama de semnal - Latch RS



- R – reset
- S – set
- Flip-flop elementar în logica pozitivă
- R aduce circuitul în 0, iar S în 1
- Variabila de stare este și variabilă de ieșire

Ordinele automatelor

- Un automat de ordin $n+1$ poate fi generat prin interconectarea unor automate de ordin inferior din care cel puțin unul este de ordin n și e conectat într-o buclă de reacție
- Automatul de ordin 0 este un circuit combinațional și e caracterizat prin absența variabilelor de stare
- Prin conectarea în serie sau în paralel a mai multor automate de ordin n se obține tot un automat de ordin n
- Până acum: orice circuite combinaționale cuplate în serie sau în paralel, dar fără reacție, formează tot un circuit combinațional

Ordinele automatelor

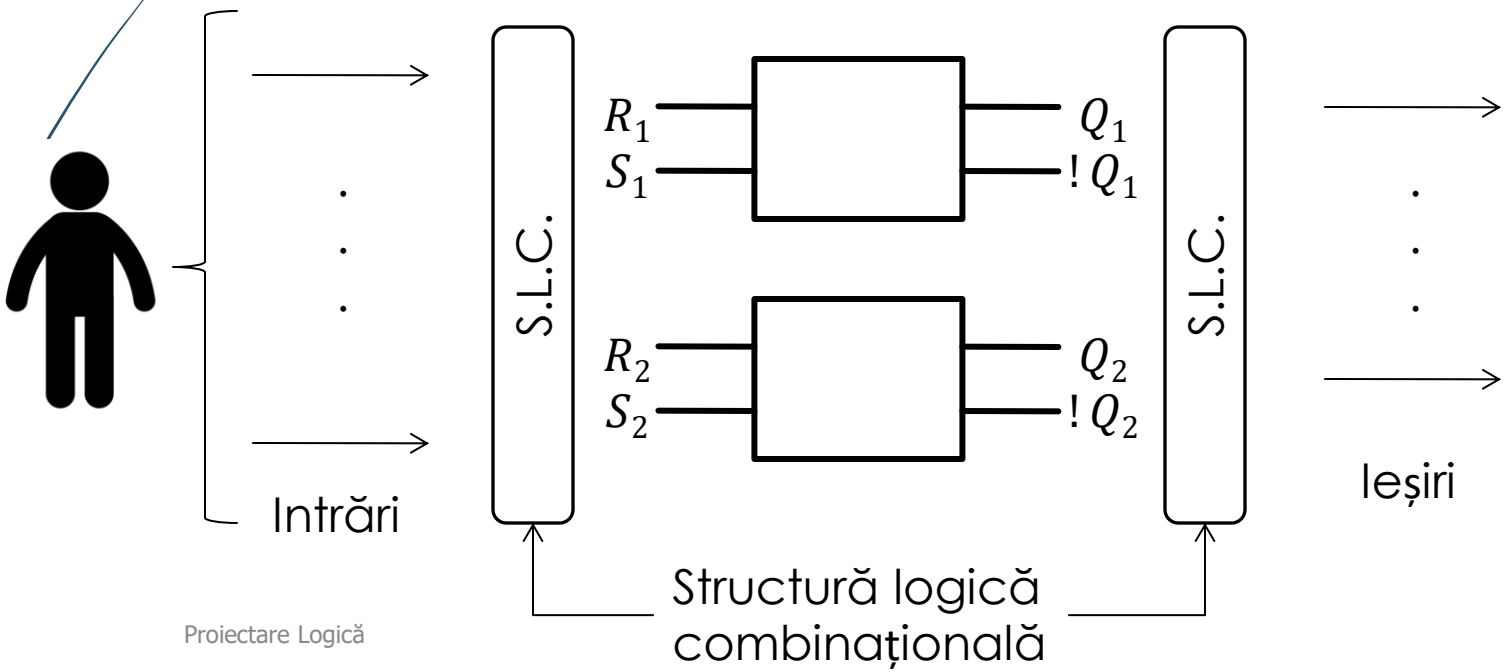
- Flip-flop am spus că este de ordin 1 – de ce?
- Pentru că am buclă de reacție
- Dacă flip-flop îl punem în bucla de reacție a altui circuit → circuit de ordin 2
- La curs vom lucra în special cu circuite de ordin 2, dar spre sfârșitul cursului o sa facem circuite programabile care sunt de ordin 3 sau mai mare.
- Circuitele programabile sunt caracterizate prin elemente de soft care sunt scrise în circuit → reacția se face prin soft (reacția se face prin acces la o anumită zonă de memorie – se revine într-un anumit punct al procesării).

Reprezentarea circuitelor Flip-Flop

- Reprezentare



- Dacă am mai multe, le dau indici pentru a le identifica



Utilizarea circuitelor Flip-Flop

Q_1	Q_2	$Stare$
0	0	S_1
0	1	S_2
1	0	S_3
1	1	S_4

- Câte stări are circuitul?
- 4
- Ce fac cu ele? La ce sunt bune?
- Comand diferite dispozitive (ex: închid/deschid uși, aprind/închid beculețe, ud gazonul, închid/deschid ecluze etc.)
- S.L.C. – urile, pe baza unei anumite logici, iau intrările și le aplică la intrările circuitelor secvențiale astfel încât la ieșire se generează stările în secvența și la momentul potrivit

Circuite secvențiale

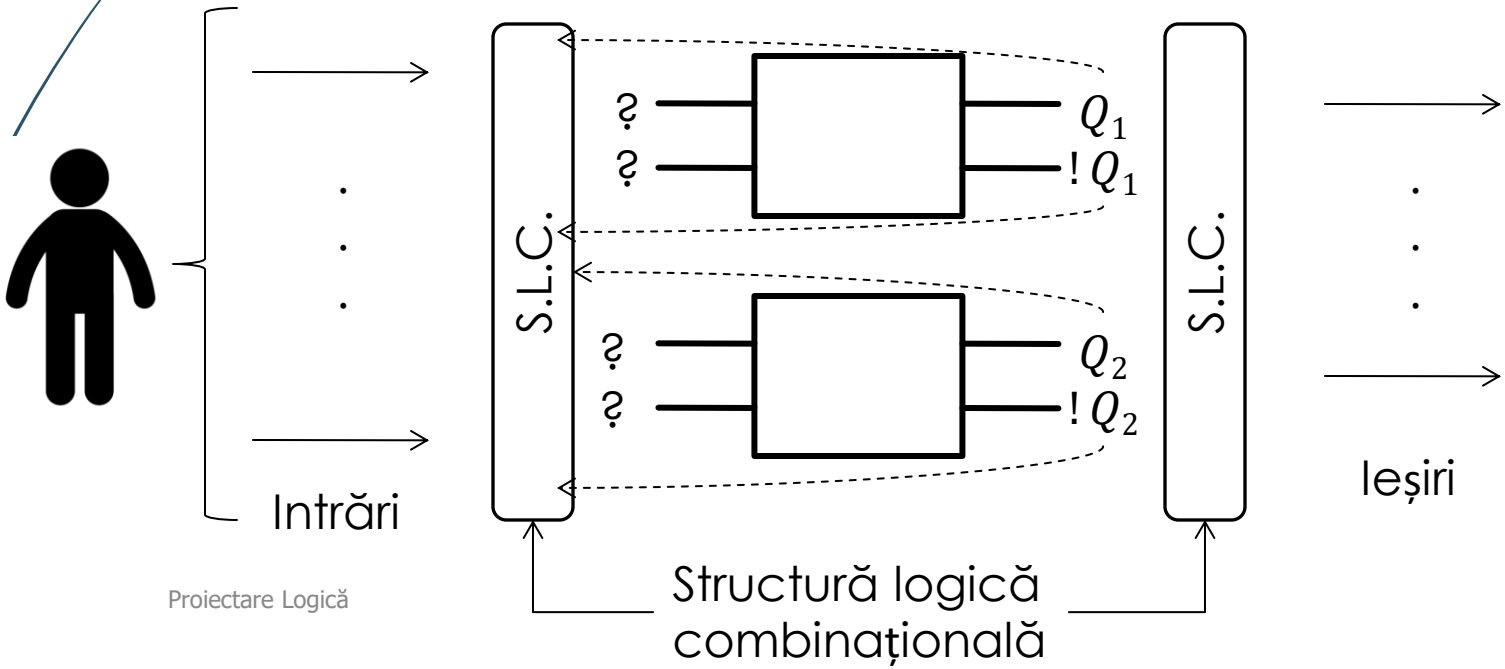
- Ce știm acum?
  - Ce funcții combinaționale să aplicăm ca să obținem rezultatele dorite → facem la fel și la circuitele secvențiale (intrările sunt stările + combinația variabilelor de intrare)
- Ce nu știm?
  - Cum să calculăm ce trebuie pus la intrarea circuitelor secvențiale astfel încât ieșirile să fie corecte
  - Nu e suficient să știm ce se întâmplă cu un singur circuit secvențial (CBB), ci trebuie să știm și ce interacțiuni există între ele

Circuite secvențiale

- Deci nu e suficient să definim
- $Q_1^{t+1} = f(Q_1^t, intrări)$
- $Q_2^{t+1} = f(Q_2^t, intrări)$
- Pentru că sunt scoase din context → mai intervine de fapt și cealaltă variabilă:
- $Q_1^{t+1} = f(Q_1^t, intrări, Q_2^t)$
- $Q_2^{t+1} = f(Q_2^t, intrări, Q_1^t)$
- În realitate, ieșirile de la Q vor fi trecute pe intrare tocmai ca să ne asigurăm că ținem cont de dependențele dintre variabilele de stare

Circuite secvențiale

- Crește ordinul circuitului secvențial → cum justificăm asta? → funcționalitatea depinde de un anumit algoritm (succesiune de acțiuni ce trebuie întreprinse pentru a ajunge la un anumit rezultat)
- Ex: un semafor la o intersecție simplă: verde → galben după un timp → roșu după un timp. La intersecția de la Leu e mai complex, avem un alt algoritm de urmat.



Circuite secvențiale

- Ex: dacă avem un circuit secvențial cu 7 stări → de câte circuite flip-flop am avea nevoie ca să îl implementăm?
- 3, față de exemplu anterior, am mai avea încă un RS și toate ar fi legate între ele prin faptul că intrările sunt realizate prin circuite combinaționale ce primesc intrările circuitului + ieșirile de la toate celelalte RS-uri
- Ce învățăm de fapt?
- Cum să combinăm intrările ca să obținem la ieșire ceea ce dorim (funcționalitatea dorită)

Sincronizarea circuitelor

- La circuite combinaționale nu are sens să vorbim despre sincronizare.
- La circuitele secvențiale, în funcție de modul de realizare a tranziției avem circuite sincrone/asincrone
- Ex: RS-ul făcut este asincron.
- Sincronizarea se face pe baza unei intrări de ceas (Tact, Clock)
- Există 2 tipuri de sincronizări:
  - Sincronizarea pe front
  - Sincronizarea de tip master slave

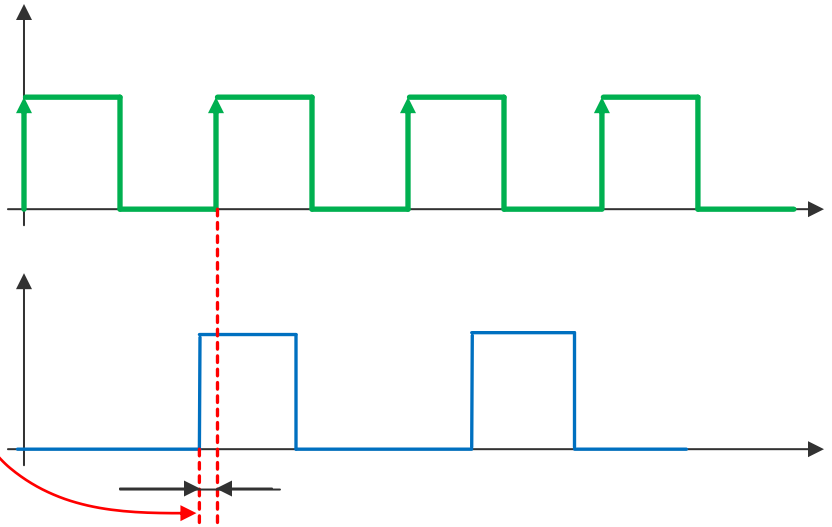
Sincronizarea pe front crescător / descrescător

- Pe unul din cele 2 fronturi are loc tranziția → se întâlnește la o serie de CBB-uri
- Realizează operația de basculare la o anumită valoare a tensiunii, atunci când are loc tranziția pozitivă/negativă
- Un dispozitiv dat nu recunoaște decât un singur tip de tranziție (fie pozitivă, fie negativă)
- Intrarea de ceas acționată pe front validează intrările de date și transferă datele sincron la ieșire

Sincronizarea pe front crescător / descrescător

- Avantaje:
  - Dacă se cunoaște momentul de timp se pot pregăti condițiile de intrare astfel încât să se realizeze tranziția când trebuie
  - În cataloage, circuitele au niște valori pentru timpul minim de stabilitate pentru a recunoaște o anumită schimbare a valorilor de intrare

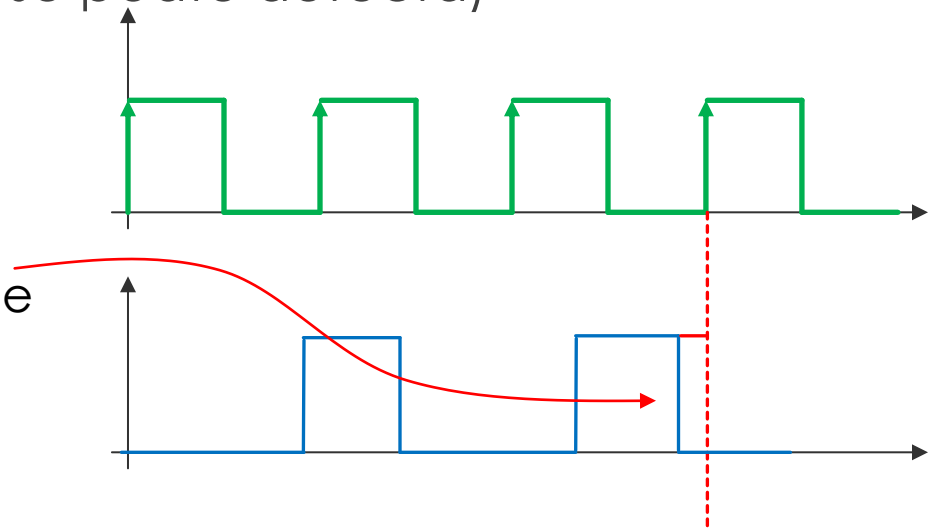
Aici nu se poate detecta pentru că semnalul nu e stabil de suficient timp



Sincronizarea pe front crescător / descrescător

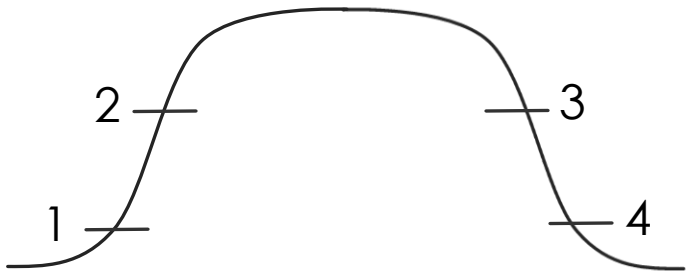
- Dezavantaje:
  - Nu mereu putem sincroniza intrările cu semnalul de sincronizare
  - Apare când mărim prea mult perioada semnalului de sincronizare
  - Există metode de rezolvare a problemelor (ex: folosire de circuite de memorare pentru prelungirea semnalului până se poate detecta)

Aici nu se poate detecta pentru că semnalul nu există pe frontul crescător al ceasului



Sincronizarea master-slave

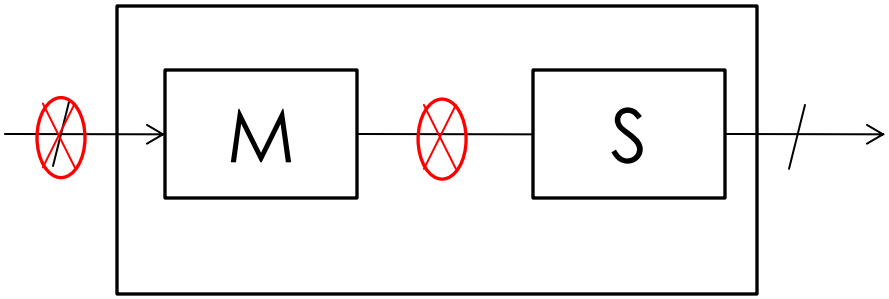
- Se numește și sincronizare în 4 timpi – avem 4 momente de timp în care se întâmplă ceva în circuit (semnalul de ceas acționează circuitul în 4 pași)
- E utilizată la acele circuite secvențiale la care putem identifica 2 module: master (M) & slave (S)
- Este format din 2 latch-uri în serie

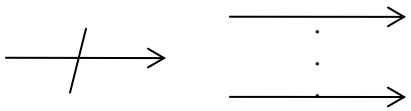


1. Izolează etajul S de M (ieșirile vechi rămân stabile)
2. Se validează intrările datelor în M
3. Se invalidează intrările de date (modificările noi nu se mai iau în calcul)
4. Se face trecerea datelor din M în S

Sincronizarea master-slave

- Observație: intrările nu sunt conectate niciodată direct la ieșiri, pentru orice moment al tactului.



-  1,2, ... N semnale
- Sincronizarea în 4 timpi protejează ieșirea de intrare (ex. La JK-uri)

Proiectare Logică

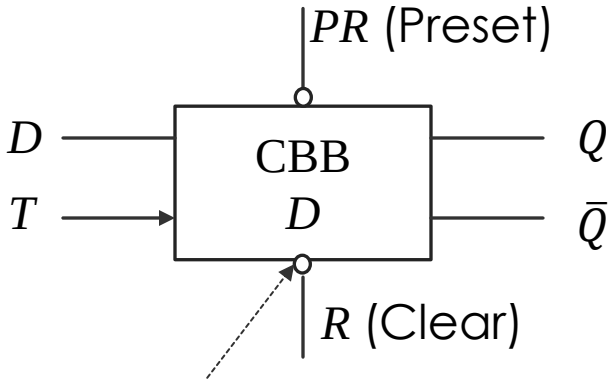
- I CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- **I CC: prof. dr. ing. Anca Morar**
  - anca.morar@cs.pub.ro

Curs 8: Circuite basculante bistabile D, RS, JK

CBB tip D

- Unul din cele mai utilizate circuite la acest nivel
- E integrat pe scară medie
- De ce învățăm circuite din astea când în practică se folosesc circuite integrate pe scară largă și VLSI? (un microprocesor e doar un pic mai mare ca un circuit de tip D sau altele similare)
- Pentru că principiile de la CBB-uri se aplică și la circuite integrate pe scară largă și VLSI
- Pentru că încă se folosesc în practică în soluții embeded (unde e nevoie, pe lângă soluții standard și de diverse adaptări / ceva în plus)

CBB tip D



- D = date
- T = tact (CLK)
- Este sincron
- Include un circuit RS asincron și are o intrare de date și una de ceas
- R & S sunt intrări asincrone ale circuitului de tip D, sunt prioritare și se aplică peste intrările sincrone (indiferent de D sau CLK)
  - dacă $R = 0, Q$ trece în 0 iar dacă $S = 0, Q = 1$
  - R & S sunt active pe 0
  - R & S nu pot fi simultan pe 0 (active)

CBB tip D

- Observație: Peste tot intrările asincrone sunt prioritare față de cele sincrone → sunt intrări de forțare. Ex: resetul de la calculator care trece toate circuitele în starea lor inițială
- Tabelul de adevăr

PR	*R*	*D*	*T*	*Q*	
0	1	*	*	1	Functie asincrona
1	0	*	*	0	
0	0				← nepermis
1	1	1	↑	1	Functie sincrona
1	1	0	↑	0	

Tranziția are loc pe frontul crescător al ceasului (depinde de front, nu de valoarea ceasului)

Front crescator al ceasului

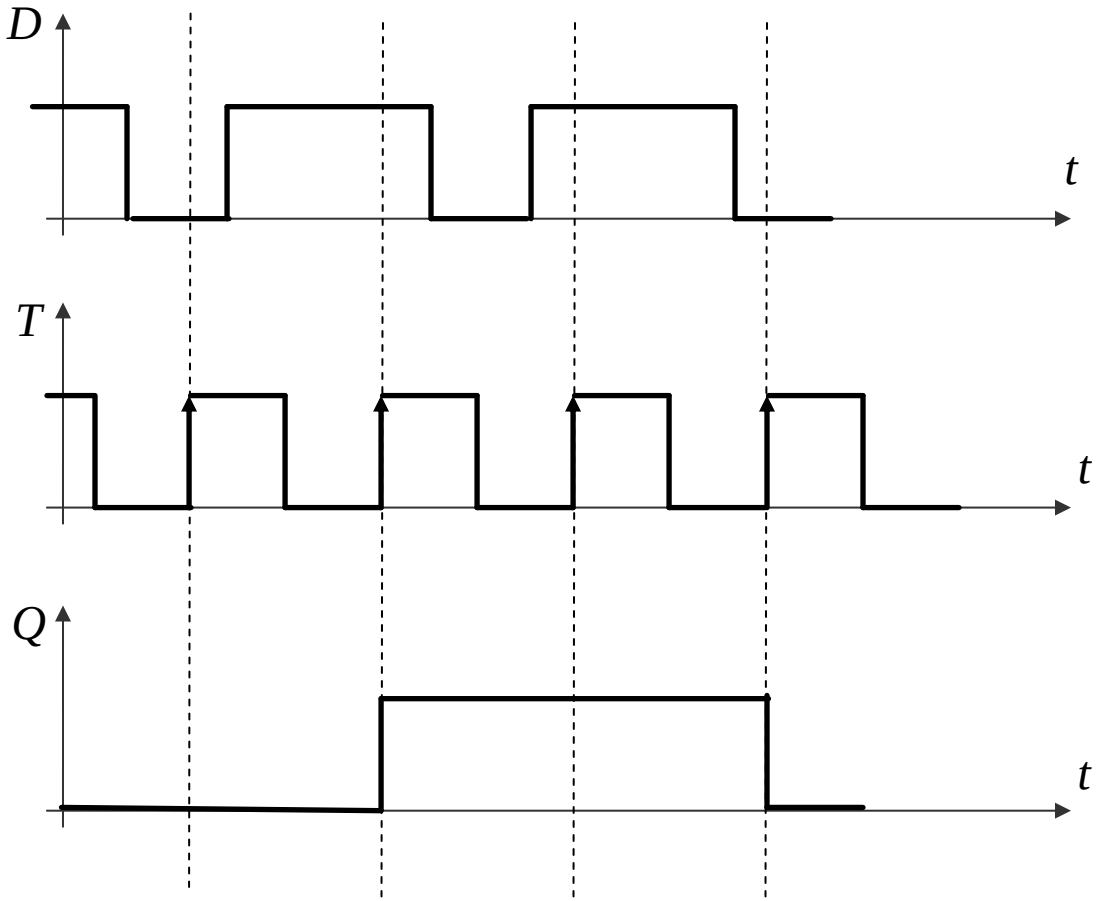
CBB tip D

D \ Q	0	1
0	0	0
1	1	1

- Ecuația circuitului este: $Q^{t+1} = D^t$
- CBB-ul de tip D ține pe o perioadă de ceas valoarea anterioară primită la intrare
- În această ecuație nu intervine RS → e ca și cum aș avea 2 circuite ce funcționează pe principiul superpoziției, dar RS e prioritar
- $Q^{t+1} = Dt \rightarrow D^t = Q^{t+1}$ → dacă știu cât vreau să obțin la ieșire, acea valoare o pun la intrare la momentul imediat anterior

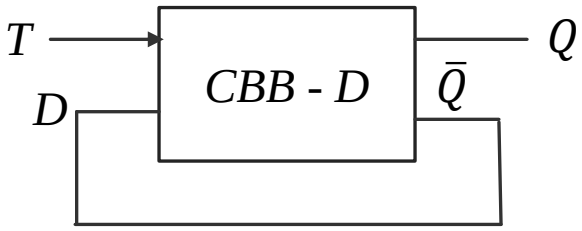


CBB tip D – diagrama de semnal

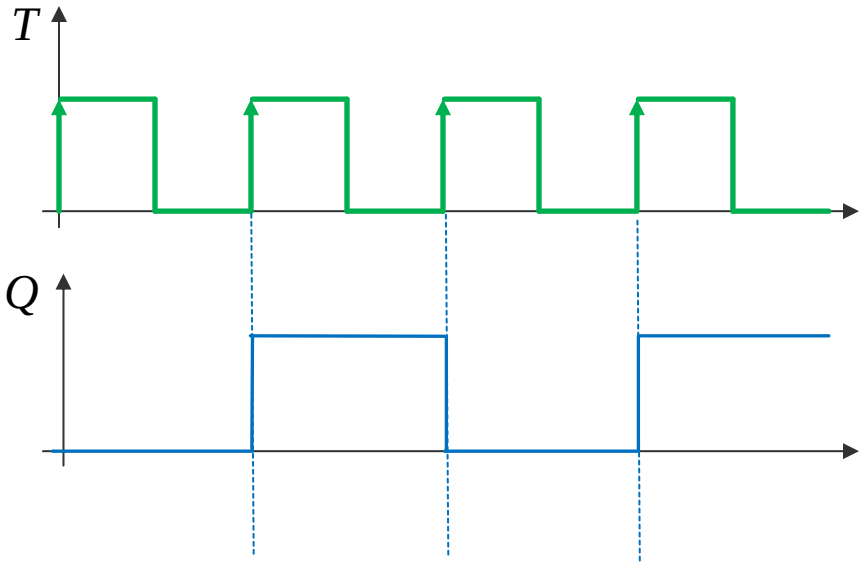


CBB tip D – aplicație: Divizor de frecvență

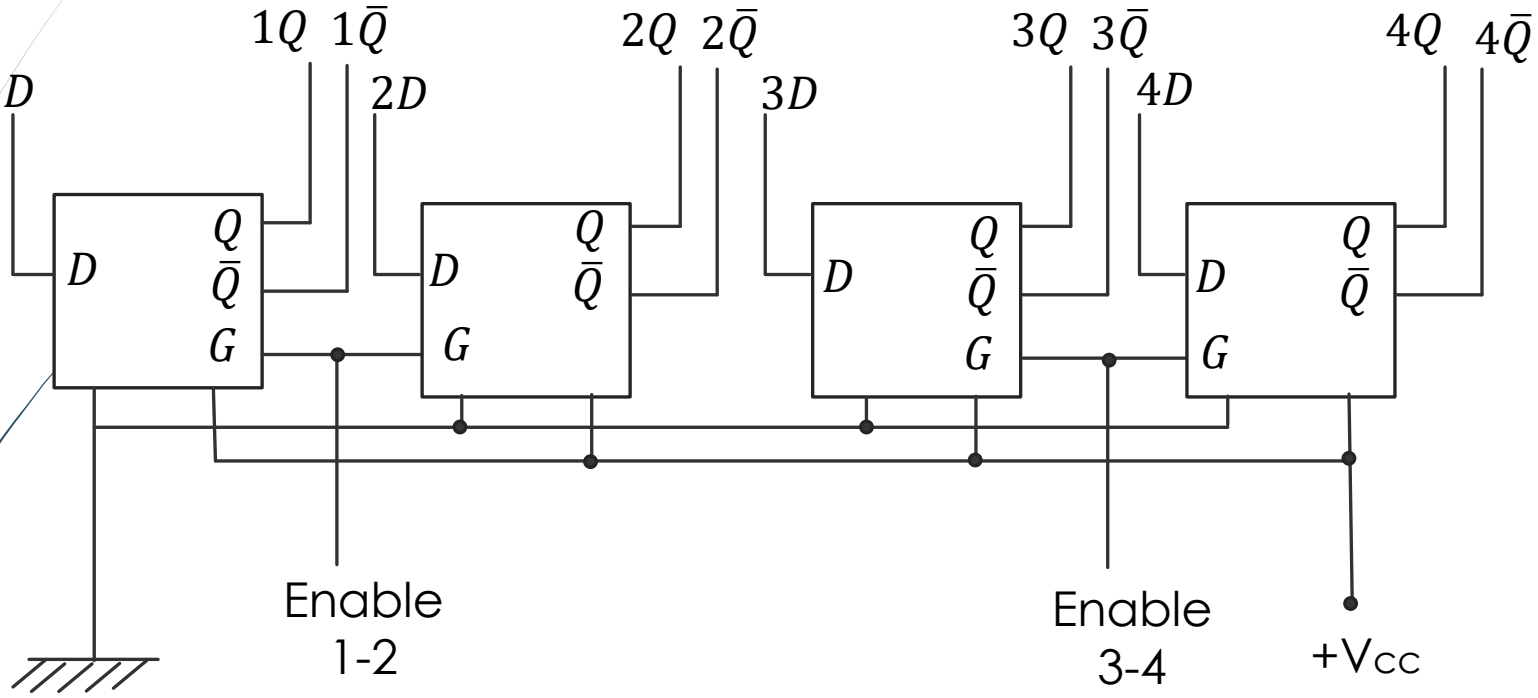
- Generează semnal de aceeași natură, dar de frecvență mai mică



$$Q^{t+1} = \overline{Q^t}$$



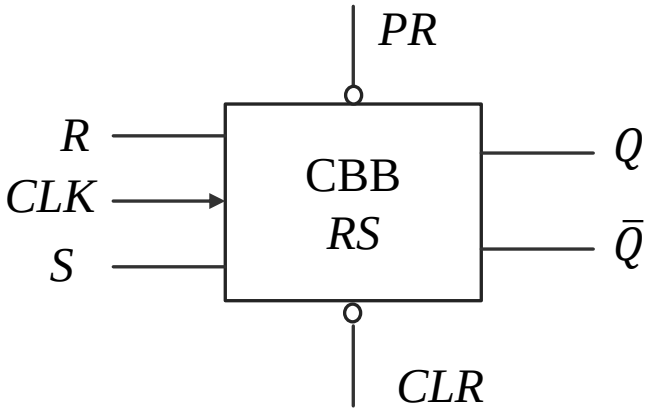
CBB tip D – circuite fizice



- Cel mai simplu registru pe 4 biți format din CBB-uri de tip D. Acestea nu sunt independente, toate sunt șterse simultan

CBB tip RS

- CBB tip D este singurul cu o singură intrare
- Există și circuit RS sincron (are în plus față de RS latch și o intrare de ceas)
- Are aceeași funcționalitate, dar acum e sincron
- Interesant pentru că sunt de fapt 2 circuite RS suprapuse – unul sincron, altul asincron
- Nu insistăm pentru că nu e prea folosit în practică din cauza combinației nepermise ($R = S = 1$)



CBB tip RS

S	R	Q^t	Q^{t+1}	
0	0	0	0	Q^t
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	1	1
1	0	1	1	
1	1	0		nedeterminat
1	1	1		

Q \ SR	00	01	11	10
0	0	0	*	1
1	1	0	*	1

S	R	Q^{t+1}
0	0	Q^t
0	1	0
1	0	1
1	1	*

CBB tip RS

- Ecuația de stare:

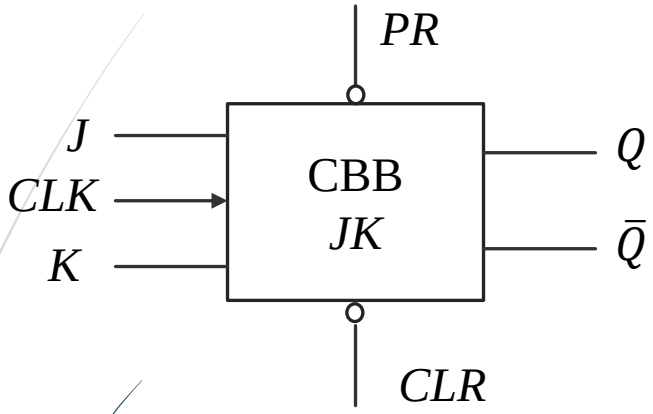
$$Q^{t+1} = \bar{R}^t Q^t + \bar{R}^t S^t$$

$$S^t R^t = 0 \quad \text{(cel putin unul este 0)}$$

- De obicei se cunoaște funcționalitatea dorită de la un aparat (Q^t și Q^{t+1}) și se dorește identificarea elementelor de circuit necesare pentru implementarea acelui aparat (R și S) → se poate detecta folosind tabelul următor:

Q^t	Q^{t+1}	R	S
0	0	–	0
0	1	0	1
1	0	1	0
1	1	0	–

Circuitul basculant bistabil JK



- 2 Intrări de date: J, K
- 1 Intrare de ceas: CLK
- 2 Ieşiri: Q şi $!Q$
- 2 Intrări de comandă: R (clear)
  S (preset)
  - asincrone, prioritare

- Implementarea fizică constă în 2 circuite pe pastilă
- Are funcționalitate asincronă precum celelalte CBB-uri
- Are un comportament derivat din funcționarea RS (unde aveam comportament neplăcut din cauza stării de incontrolabilitate)
- A apărut pentru a corecta problema de la RS (pentru orice combinație de intrare, se știe în ce stare trece CBB JK)

CBB JK – tabelul de adevăr

J	K	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

CLK	J	K	Q^{t+1}	
	0	0	Q^t	pasiv
	0	1	0	reset sincron
	1	0	1	set sincron
	1	1	$\overline{Q^t}$	toggle

- Trecem la varianta doar în funcție de J și K:

- Tabelul de adevăr este identic cu cel de la CBB RS, cu excepția $J = K = 1$, când ieșirea își modifică starea

CBB JK – ecuația de stare următoare

- Pe baza tabelului de adevăr încercăm să determinăm ecuația de stare următoare → completăm diagrama Karnaugh corespunzătoare

J	K	Q^t	Q^{t+1}	
0	0	0	0	Q^t
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	1	1
1	0	1	1	
1	1	0	1	$\overline{Q^t}$
1	1	1	0	

Q \ JK	00	01	11	10
0	0	0	1	1
1	1	0	0	1

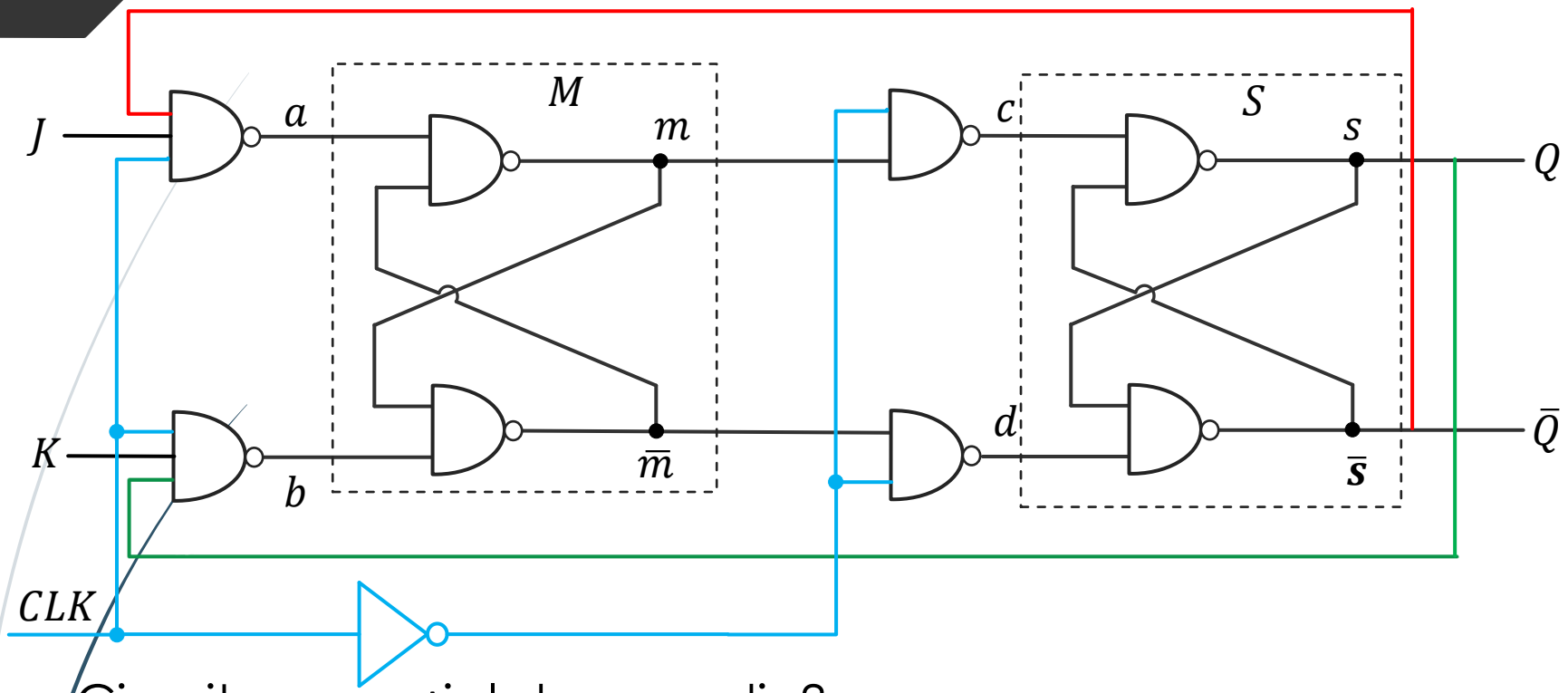
$$Q^{t+1} = \bar{K}^t Q^t + J^t \bar{Q}^t$$

CBB – determinare J și K

- Cel mai răspândit tip de CBB
- Ca la RS, în general se cunoaște funcționarea circuitului și se dorește identificarea elementelor de circuit necesare pentru a se implementa J și K. Acestea se determină pe baza tabelului

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

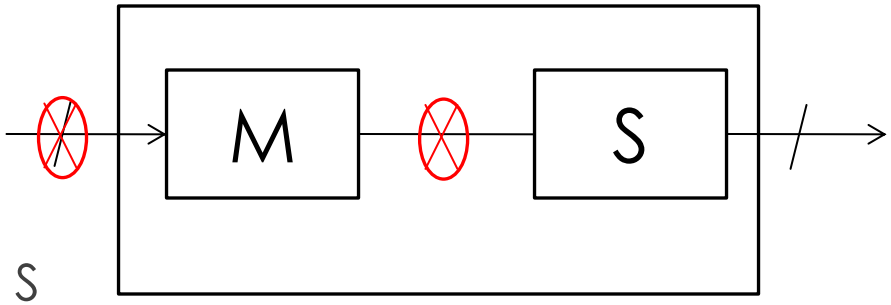
CBB JK – implementare (1)



- Circuit secvențial de ce ordin?
- O luăm pe bucățele:
  - Zonele punctate reprezintă CBB-uri elementare (de ordin 1)
  - Cum sunt legate? În serie →ordin 1
  - DAR! Observăm cele 2 conexiuni de reacție (!Q – J & Q - K)
  - → Ordinul este 2 pentru că are în bucla de reacție CBB de ordin 1

CBB JK – implementare (2)

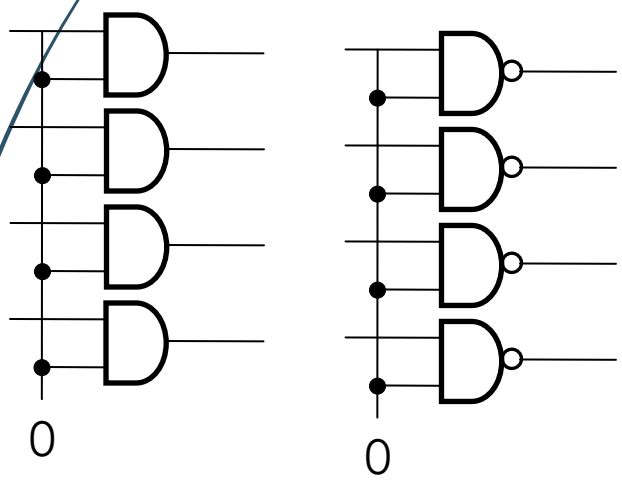
- Avem 2 părți: una de master (M) și una de slave (S)
- Am vorbit în trecut de sincronizarea
  - Pe front
  - Master-slave (în 4 timpi). Care sunt acțiunile din cei 4 timpi?
    - 1. decuplarea M de S,
    - 2. intrarea datelor în M,
    - 3. invalidarea intrărilor,
    - 4. trecerea datelor din M în S



  - Această sincronizare are ca scop protejarea circuitului de momente ce țin de timpii de propagare, timpii de stabilizare, momentele când tensiunea este între nivelurile de 0 și 1, etc.

CBB JK – sincronizare (1)

- Ce înseamnă invalidarea intrărilor? Cum se face?
- Porțile ȘI pot bloca aplicarea datelor utile prin simpla aplicare a unei valori 0. La NAND e la fel, ieșirea nefiind 0, ci 1.

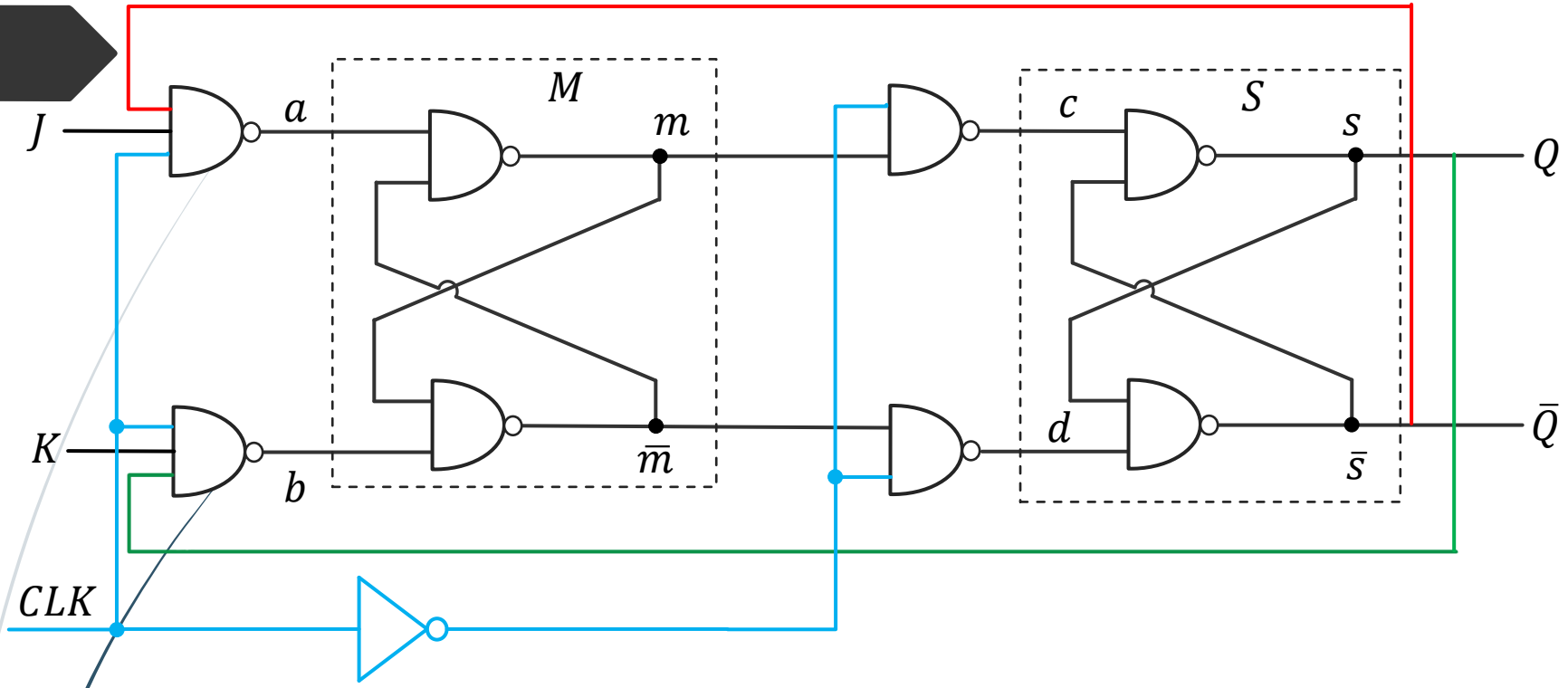


- Este foarte folosită pentru că protejează circuitul și este foarte simplă (ieftină, fără erori)
- Așa se întâmplă cu nivelul a-b / c-d de NAND-uri din CBB JK

CBB JK – sincronizare (2)

- Care sunt datele utile?
- J & K
- Cum blocăm circuitul?
- Cu ajutorul CLK
- Cum adică?
- O dată e valid un nivel, o dată celălalt
- Cum se face asta?
- Cu ajutorul inversorului → el asigură decuplarea

CBB JK – implementare (3)



$$s = \overline{c \cdot \overline{s \cdot d}} = \bar{c} + s \cdot d$$

$$m = \overline{a \cdot \overline{b \cdot m}} = \bar{a} + b \cdot m$$

$$c = \overline{m \cdot \overline{CLK}}$$

$$d = \overline{\bar{m} \cdot \overline{CLK}} = m + CLK$$

$$a = \overline{J \cdot CLK \cdot \bar{Q}} = \overline{J \cdot CLK \cdot \bar{s}}$$

$$b = \overline{K \cdot CLK \cdot s} = \bar{K} + \overline{CLK} + \bar{s}$$

Determinăm formulele pentru s și m:

$$s = m \cdot \overline{CLK} + m \cdot s + CLK \cdot s$$

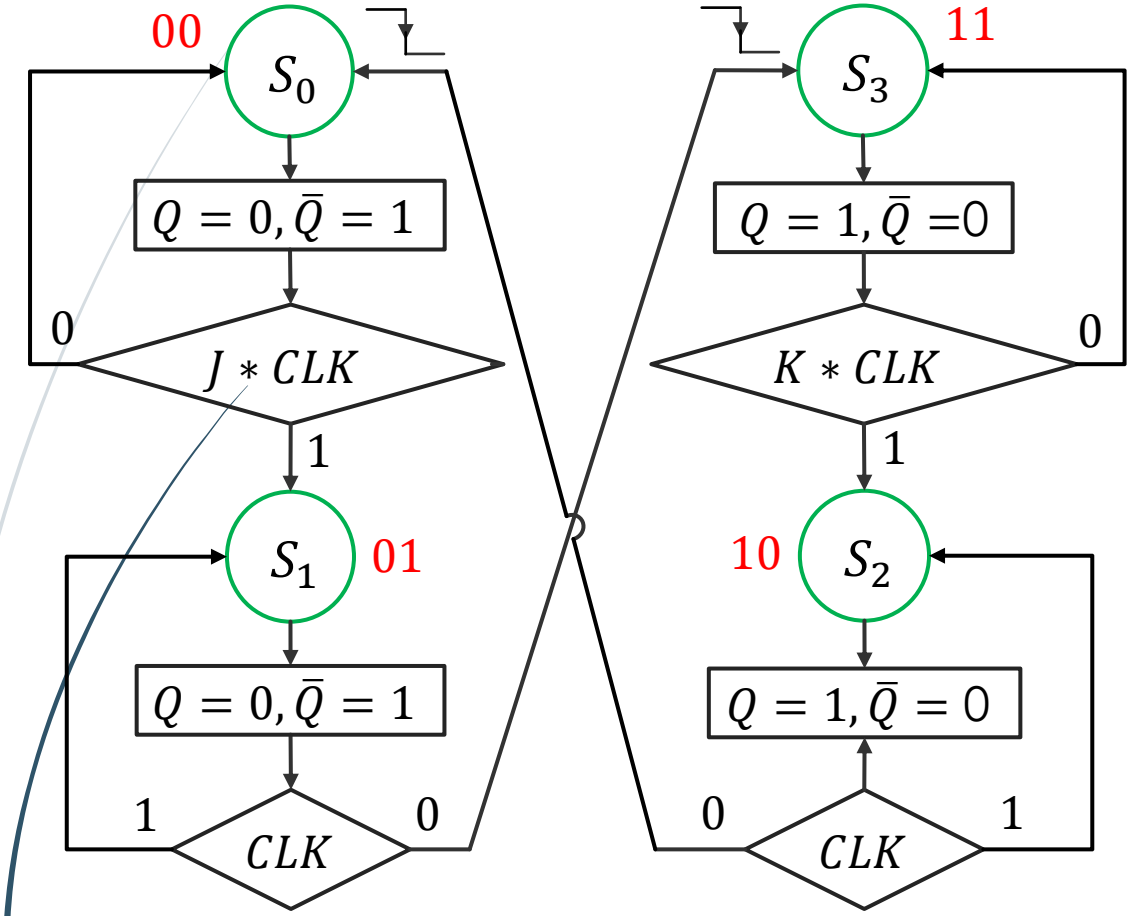
$$m = J \cdot CLK \cdot \bar{s} + \bar{K} \cdot m + \\ + \overline{CLK} \cdot m + \bar{s} \cdot m$$

CBB JK – funcționare

- Putem considera m și s ca fiind variabilele de stare ale CBB-urilor elementare → circuitul are 4 stări corespunzătoare acestor variabile de stare.
- Construim diagrama Karnaugh folosind s, m, CLK, J și K

s, m \ Clk, J, K	0 0 0		0 0 1		0 1 1		0 1 0		1 1 0		1 1 1		1 0 1		1 0 0	
00	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0
01	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1
11	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1
10	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0

CBB JK - Organigrama



- Cele 4 momente de la sincronizarea master-slave corespund celor 4 stări.
- Temă: care moment corespunde cărei stări? + Justificare.

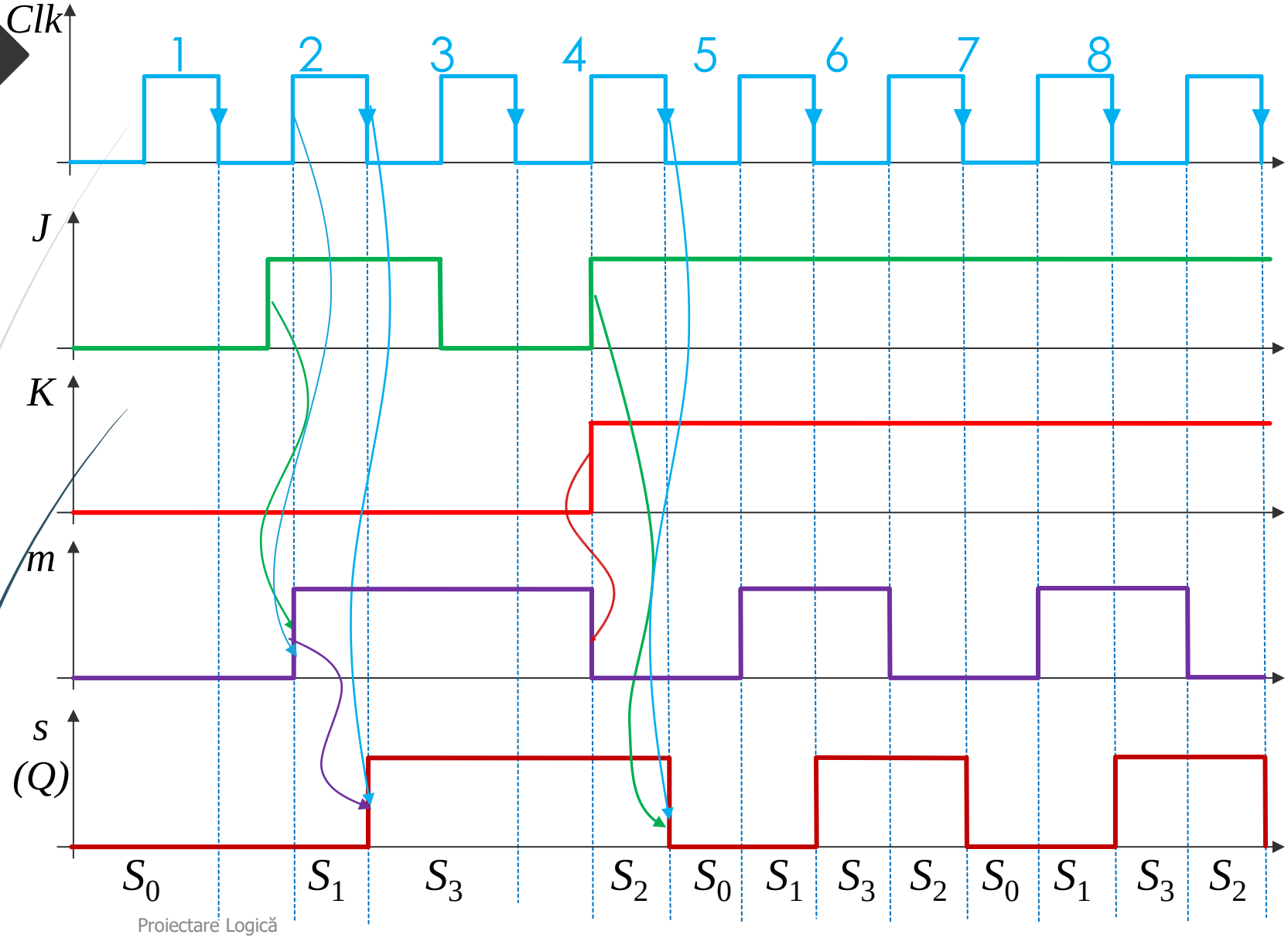
$$s = m * \overline{CLK} + m * s + CLK * s$$

$$m = J * CLK * \bar{s} + \overline{K} * m + \overline{CLK} * m + \bar{s} * m$$

CBB JK – funcționare

- Tranziția mărimilor de ieșire se manifestă pe căderea impulsului de ceas → funcționarea circuitului este sincronă pe frontul descrescător al semnalului CLK
- Când $CLK = 0$, circuitul este fie în S_0, fie în S_3, când $CLK = 1$, circuitul este fie în S_1, fie în S_2
- J controlează S_0 a circuitului (cât timp $J = 0$, circuitul rămâne blocat în S_0) → J controlează trecerea lui Q din 0 în 1
- K controlează S_3 a circuitului (cât timp K = 0, circuitul rămâne blocat în S_3) → K controlează revenirea lui Q din 1 în 0
- Obs: J și K au funcționalitate similară cu S și R, dar nu pot determina funcționarea circuitului în lipsa ceasului (acțiunea lor e de tip sincron)

CBB JK - Diagrama de semnal

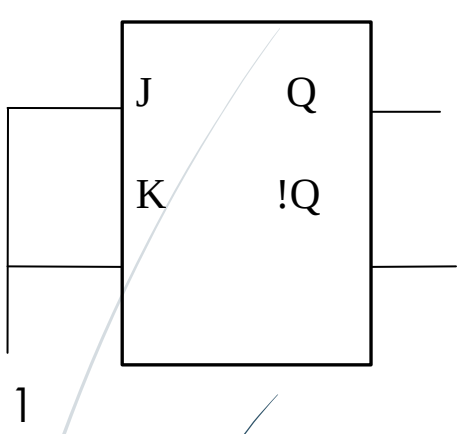


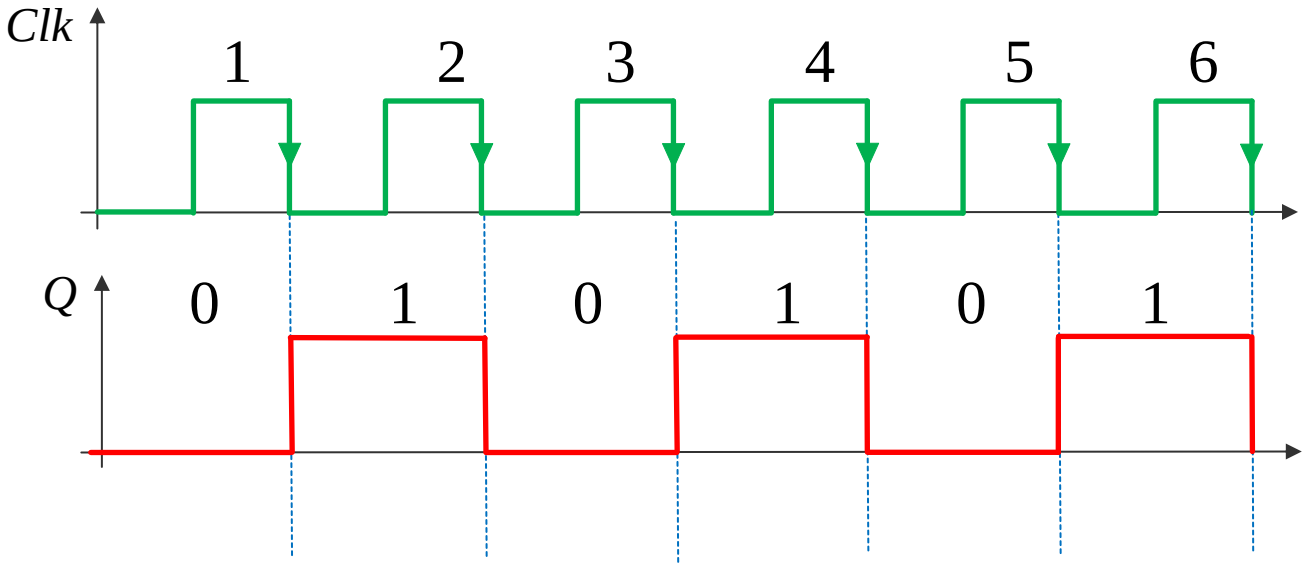
CBB JK –explicații

- Circuitul M captează informația utilă pe durata activă a lui $CLK\ (=1)$
- Circuitul S copiază informația din M în momentul trecerii ceasului în 0
- În practică, circuitul este prevăzut și cu comenzi asincrone (preset și clear) active pe 0:

PR	*CLR*	*J*	*K*	*CLK*	*Q t+1*	
1	1	*	*	↓	Ca mai devreme	← Funcționare sincronă
0	1	*	*	*	1 (set asincron)	Funcționare asincronă
1	0	*	*	*	0 (reset asincron)	Funcționare asincronă
0	0					← Nepermis

CBB JK – aplicație (1)



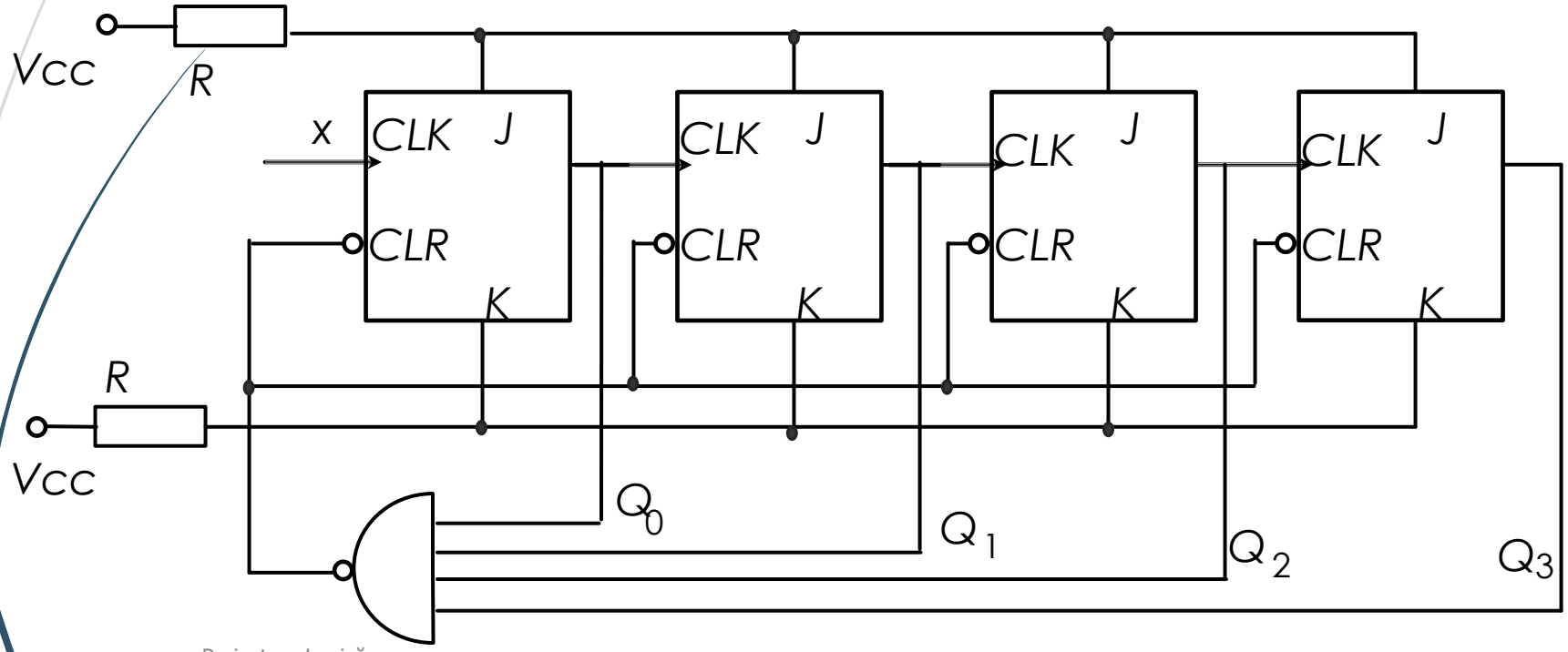


- Ce face?
- Inversează valoarea lui Q: $Q^{t+1} = !Q^t$
- Divizor de frecvență
- Am mai intâlnit?
- Da, la CBB D

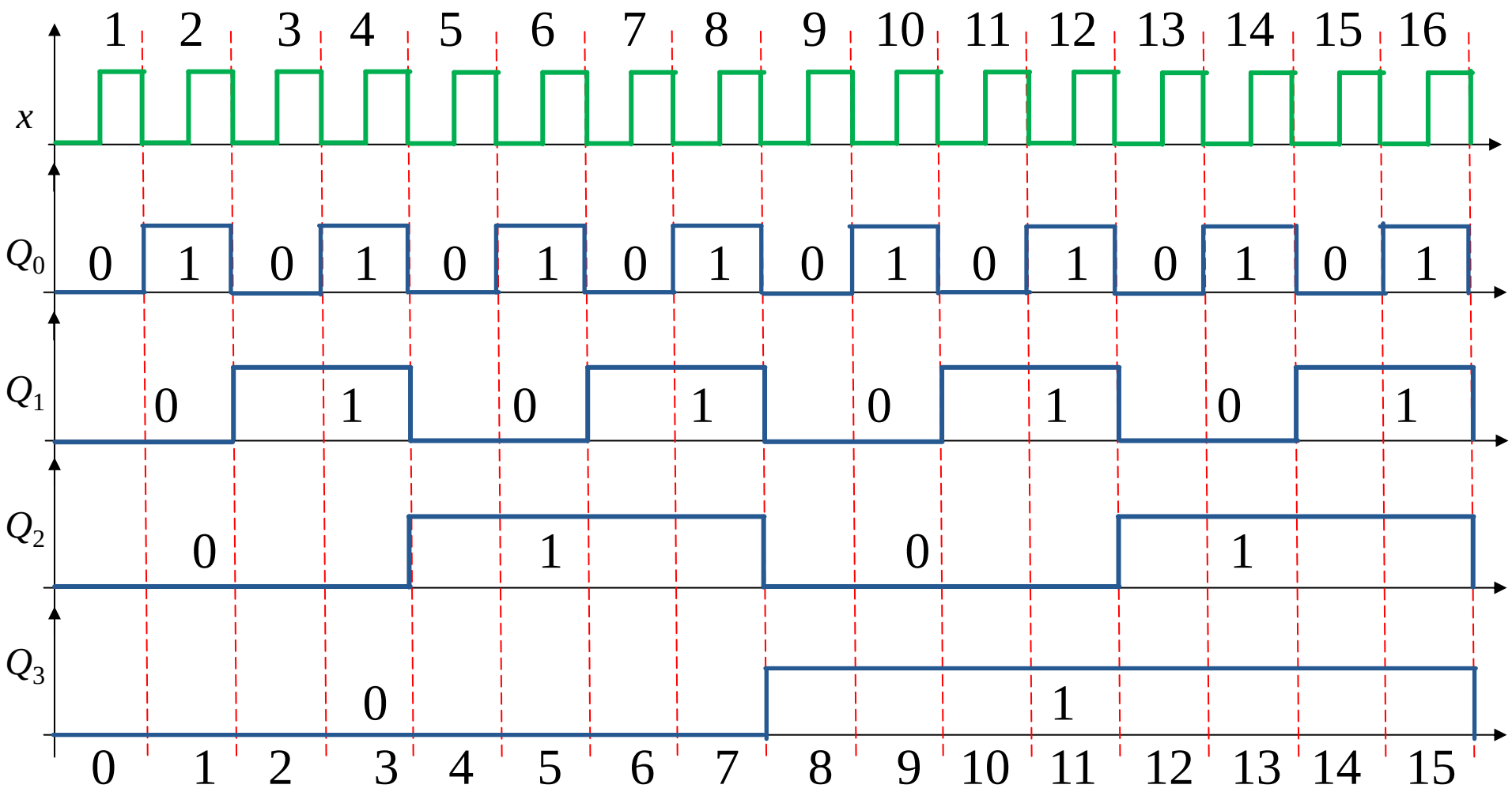
$$Q^{t+1} = \bar{K}^t Q^t + J^t \bar{Q}^t$$

CBB JK –aplicație- Numărător pe 4 biți

- Schemă clasică de numărător
- Numărător ciclic → după ce ajunge la 1111 trece în 0
- Când face reset? Când toate Q-urile sunt 1



Numărător - Diagrama de semnal



Numărător - detalii

- Care este cel mai semnificativ bit și care cel mai puțin semnificativ bit?
- MSB: Q_3, LSB: Q_0
- În funcție de ce punem în poarta de jos se modifică valoarea maximă până la cât numără
- Ex: ca să fac numărător decimal, trebuie să neg intrările de la Q_0 și Q_2
- Putem să îl extindem la mai mulți biți?
- Dezavantaj: fiecare circuit JK introduce întârzieri și dacă semnalul x este prea rapid se pierd din semnale



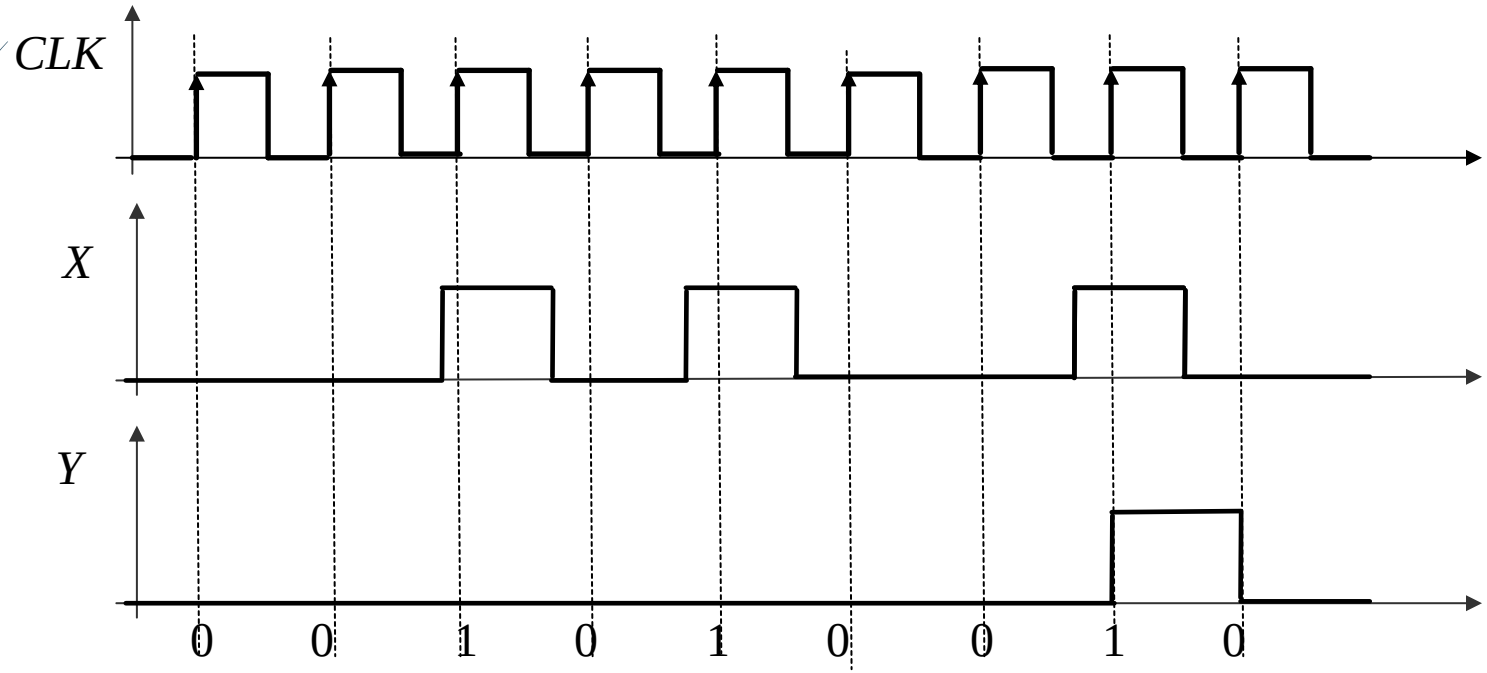
Proiectare Logică

- I CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- **I CC: prof. dr. ing. Anca Morar**
  - anca.morar@cs.pub.ro

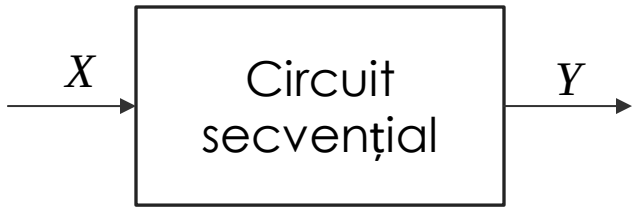
Curs 9: Circuite basculante bistabile. Exemple

Exemplu (1) – enunț

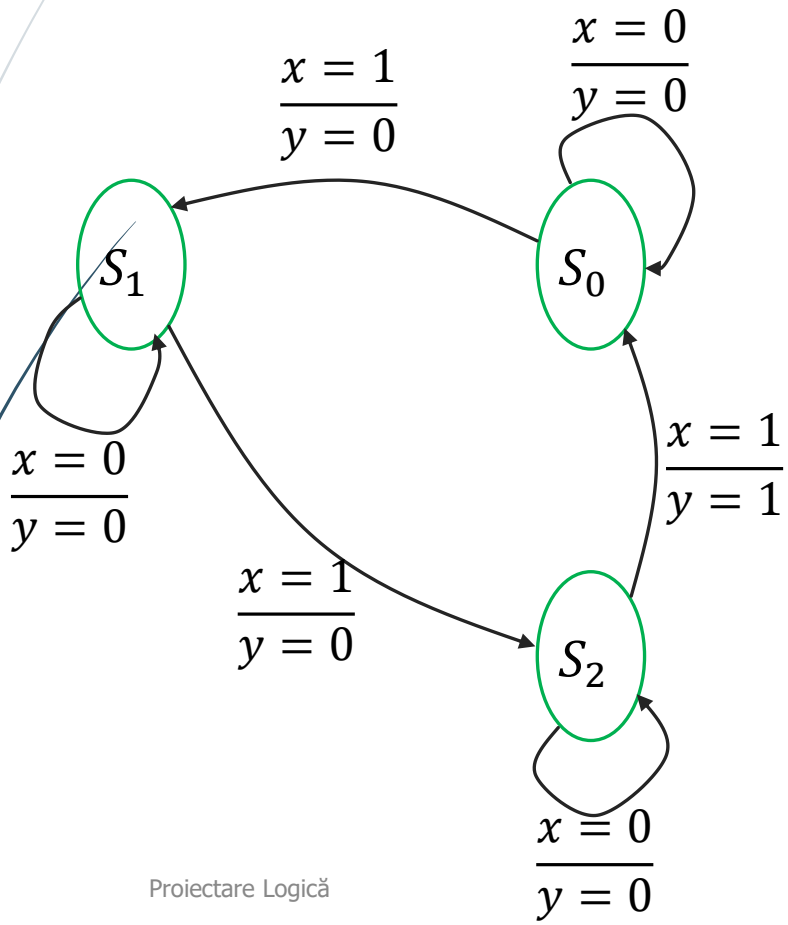
- Să se proiecteze un circuit secvențial *sincron* care să furnizeze un semnal $Y = 1$ la ieșire în momentul când a primit 3 valori de intrare egale cu 1 ale semnalului X. Aparatul va trece în starea inițială la primirea celor 3 valori 1.
- Desenăm diagrama de semnal ca să vedem ce avem de făcut:



Exemplu (2) - graf



- Pentru început, reprezentăm circuitul sub formă de graf



- S_0 – nu am primit niciun „1”
- S_1 – am primit un „1”
- S_2 – am primit doi de „1”
- Diferă de exemplul din laborator:
  - Nu se identifică secvența 111, ci momentul când au venit 3 de 1
  - Secvențele nu se suprapun (vezi enunț - e f. important să îl citiți cu atenție)
- Ce tip de circuit este?

Mealy

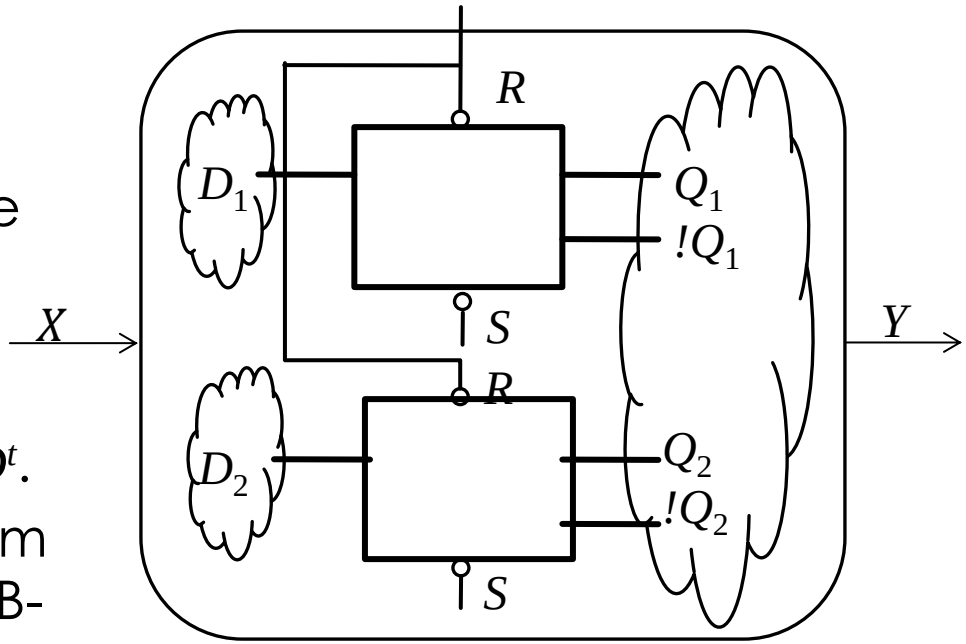
Exemplu (3) - detalii

- Câte stări avem?
- 3
- De câte variabile de stare avem nevoie?
- 2 (Q_2 & Q_1)
- De câte CBB-uri avem nevoie pentru codificare?
- 2 (unul pentru fiecare variabilă)
- Ce știm?
- Starea următoare este exprimată de niște valori ale lui Q_2 și Q_1 și există dependențele:
- ${Q_1}^{t+1} = f({Q_1}^t, X, {Q_2}^t)$ $și$ ${Q_2}^{t+1} = f({Q_2}^t, X, {Q_1}^t)$ și NU
- ${Q_1}^{t+1} = f({Q_1}^t, X)$ $și$ ${Q_2}^{t+1} = f({Q_2}^t, X)$

Exemplu (4) - schemă

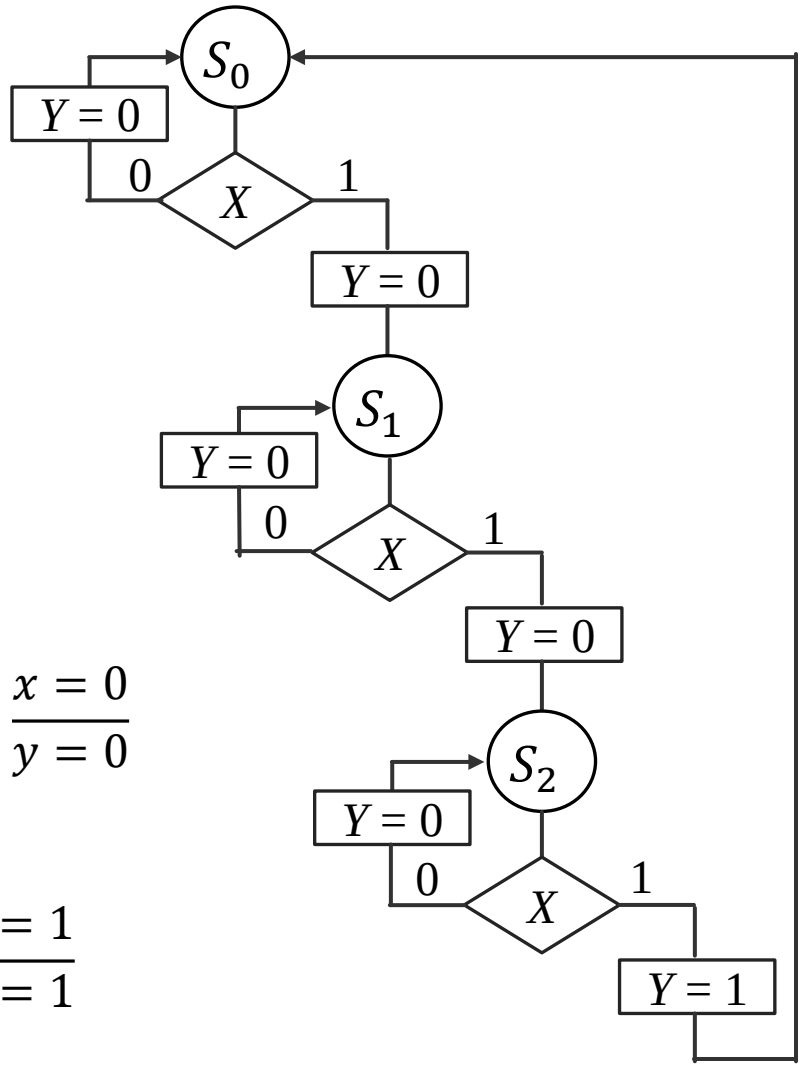
- Ce proiectăm?
- Un circuit secvențial ce are la intrare X, la ieșire Y, trece prin 3 stări ce pot fi generate de 2 variabile de stare Q_1 și Q_2, fiecare reprezentată printr-un CBB tip D.

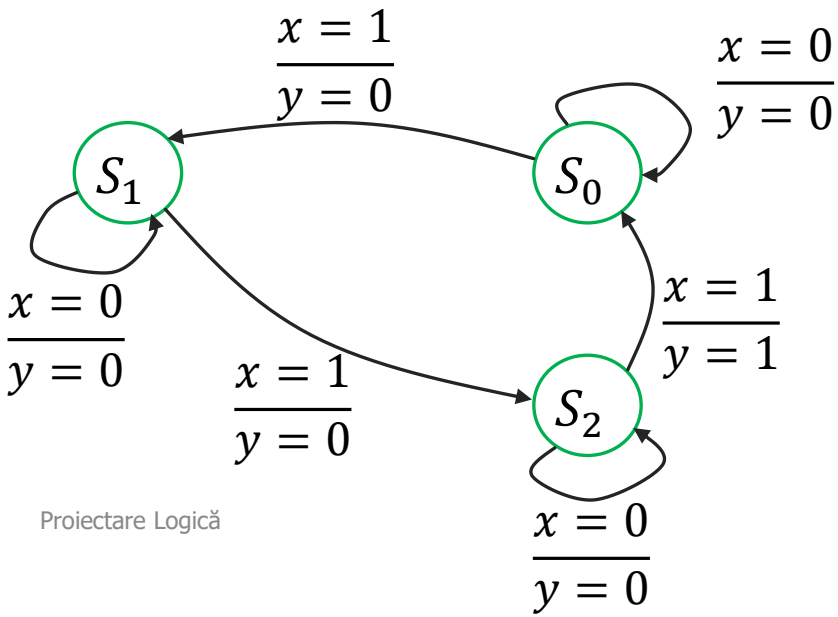
- Regulile circuitelor combinaționale ne spun ce să punem pe intrarea CBB-urilor ca să obținem ieșirea dorită la momentul dorit, folosind $Q^{t+1} = D^t$.
- Trebuie să vedem cum combinăm ieșirile CBB-urilor ca să obținem Y.



Exemplu (5) - Organigrama

- Redesenăm graful folosind simboluri ca la Mealy & Moore
- Se putea sa trec ieșirea doar atunci când $Y = 1$, ca să nu supraîncarc desenul
- Există situații când trebuie pusă ieșirea tot timpul, dar de obicei nu e nevoie





Exemplu (6) – codificarea stărilor

- Codificăm stările: $S_0 - 00, S_1 - 01, S_2 - 10$
- De ce am codificat așa?
- $S_0 - 00$ pentru că am considerat-o stare inițială și aceasta se codifică cu 0 pentru toate variabilele de stare.
- De ce?
- Pentru că știm că CBB-urile au și intrările asincrone de S/R. R-urile de obicei se leagă împreună. Dacă circuitul merge prost, atunci e resetat, ceea ce înseamnă că trec toate variabilele de stare pe 0 (e trecut în stare inițială) și de aceea starea inițială e codificată cu 0 pentru toate variabilele de stare.
- În continuare valorile sunt puse în ordine.

Exemplu (7) – tabela de adevăr

- Construim tabela de adevăr:

	Q_2^t	Q_1^t	X^t	Q_2^{t+1}	Q_1^{t+1}	Y^{t+1}
S_0	0	0	0	0	0	0
	0	0	1	0	1	0
S_1	0	1	0	0	1	0
	0	1	1	1	0	0
S_2	1	0	0	1	0	0
	1	0	1	0	0	1
*	1	1	0	*	*	*
	1	1	1	*	*	*

Stare redundantă – nu avem nevoie de ea

Exemplu (8) – valorile pentru Q și Y

- Având tabelul de adevăr, ce putem face?
- Diagramele Karnaugh
- Ce trebuie avut în vedere?
- Trebuie să determinăm Q_2^{t+1}, Q_1^{t+1} și Y pe baza Q_2^t, Q_1^t și X
- Doar pe baza grafului, știm să calculăm Q și Y?
- La Mealy am făcut funcții de tranziție și ieșire în forma algebrică → putem face același lucru și aici → e la fel
- Nu facem așa pentru că e mai simplu cu diagrame Karnaugh

Exemplu (9) - Diagramele Karnaugh

- Facem câte o diagramă Karnaugh pentru fiecare (Q_2, Q_1 și Y).
- Q_2 și Q_1 ne dau funcțiile combinaționale ce se pun la intrările lui D_1 și D_2.
- Y ne dă funcția combinațională ce ne spune cum combinăm ieșirile lui D_1 și D_2 pentru a obține ieșirea Y.

Q_2^t	Q_1^t	X^t	Q_2^{t+1}	Q_1^{t+1}	Y^{t+1}
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	0	0	1
1	1	0	*	*	*
1	1	1	*	*	*

Q_2^{t+1}

X \ Q_2Q_1	00	01	11	10
0	0	0	*	1
1	0	1	*	0

Q_1^{t+1}

X \ Q_2Q_1	00	01	11	10
0	0	1	*	0
1	1	0	*	0

Y^{t+1}

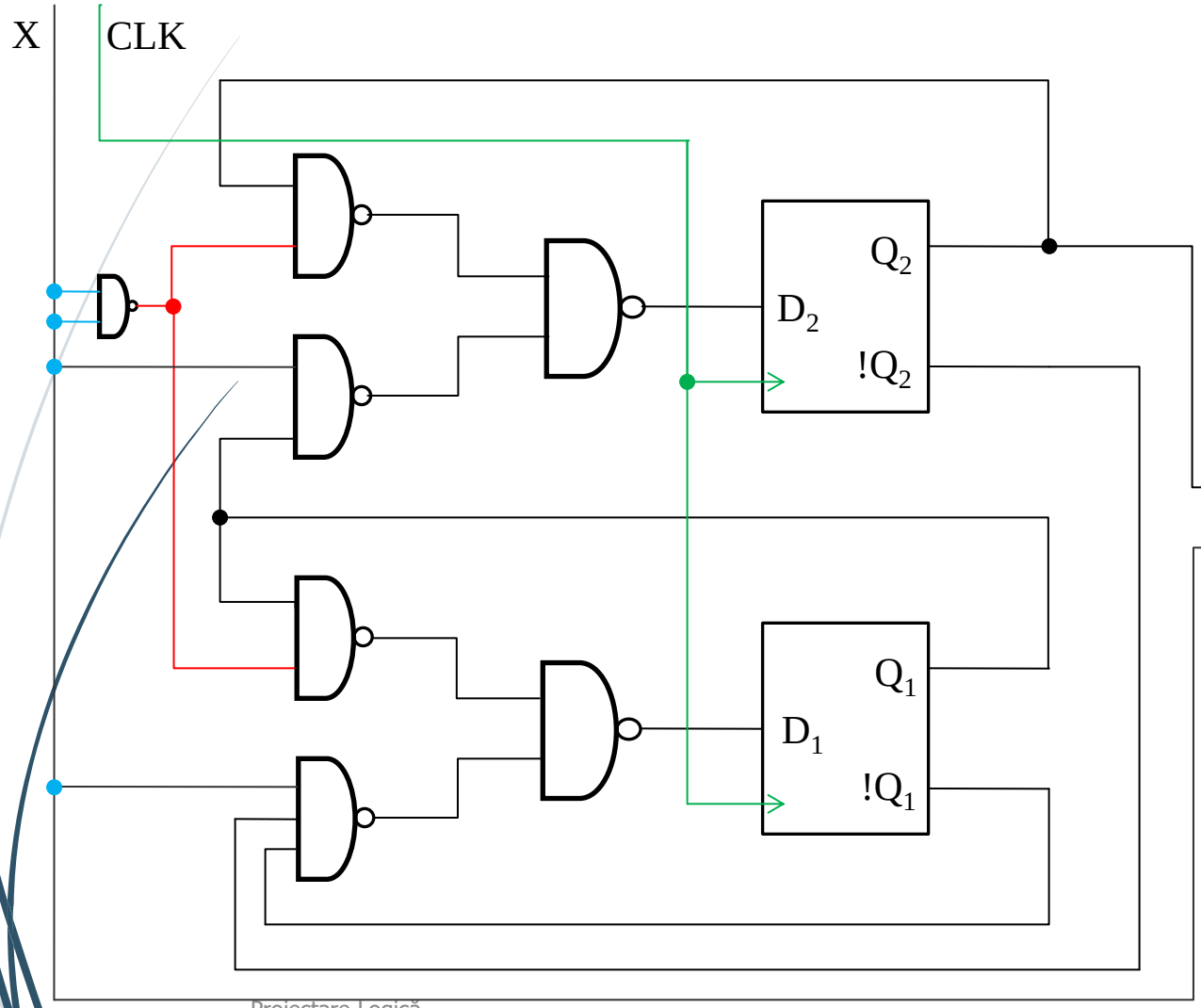
X \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	*	1

$$Q_2^{t+1} = Q_1X + Q_2!X = D_2^t$$

$$Q_1^{t+1} = Q_1!X + !Q_2!Q_1X = D_1^t$$

$$Y^{t+1} = Q_2X$$

Exemplu (10) - Implementare



- Ce fac cu valorile obținute?
- E o problemă de identificare

- Convenție:

Semnale diferite

Același semnal

Folosirea organigraamelor în sinteza circuitelor secvențiale

1. Descrierea în cuvinte și sub formă de schemă bloc a condițiilor de funcționare
2. Elaborarea organigramei
3. Codificarea stărilor
4. Construcția tabelului de adevăr
5. Construirea diagramelor Karnaugh pentru variabilele de stare
6. Obținerea ecuațiilor de stare
7. Obținerea ecuațiilor de utilizare a CBB-urilor folosite în implementare
8. Determinarea expresiilor logice ale variabilelor de ieșire
9. Implementarea cu circuite integrate (desenare schemă)
10. Analiza finală a modului de realizare a tranzițiilor și de apariție a semnalelor de ieșire (curse critice și hazard computațional)

Descrierea în cuvinte

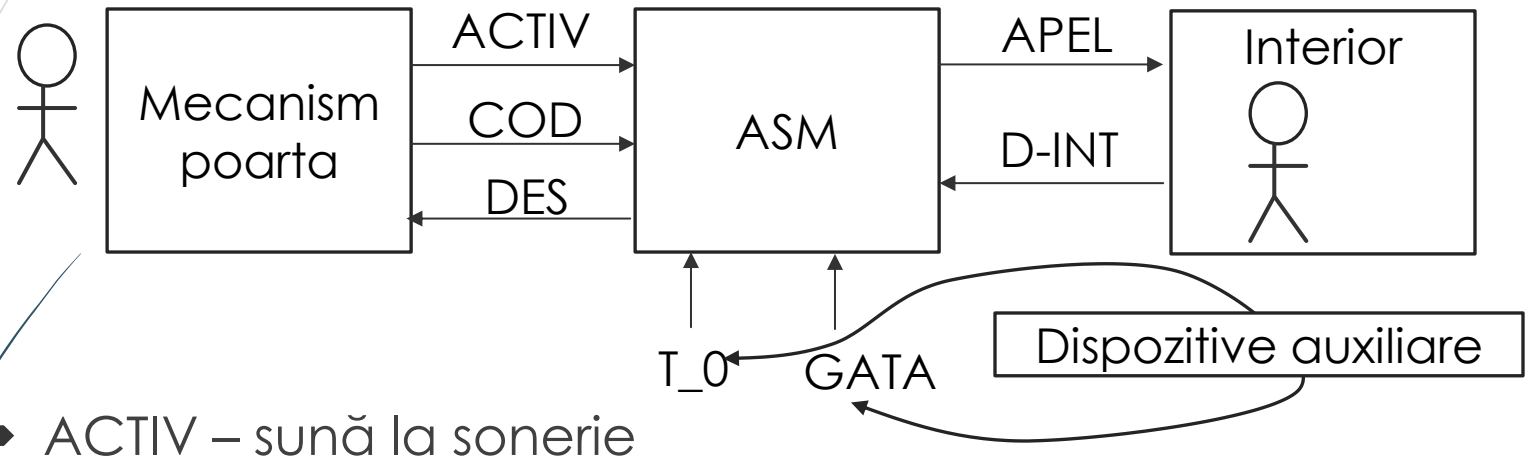
- Să se implementeze un automat care activează interfonul unei locuințe uni-familiale. Ușa se poate deschide prin formarea unui cod corect sau prin acționarea din interior. Dispozitive auxiliare asigură semnalul de *Time_out (T_O)*, respectiv pe cel de *Gata* (reînchiderea interfonului după scurgerea unui timp în care nu a venit niciun răspuns din interior, respectiv reînchiderea zăvorului după scurgerea unui timp de la deschiderea ușii).
- **Observație**: descrierea se face în limbaj uzual, fără informații referitoare la semnale, CBB etc.

Schema bloc

- Schema bloc = instrument ingineresc deosebit de util ce vă permite înțelegerea funcționării de ansamblu a schemei/ circuitului/ sistemului pe care îl aveți de realizat și care interacționează cu mediul înconjurător.
- Schema bloc reflectă într-o formă neformalizată descrierea sistemului, dar e posibil să nu surprindă toate aspectele (diverse detalii)

Schema bloc

Interacționează direct omul cu ASM-ul?
Nu! → am un mecanism la poartă pe care îl acționează omul



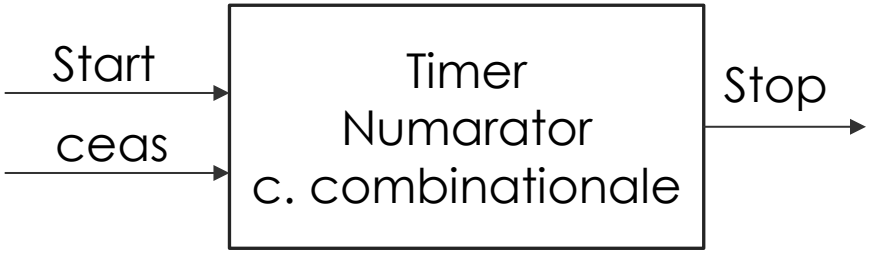
- ACTIV – sună la sonerie
- APEL – sunetul se transmite de ASM către soneria din casă
- COD – bagă direct codul (nu sună)
- DES – ASM transmite semnalul de deschidere către poartă
- D_INT – omul din casă transmite către ASM semnalul de deschidere poartă

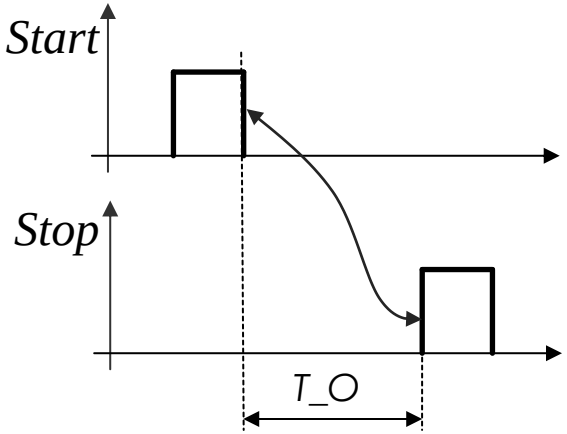
Schema bloc

- Cu ce lucrează ASM? – semnale digitale
- Cu ce lucrează mecanism poartă? – semnale analogice (electromagnet)
- Am încercat să menținem cât mai simplu exemplul ca să nu complicăm schema
- Ex: COD – parte a circuitului și vine ca semnal digital care intră în ASM, dar el este un modul auxiliar pe care nu îl înglobăm în ASM → ne bazăm pe reducerea complexității prin realizarea de mai multe componente ce comunică între ele → crește fiabilitatea și e mai ușor de implementat.

Schema bloc

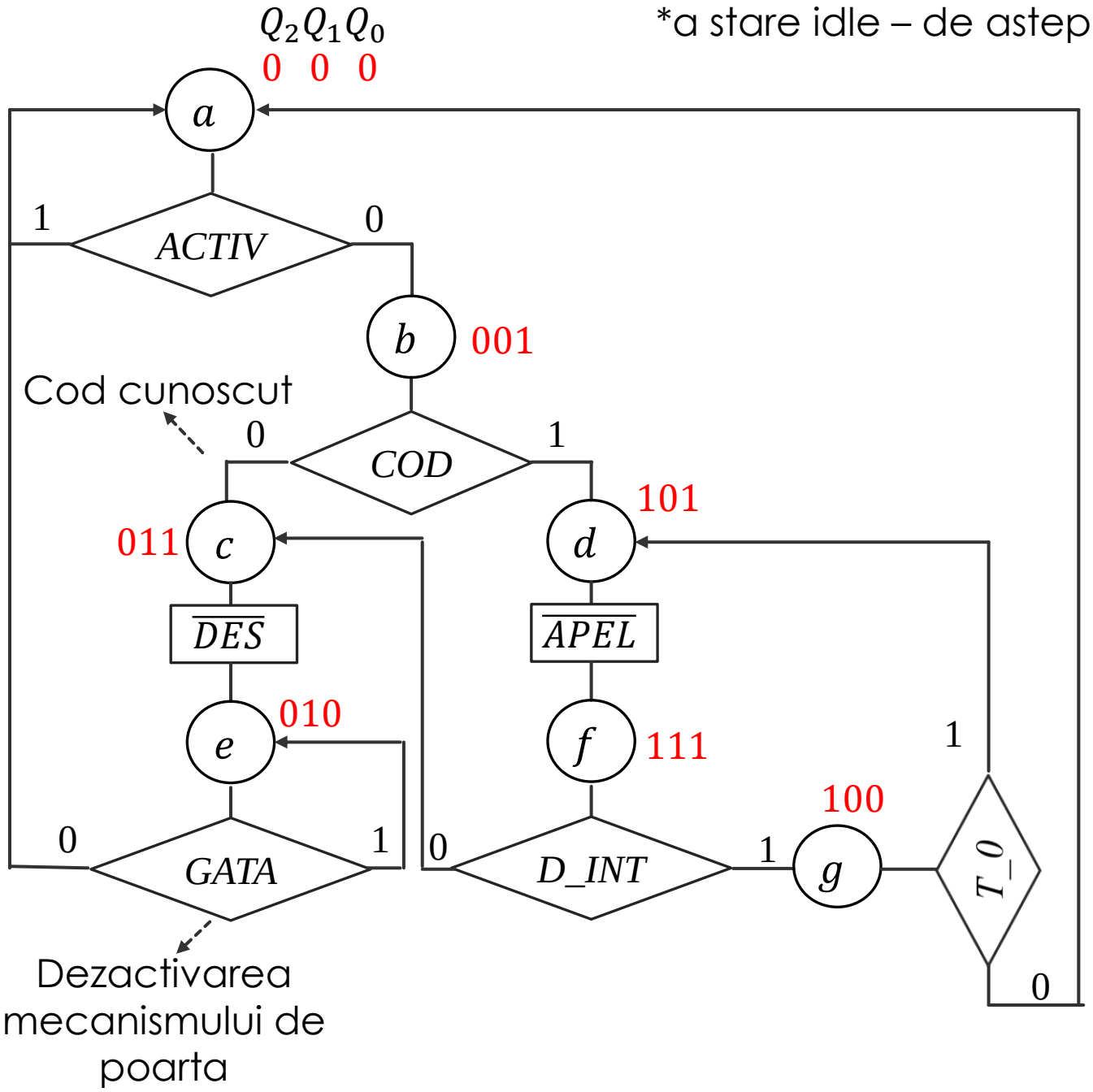
- Soluția aleasă: *decizia dacă codul e corect* o ia un circuit extern care face parte din mecanismul porții (poarta lucrează la 12/24 V → semnalul transmis e pe 5 V)
- *T_O & Gata* sunt niște semnale generate de timere, dar cum încă nu am făcut numărătoare, le băgăm de asemenea în circuite auxiliare.
- Schema bloc ne spune ce intră și ce iese din ASM și noi trebuie să identificăm algoritmul de funcționare după care acesta interacționează cu omul din casă și mecanismul de poartă





Organigrama

- Prin câte stări trece aparatul?
- Nu pot să spun dinainte – ex. Poarta are 2 stări (închis/deschis), dar nu și aparatul → facem organigrama ca să aflăm numărul de stări
- Convenții – semnalele sunt active pe 0
  - ACTIV – activ pe 0 (dacă e = 1 înseamnă că nu a venit nimeni la poartă)
  - COD = 0 (cod corect) și COD = 1 (cod incorect) → transmite semnalul către soneria din casă
  - D_INT = 0 (omul a dat drumul) D_INT = 1 (omul nu a dat drumul)
- De ce așa și nu 0 = Fals, 1 = Adevărat?
  - Există aparate care funcționează și într-un fel și în altul
  - Trebuie să vedeți și un tip, și pe celălalt
- Observație: în acest curs nu discutăm despre tipul circuitelor (am exemplificat pe TTL, dar e la fel pe MOS / CMOS/ etc.).


$Q_2Q_1Q_0$
0 0 0
*a stare idle – de asteptare
a
1
ACTIV
0
b
001
Cod cunoscut
0
COD
1
011
c
101
d
DES
APEL
010
e
f
111
100
1
0
GATA
1
0
D_INT
1
g
T_0
0
Dezactivarea mecanismului de poarta

Organigrama

- ***Observație***: Intrările în automat sunt condițiile din organigramă. Nu pot să consider decizii alte semnale.
- Acum putem spune câte stări are ASM-ul?
- 7
- Câte variabile de stare sunt necesare?
- 3 (maxim 8 stări, dacă aveam doar 2 – maxim 4 stări și erau insuficiente)
- Câte CBB-uri?
- 3
- PROIECT: Dacă funcționalitatea e prea complexă (organigramă prea stufoasă) ce putem face?
- Externalizăm din funcționalități în dispozitivele auxiliare.
- Unde ați mai întâlnit așa ceva?
  - Programare → subprograme

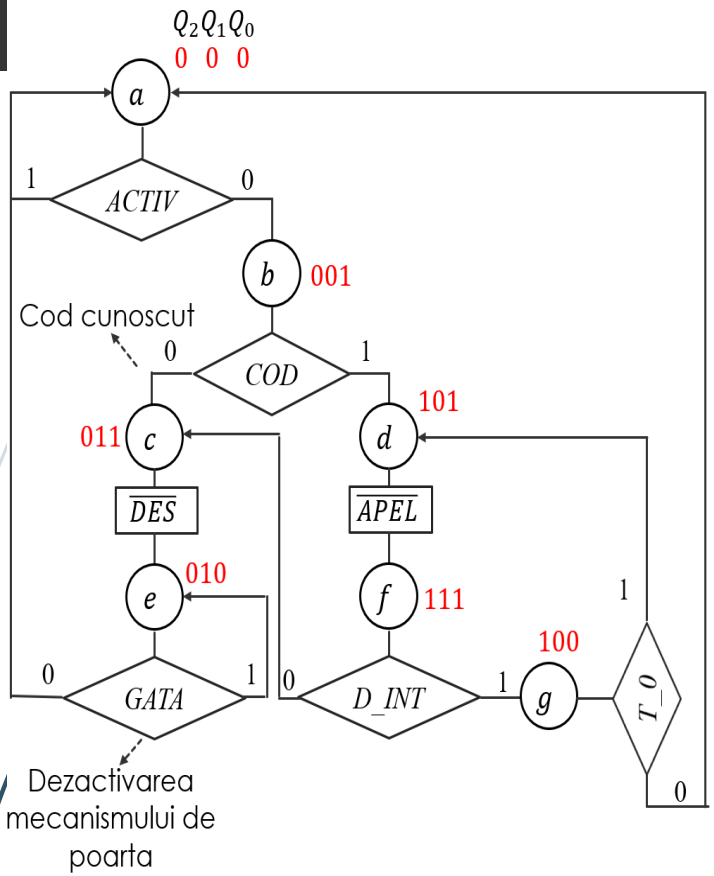
Asignarea stărilor

- 3 variabile de stare: $Q_0, Q_1 \text{ și } Q_2$ → începem cu 000 și încercăm să plasăm stările într-o diagramă Karnaugh
- La exemplul precedent am făcut $Q_2, Q_1 \text{ și } Y$ dependente de $Q_2, Q_1 \text{ și } X$ (am făcut dependența explicită de X pentru a înțelege mai ușor).
- De acum încolo, facem dependența doar de variabilele de stare ca să menținem diagramele cât mai mici.
- Puteam face așa mai devreme?
- Da, prin considerarea lui X ca variabilă externă și integrarea acesteia în diagramă (am mai făcut)

Asignarea stărilor

- Primul pas în asignarea stărilor e să plasăm stările în diagramă (care se numește spațiul stărilor)
- Regulă: stările următoare (în măsura posibilităților) trebuie să fie adiacente stării curente (trebuie să difere printr-un singur bit)
- Facem diagrama → pentru unele combinații se respectă, pentru altele nu → pentru problema de față nu se poate respecta condiția adiacenței pentru toate stările

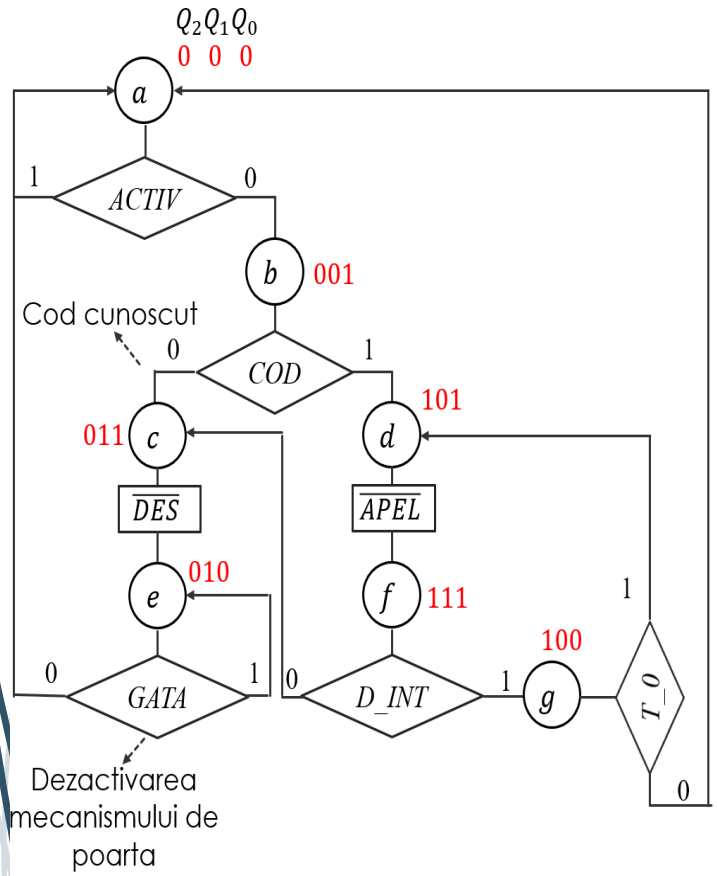
Asignarea stărilor



Q_2Q_1 / Q_0	00	01	11	10
0	a	e	*	g
1	b	c	f	d

- Din a ma duc in a sau in b
- Din b ma duc in c sau in d
- Din c ma duc in e
- Din e ma duc in e sau in a
- Din d ma duc in f
- Din f ma duc in g sau in c
- Din g ma duc in d sau in a (am pierdut adiacenta cu f; dacă pun g alături, atunci nu mai e adiacent cu d și a
- Proiect: trebuie să faceți asignarea singuri
- Examen: vi se dă asignarea și trebuie să o respectați
- Atenție ordine $Q_2Q_1Q_0 \rightarrow Q_0Q_1Q_2$

Tabela de adevăr



Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}
0	0	0	0	0	!ACTIV
0	0	1	COD	!COD	1
0	1	0	0	GATA	0
0	1	1	0	1	0
1	0	0	T_O	0	T_O
1	0	1	1	1	1
1	1	0	*	*	*
1	1	1	D_INT	!D_INT	!D_INT

- Cum completăm?
- Ne uităm la organigramă:
  - Din a mă duc fie în a, fie în b, în funcție de ACTIV
  - $Q_0 = 0$ când ACTIV = 1 și $Q_0 = 1$ când ACTIV = 0 → Q_0 = !ACTIV

Diagramele Karnaugh pentru ecuațiile de stare următoare

- Există diferențe între tranzițiile a → b, b → d și d → f?
- La primele 2 sunt salturi condiționate, pe când la ultima e un salt necondiționat
- La fiecare salt condiționat, decizia va apare în diagramă (și în tabelul de adevăr) → astfel se obțin diagramele de stare ce depind de variabilele stare și de cele externe
- Ce facem mai departe cu formulele obținute?
- Le punem la intrările CBB-urilor

Diagramele Karnaugh pentru ecuațiile de stare următoare

- Cum le completez?
- Fie pe baza tabelului de adevăr, fie mă uit la organigramă, la fel cum am construit tabelul de adevăr

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

$Q_2 = Q_2!Q_1Q_0 + COD*!Q_1Q_0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	Gata	*	0
1	!COD	1	!D_INT	1

$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q_0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1

$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$

Ecuațiile de utilizare a CBB-urilor

- Următorul pas după construirea diagramelor Karnaugh este să obținem ecuațiile de utilizare a CBB-urilor
- Folosind CBB tip D $Q^{t+1} = D^t$ →
- $D_2 = Q_2!Q_1Q_0 + COD*!Q_1Q0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$
- $D_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$
- $D_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$

Determinarea expresiilor logice ale variabilelor de ieșire

- Câte ieșiri avem?
- 2: APEL & DES (active pe 0) – trebuie negate după minimizare
- Pentru fiecare ieșire facem o diagramă Karnaugh și astfel îi determinăm ecuația

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	1	0	0

$!DES = !Q_2Q_1Q_0$

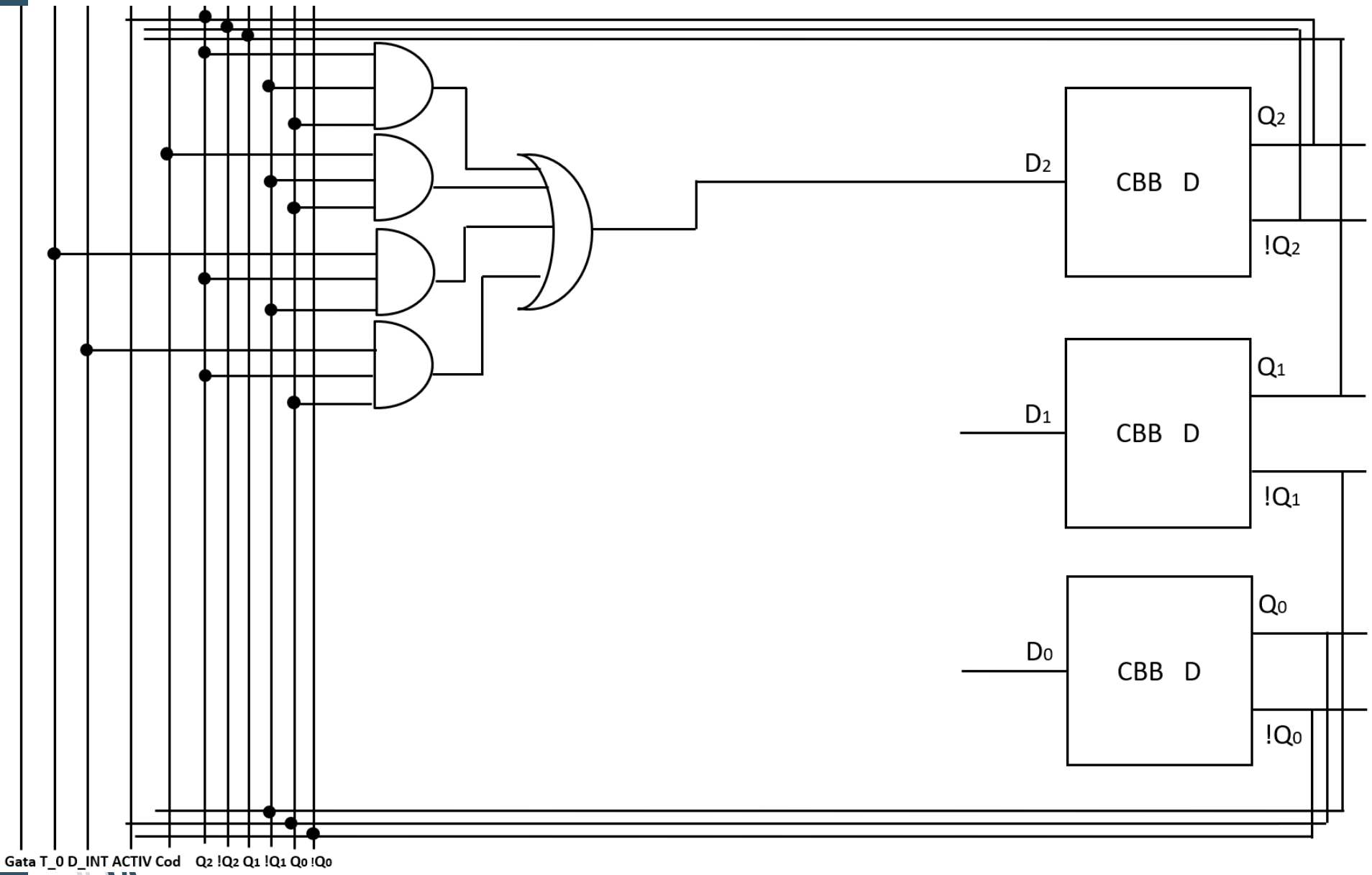
$DES = !(!Q_2Q_1Q_0)$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	0	1

$!APEL = Q_2!Q_1Q_0$

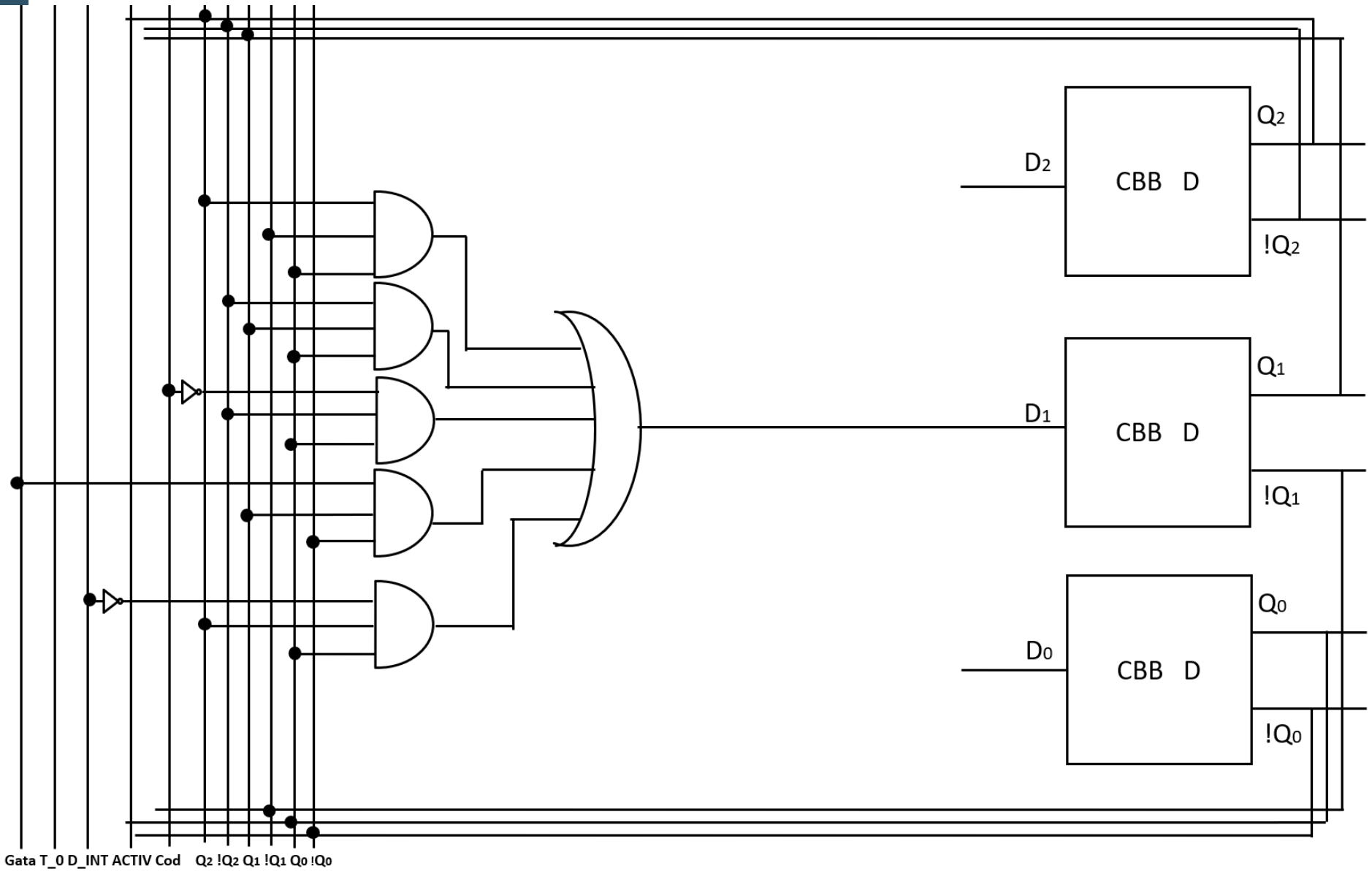
$APEL = !(Q_2!Q_1Q_0)$

- Ce mai putem face?
- Putem rezolva pasul 9 (implementarea cu circuite integrate) având toate informațiile disponibile

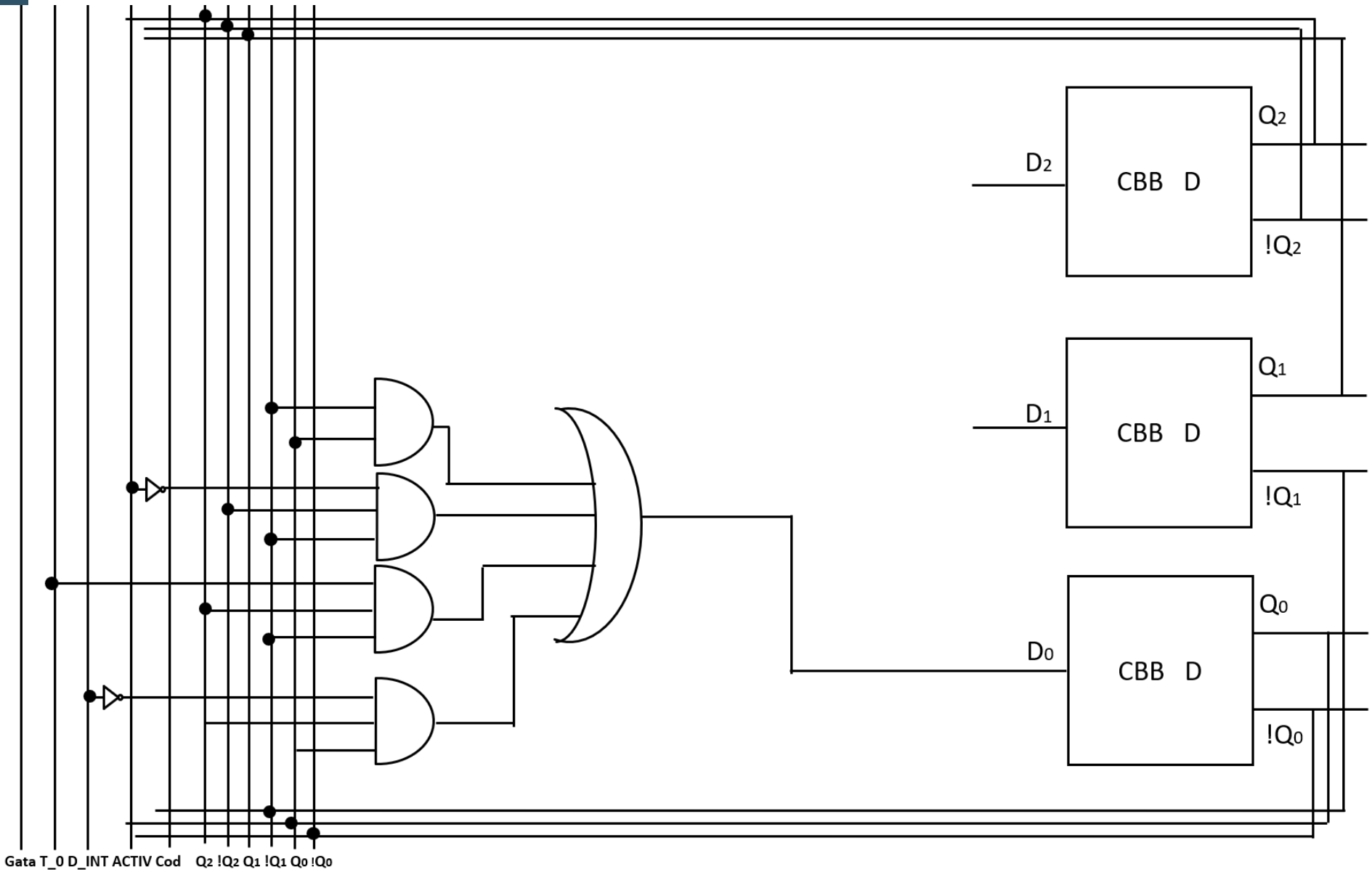


Gata T_0 D_INT ACTIV Cod Q2 !Q2 Q1 !Q1 Q0 !Q0

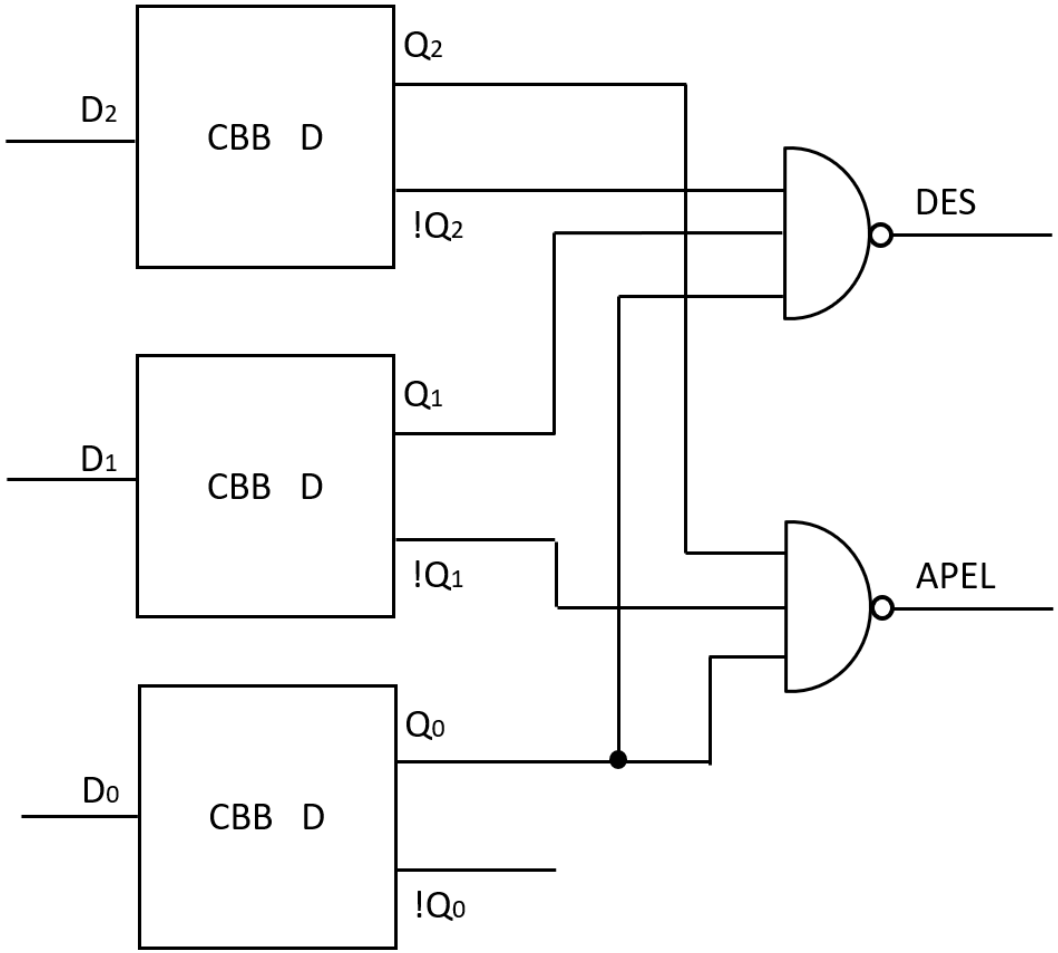
$$D_2 = Q_2!Q_1Q_0 + COD*!Q_1Q0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$$



$$D_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$$



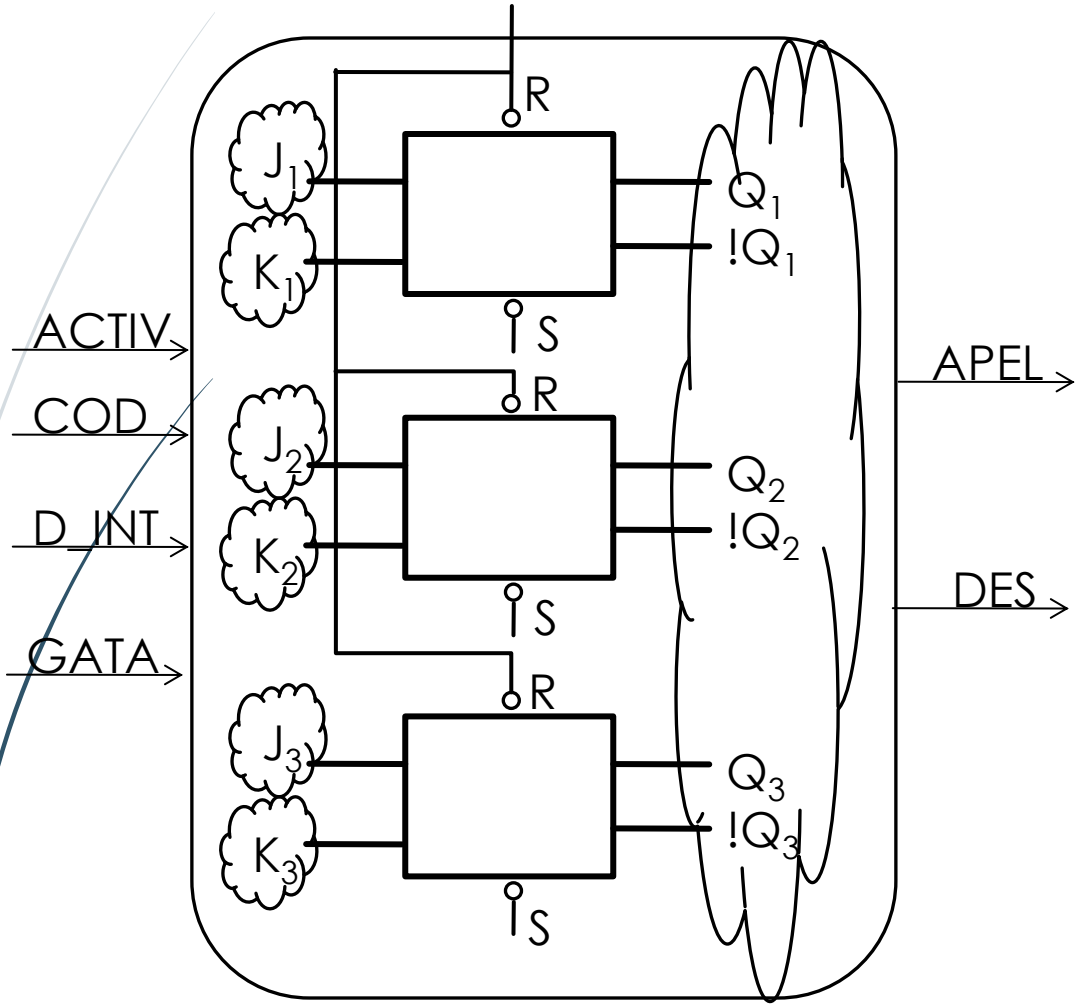
$$D_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$$



$$DES = !(!Q_2 Q_1 Q_0)$$

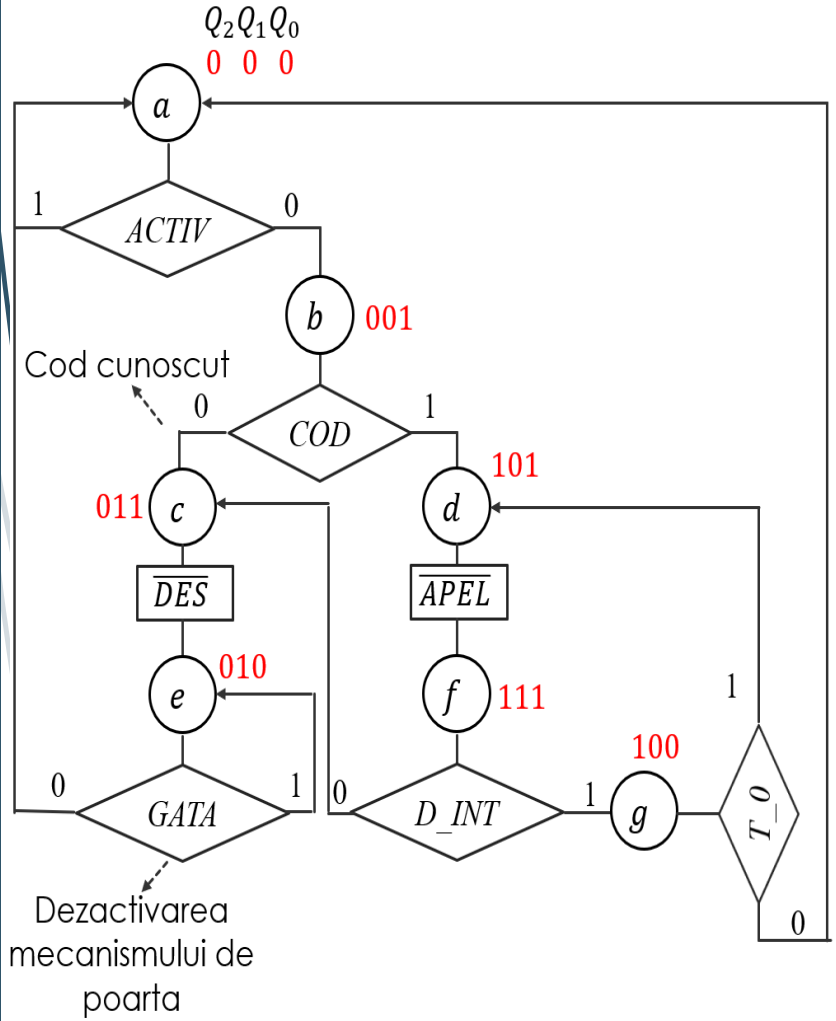
$$APEL = !(Q_2 !Q_1 Q_0)$$

Implementare Interfon CBB JK



- Similar cu CBB de tip D, trebuie să detectăm ce punem pe intrări (J,K) și cum combinăm ieșirile (Q,!Q)
- La CBB D era foarte simplu de determinat intrările ($Q^{t+1} = D^t$)
- La CBB JK trebuie să împărțim Q^{t+1} în intrarea lui J și a lui K

Interfon – Organigrama și tabel de adevăr



Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}
0	0	0	0	0	!ACTIV
0	0	1	COD	!COD	1
0	1	0	0	GATA	0
0	1	1	0	1	0
1	0	0	T_O	0	T_O
1	0	1	1	1	1
1	1	0	*	*	*
1	1	1	D_INT	! D_INT	! D_INT

Ecuația de stare pentru Q2

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

- Cum împărțim?

$$Q_2 = Q_2!Q_1Q_0 + T_O*Q_2!Q_1 + COD*!Q_1Q0 + D_INT*Q_2Q_0$$

- Știm că $Q^{t+1} = J!Q^t + !KQ^t$
- Ce se întâmplă cu Q^{t+1} dacă $Q^t = 0$? $Q^{t+1} = J$
- Ce se întâmplă cu Q^{t+1} dacă $Q^t = 1$? $Q^{t+1} = !K$
- Citim invers: $J = Q^{t+1} \mid_{Q^t = 0}$ și $!K = Q^{t+1} \mid_{Q^t = 1}$
- J = Q^{t+1} dar numai partea corespunzătoare lui $Q^t = 0$
  → unde e $Q^t = 0$? Pentru Q_2 = stânga

Determinare J și K

- Arătăm că ecuația de stare exprimă o combinare a 2 componente (când $Q^t = 0$ și $Q^t = 1$).
- Cele 2 situații nu se suprapun și acoperă întreaga diagramă → putem defini pe J și K fără ca cele 2 componente să se influențeze una pe alta →
- Am mai întâlnit asta? Da! Superpoziția de la diagrame Karnaugh cu variabile externe
- ca să știm ce punem pe intrările CBB JK lucrăm cu diagrama de stare următoare și o spargem în 2: zona pentru J și zona pentru !K

Determinare J_2 și K_2

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

Q_0 Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

Q_0 Q_2Q_1	00	01	11	10
0	0	0	*	*
1	COD	0	*	*

$J_2 = COD!Q_1Q_0$

Q_0 Q_2Q_1	00	01	11	10
0	*	*	*	T_O
1	*	*	D_INT	1

$!K_2$

Q_0 Q_2Q_1	00	01	11	10
0	*	*	*	!T_O
1	*	*	!D_INT	0

$K_2 = !T_O\ Q_0 + !D_INT\ Q_1$

- Pornim de la diagrama de stare
- Care e zona pentru J_2?
- Primele 2 coloane → restul nu ne interesează → punem *
- La fel pentru $!K_2$
- Putem face direct diagrama lui K prin negarea fiecărei căsuțe în parte → de obicei așa se face ca să nu greșiți!!!

Determinare J_1 și K_1

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

- Cum se face împărțirea pentru J_1 și K_1?
- E la fel ca pentru Q_2?

Q_0 Q_2Q_1	00	01	11	10
0	0	Gata	*	0
1	!COD	1	!D_INT	1

$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$

Q_0 Q_2Q_1	00	01	11	10
0	0	*	*	0
1	!COD	*	*	1

$J_1 = Q_2Q_0 + !COD*Q0$

Q_0 Q_2Q_1	00	01	11	10
0	*	!Gata	*	*
1	*	0	D_INT	*

$K_1 = !Gata*!Q_0 + !D_INT*Q_2$

Determinare J_0 și K_0

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

- Cum se face împărțirea pentru J_0 și K_0?

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1

$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	*	*	*	*

$J_0 = !ACTIV*!Q_2!Q_1 + T_O*Q_2$

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	*
1	0	1	D_INT	0

$K_0 = !Q_2Q_1 + D_INT*Q_1$

Implementare cu CBB JK

- OBSERVAȚIE: prin spargerea lui Q^{t+1} în zonele corespunzătoare lui J și K avem niște formule care ne duc la o complexitate mai mică → circuitele pe intrările lui J și K sunt mai simple decât ce puneam pe intrarea lui D, dar trebuie să fim atenți cum facem împărțirea (și să nu uităm să negăm)

Implementare cu CBB JK

- Ieșirile circuitului rămân la fel ca în cazul CBB D (ca să fie active pe 0, trebuie negate după minimizare):

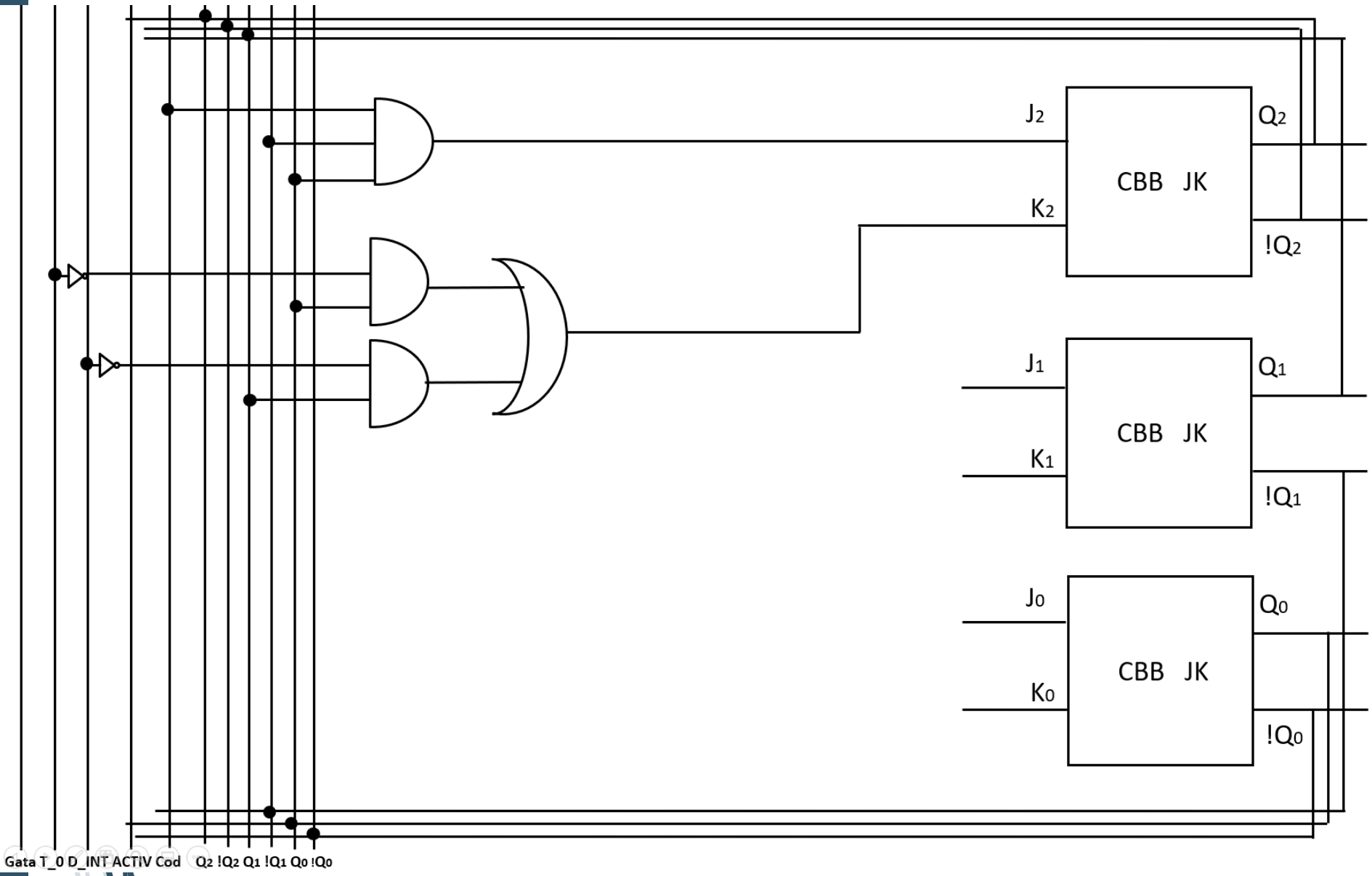
Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	1	0	0

$!DES = !Q_2Q_1Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	0	1

$!APEL = Q_2!Q_1Q_0$

- Acum putem trece la implementarea circuitului cu CBB de tip JK



$J_2 = COD!Q_1Q_0$

$K_2 = !T_O\ Q_0 + !D_INT\ Q_1$



Proiectare Logică

- I CB & CB: prof. dr. ing. Mariana Mocanu
  - mariana.mocanu@cs.pub.ro
- I CA & CD: conf. dr. ing. Costin Chiru
  - costin.chiru@cs.pub.ro
- **I CB: prof. dr. ing. Anca Morar**
  - anca.morar@cs.pub.ro

Curs 10: Circuite basculante bistabile. Exemple (2)

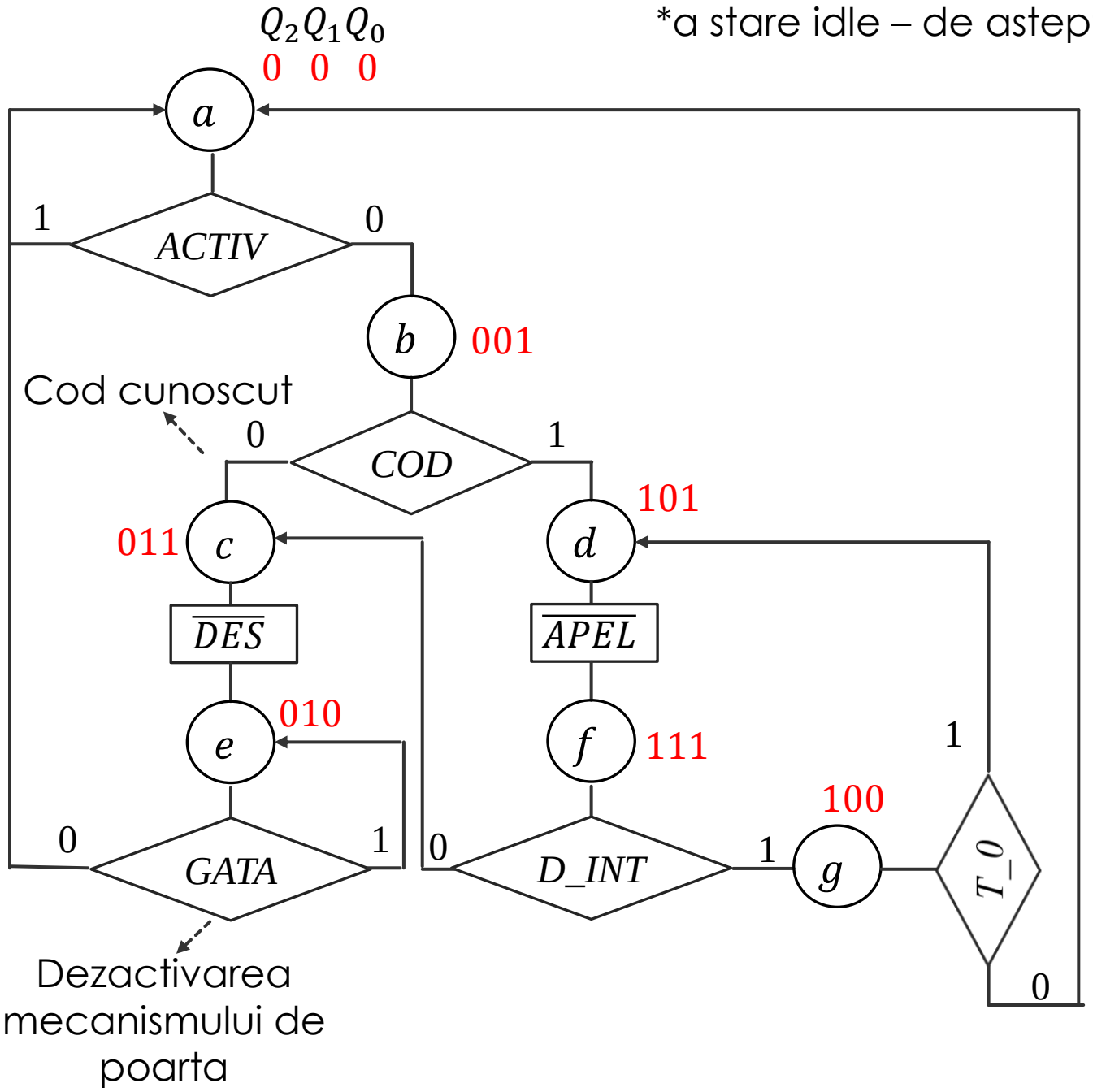
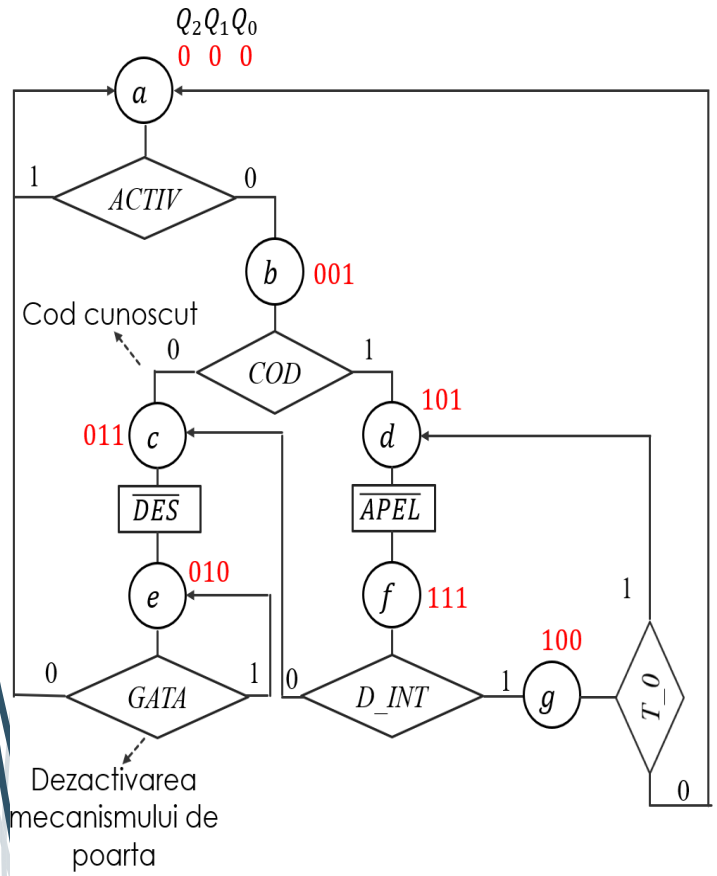

$Q_2Q_1Q_0$
0 0 0
*a stare idle – de asteptare
a
1
ACTIV
0
b
001
Cod cunoscut
0
COD
1
011
c
101
d
DES
APEL
010
e
f
111
100
1
0
GATA
1
0
D_INT
1
g
T_0
0
Dezactivarea mecanismului de poarta

Tabela de adevăr



Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}
0	0	0	0	0	!ACTIV
0	0	1	COD	!COD	1
0	1	0	0	GATA	0
0	1	1	0	1	0
1	0	0	T_O	0	T_O
1	0	1	1	1	1
1	1	0	*	*	*
1	1	1	D_INT	!D_INT	!D_INT

- Cum completăm?
- Ne uităm la organigramă:
  - Din a mă duc fie în a, fie în b, în funcție de ACTIV
  - $Q_0 = 0$ când ACTIV = 1 și $Q_0 = 1$ când ACTIV = 0 → Q_0 = !ACTIV

Diagramele Karnaugh pentru ecuațiile de stare următoare

- Cum le completez?
- Fie pe baza tabelului de adevăr, fie mă uit la organigramă, la fel cum am construit tabelul de adevăr

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

$Q_2 = Q_2!Q_1Q_0 + COD*!Q_1Q_0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	Gata	*	0
1	!COD	1	!D_INT	1

$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q_0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1

$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$

Ecuațiile de utilizare a CBB-urilor

- Următorul pas după construirea diagramelor Karnaugh este să obținem ecuațiile de utilizare a CBB-urilor
- Folosind CBB tip D $Q^{t+1} = D^t$ →
- $D_2 = Q_2!Q_1Q_0 + COD*!Q_1Q0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$
- $D_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$
- $D_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$

Determinarea expresiilor logice ale variabilelor de ieșire

- Câte ieșiri avem?
- 2: APEL & DES (active pe 0) – trebuie negate după minimizare
- Pentru fiecare ieșire facem o diagramă Karnaugh și astfel îi determinăm ecuația

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	1	0	0

$!DES = !Q_2Q_1Q_0$

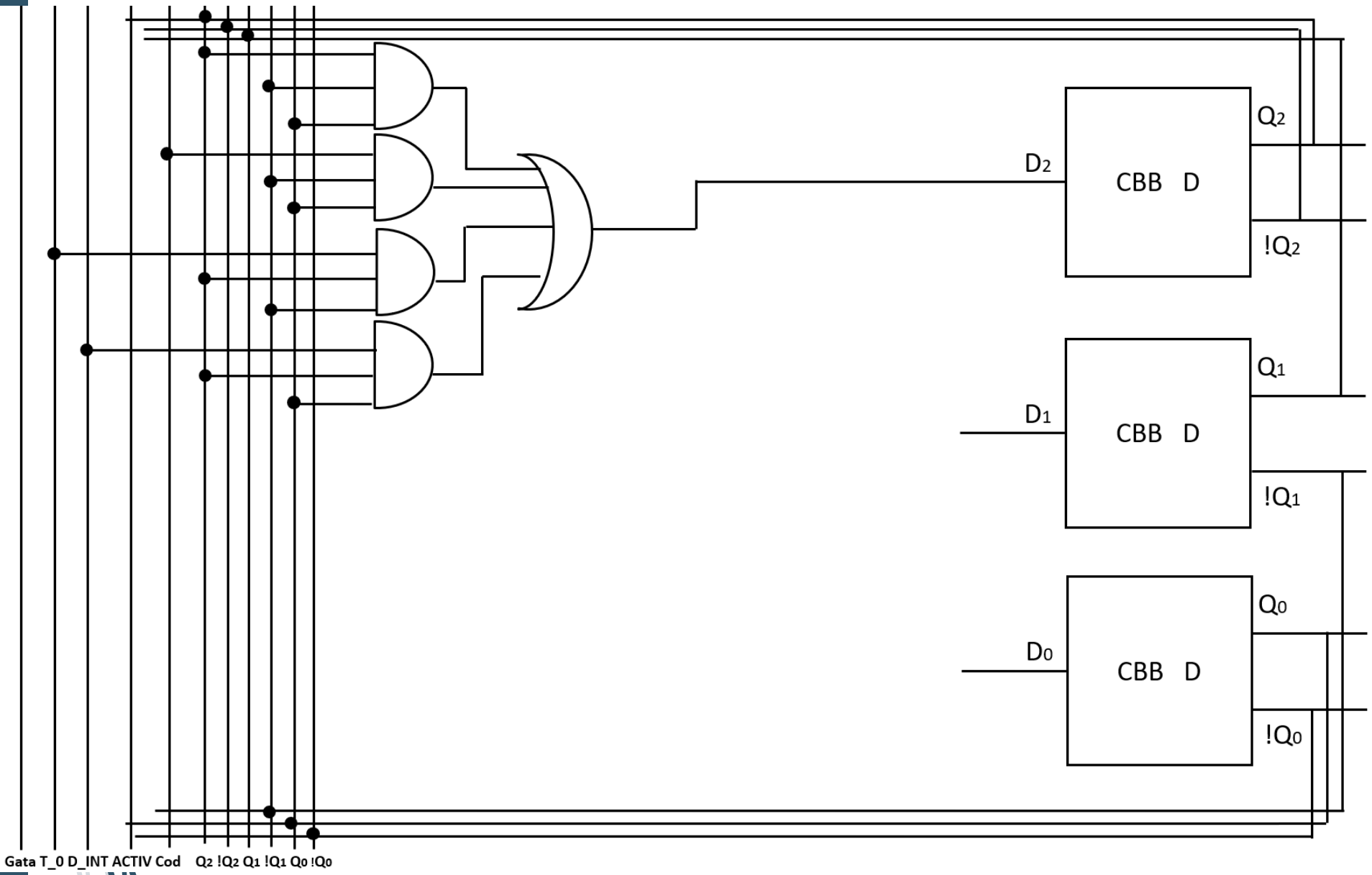
$DES = !(!Q_2Q_1Q_0)$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	0	1

$!APEL = Q_2!Q_1Q_0$

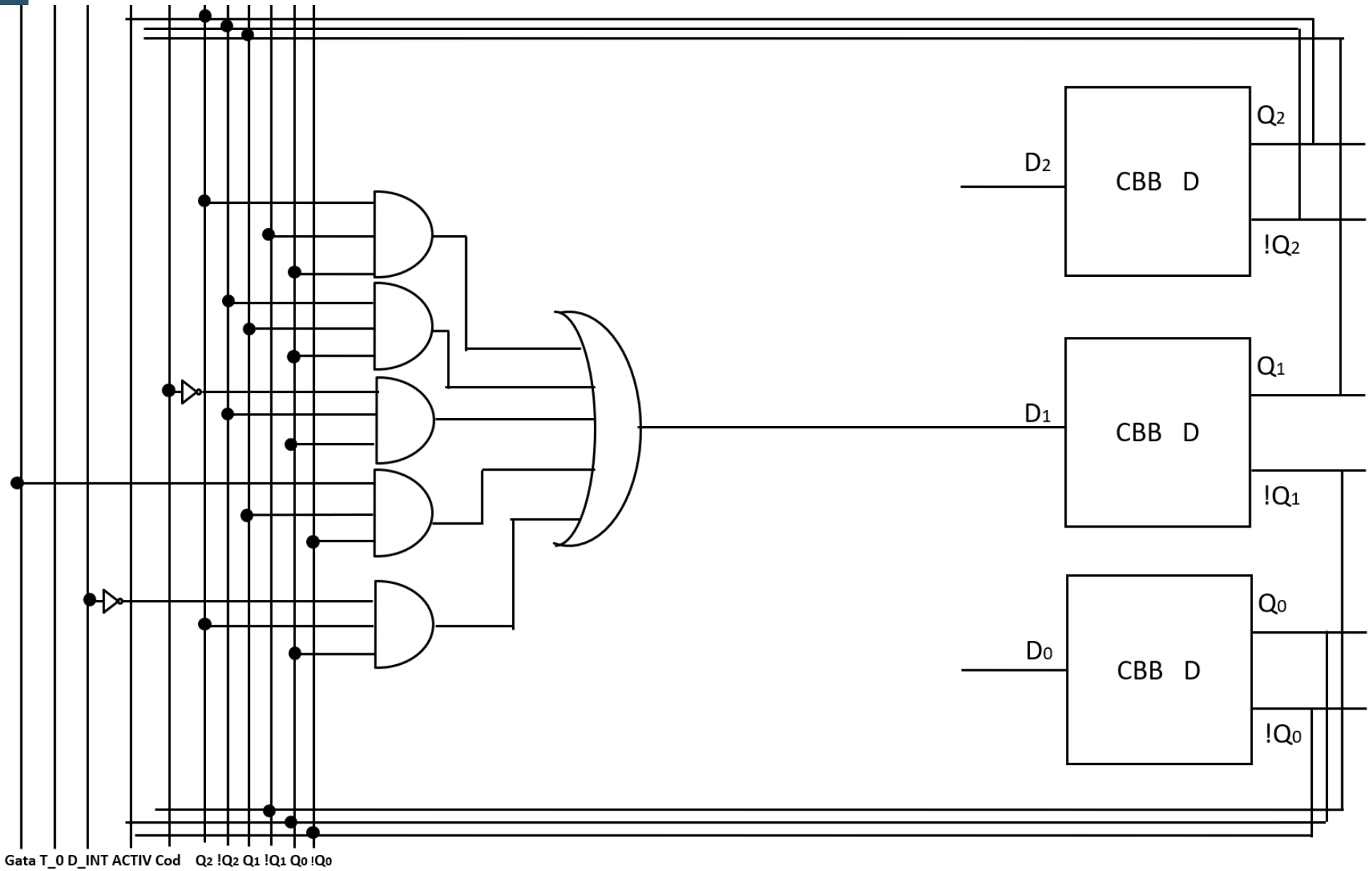
$APEL = !(Q_2!Q_1Q_0)$

- Ce mai putem face?
- Putem rezolva pasul 9 (implementarea cu circuite integrate) având toate informațiile disponibile

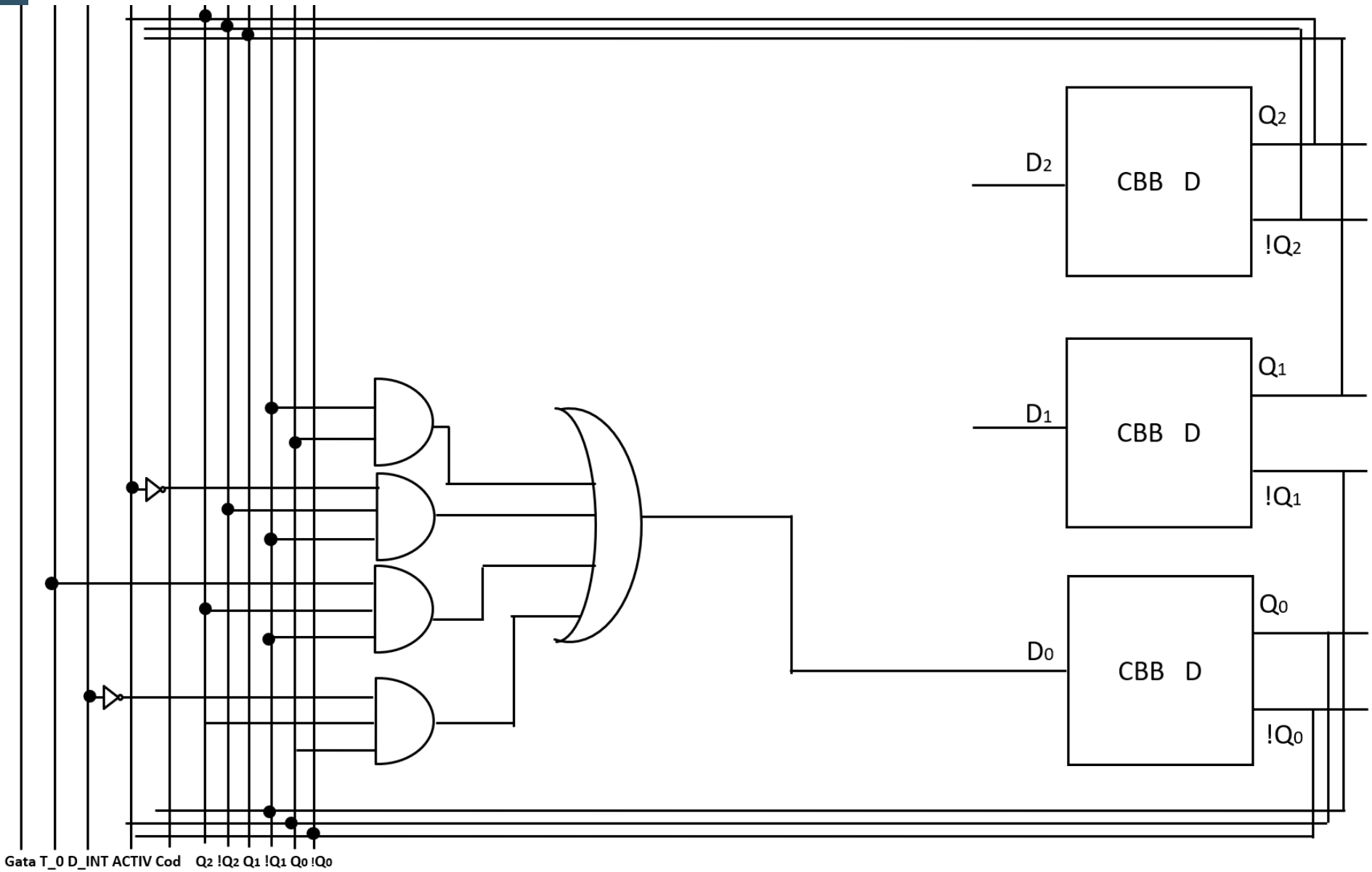


Gata T_0 D_INT ACTIV Cod Q2 !Q2 Q1 !Q1 Q0 !Q0

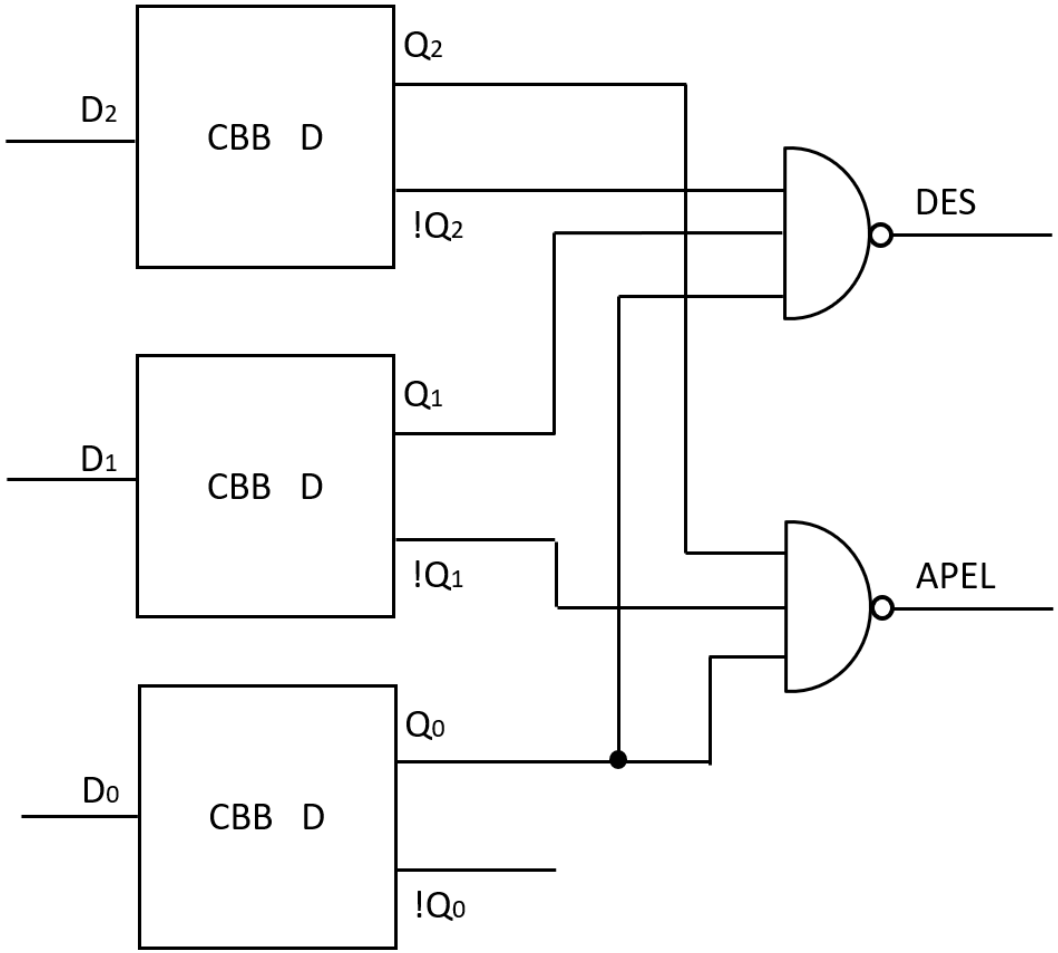
$$D_2 = Q_2!Q_1Q_0 + COD*!Q_1Q0 + T_O*Q_2!Q_1 + D_INT*Q_2Q_0$$



$$D_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$$



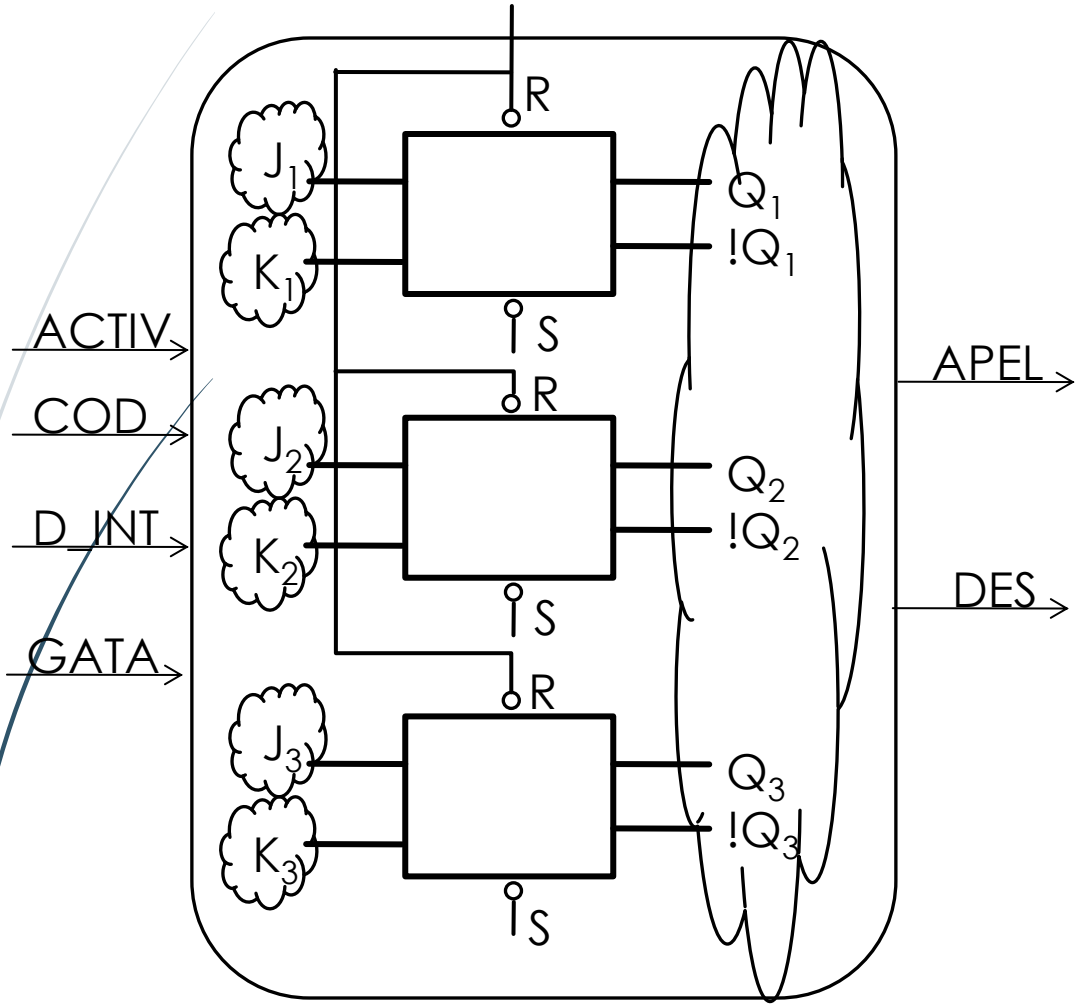
$$D_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$$



$$DES = !(!Q_2 Q_1 Q_0)$$

$$APEL = !(Q_2 !Q_1 Q_0)$$

Implementare Interfon CBB JK



- Similar cu CBB de tip D, trebuie să detectăm ce punem pe intrări (J,K) și cum combinăm ieșirile (Q,!Q)
- La CBB D era foarte simplu de determinat intrările ($Q^{t+1} = D^t$)
- La CBB JK trebuie să împărțim Q^{t+1} în intrarea lui J și a lui K

Determinare J_2 și K_2

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

Q_0 Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

Q_0 Q_2Q_1	00	01	11	10
0	0	0	*	*
1	COD	0	*	*

$J_2 = COD!Q_1Q_0$

Q_0 Q_2Q_1	00	01	11	10
0	*	*	*	T_O
1	*	*	D_INT	1

$!K_2$

Q_0 Q_2Q_1	00	01	11	10
0	*	*	*	!T_O
1	*	*	!D_INT	0

$K_2 = !T_O\ Q_0 + !D_INT\ Q_1$

- Pornim de la diagrama de stare
- Care e zona pentru J_2?
- Primele 2 coloane + restul nu ne interesează → punem *
- La fel pentru $!K_2$
- Putem face direct diagrama lui K prin negarea fiecărei căsuțe în parte → de obicei așa se face ca să nu greșiți!!!

Determinare J_1 și K_1

- Cum se face împărțirea pentru J_1 și K_1?
- E la fel ca pentru Q_2?

Q_0 Q_2Q_1	00	01	11	10
0	0	Gata	*	0
1	!COD	1	!D_INT	1

$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$

Q_0 Q_2Q_1	00	01	11	10
0	0	*	*	0
1	!COD	*	*	1

$J_1 = Q_2Q_0 + !COD*Q0$

Q_0 Q_2Q_1	00	01	11	10
0	*	!Gata	*	*
1	*	0	D_INT	*

$K_1 = !Gata*!Q_0 + !D_INT*Q_2$

Determinare J_0 și K_0

- Cum se face împărțirea pentru J_0 și K_0?

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1

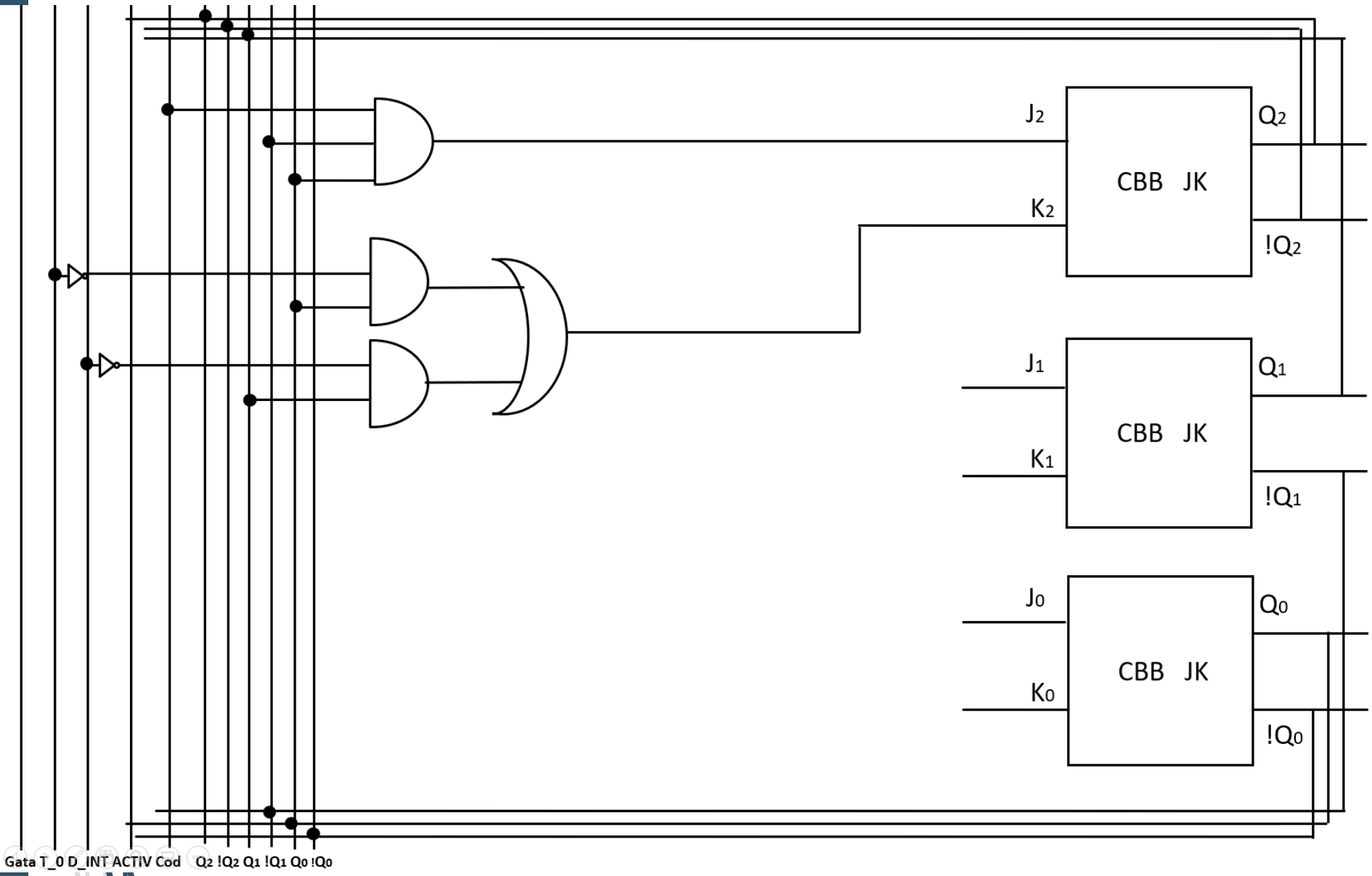
$$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$$

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	*	*	*	*

$$J_0 = !ACTIV*!Q_2!Q_1 + T_O*Q_2$$

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	*
1	0	1	D_INT	0

$$K_0 = !Q_2Q_1 + D_INT*Q_1$$



$J_2 = COD!Q_1Q_0$

$K_2 = !T_O\ Q_0 + !D_INT\ Q_1$

Interconectare MUX - CBB

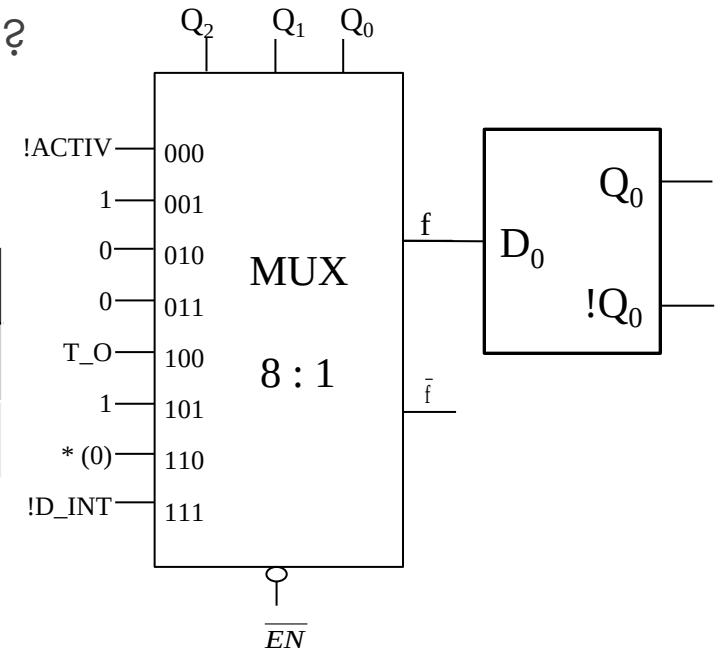
- La examen, în enunțul problemelor pot varia doar tipul de CBB și modul de realizare a circuitelor combinaționale.
- Ex: implementați Qi folosind
  - CBB D și porți (eventual de un anume fel);
  - CBB D și MUX (de un anume tip);
  - JK și porți / MUX
  - JK și porți pentru J, respectiv MUX pentru K
- În cei 10 pași am delimitat ecuațiile de stare următoare de implementarea cu CBB-uri (la tip D era același lucru, dar la JK / RS trebuie să împărțim în 2)
- Care e cea mai simplă implementare?

D + MUX 8:1

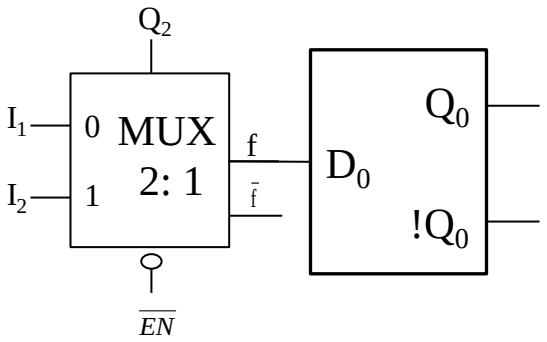
CBB D - MUX

- Care e cea mai simplă implementare?
- D + MUX 8:1
- Ex:

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1



- Altă implementare: D+MUX 2:1
- Întrebare: câte MUX 2:1? (1/7)
- Întrebare 2: ce variabile de selecție? Q_2 / Q_1 / Q_0?
- Temă: Exersați toate combinațiile posibile (MUX 2:1, MUX 4:1, var sel)

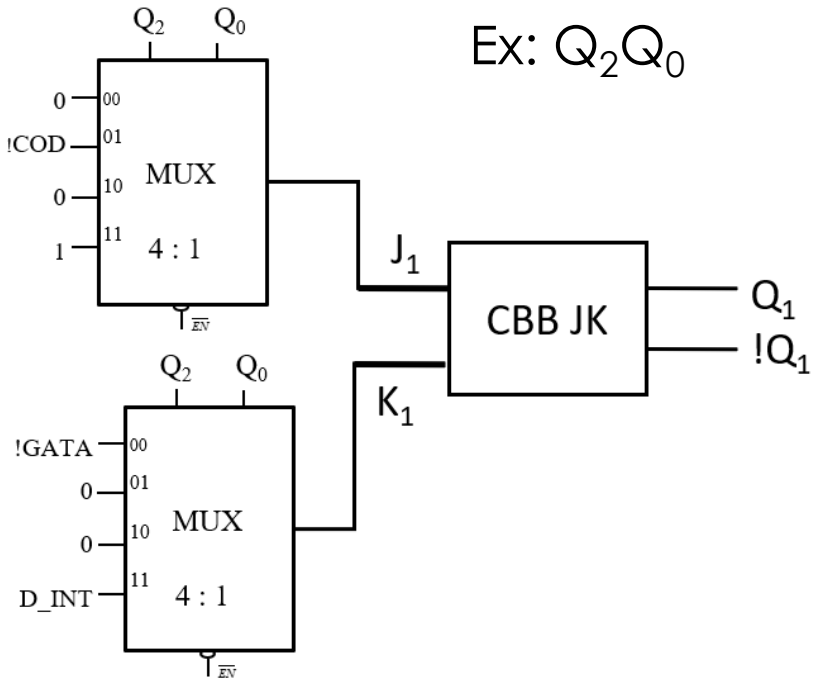


CBB JK - MUX

Q_0 \ Q_2Q_1	00	01	11	10
0	0	*	*	0
1	!COD	*	*	1

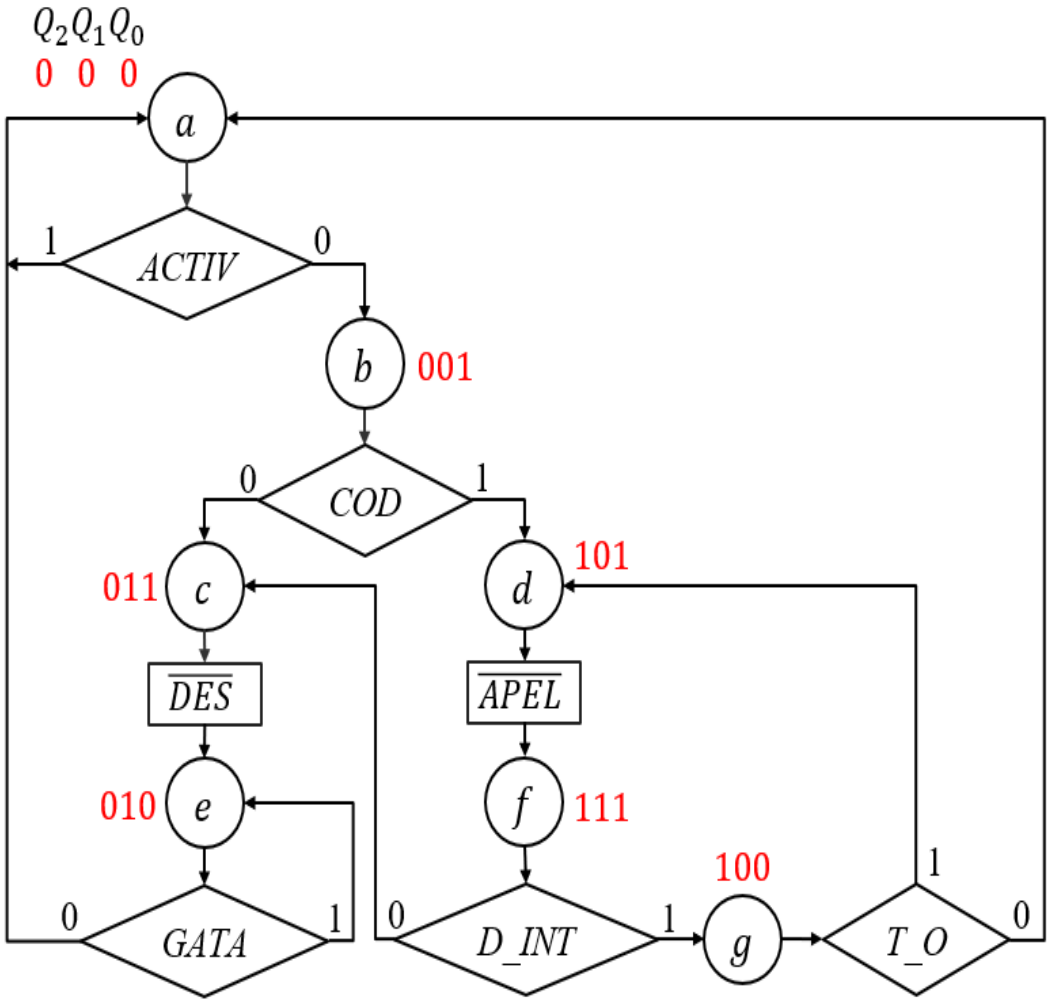
Q_0 \ Q_2Q_1	00	01	11	10
0	*	!Gata	*	*
1	*	0	D_INT	*

Ex: Q_2Q_0



- În practică, pentru cele 2 MUX se aleg aceleași variabile de selecție dar la examen le putem cere diferite (în scop didactic)
- Atenție: Pe condițiile de la MUX nu se pun variabile externe (ex: Gata, Cod etc.), ci doar variabile de stare
- Mereu ne ocupăm de partea secvențială și apoi de cea combinațională. Începem cu CBB și apoi MUX → reverse engineering
- Verificare: nu pot apărea variabilele de selecție pe intrări → termenul minimizat trebuie să conțină termenul de pe intrare + variabilele de selecție în combinația dată

Implementarea semnalelor de ieșire folosind decodificatoare



- Revenim la decodificatoare
- De ce avem nevoie de decodificatoare?
- Când se genera !DES / !APEL?
- În c (011) / d (101)
- Semnalele de ieșire se pot implementa cu porți dar și cu decodificator

Q_0 \ Q_2Q_1	00	01	11	10
0	a	e	*	g
1	b	c	f	d

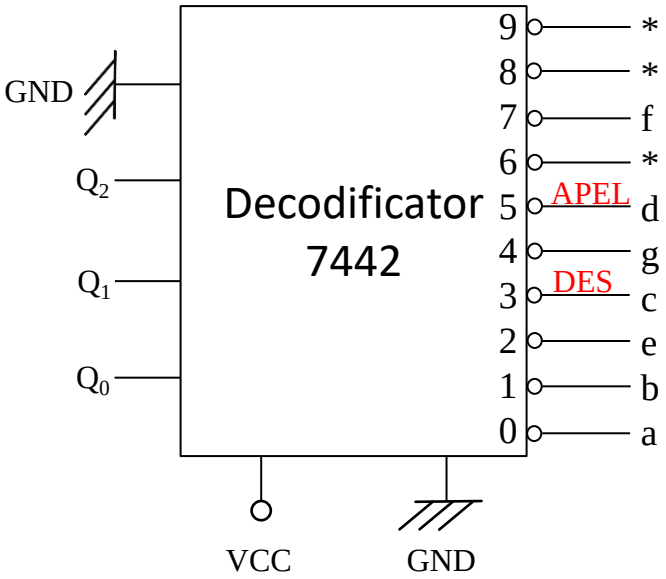
Determinarea semnalelor de ieșire folosind decodificatoare

- Cum se face implementarea semnalele de ieșire folosind decodificator?
  - Identificăm semnalele care ies din ASM
  - Din organigramă identificăm în ce stări se generează fiecare semnal de ieșire
  - Semnalele de ieșire vor fi pe pozițiile corespunzătoare stărilor identificate
- Dacă se pun variabilele de stare pe intrare, la ieșire se activează una și numai una din ieșiri
- Pentru a obține !DES la ieșirea decodificatorului, trebuie să avem la intrare 011, iar pentru !APEL vom avea 101
- Observație: dacă avem mai multe ieșiri, este necesar 1 SINGUR decodificator pentru a le implementa pe toate!

Decodificatorul 7442

- 7442 - Nu are nevoie de EN, ci doar de VCC și GND, e activ pe 0 și e folosit pentru decodificarea spațiului stărilor în circuitele secvențiale și pentru generarea ieșirilor la circuite de tip Moore
- Ce se întâmplă dacă pe intrări pun D - GND, C – Q_2, B – Q_1, A – Q_0?
- În enunțul problemei am cerut ca ieșirile să fie active pe 0 → exact cum sunt generate de 7442

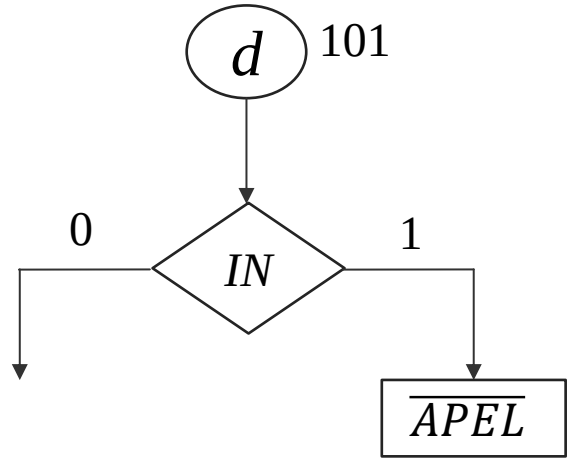
- Ieșirile stau în 1 atâta timp cât decodificatorul nu primește pe intrare 011 (când DES devine 0) sau 101 (când APEL devine 0)

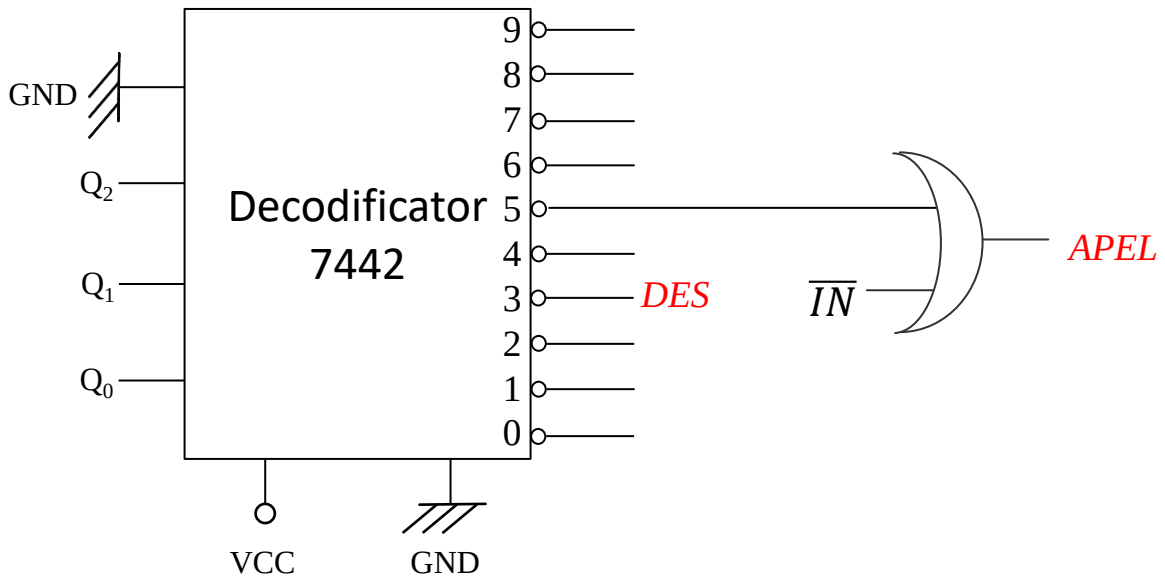


Decodificatorul 7442

Ex 2: Dacă ieșirea depinde și de o variabilă de intrare

$$\overline{APEL} = d \cdot IN$$
$$APEL = \overline{d \cdot IN} = \bar{d} + \overline{IN}$$





Utilitatea decodificatoarelor

1. Există multe situații în care nu am acces la stările circuitelor, dar sunt și situații când am acces la ele → în acest caz, în partea de proiectare, circuitele se testează folosind testere ce conțin astfel de decodificatoare (ce au și ieșiri luminoase) a.î. dacă generez manual impulsuri de ceas să se poată verifica trecerea aparatului dintr-o stare în alta
2. Generarea semnalelor de ieșire (decodifică spațiul stărilor) - foarte simplă, ca în exemplu

Exemplul 2 - Organigrama

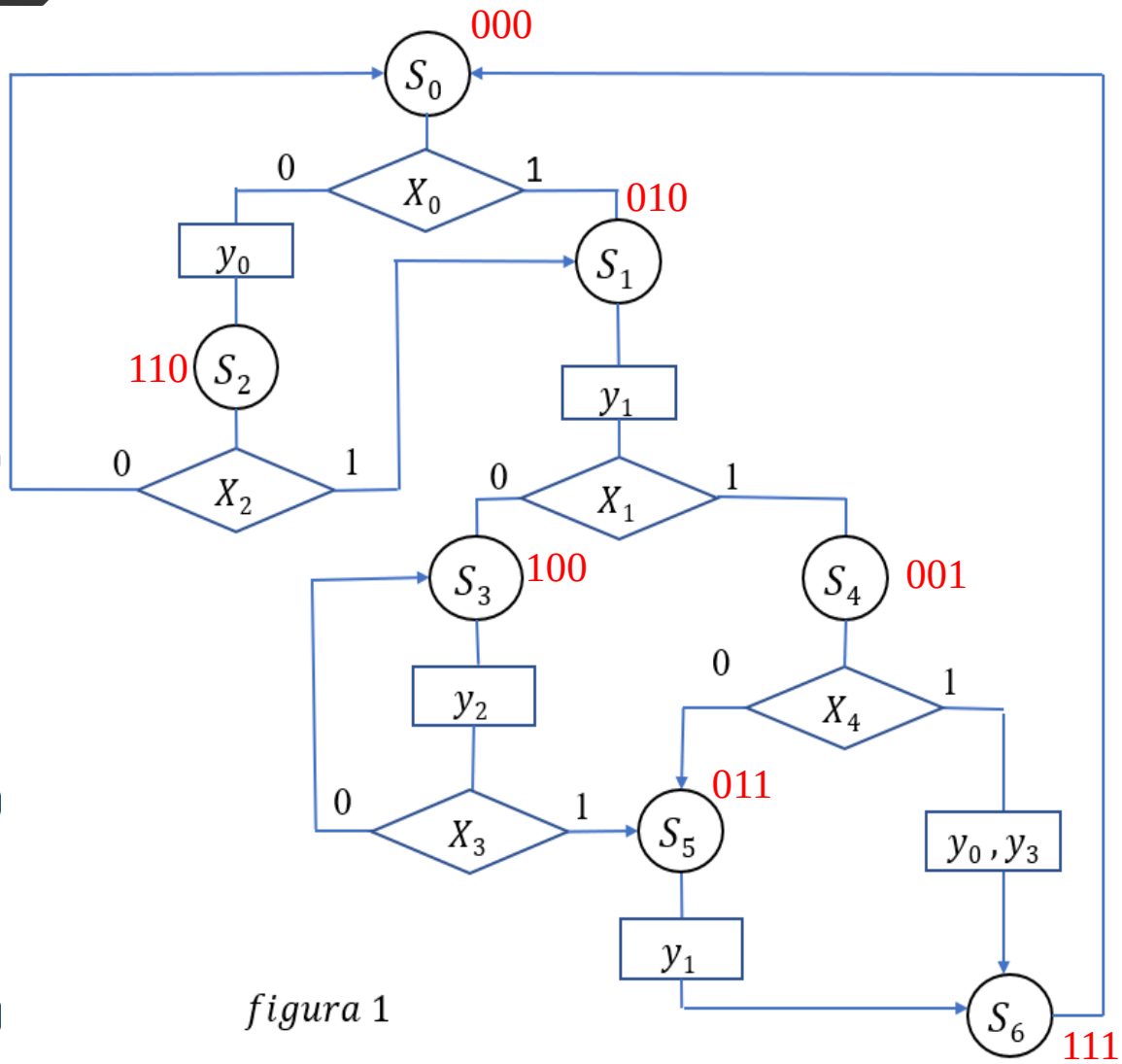
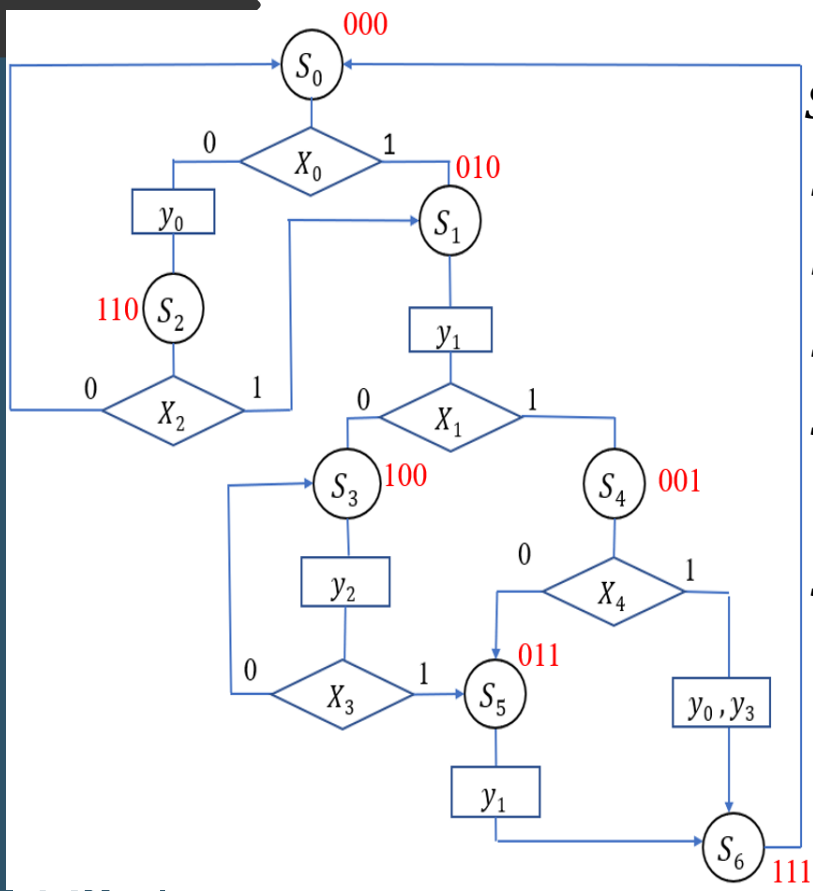


figura 1

Q_2Q_1 / Q_0	00	01	11	10
0	S_0	S_1	S_2	S_3
1	S_4	S_5	S_6	*

Codificare stari

Tabelul de tranziții (de adevăr)



	Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}	y_0	y_1	y_2	y_3
S_0	0	0	0	$\bar{x}_0$	1	0	$\bar{x}_0$	0	0	0
S_4	0	0	1	x_4	1	1	x_4	0	0	x_4
S_1	0	1	0	$\bar{x}_1$	0	x_1	0	1	0	0
S_5	0	1	1	1	1	1	0	1	0	0
S_3	1	0	0	$\bar{x}_3$	x_3	x_3	0	0	1	0
	1	0	1	-	-	-	-	-	-	-
S_2	1	1	0	0	x_2	0	0	0	0	0
S_6	1	1	1	0	0	0	0	0	0	0

S_0 (000): $x_0 = 0 \rightarrow S_2$ (110)
$x_0 = 1 \rightarrow S_1$ (010)

S_1 (010): $x_1 = 0 \rightarrow S_3$ (100)
$x_1 = 1 \rightarrow S_4$ (001)

S_2 (110): $x_2 = 0 \rightarrow S_0$ (000)
$x_2 = 1 \rightarrow S_1$ (010)

S_3 (100): $x_3 = 0 \rightarrow S_3$ (100)
$x_3 = 1 \rightarrow S_5$ (011)

S_4 (001): $x_4 = 0 \rightarrow S_5$ (011)
$x_4 = 1 \rightarrow S_6$ (111)

Ecuațiile de stare

Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}	y_0	y_1	y_2	y_3
0	0	0	$\bar{x}_0$	1	0	$\bar{x}_0$	0	0	0
0	0	1	x_4	1	1	x_4	0	0	x_4
0	1	0	$\bar{x}_1$	0	x_1	0	1	0	0
0	1	1	1	1	1	0	1	0	0
1	0	0	$\bar{x}_3$	x_3	x_3	0	0	1	0
1	0	1	-	-	-	-	-	-	-
1	1	0	0	x_2	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0

Q_2^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	0	$\bar{x}_3$
1	x_4	1	0	-

Q_1^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	1	0	x_2	x_3
1	1	1	0	-

Q_0^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	x_1	0	x_3
1	1	1	0	-

$$Q_2^{t+1} = \bar{Q}_2 Q_1 Q_0 + x_4 \bar{Q}_2 Q_0 + \bar{x}_1 \bar{Q}_2 Q_1 + \bar{x}_0 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 + \bar{x}_3 Q_2 \bar{Q}_1$$

$$Q_1^{t+1} = \bar{Q}_2 Q_0 + \bar{Q}_2 \bar{Q}_1 + x_2 Q_2 Q_1 \bar{Q}_0 + x_3 \bar{Q}_1$$

$$Q_0^{t+1} = \bar{Q}_2 Q_0 + x_1 \bar{Q}_2 Q_1 + x_3 Q_2 \bar{Q}_1$$

Implementare Q_2^{t+1}

Q_2^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	0	$\bar{x}_3$
1	x_4	1	0	-

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

$$Q_2^{t+1} = \bar{Q}_2 Q_1 Q_0 + x_4 \bar{Q}_2 Q_0 + \bar{x}_1 \bar{Q}_2 Q_1 + \bar{x}_0 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 + \bar{x}_3 Q_2 \bar{Q}_1$$

$$D_2 = \bar{Q}_2 Q_1 Q_0 + x_4 \bar{Q}_2 Q_0 + \bar{x}_1 \bar{Q}_2 Q_1 + \bar{x}_0 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 + \bar{x}_3 Q_2 \bar{Q}_1$$

J_2:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	-	-
1	x_4	1	-	-

K_2 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	-	1	x_3
1	-	-	1	-

$$J_2 = Q_1 Q_0 + x_4 Q_0 + \bar{x}_1 Q_1 + \bar{x}_0 \bar{Q}_1 \bar{Q}_0$$

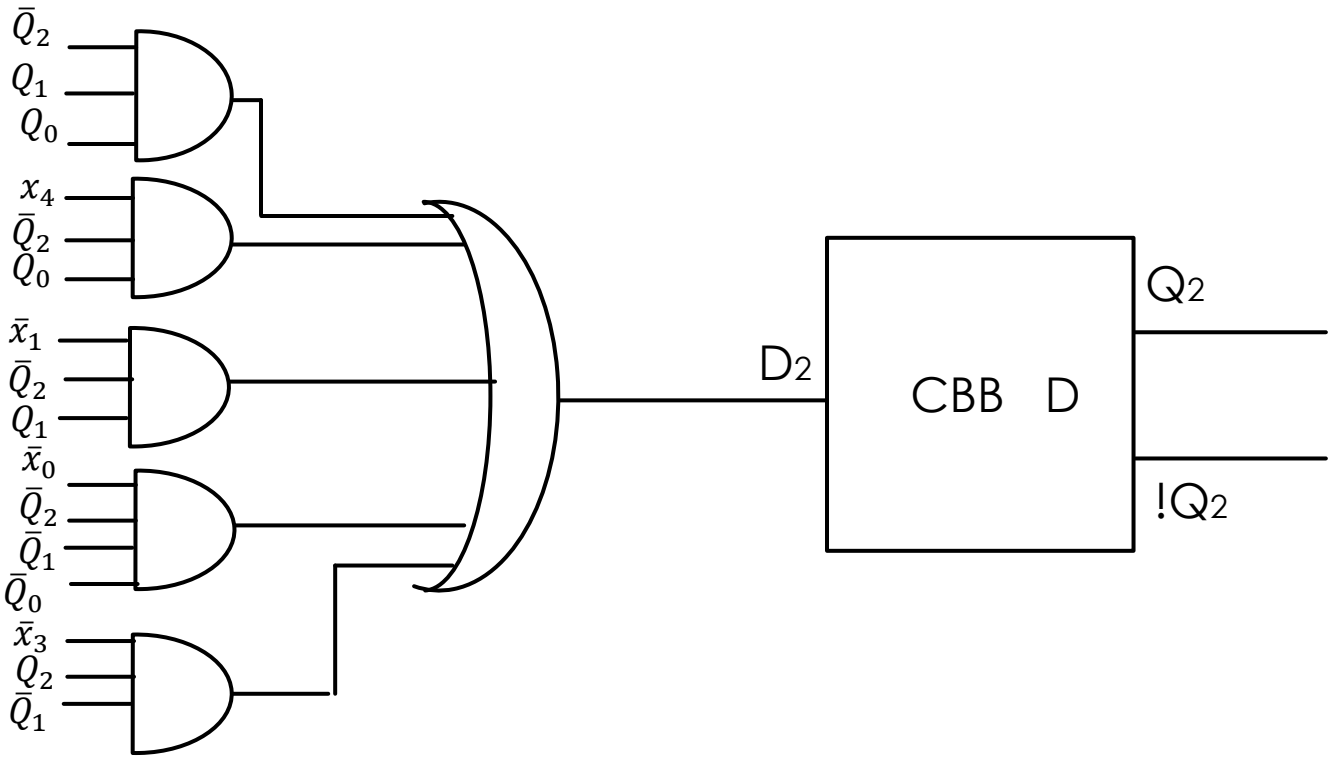
$$K_2 = Q_1 + x_3$$

Implementare Q_2^{t+1} cu CBB D și porți

Q_2^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	0	$\bar{x}_3$
1	x_4	1	0	-

$$D_2 = \bar{Q}_2 Q_1 Q_0 + x_4 \bar{Q}_2 Q_0 + \bar{x}_1 \bar{Q}_2 Q_1 + \bar{x}_0 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 + \bar{x}_3 Q_2 \bar{Q}_1$$

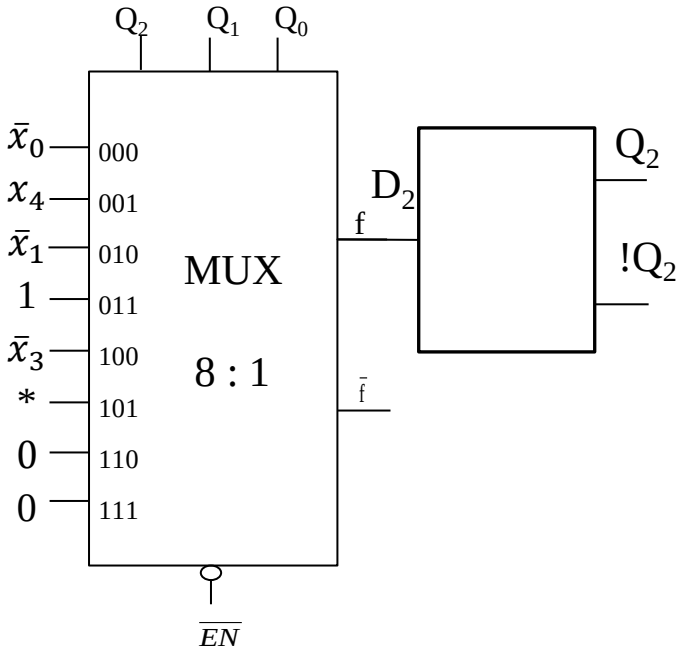


Implementare Q_2^{t+1} cu CBB D și MUX 8:1

Q_2^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	0	$\bar{x}_3$
1	x_4	1	0	-

$$D_2 = \bar{Q}_2 Q_1 Q_0 + x_4 \bar{Q}_2 Q_0 + \bar{x}_1 \bar{Q}_2 Q_1 + \bar{x}_0 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 + \bar{x}_3 Q_2 \bar{Q}_1$$



Implementare Q_2^{t+1} cu CBB JK si MUX 8:1

J_2:

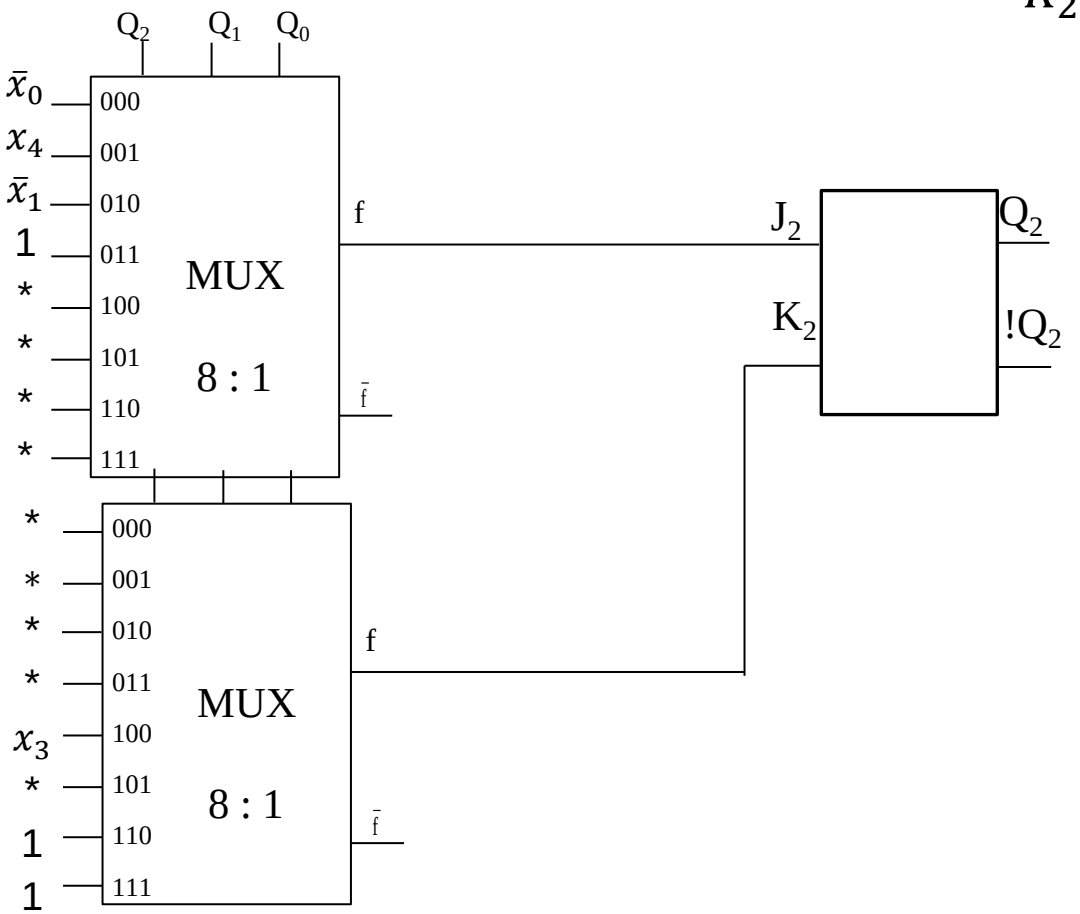
Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	$\bar{x}_1$	-	-
1	x_4	1	-	-

K_2 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	-	1	x_3
1	-	-	1	-

$$J_2 = Q_1Q_0 + x_4Q_0 + \bar{x}_1Q_1 + \bar{x}_0\bar{Q}_1\bar{Q}_0$$

$$K_2 = Q_1 + \bar{x}_3$$



Implementare Q_1^{t+1}

Q_1^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	1	0	x_2	x_3
1	1	1	0	-

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

$$Q_1^{t+1} = \bar{Q}_2 Q_0 + \bar{Q}_2 \bar{Q}_1 + x_2 Q_2 Q_1 \bar{Q}_0 + x_3 \bar{Q}_1$$

$$D_1 = \bar{Q}_2 Q_0 + \bar{Q}_2 \bar{Q}_1 + x_2 Q_2 Q_1 \bar{Q}_0 + x_3 \bar{Q}_1$$

J_1:

Q_0 \ Q_2Q_1	00	01	11	10
0	1	-	-	x_3
1	1	-	-	-

K_1 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	1	$\bar{x}_2$	-
1	-	0	1	-

$$J_1 = \bar{Q}_2 + x_3$$

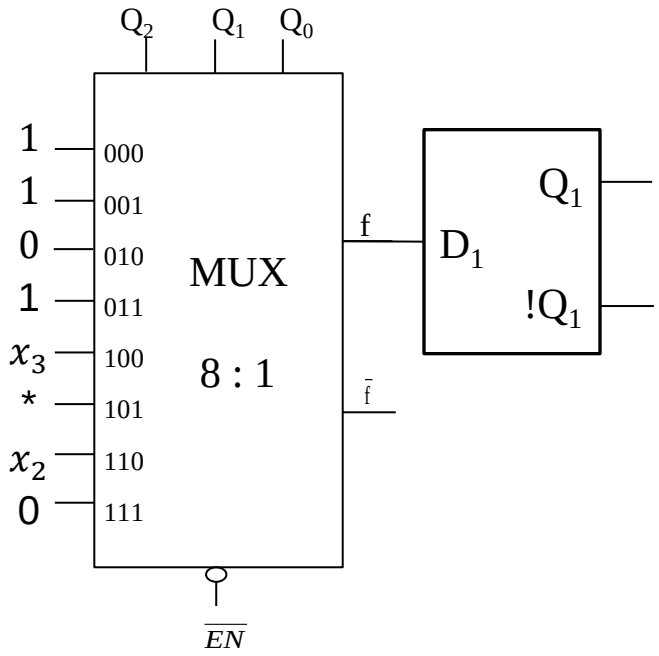
$$K_1 = \bar{Q}_2 \bar{Q}_0 + Q_2 Q_0 + \bar{x}_2 Q_2$$

Implementare Q_1^{t+1} cu CBB D și MUX 8:1

Q_1^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	1	0	x_2	x_3
1	1	1	0	-

$$D_1 = \bar{Q}_2 Q_0 + \bar{Q}_2 \bar{Q}_1 + x_2 Q_2 Q_1 \bar{Q}_0 + x_3 \bar{Q}_1$$



Implementare Q_1^{t+1} cu CBB JK și MUX 8:1

J_1:

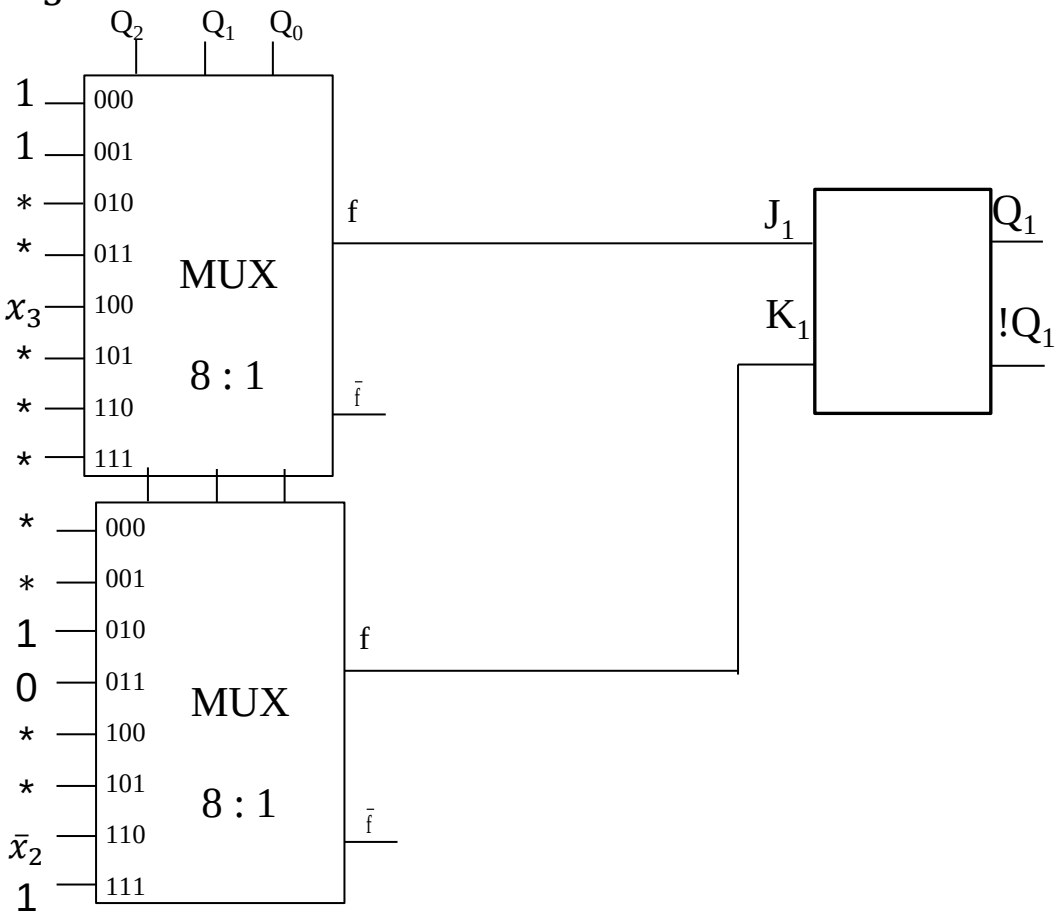
Q_0 \ Q_2Q_1	00	01	11	10
0	1	-	-	x_3
1	1	-	-	-

$J_1 = Q_2 + x_3$

K_1 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	1	$\bar{x}_2$	-
1	-	0	1	-

$K_1 = \bar{Q}_2\bar{Q}_0 + Q_2Q_0 + \bar{x}_2Q_2$



Implementare Q_1^{t+1} cu CBB JK și porți

J_1:

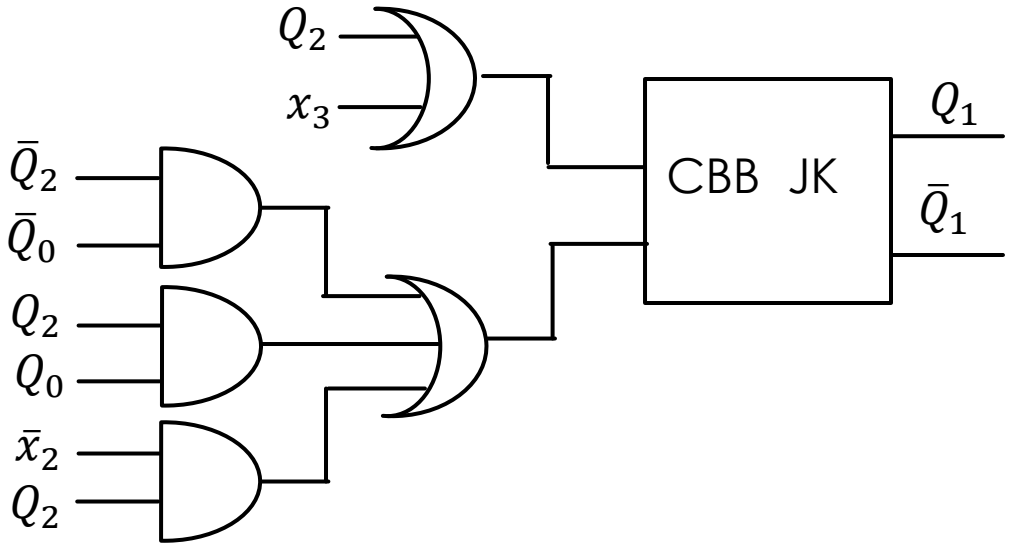
Q_0 \ Q_2Q_1	00	01	11	10
0	1	-	-	x_3
1	1	-	-	-

$$J_1 = Q_2 + x_3$$

K_1 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	1	$\bar{x}_2$	-
1	-	0	1	-

$$K_1 = \bar{Q}_2\bar{Q}_0 + Q_2Q_0 + \bar{x}_2Q_2$$



Implementare Q_0^{t+1}

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

Q_0^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	x_1	0	x_3
1	1	1	0	-

$$Q_0^{t+1} = \bar{Q}_2 Q_0 + x_1 \bar{Q}_2 Q_1 + x_3 Q_2 \bar{Q}_1$$

$$D_0 = \bar{Q}_2 Q_0 + x_1 \bar{Q}_2 Q_1 + x_3 Q_2 \bar{Q}_1$$

J_0:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	x_1	0	x_3
1	-	-	-	-

K_0 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	-	-	-
1	0	0	1	-

$$J_0 = x_1 \bar{Q}_2 Q_1 + x_3 Q_2 \bar{Q}_1$$

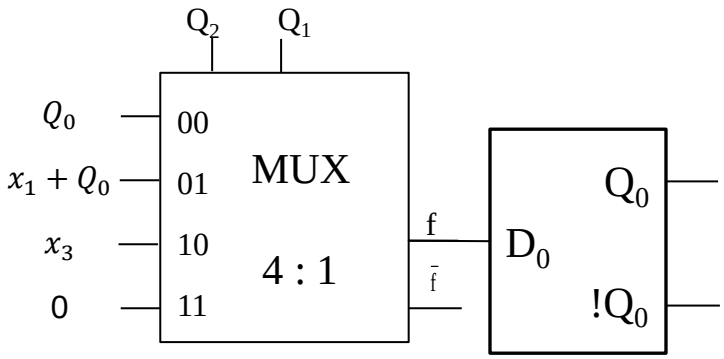
$$K_0 = Q_2$$

Implementare Q_0^{t+1} cu CBB tip D și MUX 4:1

Q_0^{t+1}:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	x_1	0	x_3
1	1	1	0	-

$$D_0 = \bar{Q}_2 Q_0 + x_1 \bar{Q}_2 Q_1 + x_3 Q_2 \bar{Q}_1$$



Implementare Q_0^{t+1} cu CBB JK si MUX 2:1

J_0:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	x_1	0	x_3
1	-	-	-	-

K_0 :

Q_0 \ Q_2Q_1	00	01	11	10
0	-	-	-	-
1	0	0	1	-

Q_0 \ Q_2	0	1
0	0	x_3
1	-	-

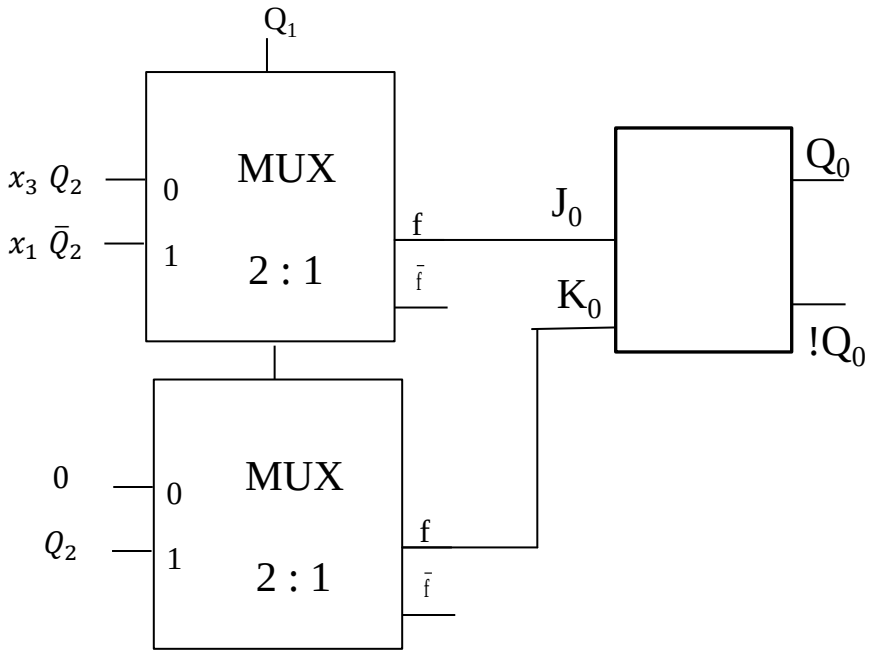
Q_0 \ Q_2	0	1
0	x_1	0
1	-	-

Q_0 \ Q_2	0	1
0	-	-
1	0	-

Q_0 \ Q_2	0	1
0	-	-
1	0	1

$$K_0 = Q_2$$

$$J_0 = x_1\bar{Q}_2Q_1 + x_3Q_2\bar{Q}_1$$



Ieșiri

Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}	y_0	y_1	y_2	y_3
0	0	0	$\bar{x}_0$	1	0	$\bar{x}_0$	0	0	0
0	0	1	x_4	1	1	x_4	0	0	x_4
0	1	0	$\bar{x}_1$	0	x_1	0	1	0	0
0	1	1	1	1	1	0	1	0	0
1	0	0	$\bar{x}_3$	x_3	x_3	0	0	1	0
1	0	1	-	-	-	-	-	-	-
1	1	0	0	x_2	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0

$$y_0 = x_4\bar{Q}_1Q_0 + \bar{x}_0\bar{Q}_2\bar{Q}_1\bar{Q}_0$$
$$y_1 = \bar{Q}_2Q_1$$
$$y_2 = Q_2\bar{Q}_1$$
$$y_3 = x_4\bar{Q}_1Q_0$$

y_0:

Q_0 \ Q_2Q_1	00	01	11	10
0	$\bar{x}_0$	0	0	0
1	x_4	0	0	-

y_1:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	1	0	0
1	0	1	0	-

y_2:

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	0	1
1	0	0	0	-

y_3:

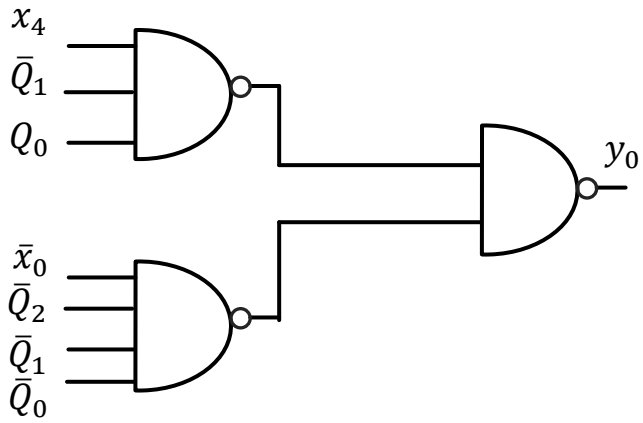
Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	0	0
1	x_4	0	0	-

Ieșiri

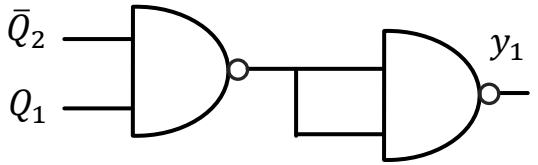
$$y_0 = x_4\bar{Q}_1Q_0 + \bar{x}_0\bar{Q}_2\bar{Q}_1\bar{Q}_0$$

$$= \overline{\overline{x_4\bar{Q}_1Q_0 + \bar{x}_0\bar{Q}_2\bar{Q}_1\bar{Q}_0}}$$

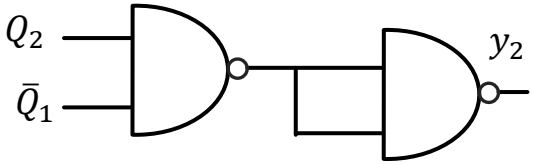
$$= \overline{\overline{x_4\bar{Q}_1Q_0} \cdot \overline{\bar{x}_0\bar{Q}_2\bar{Q}_1\bar{Q}_0}}$$



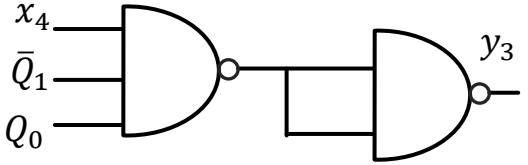
$$y_1 = \bar{Q}_2Q_1$$



$$y_2 = Q_2\bar{Q}_1$$

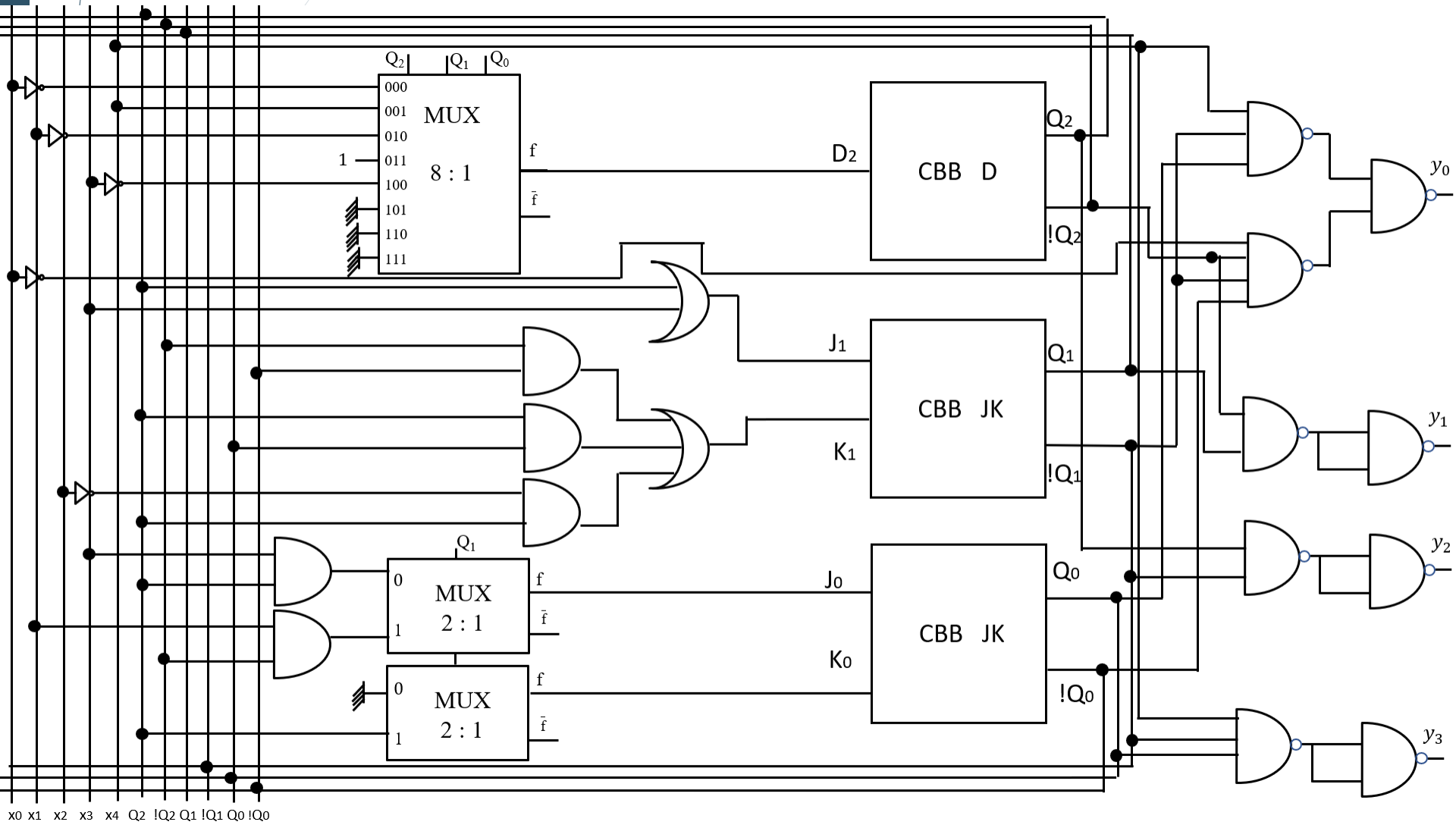


$$y_3 = x_4\bar{Q}_1Q_0$$



Desen final

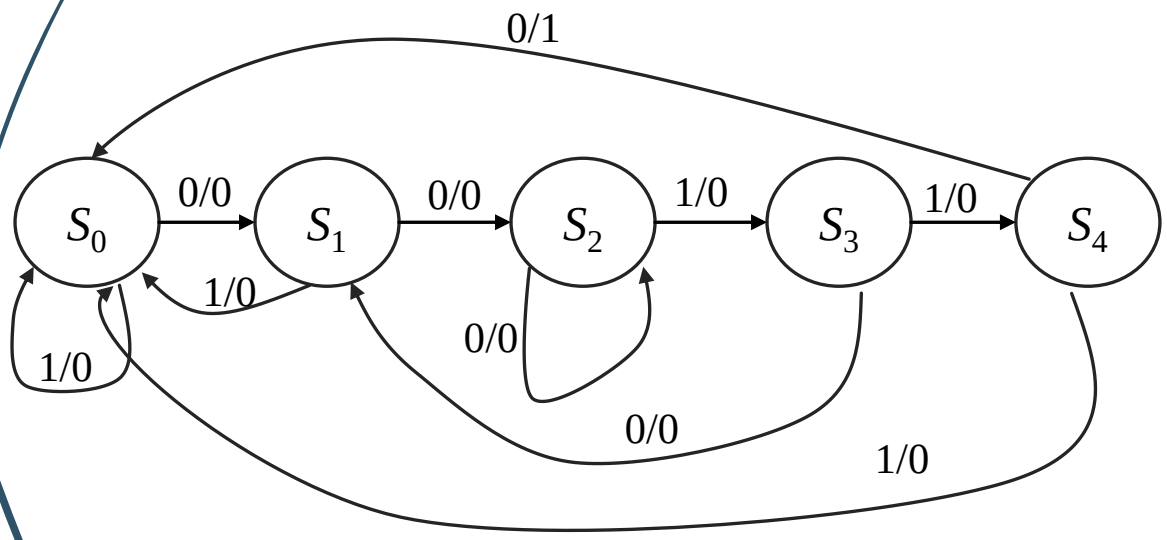
- Q_2^{t+1} cu CBB D și MUX 8:1 cu var selecție Q_2, Q_1, Q_0
- Q_1^{t+1} cu CBB JK și porți ȘI și SAU
- Q_0^{t+1} cu CBB D și MUX 2:1 cu var selecție Q_1
- Ieșirile cu porți NAND



Automate - Exemple

- Construiti un automat care recunoaste secventa 00110, fără suprapuneri
- Care sunt stările prin care trece automatul si care sunt ieșirile pentru urmatoare secventa de intrare:

	1	**0**	**0**	**1**	**1**	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**	**0**	**1**	**0**
S_0	S_0	S_1	S_2	S_3	S_4	S_0	S_1	S_2	S_2	S_3	S_1	S_2	S_3	S_4	S_0	S_1	S_0	S_0	S_1	S_0	S_1
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

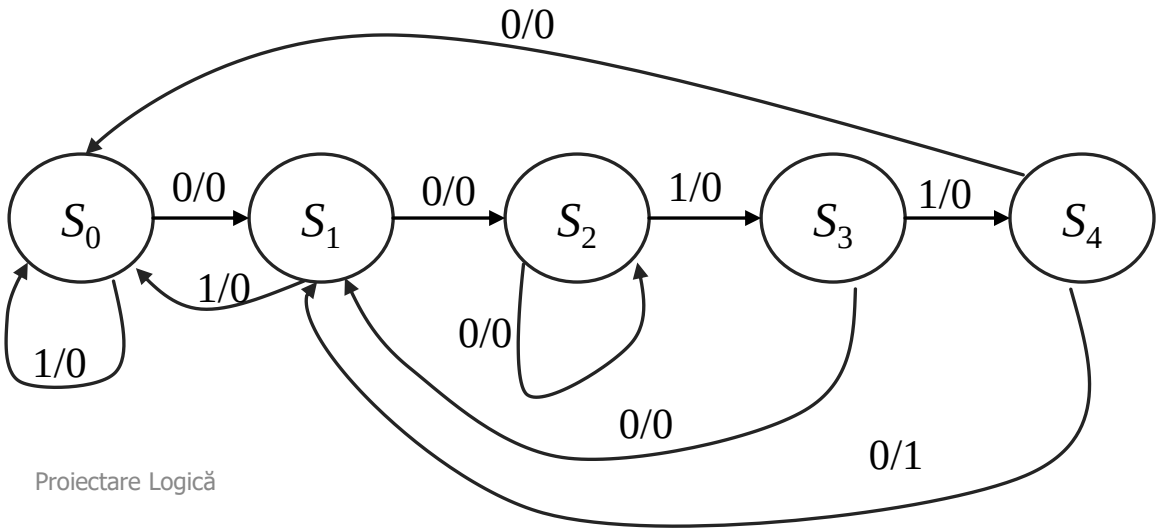


S0 – starea initiala
S1 – starea in care am primit un 0
S2 – starea in care am primit 00
S3 – starea in care am primit 001
S4 – starea in care am primit 0011

Automate - Exemple

- Construiti un automat care recunoaste secventa 00110, cu suprapuneri
- Care sunt stările prin care trece automatul si care sunt ieșirile pentru urmatoare secventa de intrare:

	1	**0**	**0**	**1**	**1**	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**	**0**	**1**	**0**
S_0	S_0	S_1	S_2	S_3	S_4	S_0	S_1	S_2	S_2	S_3	S_1	S_2	S_3	S_4	S_1	S_2	S_3	S_4	S_1	S_0	S_1
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0



Proiectare Logică

- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- I CA & CD: Costin Chiru costin.chiru@cs.pub.ro
- **I CC: Anca Morar** anca.morar@cs.pub.ro

➨**Curs 8: Clase de circuite secventiale: Numărătoare și registre.**

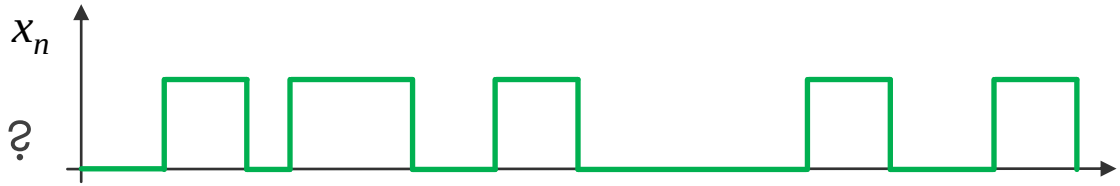
Numărătoare

- o clasă foarte importantă de circuite secvențiale de ordin 2 având la bază circuitele basculante bistabile.
- se găsesc sub formă de circuite integrate de tip MSI sau pot fi sintetizate în laborator (se pot realiza) din CBB-uri.
- realizează o secvență de numărare.
- prezintă un număr de ieşiri care prin valoarea lor logică indică numărul de impulsuri recepționate pe intrări, din momentul unei stări inițiale cunoscute.
- Capacitatea numărătorului se defineşte în raport cu numărul de ieşiri: un numărător cu n ieşiri are capacitatea maximă de 2^n.
- Cele mai utilizate - cele binare, în care la ieşire avem codul binar al stării numărătorului.

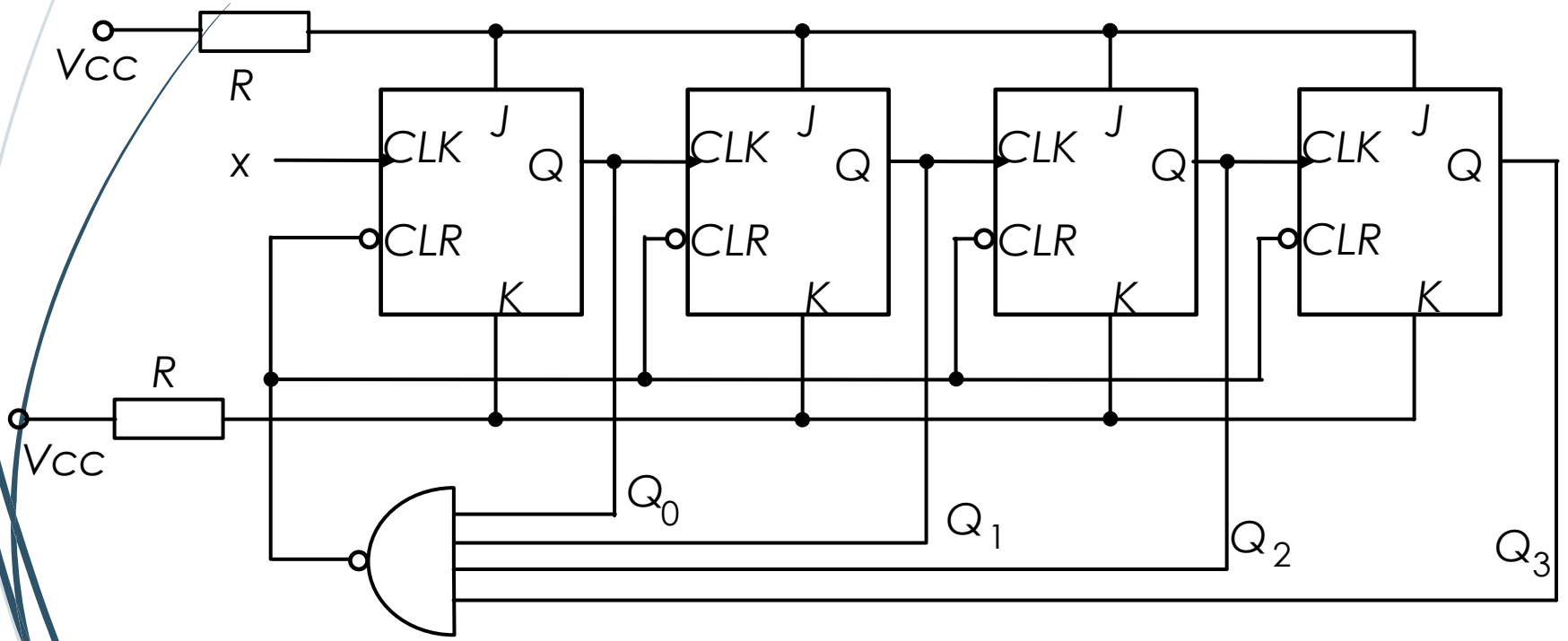
Clasificarea numărătoarelor

1. După secvența de numărare: hexazecimale, zecimale, 12
2. După modul de aplicare al semnalului de tact:
   - asincrone (impulsul de tact nu se aplică în acelaşi moment la toate automatele elementare)
   - sicrone (impulsul de tact se aplică simultan tuturor automatelor elementare care compun numărătorul)

Numărătoare

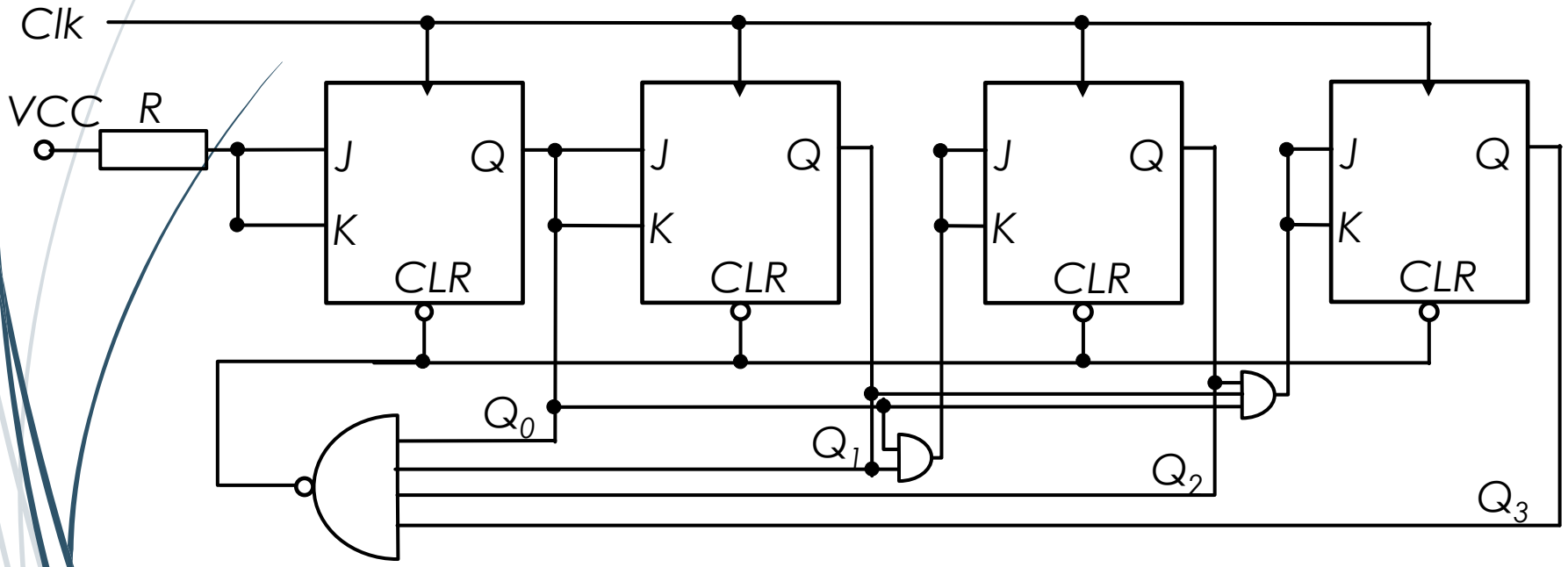


- E sincron sau nu?
- Intrarea poate să fie de forma:
- Ieșirea este asincronă pentru că momentul la care vin semnalele de intrare e aleator

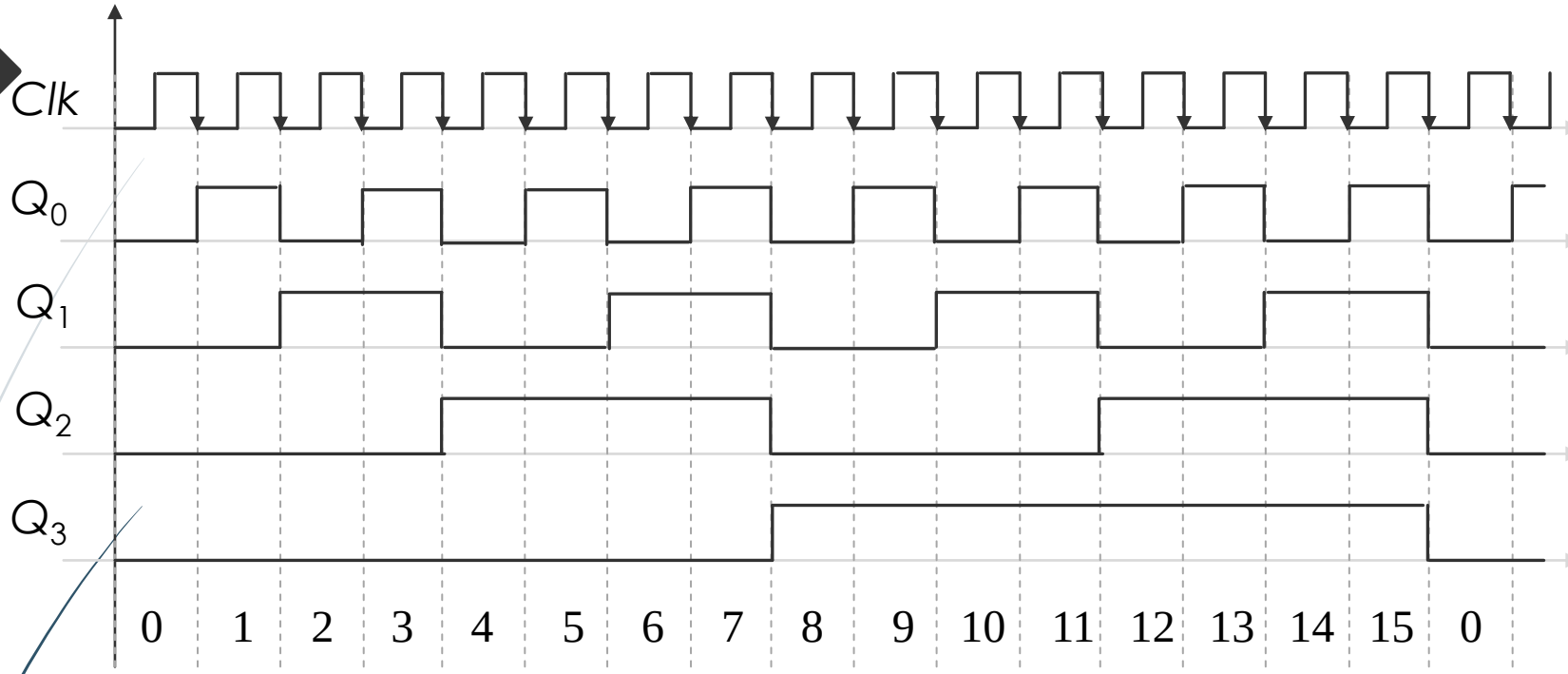


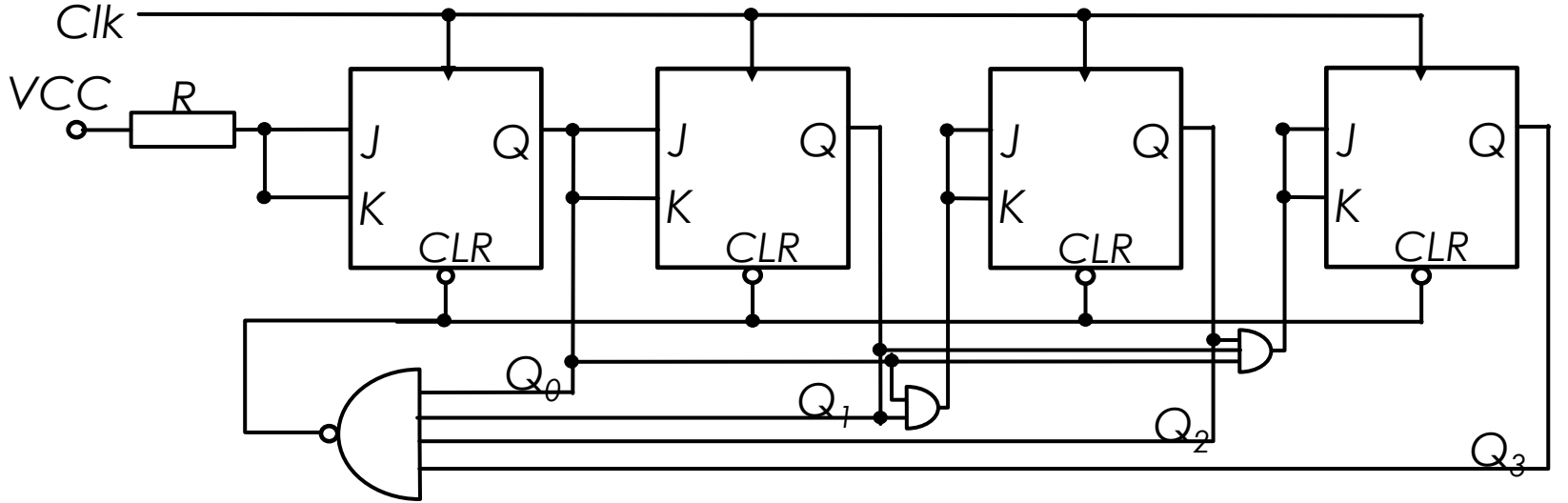
Numărătoare

- Ca să devină sincron, aparatul trebuie modificat ca mai jos:
- Cel asincron e mai ușor de realizat, dar dacă am mai multe celule, e posibil să avem întârzieri



Numărătoare





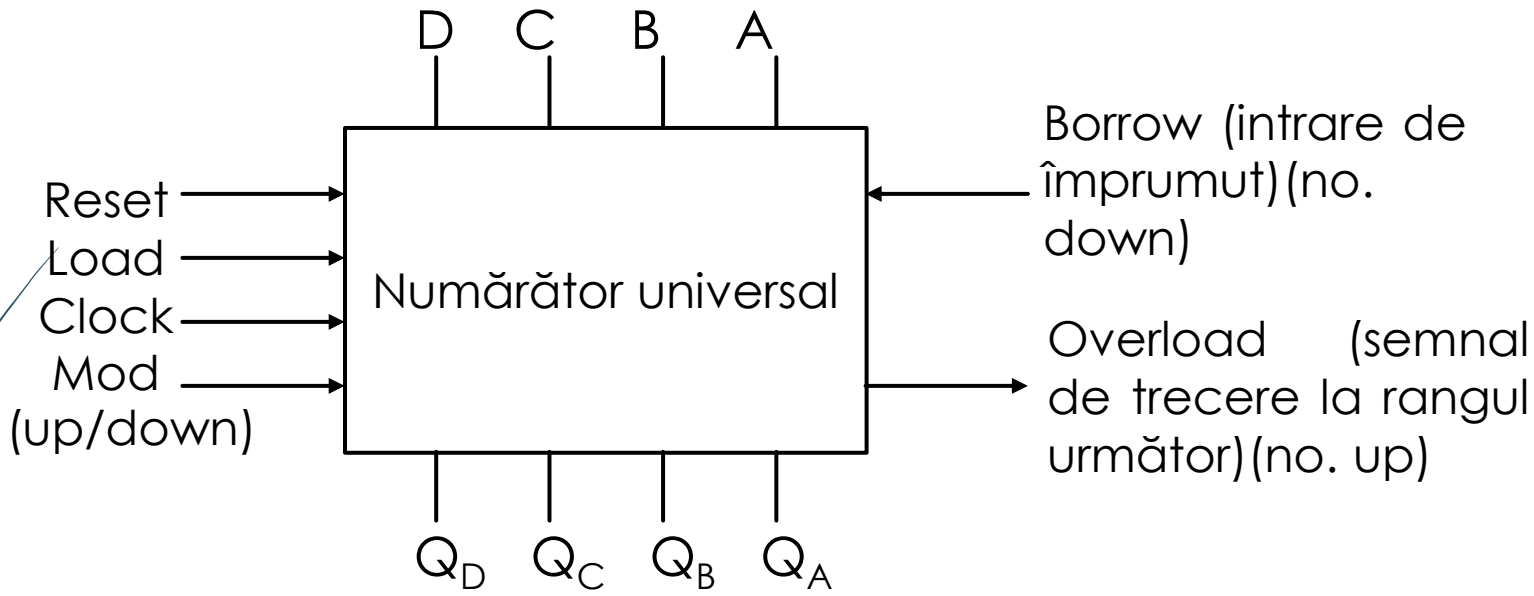
Clasificarea numărătoarelor

3. După modul de numărare
   - direct (1 → 16),
   - invers (16 → 1), foarte ușor de realizat pentru că CBB-urile au și Q și !Q și pentru numărare inversă luăm !Q în loc de Q
   - în ambele sensuri

4. După ciclicitate: ciclice sau nu (trebuie resetate în mod explicit pentru a reveni la starea inițială)

Numărătorul universal (4 biți)

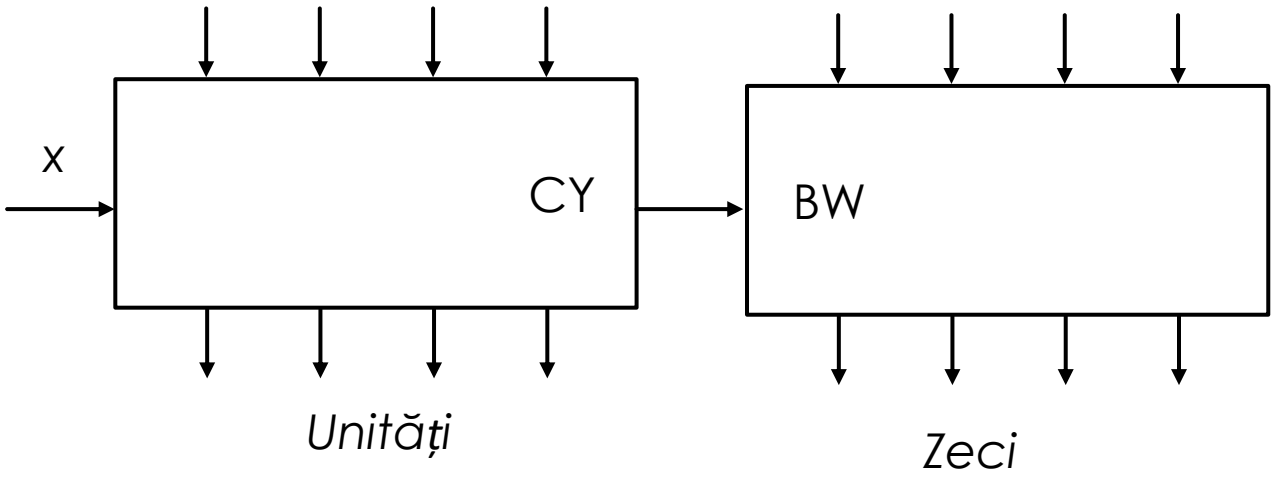
- Exemplu de numărător – există constructiv (pe capsulă)



- Overload permite concatenarea mai multor numărătoare universale ca să nu mai numere pe 4 biți, ci pe mai mulți

Concatenarea numărătoarelor

- Overload permite concatenarea mai multor numărătoare universale care să nu mai numere pe 4 biți, ci pe mai mulți biți

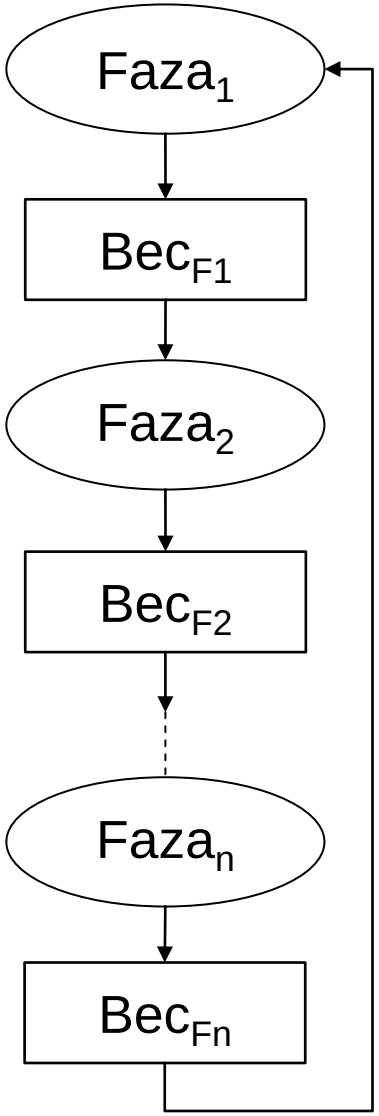


Funcții ale numărătorului universal

- Funcția de numărare, în sens direct sau invers
- Funcția de încărcare a unei valori inițiale (presetare)
  - dacă vrem să pornim de la o anumită valoare (în loc de 0), aceasta se face prin activarea lui LOAD ce încarcă în numărător în mod paralel (toate odată) valorile D, C, B, A
- Funcția de ştergere a conținutului (resetare sau aducere la zero)

Numărătoare în loc de CBB-uri

- Pentru organigrame foarte simple
- Ex: semaforul
- Secvență de numărare modulo n fără salturi, foarte simplă
- Cum fac implementarea?
- Iau numărătorul, bag ieșirile în decodificator și astfel se aprinde pe rând doar un anumit set de becuri.



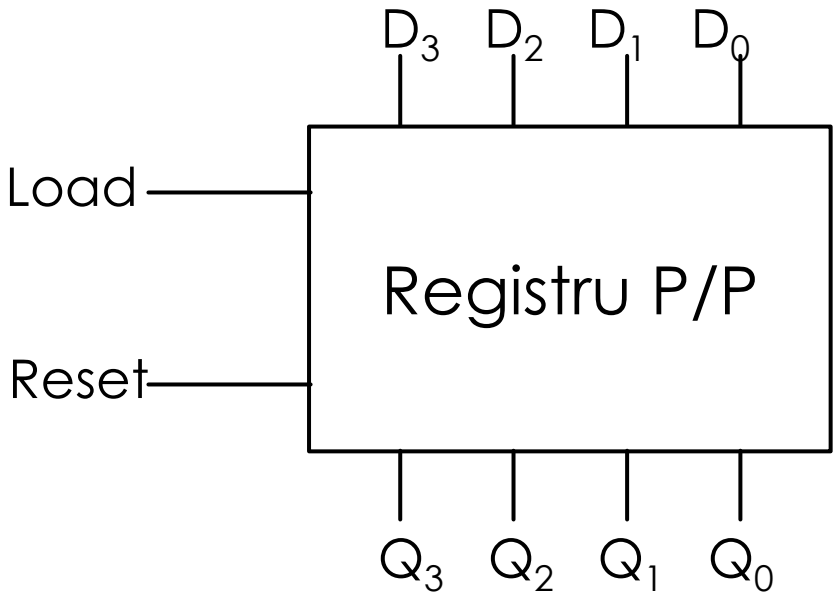
Registre

- Ați auzit de registre? Unde? Ce sunt?
- Ați auzit că unitatea centrală a unui calculator conține obligatoriu niște regiștrii și UAL?
- Reprezintă o clasă de circuite secvențiale realizate pe baza CBB-urilor cu scopul de a memora și pune la dispoziție informația formată din semnale elementare.
- Numărul semnalelor elementare este egal cu cel al circuitelor basculante bistabile care compun registrul
- Capacitatea registrului = numărul de biți care pot fi stocați sau memorați în registru

Clasificare registre

- În funcție de modul de realizare al intrărilor și ieșirilor (serie / paralel)

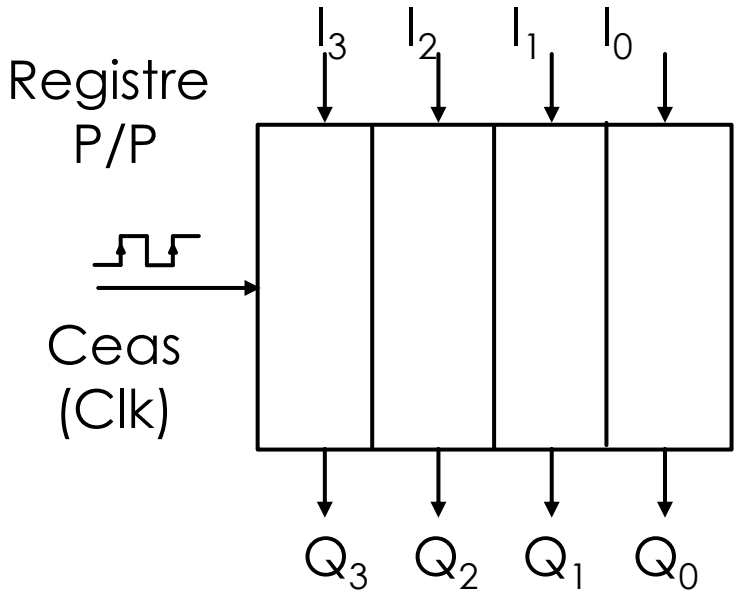
Intrare	Ieșire
P	P
P	S
S	P
S	S



Registre P/P

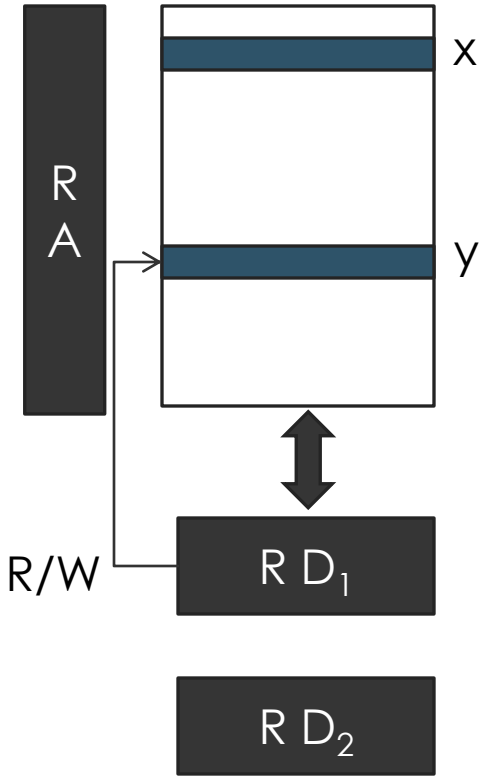
- La Load încarcă intrările în registru și le pune la dispoziția ieșirilor până la un nou load
- Seamănă cu funcția de presetare de la numărătorul universal
- Registrele sunt foarte utile în anumite zone ale calculatorului, dar și în interacțiunea dintre calculator și dispozitive periferice

- Ex: imprimanta
  - Acum: USB
  - În trecut:conectori cu mulți pini → comunicare P-P (un cuvânt se putea transmite simultan)



Registre P/P

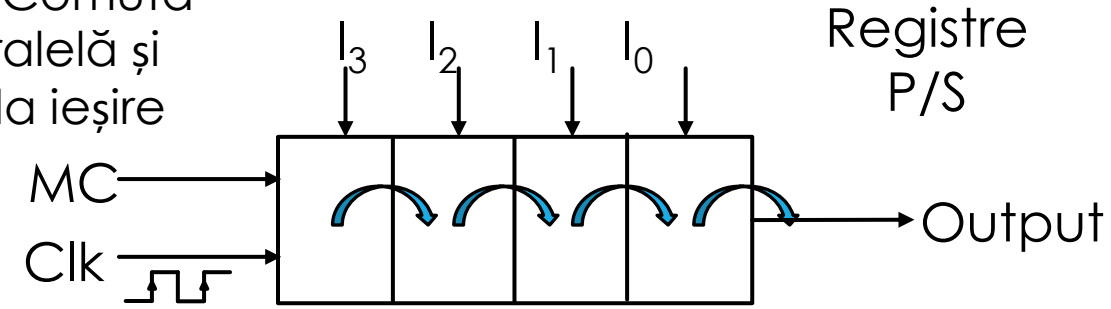
- Cum e organizată memoria?
- Pe adrese, la anumite locații avem diverse cuvinte
- Dacă vrem să facem operații cu 2 cuvinte din memorie, unul trebuie să fie salvat undeva → unde? în registrul de date
- De asemenea, există registru de adrese
- Avem nevoie de registre P/P pentru a prelua informația, a o stoca pentru o anumită perioadă de timp pentru ca ulterior să o pună la dispoziție.



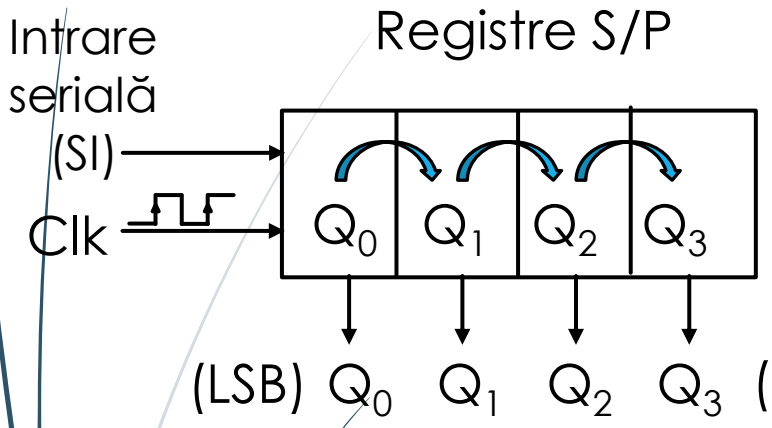
Registre P/S

- Serializare – serializează semnalul de intrare
- Datele de la intrare sunt încărcate paralel
- Prelucrarea se face bit cu bit
- În calculator toate operațiile se fac în mod paralel – serializarea se face pentru transmiterea datelor (ex: imprimantă, fibră optică) → la celălalt capăt semnalul trebuie deserializat cu ajutorul registrelor S/P

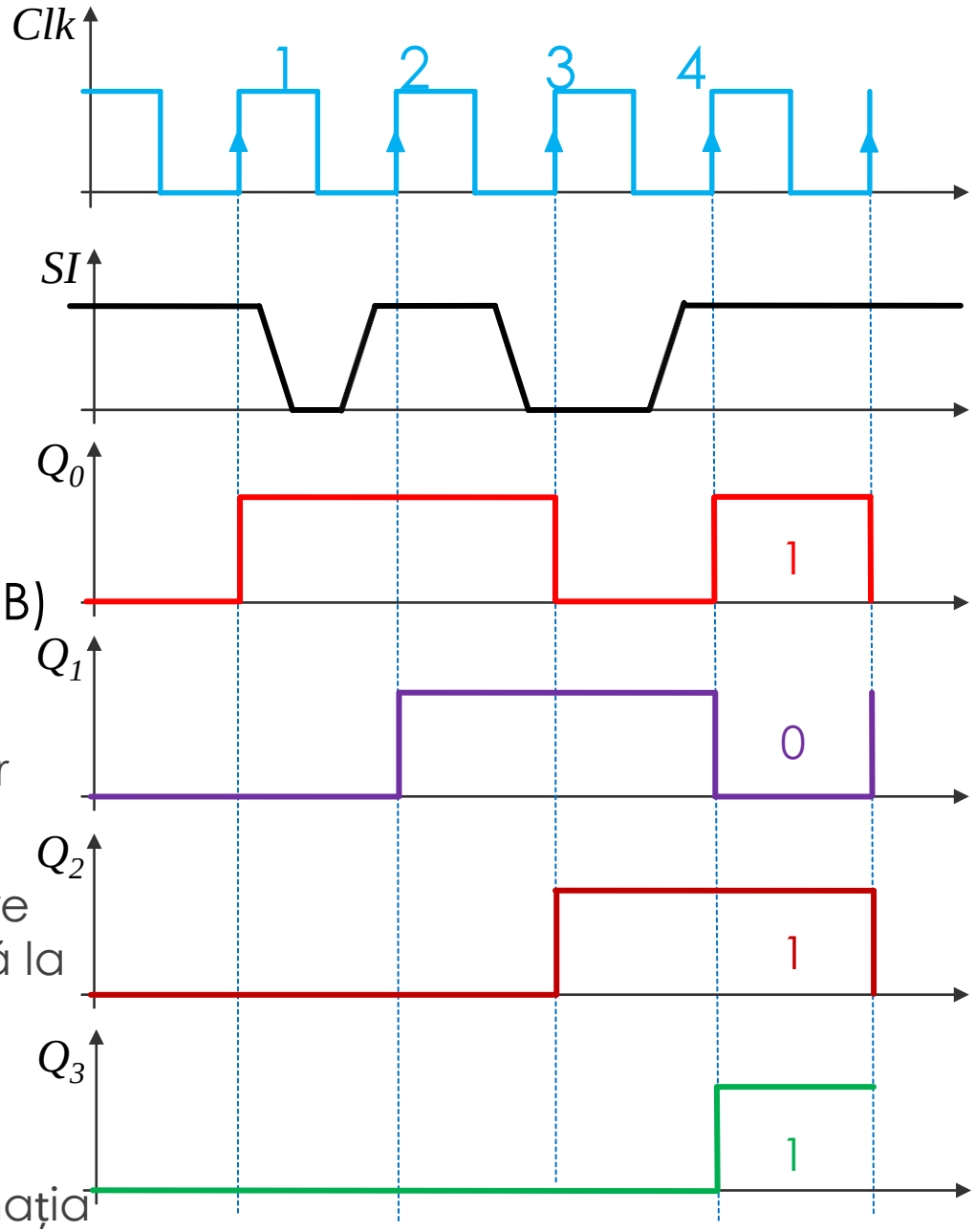
Mode Control (MC) - Comută între încărcarea paralelă și exploatarea serială la ieșire



Registre S/P



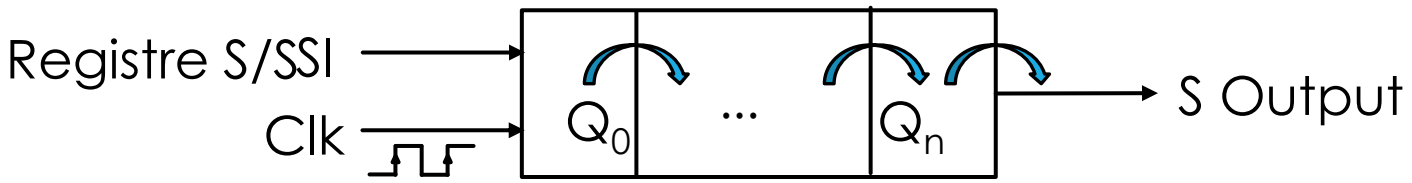
- Rol:deserializarea datelor recepționate
- Alt rol: sincronizarea dintre aparate ce funcționează la frecvențe diferite (ex: procesor și dispozitive auxiliare)
- Buffer – transportă informația între aparate cu frecvențe diferite



Ex: memorarea numărului 1101

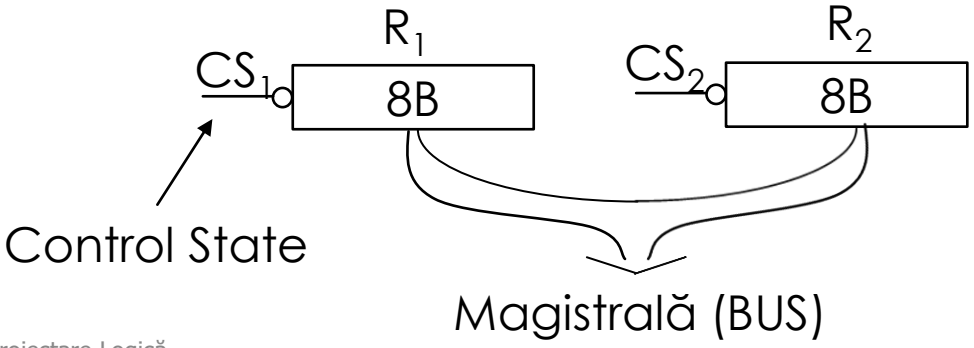
Registre S/S

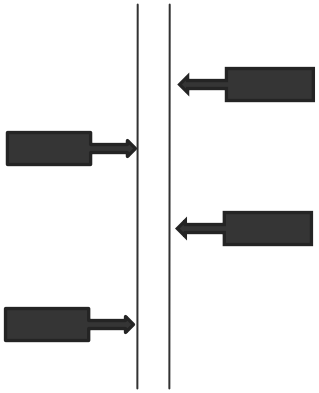
- Cozile de la programare (FIFO)
- Aceste registre se pot construi pe baza CBB-urilor
- Semnalul este transmis sincron în funcție de semnalul de ceas
- Accesul la informație este posibil doar după n tacturi de ceas (n = capacitatea registrului)
- Există și registre care funcționează după principiul stivei (LIFO)



Registre three-state (TS)

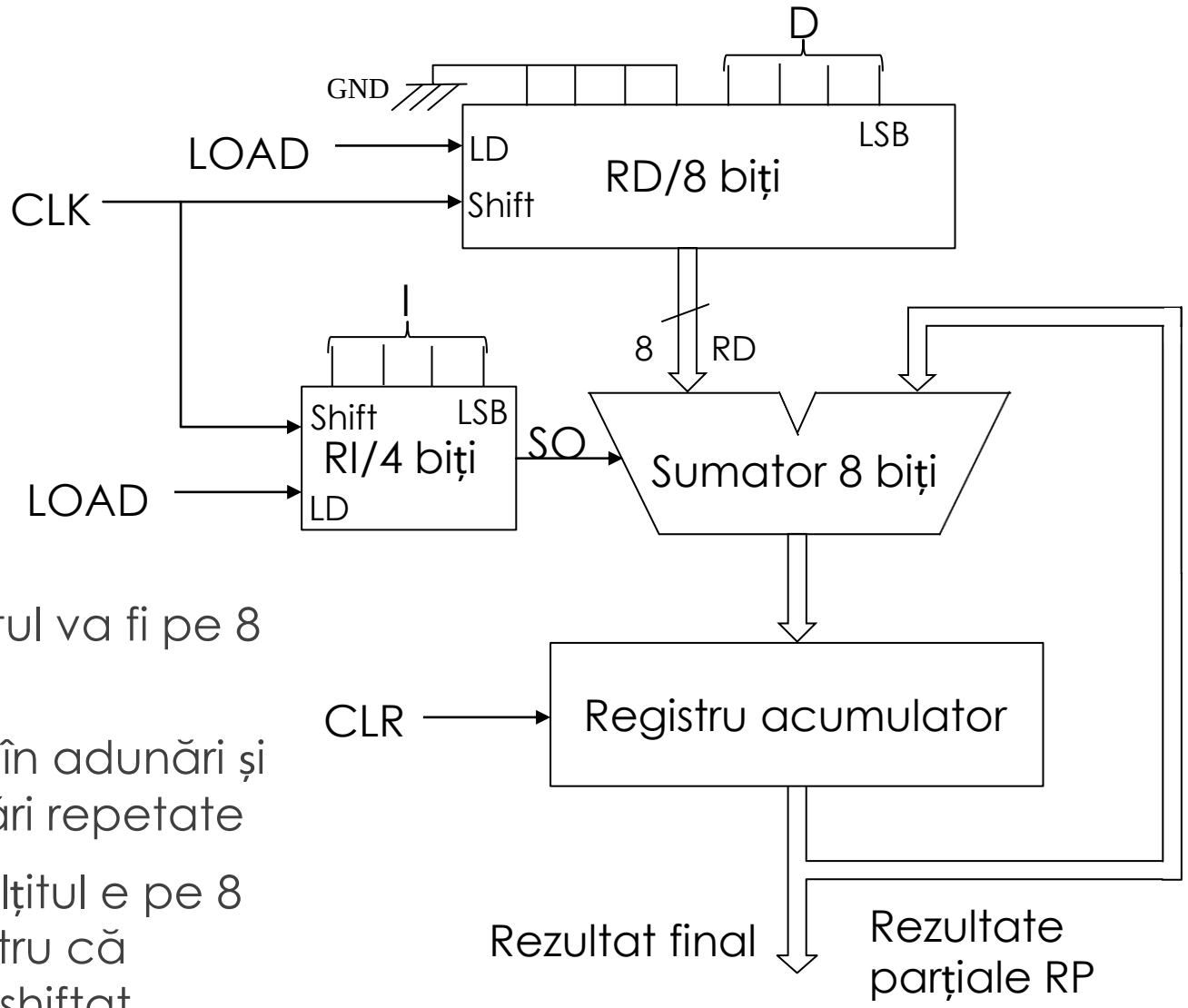
- În mod normal nu avem voie să legăm ieșirile de la 2 circuite diferite, dar acest tip de registre permit acest lucru (fac un ȘI-cablat)
- Folosit pentru a permite accesul la magistrală (transmiterea datelor de la registre la magistrală) → decid cine transmite și cine recepționează informația pe magistrală
- La activare, conținutul registrului trece pe magistrală. Dacă niciun registru nu e activ, magistrala este în TS.





Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (1)

```
  1011*
  1101
  1011
 0000
1011
1011
10001111
```



- Rezultatul va fi pe 8 biți
- Constă în adunări și deplasări repetate
- Deînmulțitul e pe 8 biți pentru că trebuie shiftat

Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (2)

Ex: 1011* 1101

Pas	RI	RD	RP

Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (3)

- Verificare:

```
   1011*
   1101
   1011
  0000
 1011
1011
10001111
```

Proiectare Logică

- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- I CC: Anca Morar anca.morar@cs.pub.ro
- **I CA & CD: Costin Chiru** costin.chiru@cs.pub.ro

Curs 12: Structuri microprogramate: Microinstructiuni cu format fix

Introducere

- Îmbină algoritmii de la programare cu circuitele învățate deja.
- Ce am făcut până acum (ASM-uri) se pretează la un număr mic de stări (ce pot fi implementate folosind maxim 4-5-6 variabile de stare).
- Structurile microprogramate sunt folosite pentru aparate cu un număr mai mare de stări.
- Partea de logică a structurii microprogramate este conținută într-un microprogram

Recapitulare (1)

- La circuitele secvențiale avem 2 elemente importante:
  1. **Stările circuitului**
     - Caracterizate
       - Printr-o configurație unică a variabilelor de stare;
       - De condițiile necesare pentru a ajunge în starea următoare plecând din starea curentă;
       - De ieșirile de la momentul curent de timp.
     - Au o anumită durată (în funcție de ceas la circuitele sincrone sau de durata dintre 2 tranziții la circuitele asincrone).
     - Memorează un anumit parcurs al stărilor (ex: ce biți au fost citiți deja)

Recapitulare (2)

- La circuitele secvențiale avem 2 elemente importante:
  2. **Tranzițiile dintre stări**
     - Pot fi de 2 feluri:
       - Condiționate de variabilele de intrare
       - Necondiționate: se trece direct în starea următoare
     - Sunt considerate a fi instantanee

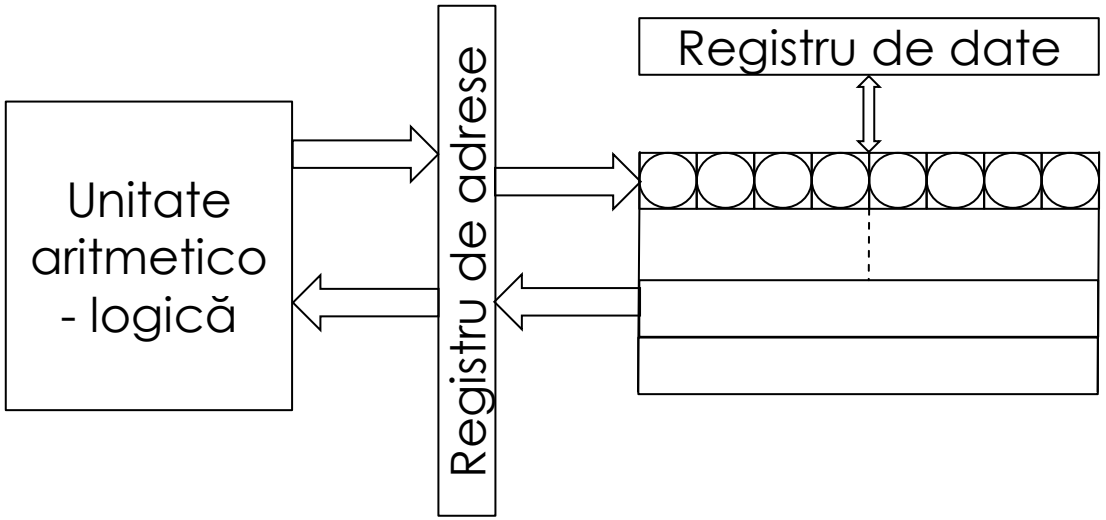
Memoria

- Ce știm despre ea?
- Nu ne-am ocupat explicit de memorii, dar am folosit elemente de memorare
- Unde am întâlnit?
- La circuitele secvențiale: orice circuit secvențial are un element de memorare
- Unde am mai vorbit despre memorii?
- 4 CBB-uri legate între ele pot reprezenta un cuvânt de memorie (de 4 biți)
- Lărgim spectrul – ce știți despre memorie? Unde ați mai întâlnit?
- Programare – stive; alocarea din C

Memoria

- Un circuit de memorie este alcătuit din locații de memorie
- Cum pot să ajung la una sau alta din locațiile de memorie?
- Folosind registrul de adrese RA (orice memorie trebuie să aibă structural un RA)
- Ce facem cu locația de memorie?

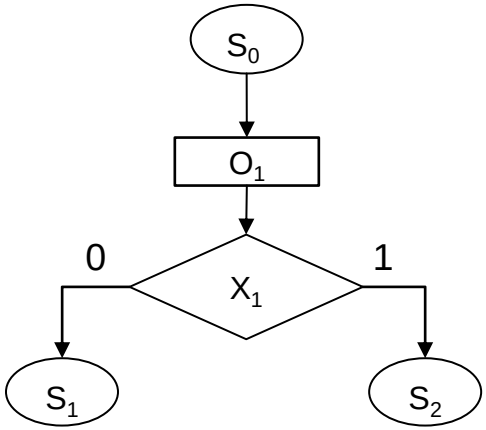
- O punem în registrul de date RD unde salvăm / de unde preluăm informații



Memoria

- Ex: pentru un cuvânt (o locație) de memorie de 8 biți putem avea 8 CBB de tip D, pentru următoarea alte 8 CBB-uri etc.
- În realitate, circuitele sunt LSI sau VLSI și nu mai putem să vedem individual fiecare circuit, dar în principiu fiecare bit este realizat printr-un circuit secvențial

- În fiecare stare dintr-un aparat este memorată informația despre care e starea următoare și eventual despre ieșirile din starea curentă



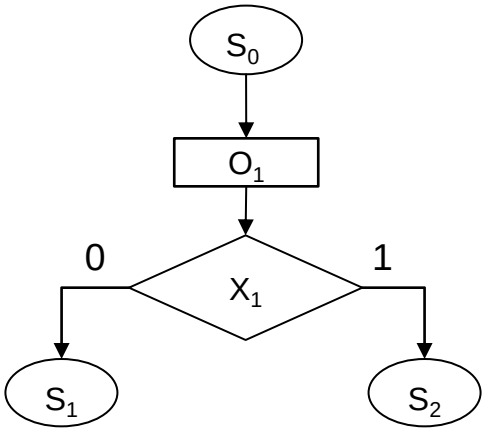
Structuri microprogramate

- Structurile microprogramate merg pe ideea că într-o organigramă avem o succesiune de stări și informația asociată fiecărei stări în parte o memorăm într-o microinstrucțiune
- **Microinstrucțiune** = toate informațiile legate de stare, ce se memorează într-o memorie cu caracter permanent
- Fiecare microinstrucțiune se găsește la o anumită adresă în memorie
- Ansamblul microinstrucțiunilor formează **microprogramul** și se memorează ca o succesiune de microinstrucțiuni

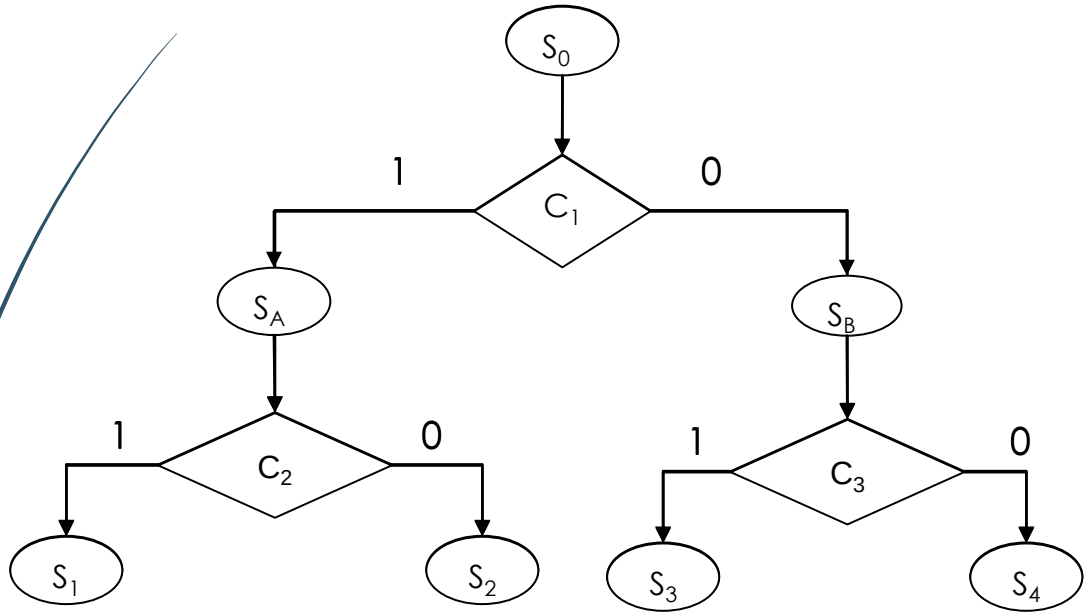
Memorie necesară

- Ex: dacă avem 3 variabile de stare → 8 stări → avem nevoie de o memorie de 8 cuvinte
- dacă avem un ASM cu 10 variabile de stare → 2^{10} = 1024 stări → avem nevoie de o memorie de 1024 de cuvinte (locații)
- Nu știm ce înseamnă o microinstrucțiune, dar știm că fiecare stare are asociată o microinstrucțiune → similar cu liniile dintr-un program (instrucțiuni): fiecare instrucțiune spune ce trebuie să se execute și dacă se trece la următoarea instrucțiune sau sărim la o alta în urma unor decizii (de ex. if).

Microinstrucțiuni



- Structura tipică este
- Ce se întâmplă dacă avem condiții complexe?

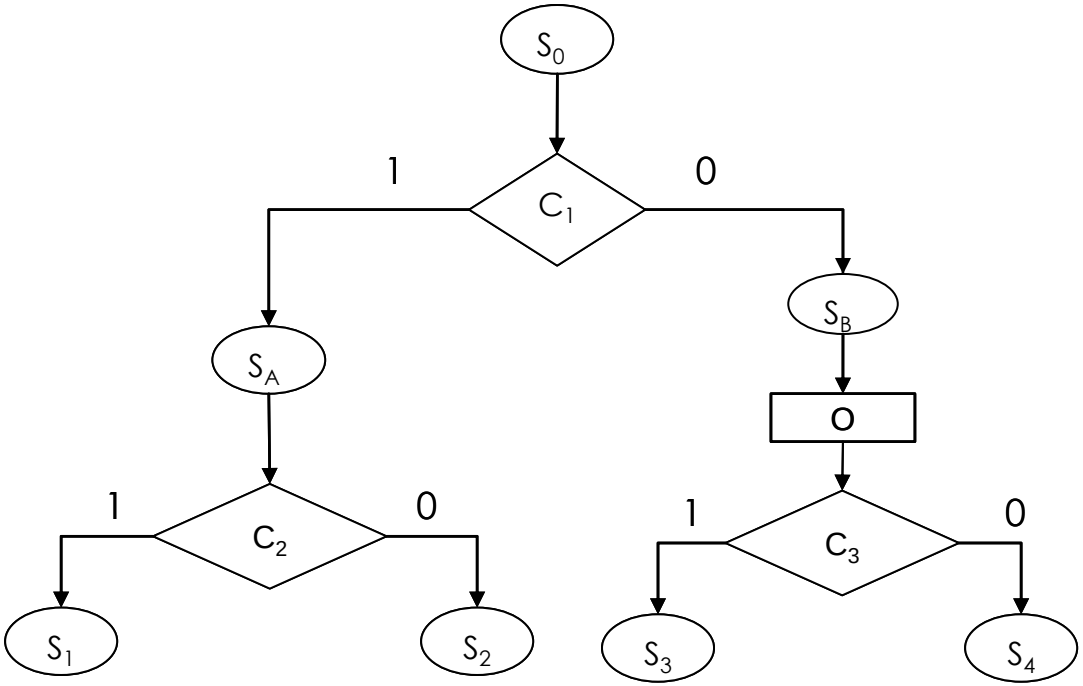


- Aducem organigrama la structura clasică
- Cum?
- Introducând stări noi

Microinstrucțiuni

- Prima diferență față de ASM:
  - La ASM trebuia să facem organigrama cât mai compactă pentru a limita numărul de stări → variabile de stare → Diagrame Karnaugh cât mai mici
  - La structuri microprogramate se lucrează de obicei cu aparate de tip Moore (se poate și cu Mealy dar e mai complicat → se pune mască pe ieșiri și se complică schema). Drept urmare, trebuie să modific schema pentru a transforma zonele de tip Mealy în zone de tip Moore

Mealy - Moore

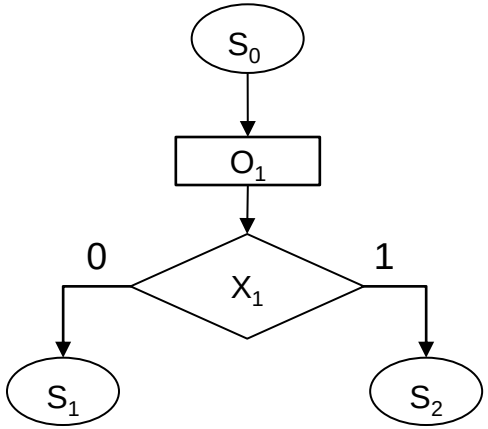


Structura microinstrucțiunii

- O microinstrucțiune are mai multe câmpuri pentru a reține informațiile legate de semnalul de intrare care cauzează tranziția, adresele stărilor următoare si ieșirile generate:

Input Code	ADR 0	ADR 1	OUT

- O parte din informații le scriem codificat, altele le scriem decodificat
- Intrările și stările → codificate
- Ieșirile → decodificate (un bit/ieșire)



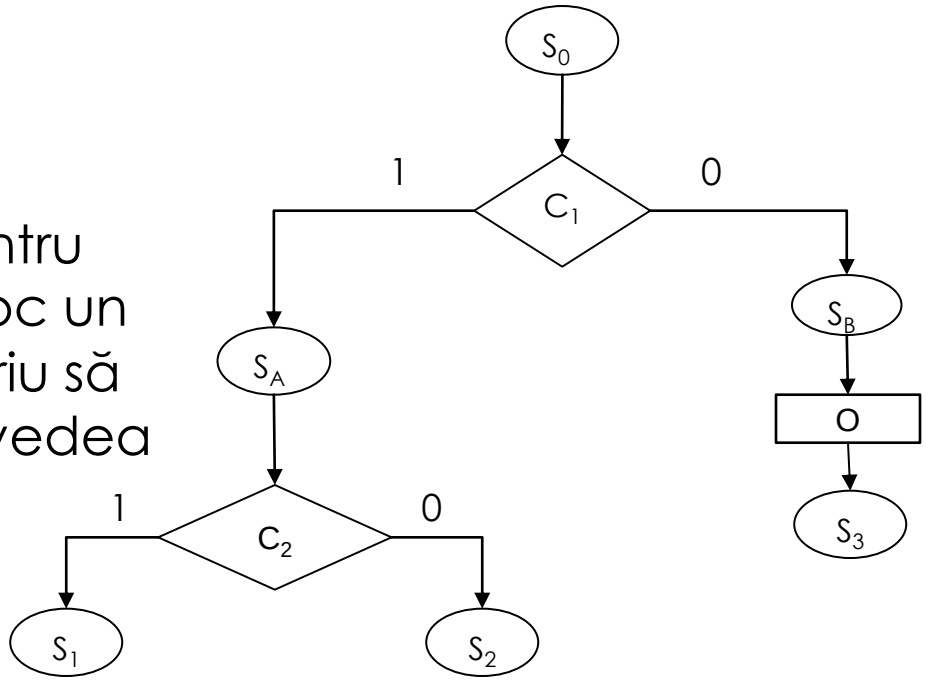
Input code

- Input Code (INPCOD) = codul semnalelor de intrare
- Cum se obține codul?
- Ex: avem 3 intrări: I_1, I_2, I_3 și încerc să le dau coduri binare. Câți biți sunt necesari?
- 2: I_1 - 01, I_2 - 10, I_3 – 11 (codificăm la rând și dacă se poate lăsăm combinația formată doar din 0-uri nefolosită)
- De ce codificăm? Ca să câștigăm biți (reprezentare pe 2 biți în loc de 3)
- Ex2: avem 14 intrări. Câți biți sunt necesari?
- 4 biți → economie de 10 biți

Starea următoare

- Presupunem că avem ceva de genul:
- Din S_B mă duc în S_3, dar din S_0 și S_A mă pot duce în 2 stări și atunci:
  - ADR0 = adresa următoare pentru semnalul de intrare egal cu 0
  - ADR1 = adresa următoare pentru semnalul de intrare egal cu 1

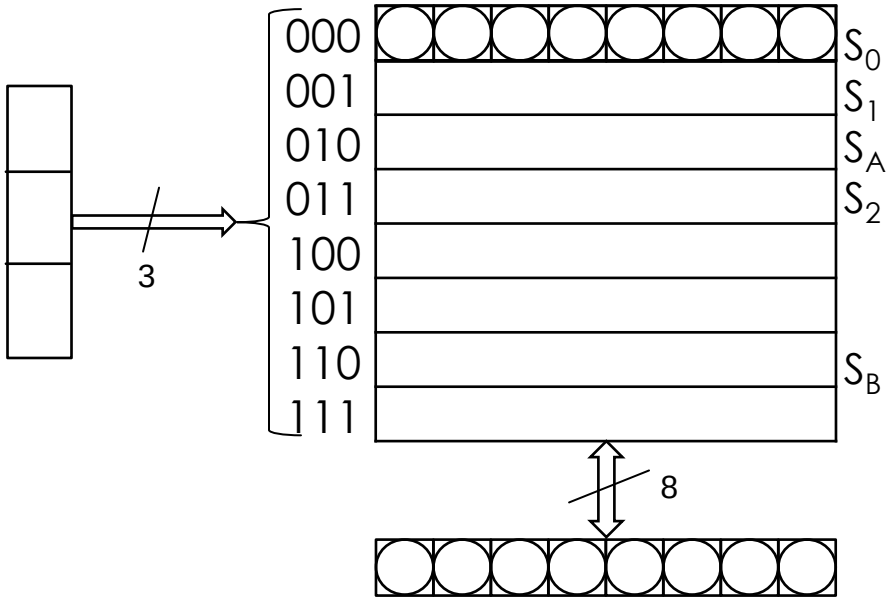
- În memoria microprogramului, pentru fiecare instrucțiune aloc un cuvânt (nu e obligatoriu să fie următorul) → vom vedea mai târziu



Starea următoare

- Exemplu: memorie de 8 cuvinte (3 variabile de stare). Câți biți conține adresa?
- 3
- Ce face adresa?
- Selectează una din instrucțiuni (stări) → e un cod
- Cum știu unde să mă duc mai departe?

- Trebuie să selectez adresa pe care merg
- Ex: dacă sunt în S_0, trebuie să am un aparat la care dacă condiția e 1, RA să genereze codul lui S_A, iar dacă e 0 codul lui S_B



Ieșiri

- OUT = semnalele de ieșire generate în starea curentă
- Nu se folosesc codificări pentru că în fiecare stare pot genera mai multe ieșiri și atunci pot avea ceva de forma:
  - S_0: O1, O4
  - S_1: O1, O2
- Nu se câștigă nimic codificându-le pentru că trebuie să se țină cont și de posibilele combinații de ieșiri → reprezentare fiecare ieșire printr-un bit 0/1

Lungimea microinstrucțiunii

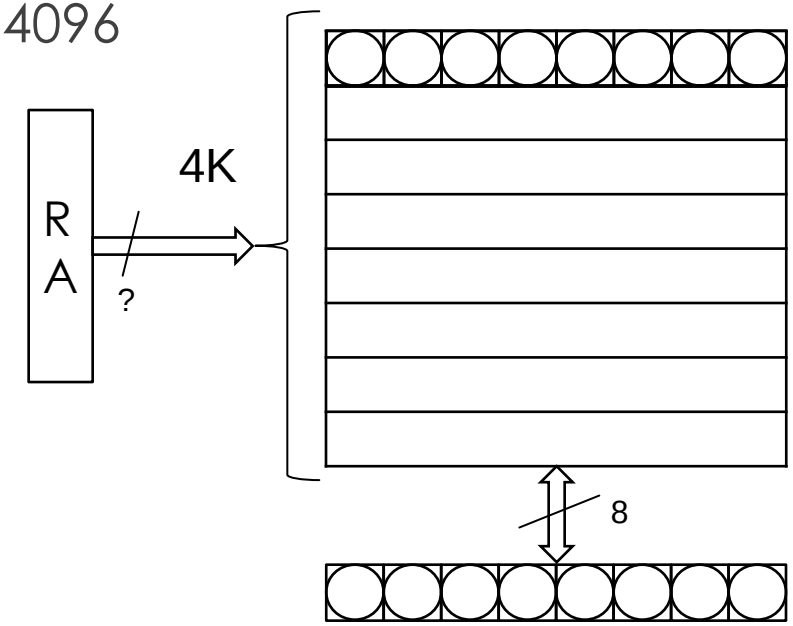
- Având forma microinstrucțiunii, trebuie să calculăm lungimea acesteia, folosind formula:
  - $L_{\mu i} = n_{inpcod} + 2 * n_{ADR} + n_{OUT}$
- Ex: 7 intrări, 25 stări, 5 ieșiri. Care e lungimea microinstrucțiunii?
- Câți biți sunt necesari pentru codificarea intrărilor? 3
- Câți biți sunt necesari pentru codificarea stărilor?
- Fiecare stare = 1 cuvânt din memorie → avem 25 de cuvinte în memorie → 5 biți necesari pentru reprezentare → 5 biți pentru RA
- $L_{\mu i}$ = 3 + 5 + 5 + 5 = 18 biți
- Teoretic am nevoie de o memorie cu RA de 5 biți, RD de 18 și capacitate de 25 de cuvinte a câte 18 biți fiecare
- În realitate, memoriile sunt standardizate → nu pot avea orice număr de biți pentru RA & RD & capacitate memorie

Problemă (1)

- Fie un circuit de memorie de 4K * 8 biți. Calculați lungimea microinstrucțiunii și stabiliți câte astfel de circuite de memorie sunt necesare dacă se știe că avem 37 de semnale de intrare și 25 de semnale de ieșire.
- Ce știm?
- Memoria are 4K * 8 (lungimea cuvântului)
- Ce lipsește?
- Numărul de stări.
- Care e numărul maxim de stări?

Problemă (2)

- Câte stări poate avea microprogramul?
- Maxim 4K adrese în memorie = maxim $2^{12} = 4096$ stări posibile → numărul maxim de stări
- Pentru adresarea lor: 12 biți
- RA → 12 biți → pot adresa 4096 poziții → aparatul poate avea maxim 4096 stări
- $L_{\mu i} = n_{ci} + 2 * n_{adr} + n_{out}$
- 37 intrări, 25 ieșiri
- $L_{\mu i} = 6 + 2 * 12 + 25 = 55$ biți



Problemă (3)

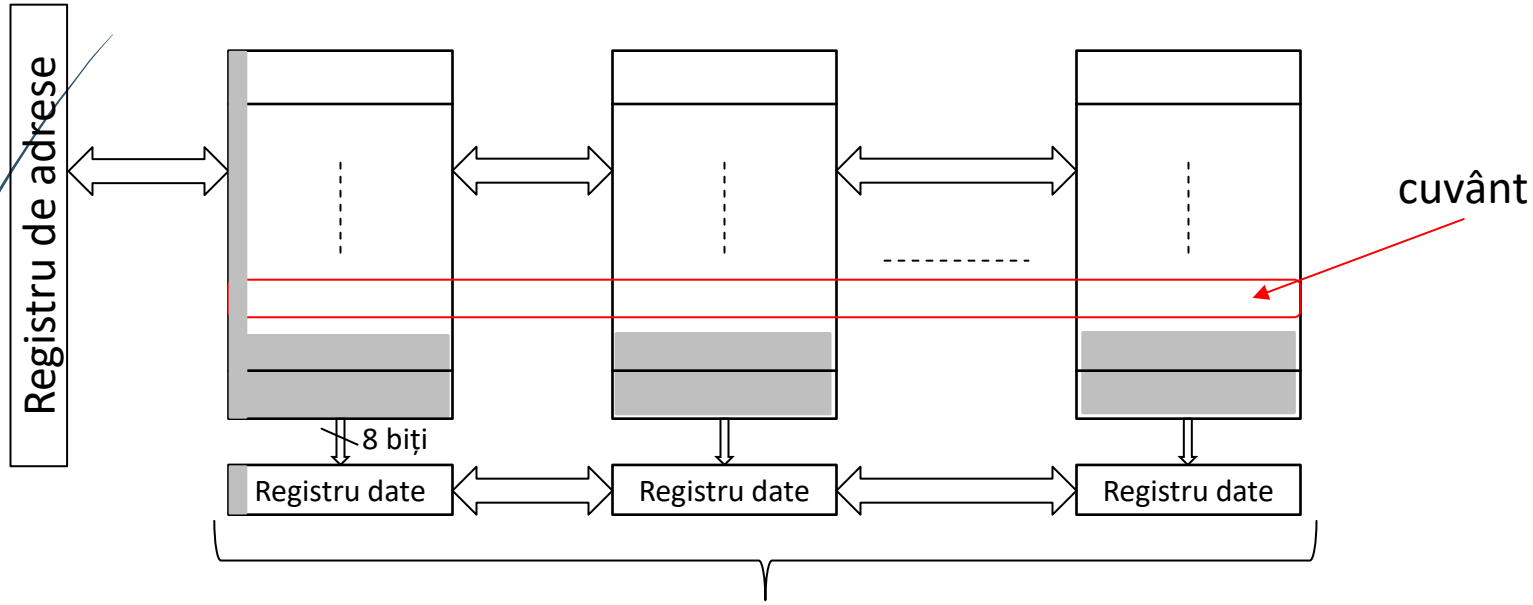
- Pot folosi această memorie pentru un aparat cu 16 stări?
- Da
- Se modifică lungimea microinstrucțiunii?
- Nu, pentru că ele sunt adresate în continuare folosind RA care trebuie să poată să numere pâna la 4095 (chiar dacă majoritatea locațiilor vor fi libere).

Problemă (4)

- De unde vine întrebarea a doua: de câte circuite avem nevoie?
- $L_{\mu i}$ = 55 biți, dar în memorie eu am cuvinte de maxim 8 biți → ce facem?
- Luăm mai multe astfel de circuite și le cuplăm în paralel (le „concatenăm")
- Un cuvânt este reprezentat de concatenarea RD-urilor
- Câte circuite sunt necesare? [55/8] = 7 (1 bit din RD cel mai semnificativ rămâne nefolosit)
- Toate RD sunt comandate simultan de același RA și datele sunt selectate și copiate simultan în RD - uri. Concatenarea lor conduce la cuvântul microinstrucțiunii

Problemă (5)

- Ce se întâmplă dacă aparatul are mai puține stări (nu 4096)?
- Există o zonă de memorie nefolosită în fiecare dintre circuite
- Dacă $L_{\mu i}$ nu este multiplu al dimensiunii cuvântului memoriei, rămâne o zonă de memorie nefolosită în memoria corespunzătoare MSB



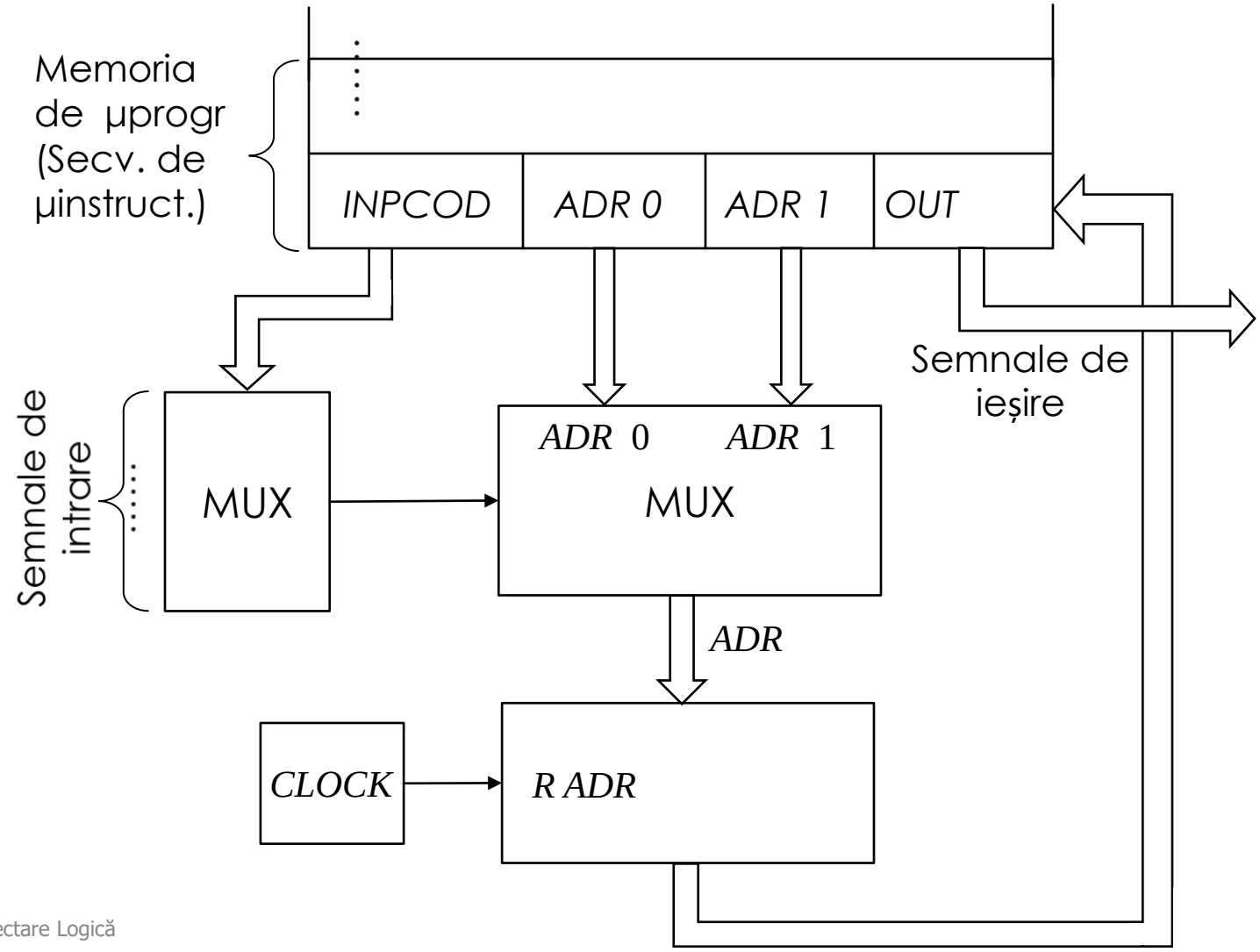
55 biți
7 capsule
8*7 = 56 : 55

Problemă (6)

- Alt exemplu: memorii 1K * 8 → avem 1024 stări → RA are 10 biți
- $L_{\mu i} = 6 + 2 * 10 + 25 = 51$ biți
- Avem nevoie tot de 7 circuite, 5 biți din memoria corespunzătoare MSB rămânând nefolosiți
- Alt exemplu: 1024 stări, dar memoria 4K * 8 → $L_{\mu i} = 55$ biți → 7 circuite, 1 bit nefolosit în memoria corespunzătoare MSB și ¾ din fiecare din cele 7 circuite de memorie rămane nefolosită

Structura de comandă microprogramată (1)

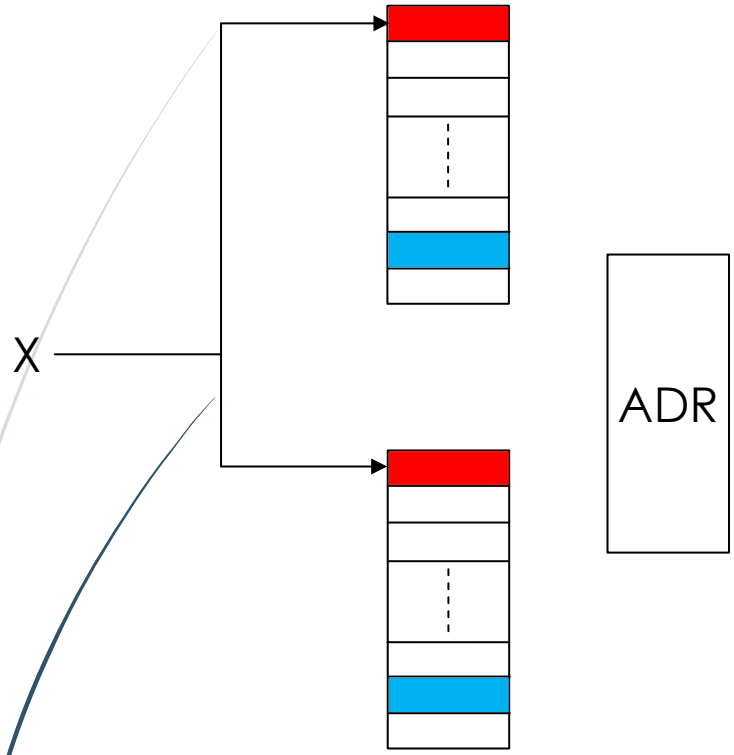
- Până acum am văzut cum se organizează memoria de microprogram → am văzut unde se află informația.
- Acum vom vedea ce informație punem în fiecare locație de memorie pentru a se înlănțui corect stările și pentru a scoate corect ieșirile

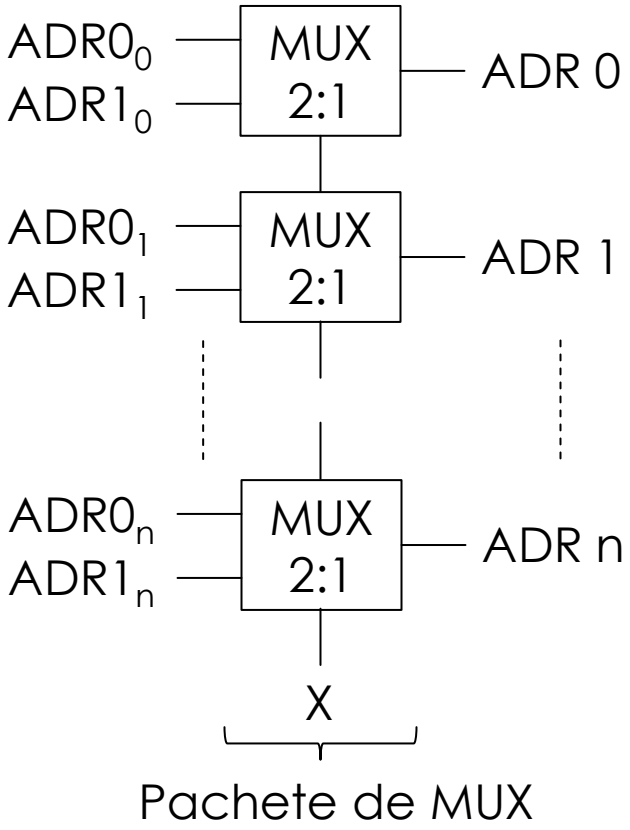


Structura de comandă microprogramată (2)

- INPCOD reprezintă codul de selecție pentru MUX (îmi va selecta una dintre intrări)
- Pe intrările acestuia se vor pune intrările circuitului (semnalele de intrare): I_0 ... I_n (în cazul nostru I_0 ... I_{36})
- Ieșirea MUX va reprezenta valoarea uneia dintre decizii (→ cea care decide dacă mă duc la ADR0 sau la ADR1) și intră într-o structură mai specială MUX 2:1. Această structură este de fapt o baterie de MUX, dar care nu sunt cuplate în cascadă

Bateria de MUX





- Pt X = 0 se obțin ADR0$_0$, ADR0$_1$ ... ADR0$_n$
- Pt X = 1 se obțin ADR1$_0$, ADR1$_1$ ... ADR1$_n$
- Concatenând acești biți se obține următoarea adresă

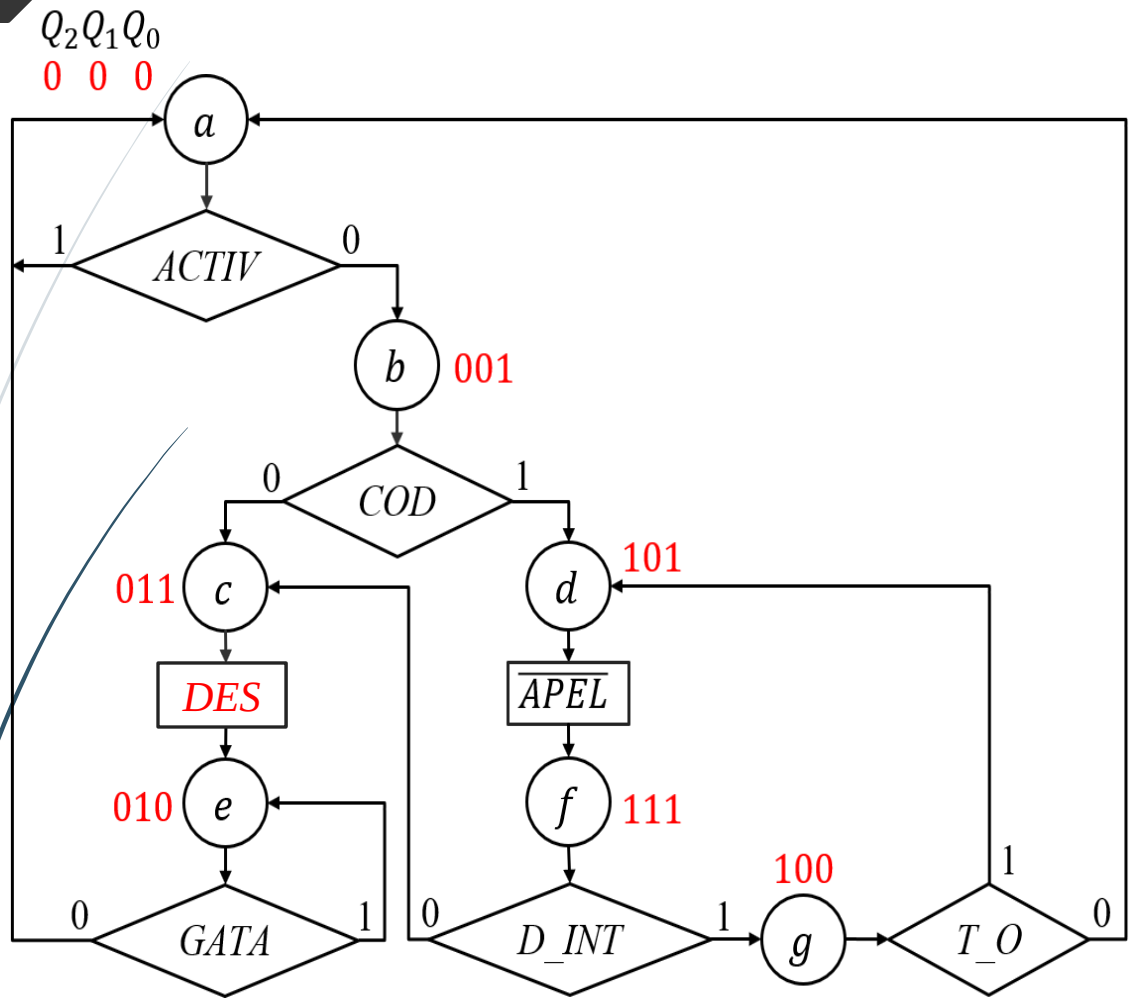
Structura de comandă microprogramată (3)

- În microprogram avem o succesiune de microinstrucțiuni
- La un moment dat, se activează o microinstrucțiune
- Din câmpul INPCOD se selectează una dintre intrări, aceasta putând fi 0 sau 1
- Mai departe, bateria de MUX generează în RADR adresa corespunzătoare din microinstrucțiune și microprogramul continuă cu acea adresă
- Așa se trece prin întregul microprogram
- Dacă am prea multe intrări, MUX inițial poate fi de fapt o cascadă de MUX-uri astfel încât să se acopere toate intrările

Examen

- Dacă vi se cere să implementați cu structuri microprogramate, trebuie neapărat să:
- Calculați lungimea microinstrucțiunii
- Completați memoria microprogramului:
  - alegeți tipul de memorie (e posibil să vi se ceară explicit un anumit tip de memorie)
  - să spuneți câte circuite de memorie sunt necesare

Problema cu interfonul



- Câte intrări?
- 5
- Câți biți?
- 3
- Care sunt intrările?
- ACTIV, COD, GATA, D_INT, T_O
- Câte stări?
- 7
- Câți biți?
- 3
- Câte ieșiri?
- 2 (!APEL, !DES)
- Cât e $L_{\mu i}$?

$L_{\mu i} = 3 + 2 * 3 + 2 = 11$ biți

Problema cu interfonul – memoria de microprogram (1)

- Am făcut o modificare: am pus DES și !APEL în organigramă ca să vedem cum sunt tratate atât ieșirile active pe 0 cât și cele active pe 1
- Trebuie să completăm tabelul:

- Atenție! Codificarea stărilor nu are nicio legătură cu numărul cuvântului din memorie.
- Se poate folosi același cod în memorie ca în organigramă, dar acest lucru nu e obligatoriu
- Noi vom folosi aceeași codificare

	INPCOD | ADR0 | ADR1	DES	APEL
000			
001			
010			
011			
100			
101			
110			
111			

Problema cu interfonul – memoria de microprogram (2)

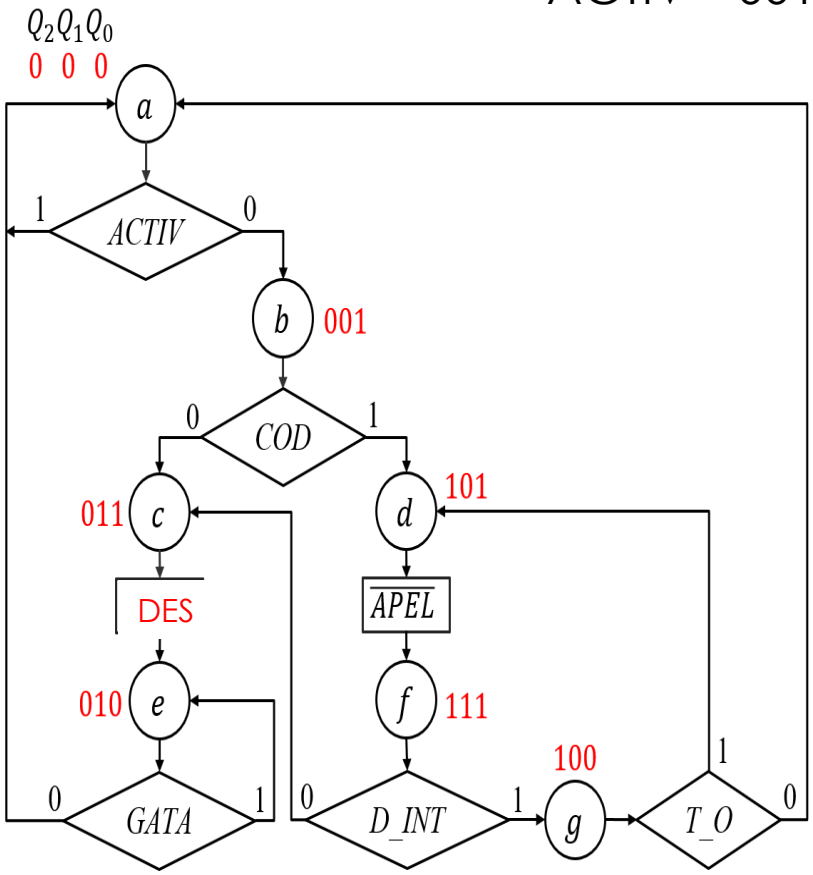
- Primul lucru: trebuie să codificăm intrările → când facem tabelul, trebuie specificat codul fiecărei intrări
- ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101

- La ieșiri: în 000 nu se generează ieșiri → DES (activ pe 1) rămâne în 0 în timp ce APEL (activ pe 0) rămâne în 1

	INPCOD	ADR0	ADR1	DES	APEL
a – 000	001	001	000	0	1
001					
010					
011					
100					
101					
110					
111					

Problema cu interfonul – memoria de microprogram (3)

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101



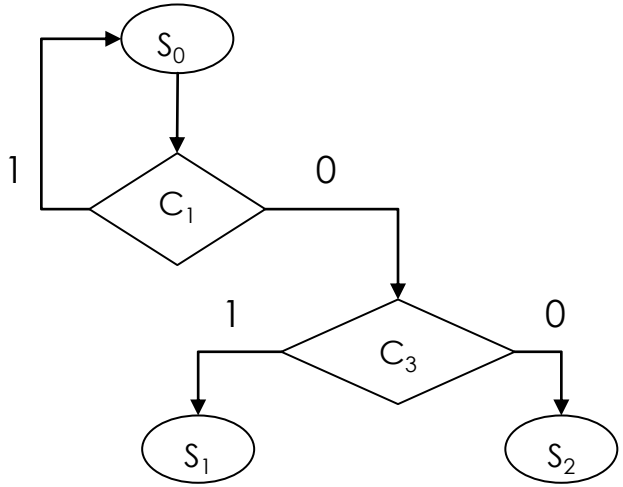
	INPCOD	ADR0	ADR1	DES	APEL
a – 000	001	001	000	0	1
b – 001	010	011	101	0	1
e - 010	011	000	010	0	1
c - 011	000	010	010	1	1
g - 100	101	000	101	0	1
d - 101	000	111	111	0	0
- - 110	xxx	xxx	xxx	x	x
f - 111	100	011	100	0	1

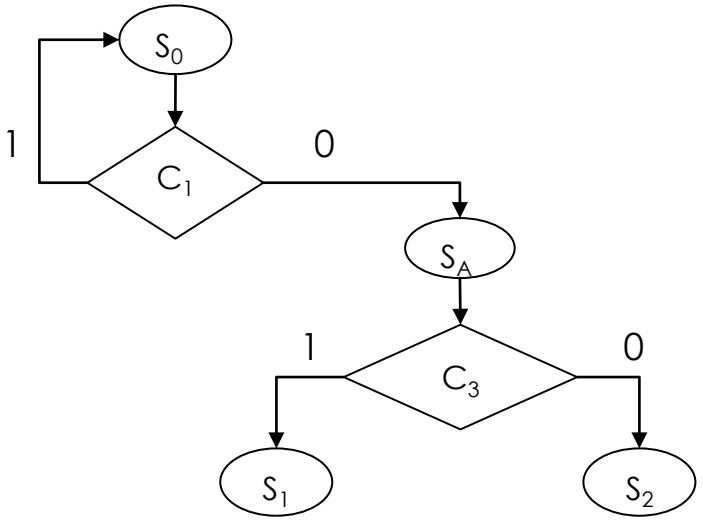
- Nu rămân mereu coduri nefolosite (intrări = $2^{\text{biți}}$), iar dacă nu rămân, punem un cod oarecare
- La salturile necondiționate se pune același lucru pe ADR0 și ADR1

- Ce se întâmplă în C?
- Putem pune XXX sau, dacă rămân coduri nefolosite, pun 000 → de asta începem codificarea de la 001
- 000 va fi codul pentru salturile necondiționate

Examen

- Mai pot apărea probleme de genul
- Cum rezolvăm?
- Trebuie să adăugăm stări suplimentare (S_A) pentru că în MUX-ul inițial nu se generează semnal decât de pe o singură intrare și de aceea nu pot avea condiții compuse





Dezavantaj microinstrucțiuni cu format fix

- Lungimea microinstrucțiunii poate să devină prea mare
  - Interfon: 7 stări, $L_{\mu i}$ = 11 → dacă avem circuite de memorare pe 4 biți avem nevoie de 3, dacă sunt pe 8 biți avem nevoie de 2
- Data viitoare vedem cum se poate scrie microinstrucțiunea pe mai puțini biți
- Observație legată de ieșiri:
  - În exemplul nostru, informația utilă apare doar in 2 locuri: când !APEL devine 0 și când DES devine 1
  - În rest, este informație irosită
  - Vom vedea cum în loc să punem această informație, punem altceva astfel încât să se micșoreze lungimea microinstrucțiunii



Proiectare Logică

- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- **I CC: Anca Morar** anca.morar@cs.pub.ro
- I CA & CD: Costin Chiru costin.chiru@cs.pub.ro

Curs 13: Structuri microprogramate: microinstructiuni cu format variabil

Recapitulare (1)

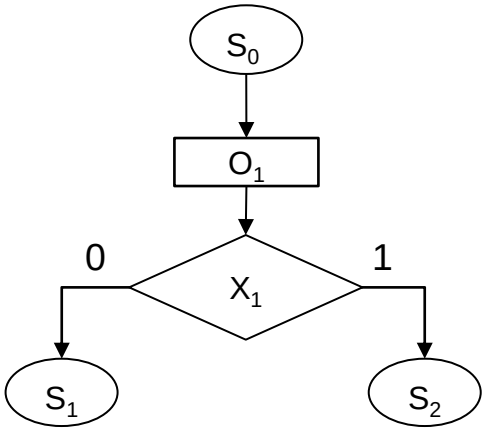
- De-a lungul semestrului ați învățat să faceți sinteza circuitelor folosind CBB-uri și identificând ecuațiile de stare și ecuațiile de intrare/ieșire din CBB-uri
  - Problemă: se poate aplica această metodă pentru un număr relativ mic de stări (altfel diagramele Karnaugh devin prea mari)
- Am făcut cursul trecut o altă metodă de sinteză a circuitelor folosind structuri microprogramate
- Ideea de bază e că avem o succesiune de stări și tranziții (condiționate de variabile de intrare sau necondiționate) ce pot fi salvate în structurile microprogramate

Recapitulare (2)

- În practică, automatele cu multe stări sunt frecvente și nu putem să le „spargem” pentru că nu știm să lucrăm cu mai mult de 5 variabile → de aceea se folosesc structurile microprogramate
- Alt avantaj al structurilor microprogramate este fiabilitatea crescută
- Microinstrucțiunile sunt cuprinse în microprogram, acesta fiind salvat în memorie
- Data trecută am făcut microinstrucțiuni cu format fix și am spus că vom învăța o metodă de a elimina informația redundantă

Microinstrucțiuni cu format fix (1)

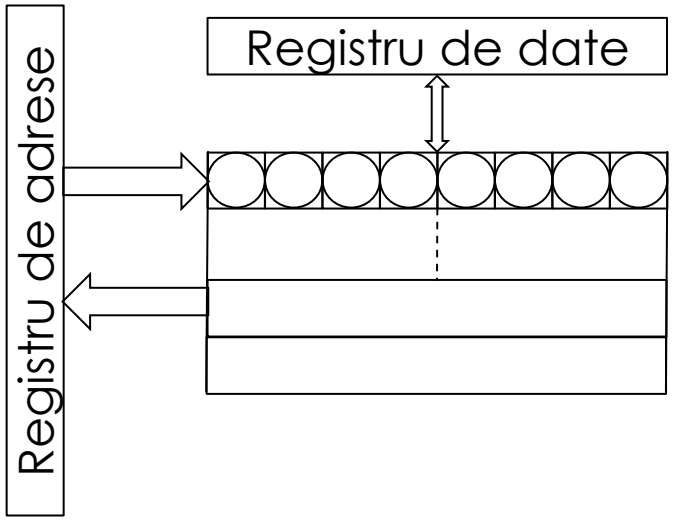
- Ce trebuie să conțină?
- Adresa stării curente, informații referitoare la condiție, unde pot merge mai departe și informații despre eventualele ieșiri



Input Code	ADR 0	ADR 1	OUT

- $L_{\mu i} = n_{inpcod} + 2 * n_{ADR} + n_{OUT}$

- RA selectează adresa, iar prin intermediul RD se scriu/citesc date în/din memorie



Microinstrucțiuni cu format fix (2)

Input Code	ADR 0	ADR 1	OUT



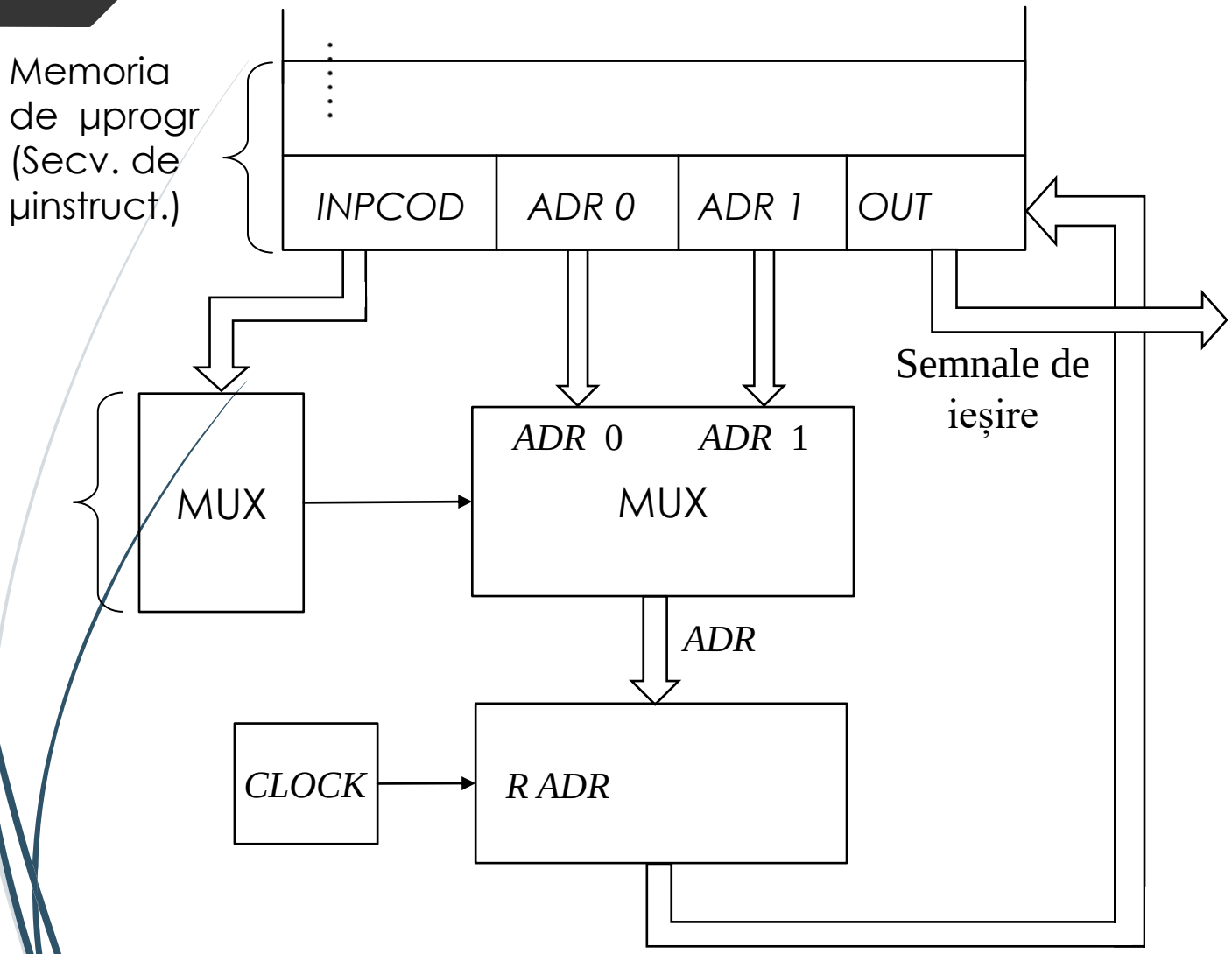
- Cele n intrări sunt codificate, iar codul lor e folosit în prima parte a microinstrucțiunii pentru a stabili care dintre intrări e responsabilă de tranziție
- ADR0, ADR1 – adresele pe 0/1 = locul unde se „sare" cu fluxul de control al aparatului
- OUT = ieșirile, nu se codifică → numărul de ieșiri = numărul de biți OUT necesari
- $L_{\mu i}$ este suma biților necesari pentru fiecare câmp în parte

Microinstrucțiuni cu format fix (3)

- Avem 2 situații posibile:

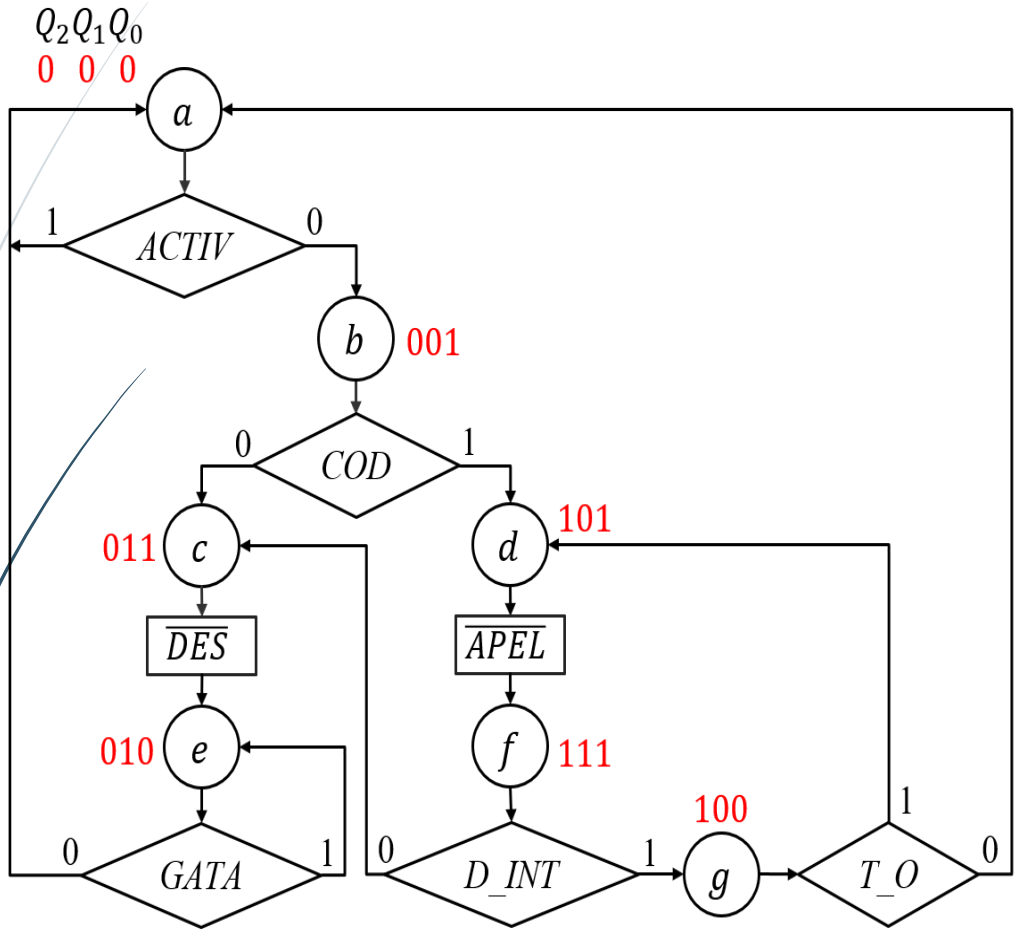
1. Când memoria este impusă și atunci pe baza acesteia determinăm numărul de biți pentru RA. Lungimea cuvântului de memorie poate fi oricât de lung și atunci folosim mai multe circuite de memorie în paralel.
   Ex: memorie de 1k = 2^{10} → RA are 10 biți
   Ex2: memorie de 8 cuvinte = 2^3 → RA are 3 biți
2. Pornind de la organigramă → numărul de stări → numărul de variabile de stare → numărul de biți al RA. Dacă sunt mai multe zone de memorie decât stări, atunci rămân zone nefolosite.
   (Nu este necesar să stabilim coduri pentru stări care să difere printr-un singur bit, ca la CBB-uri!)

Structura de comandă microprogramată



Observație: trebuie să știți să explicați structura de comandă microprogramată, dar nu e nevoie să o reproduceți → trebuie doar să știți să calculați $L_{\mu i}$ și să completați memoria de microprogram

Problema cu interfonul

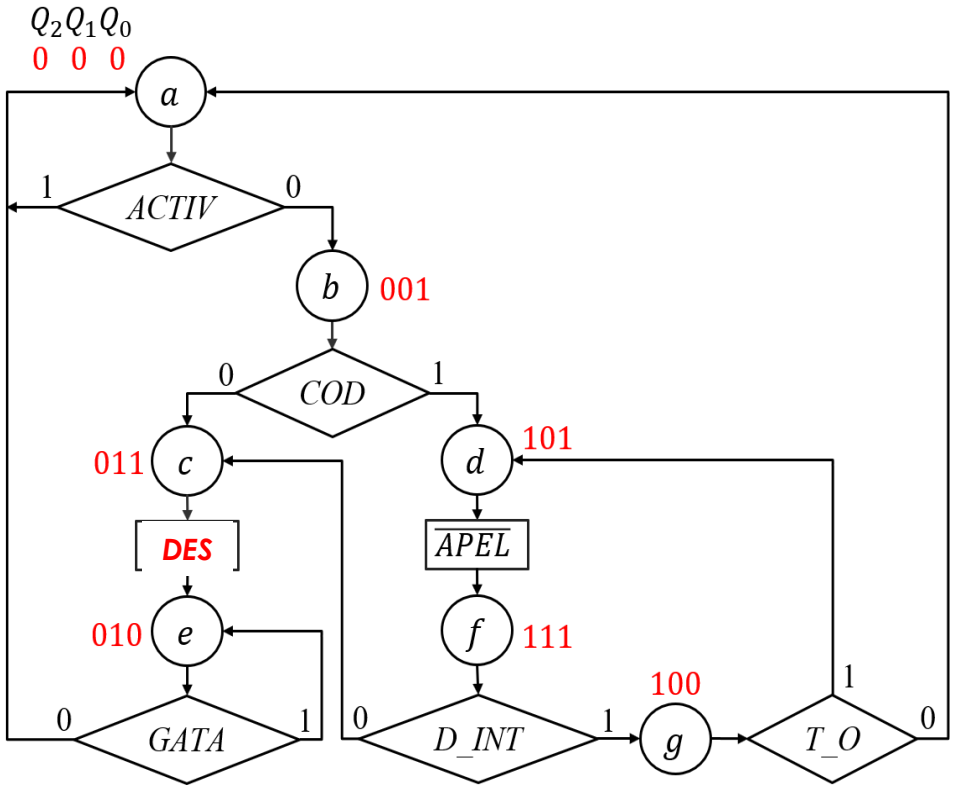


- Câte intrări?
- 5
- Câți biți?
- 3
- Care sunt intrările?
- ACTIV, COD, GATA, D_INT, T_O
- Câte stări?
- 7
- Câți biți?
- 3
- Câte ieșiri?
- 2 (!APEL, !DES)
- Cât e $L_{\mu i}$?

$L_{\mu i} = 3 + 2 * 3 + 2 = 11$ biți

Problema cu interfonul – memoria de microprogram (3)

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101



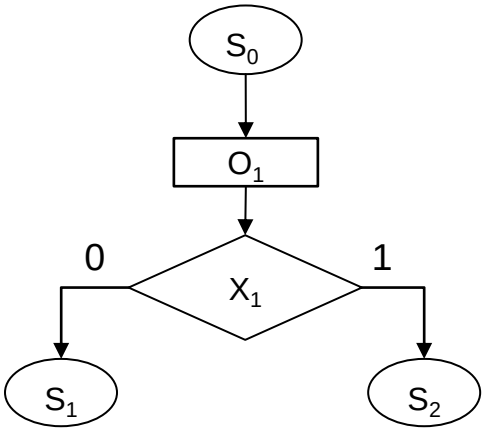
	INPCOD	ADR0	ADR1	DES	APEL
a - 000	001	001	000	0	1
b - 001	010	011	101	0	1
e - 010	011	000	010	0	1
c - 011	000	010	010	1	1
g - 100	101	000	101	0	1
d - 101	000	111	111	0	0
- - 110	xxx	xxx	xxx	x	x
f - 111	100	011	100	0	1

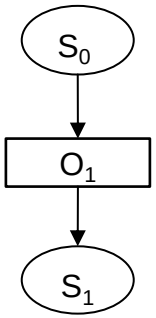
- Nu rămân mereu coduri nefolosite (intrări = $2^{\text{biți}}$), iar dacă nu rămân, punem un cod oarecare
- La salturile necondiționate se pune același lucru pe ADR0 și ADR1

- Ce se întâmplă în C?
- Putem pune XXX sau, dacă rămân coduri nefolosite, pun 000 → de asta începem codificarea de la 001
- 000 va fi codul pentru salturile necondiționate

Microinstrucțiuni cu format variabil (1)

- În cazurile semnalate se pierde spațiul de memorie fie prin specificarea de ieșiri inutile, fie prin dublarea adresei următoare → încercăm să optimizăm memoria folosită prin folosirea microinstrucțiunilor cu format variabil
- Există 2 tipuri de structuri în cazul microinstrucțiunilor cu format variabil:

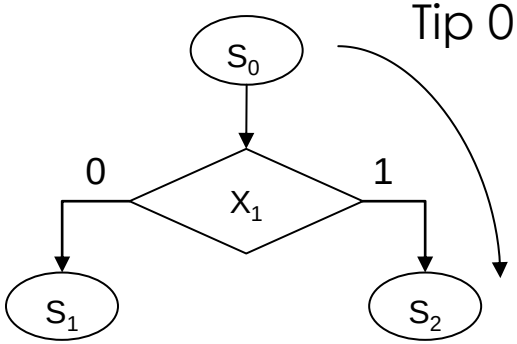




Tip 1

adr S1 = adr S0 +1

Se generează doar ieșiri
Saltul este necondiționat



Tip 0

adr S_2 = adr S_0 + 1

Doar decizie
Nu avem ieșiri

Microinstrucțiuni cu format variabil (2)

- Dacă avem o structură de Tip 1, avem nevoie doar de adresa următoare și de ieșiri, nu de 2 adrese + inpcod + ieșiri
- Dacă avem o structură de Tip 0, atunci avem nevoie de 2 adrese + inpcod, dar nu mai avem ieșiri
- În plus, dacă la RA de la memorie avem un numărător, atunci am putea avea adrese consecutive:
  - la Tip 1: adr S_1 = adr S_0 + 1 și ne interesează doar ieșirile,
  - la Tip 0: adr S_3 = adr S_1 + 1 și ne interesează doar intrarea (inpcod) și ADR_0
- Convenție: la microinstrucțiuni de Tip 0, când intrarea este 1, adresa următoare este adresa curentă + 1

Microinstrucțiuni cu format variabil (3)

- Având o astfel de memorie cu numărător la RA:
- $L_{\mu i\ Tip\ 1} = 1 + n_{out}$

1 | ieșiri

- 1 este pentru tipul instrucțiunii
- $L_{\mu i\ Tip\ 0} = 1 + n_{ci} + n_{adr0}$

0 | INPCOD | ADR0

- De cele mai multe ori microinstrucțiunile de Tip 1 și Tip 0 vor avea lungime diferită pentru că depind de logica programului (număr de stări, număr de ieșiri, număr de variabile de intrare)
- Pentru a determina $L_{\mu i}$ se va considera maximul dintre cele 2 valori:

$$L_{\mu i} = \max\left(L_{\mu i\ Tip\ 1}, L_{\mu i\ Tip\ 0}\right)$$

Microinstrucțiuni cu format variabil (4)

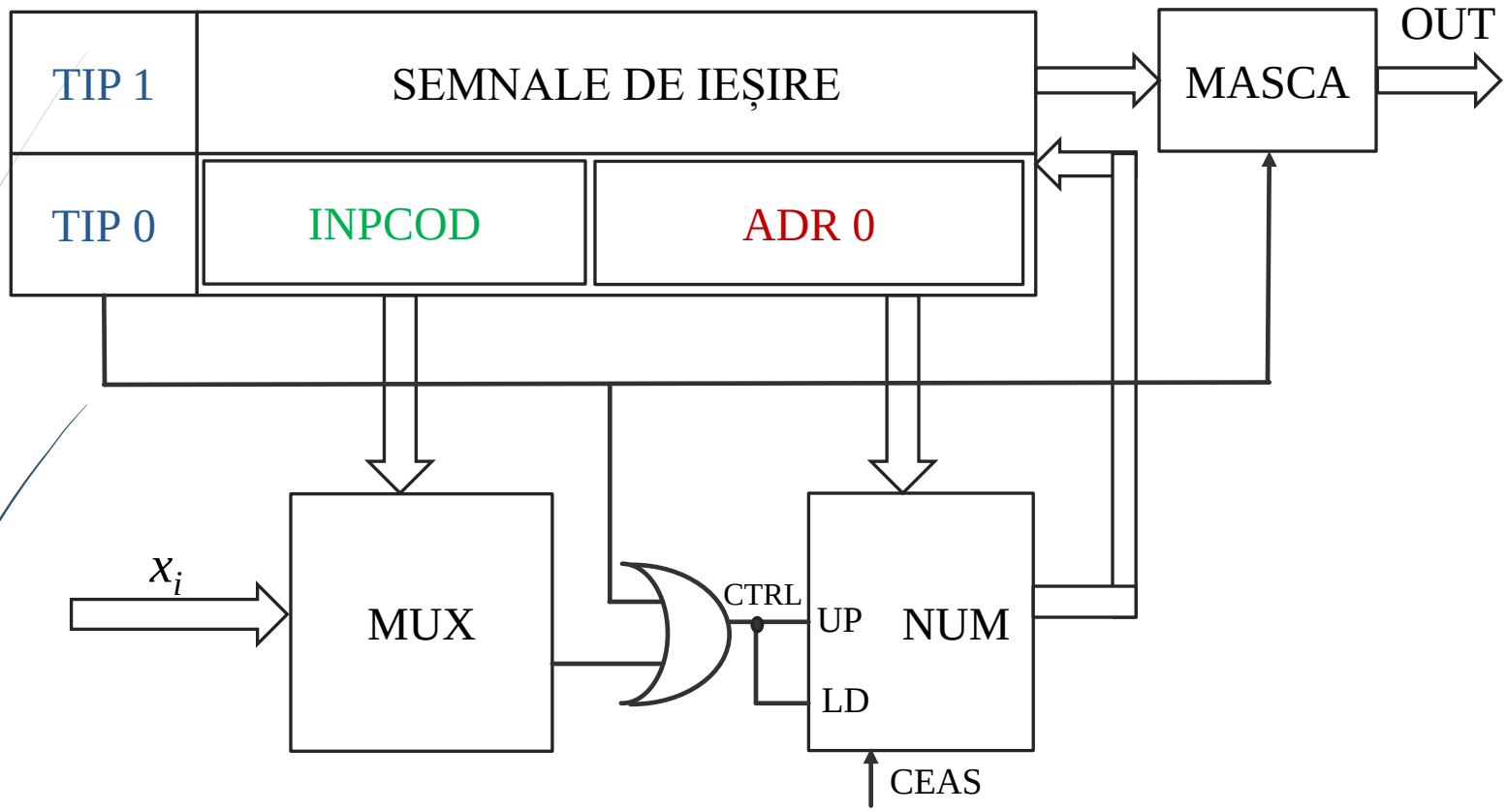
- Ex: adrese pe 4 biți, tot 4 biți pentru codul variabilelor de intrare și 30 ieșiri:
  - $L_{\mu i\ Tip\ 1} = 1 + 30 = 31$
  - $L_{\mu i\ Tip\ 0} = 1 + 4 + 4 = 9$
  - $L_{\mu i} = \max(L_{\mu i\ Tip\ 1}, L_{\mu i\ Tip\ 0}) = 31$

- Pentru asta:
  1. Memoria trebuie să știe să diferențieze microinstrucțiunile de tip 1 și 0 între ele (se modifică unitatea de comandă) și
  2. Trebuie modificată organigrama pentru a avea numai instrucțiuni de tip 1 și 0 prin adăugare de stări.

Structura de comandă microprogramată (1)

- Trebuie să ne asigurăm că putem trece de la adresa curentă la adresa incrementată, dar și că se poate face salt la o altă adresă dacă avem instrucțiune de tip 0 și intrarea este 0 → se folosește un numărător universal pentru că permite incrementarea, precum și presetarea (folosind LOAD) unei anumite valori și continuarea incrementării de la acel punct
- Acest numărător joacă de fapt rolul RA

Structura de comandă microprogramată (2)

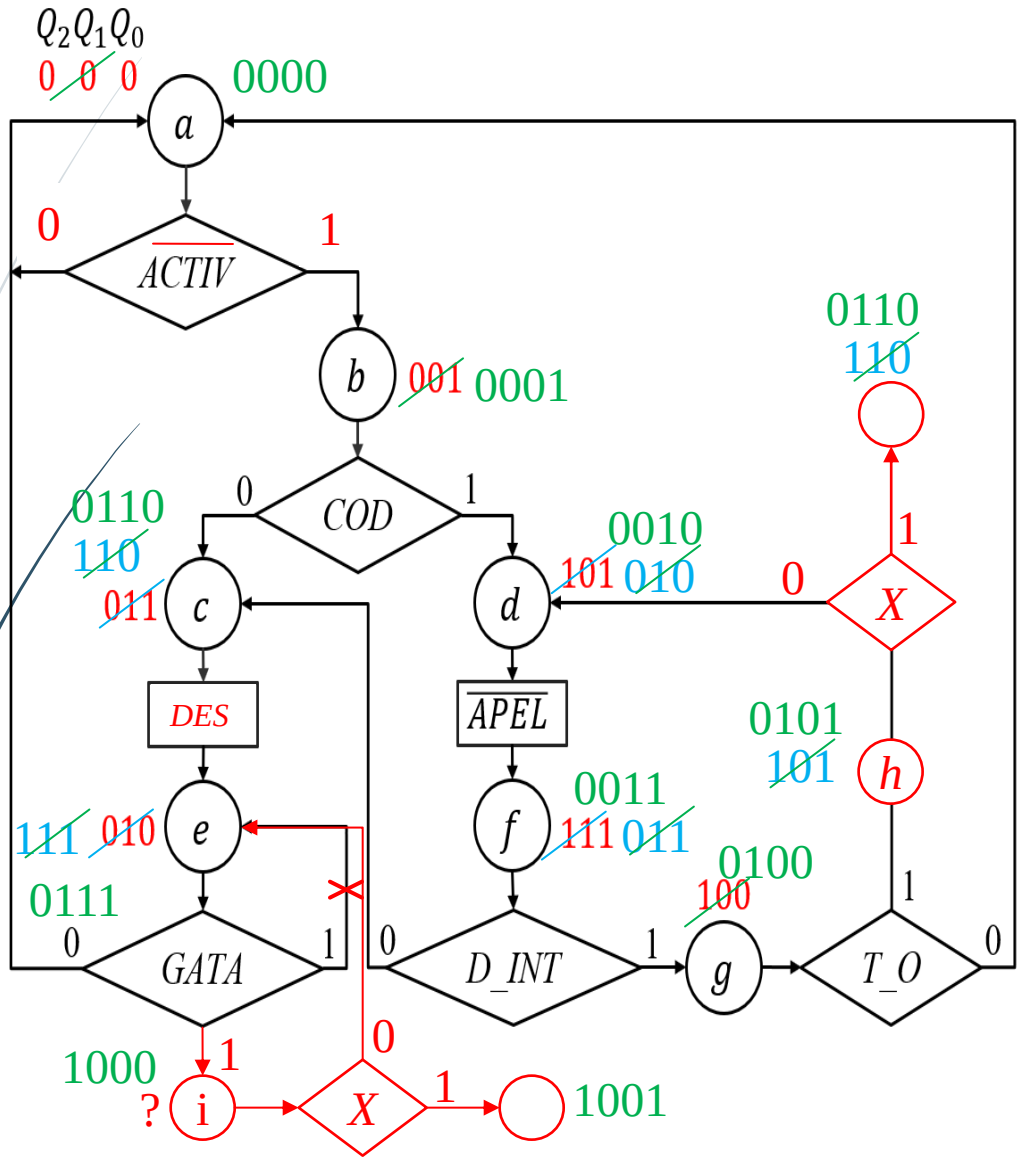


- Nu trebuie reprodusă la examen! Trebuie doar să o înțelegeți

Structura de comandă microprogramată (3)

- Dacă Tip = 1, Ctrl = 1
  - adresa curentă se incrementează (Up = 1)
  - se activează masca pentru a putea citi ieșirile (acestea se citesc direct)
- Dacă tip = 0
  - se dezactivează masca pentru a nu putea citi ieșirile
  - Dacă intrarea este = 1, Ctrl = 1 și adresa curentă se incrementează (Up = 1)
  - Dacă intrarea este = 0, Ctrl = 0, Load se activează și se încarcă ADR0 în numărător

Modificarea organigramei (1)



- Pas1: verificăm că avem doar instrucțiuni de tip 1 sau 0
- Pas 2: verificăm Tip 1: adresa următoare = adresa curentă + 1 și același lucru la Tip 0 când variabila de intrare = 1

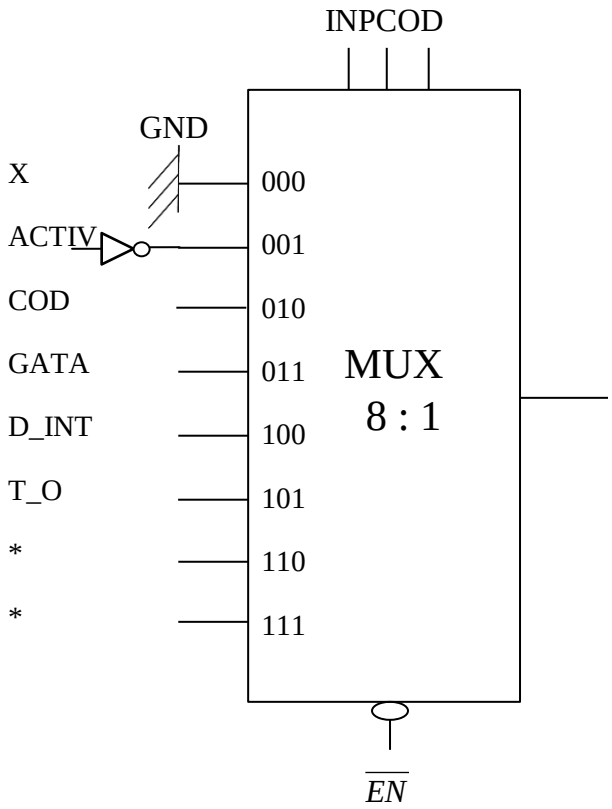
$X = 0$ → e posibil să se mărească numărul de biți pentru codificarea variabilelor de intrare

Modificarea organigramei (2)

- Din g oricum aș pune nu pot să fac tranziția pe 1 și atunci mai introduc o stare și o variabilă de intrare care va fi legată la masă (= 0 tot timpul). Astfel, folosesc din nou tranziția pe 0 ca să ajung unde doresc, iar tranziția pe 1 se duce către o stare unde nu ajung niciodată.
- Observații:
  - 1. codul stărilor unde nu se ajunge niciodată poate fi refolosit
  - 2. variabila de intrare legată la masă poate și ea fi refolosită de oricâte ori este nevoie pentru astfel de tranziții

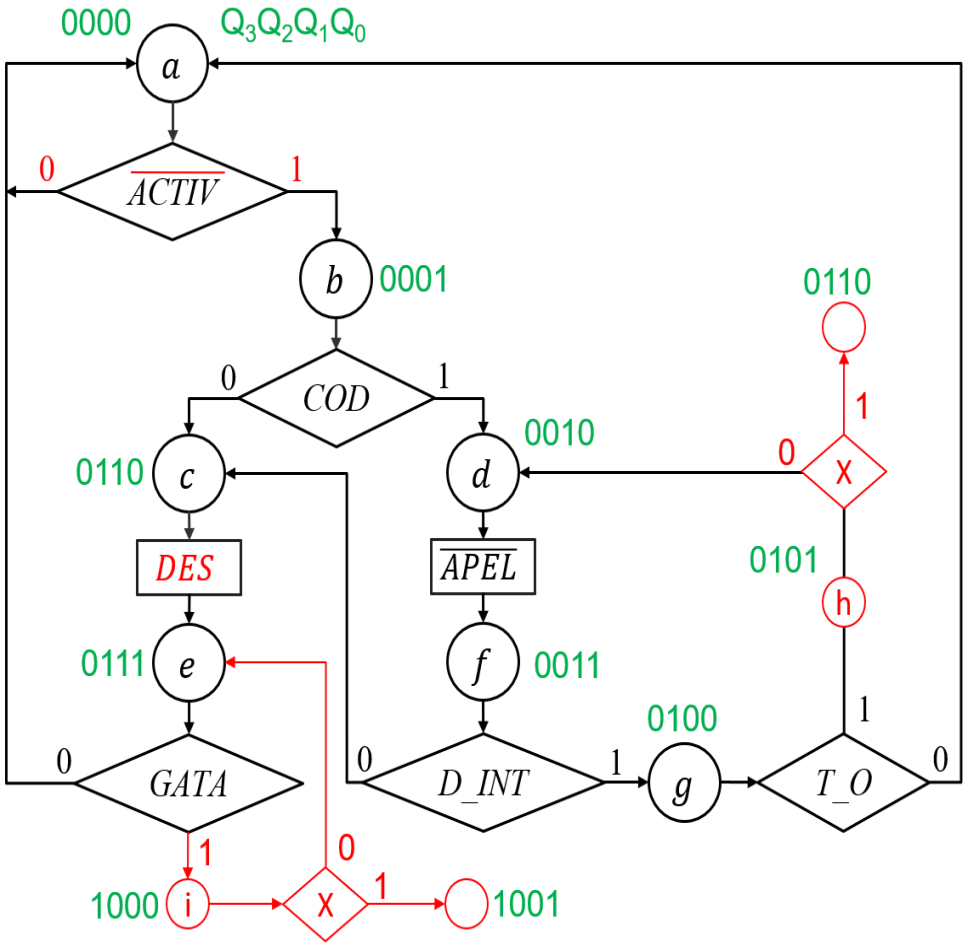
Modificarea organigramei (3)

- Ce înseamnă că modific ACTIV în !ACTIV?
- În unitatea de comandă, în MUX, în loc să pun ACTIV, pun !ACTIV
- !ACTIV va avea același cod ca ACTIV sau trebuie să neg biții?
- Are același cod și doar neg intrarea
- Pot să o fac mereu?
- Nu! Doar atunci când respectiva variabilă de intrare este folosită o singură dată (într-un singur loc).
- Altfel ce fac?
- Introduc o altă variabilă de intrare!



Memoria de microprogram

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101



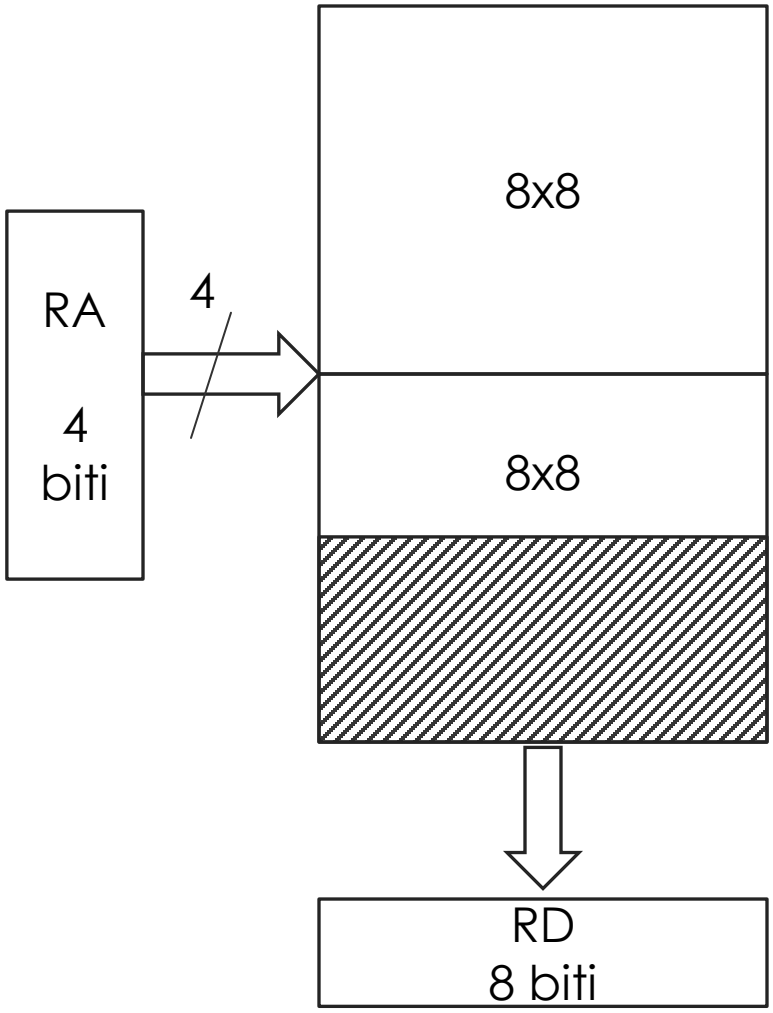
- $L_{\mu i\ tip\ 1} = 1 + 2 = 3$
- $L_{\mu i\ tip\ 0} = 1 + 3 + 4 = 8$
- $L_{\mu i} = \max(L_{\mu i\ tip\ 1}, L_{\mu i\ tip\ 0}) = 8$

Adr	*TIP* | *DES + APEL* (*COD* + *ADR0*)
a - 0000	0 | 001 0000
b - 0001	0 | 010 0110
d - 0010	1 | 00x xxxx
f - 0011	0 | 100 0110
g - 0100	0 | 101 0000
h - 0101	0 | 000 0010
c – 0110	1 | 11x xxxx
e - 0111	0 | 011 0000
i - 1000	0 | 000 0111
* - 1001	x | xxx xxxx

Memoria de microprogram

- Având în vedere că de cele mai multe ori cele 2 tipuri de microinstrucțiuni au dimensiuni diferite, trebuie ca una dintre ele să se completeze cu x-uri. Acest lucru se poate face fie la MSB, fie la LSB – e la latitudinea voastră
- Memorie consumată:
  - Format fix: 7 stări *11 biți = 77 biți
  - Format variabil: 9 stări * 8 biți = 72 biți

Memorii 8 x 8 – pentru μi cu format variabil

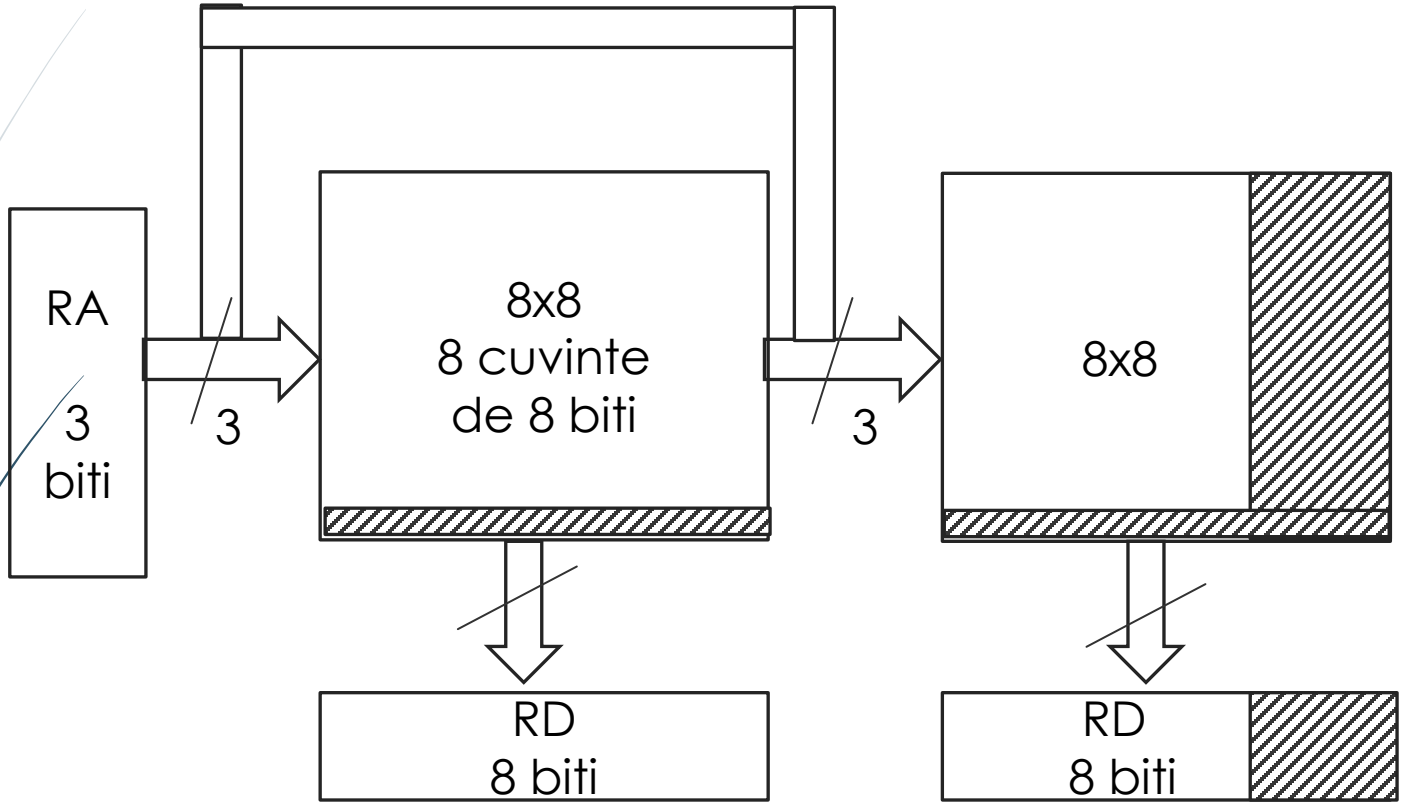


Memorii 8 x 8

μi cu format variabil:
11 stări x 8 biți

- 2 chip-uri de memorie

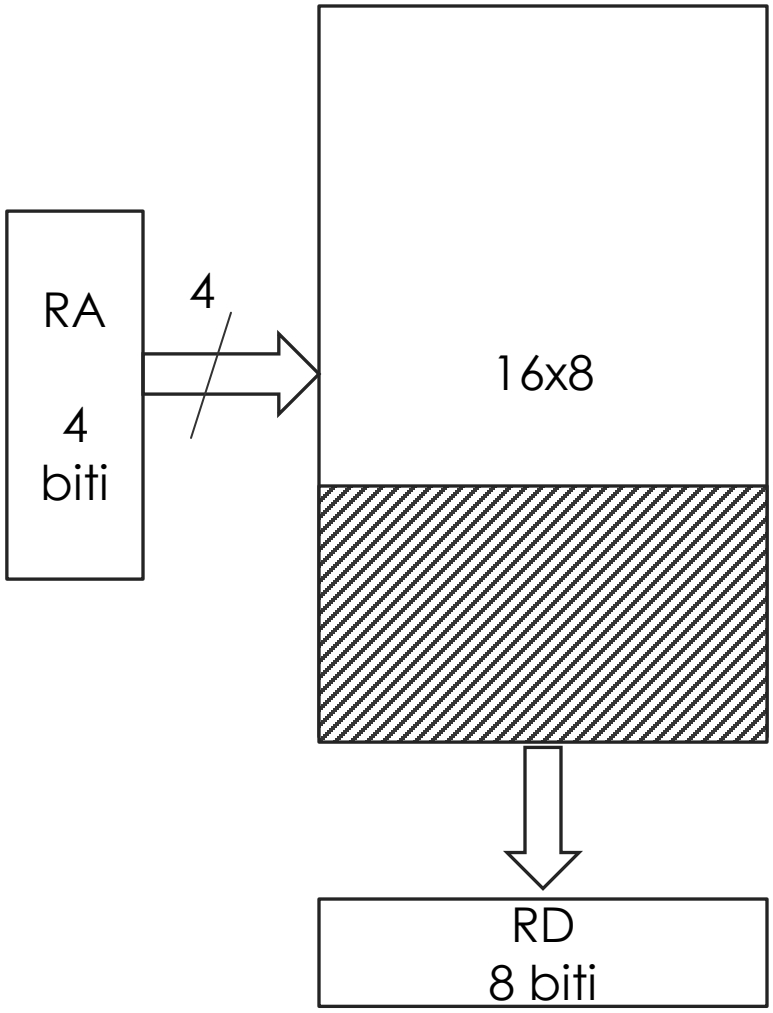
Memorii 8 x 8 – pentru μi cu format fix



Memorii 8 x 8

μi cu format fix: 7 stări * 14 biți
- 2 chip-uri de memorie

Memorii 16 x 8 – pentru μi cu format variabil

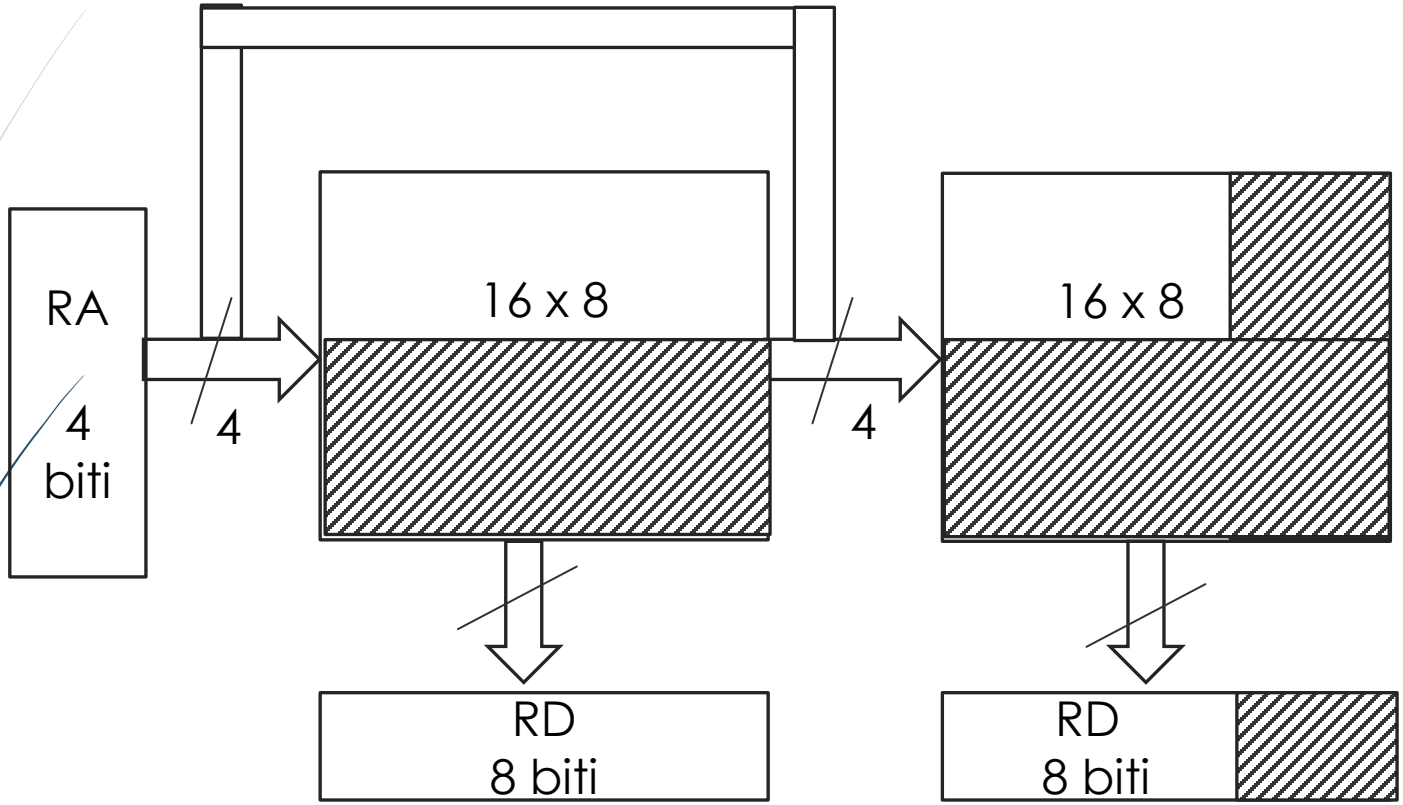


Memorii 16 x 8

μi cu format variabil:
9 stări x 8 biți

- 1 chip de memorie

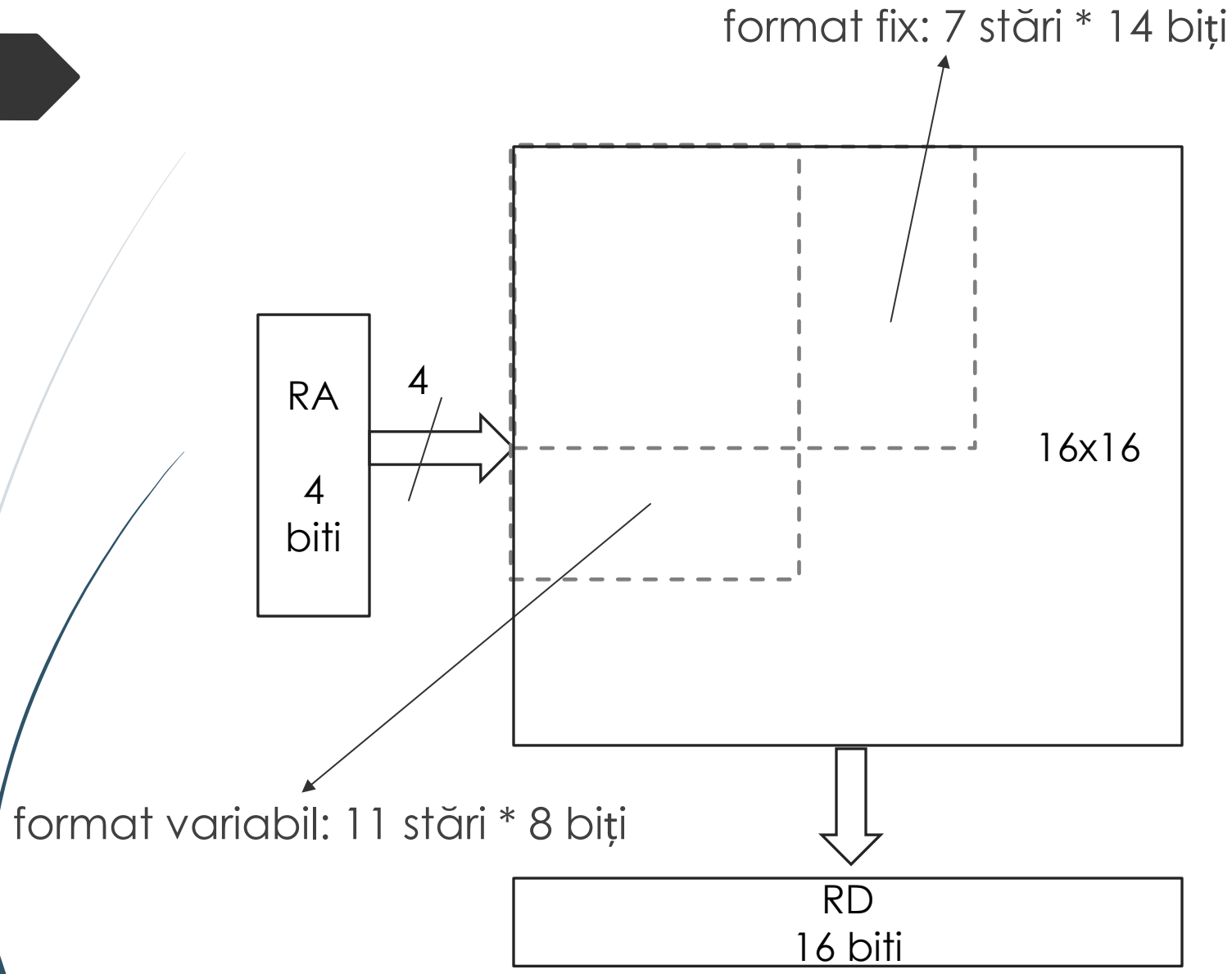
Memorii 16 x 8 – pentru μi cu format fix



Memorii 16 x 8

μi cu format fix: 7 stări * 14 biți
- 2 chip-uri de memorie

Memorii 16 x 16





Proiectare Logică

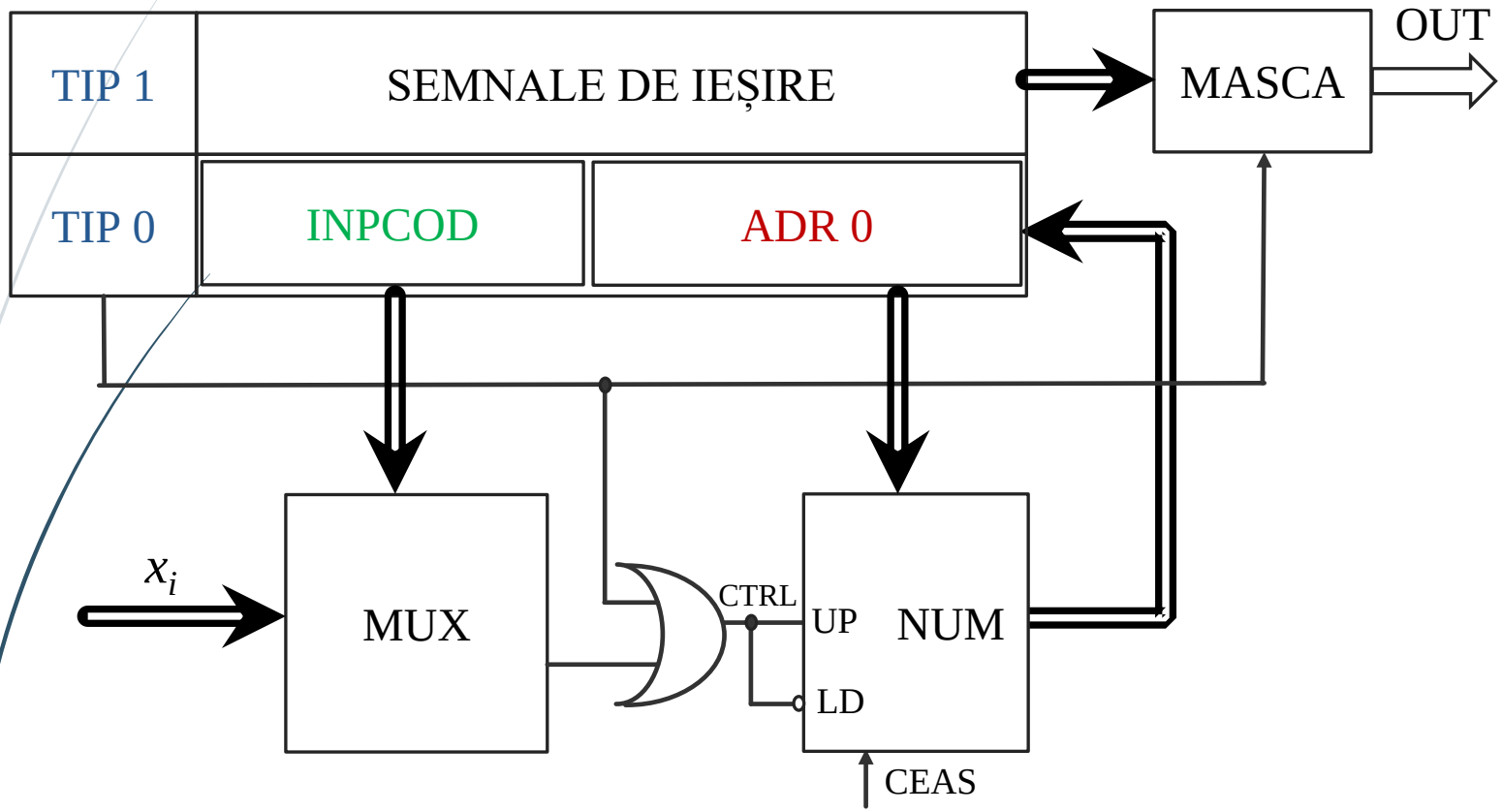
- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- **I CC: Anca Morar** anca.morar@cs.pub.ro
- I CA & CD: Costin Chiru costin.chiru@cs.pub.ro

Curs 14: Proiectarea dispozitivelor de comandă. Exemplu de implementare pentru proiectul în echipă – Curs Aplicativ

Reminder - Conținutul minimal al proiectului

- Descrierea funcționării automatului de comandă realizat
- Organigrama automatului (se recomandă ca automatul să aibă 5 variabile de stare)
- Calculul lungimii microinstrucțiunii
- Conținutul memoriei de microprogram
- Schema unității de comandă microprogramată
- Desenul cablajului circuitului proiectat

Schema unității de comandă microprogramată



Exemplu

- Automat cu:
  - 20 stări
  - 7 intrări: $x_1, x_2, x_3, x_4, x_5, x_6, x_7$
  - 12 ieșiri: $y_1, y_2, \ldots. y_{12}$

- Piese disponibile:
  - Circuite de memorie 256 x 4 MWS5101
  - Multiplexoare 16:1 54150
  - Numărătoare pe 4 biți SN74193
  - Chip-uri 7408 cu porți AND
  - Chip-uri 7432 cu porți OR
  - Chip-uri 7404 cu porți NOT (Inversor)

Calculul lungimii microinstrucțiunii (1)

- Automat cu:
  - 20 stări
  - 7 intrări: $x_1, x_2, x_3, x_4, x_5, x_6, x_7$
  - 12 ieșiri: $y_1, y_2, \ldots. y_{12}$

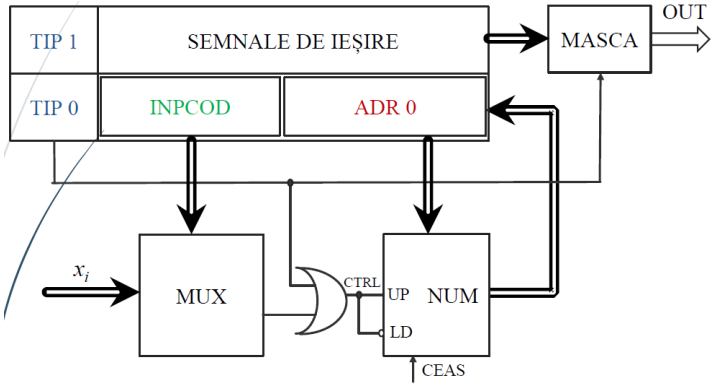
- Dacă nu țin cont de memoria aleasă:
  - *μi* de tip 1: $1 + n_{out} = 13$
  - *μi* de tip 0: $1 + n_{ci} + n_{adr} = 1 + 3 + 5 = 9$
  - $L_{\mu i} = \max(13, 9) = 13$

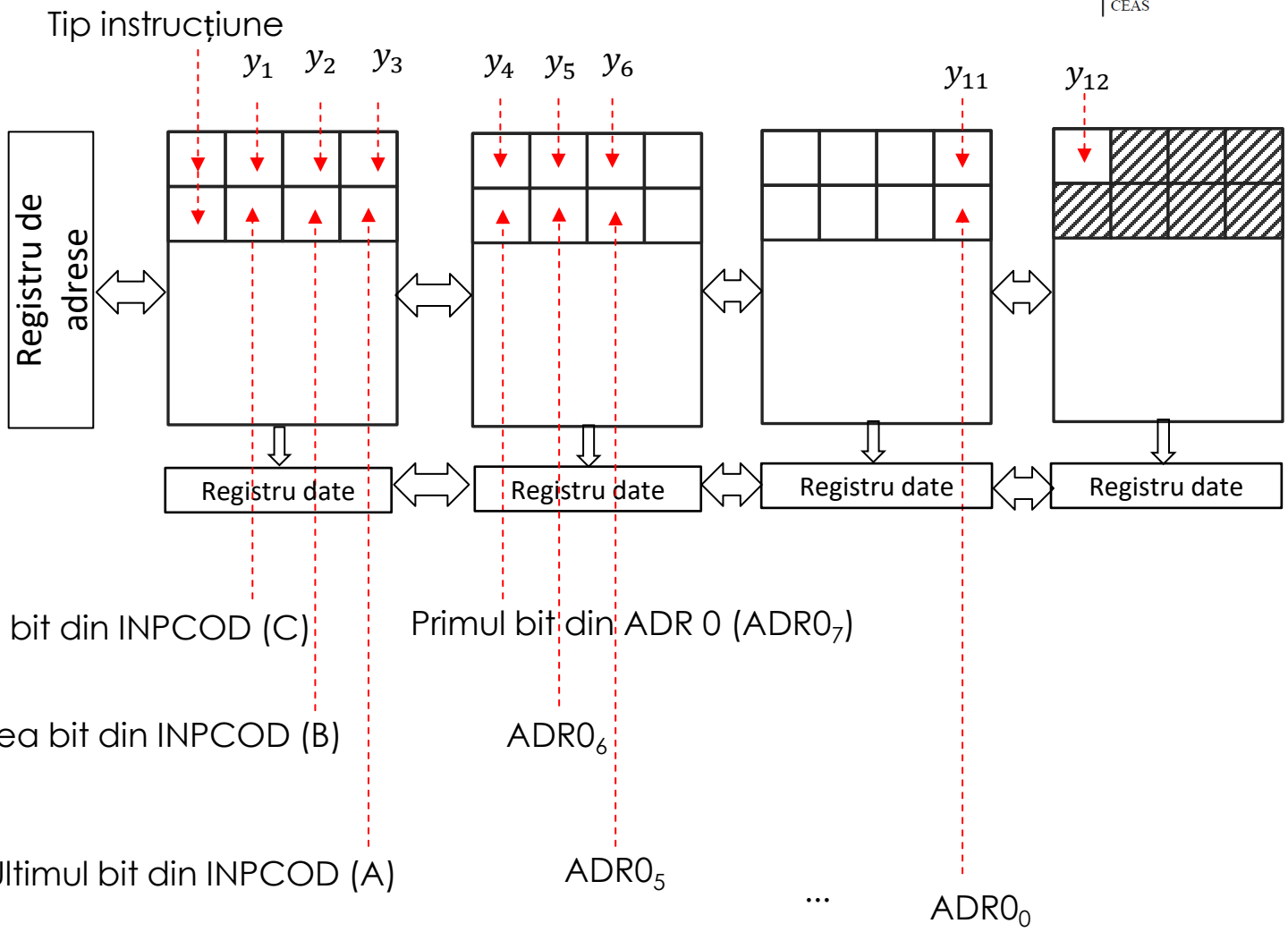
Calculul lungimii microinstrucțiunii (2)

- Automat cu:
  - 20 stări
  - 7 intrări: $x_1, x_2, x_3, x_4, x_5, x_6, x_7$
  - 12 ieșiri: $y_1, y_2, \ldots. y_{12}$

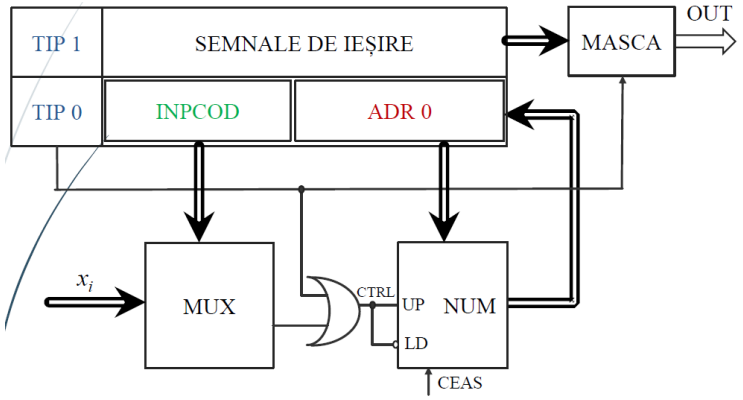
- Dacă țin cont de memoria aleasă: 256 x 4 => pot adresa maxim 256 de stări, cu un RA pe 8 biți => 8 biți pentru stări:
  - μi de tip 1: $1 + n_{out} = 13$
  - μi de tip 0: $1 + n_{ci} + n_{adr} = 1 + 3 + 8 = 12$
  - $L_{\mu i} = \max(13, 12) = 13$ => am nevoie de 4 circuite de memorie

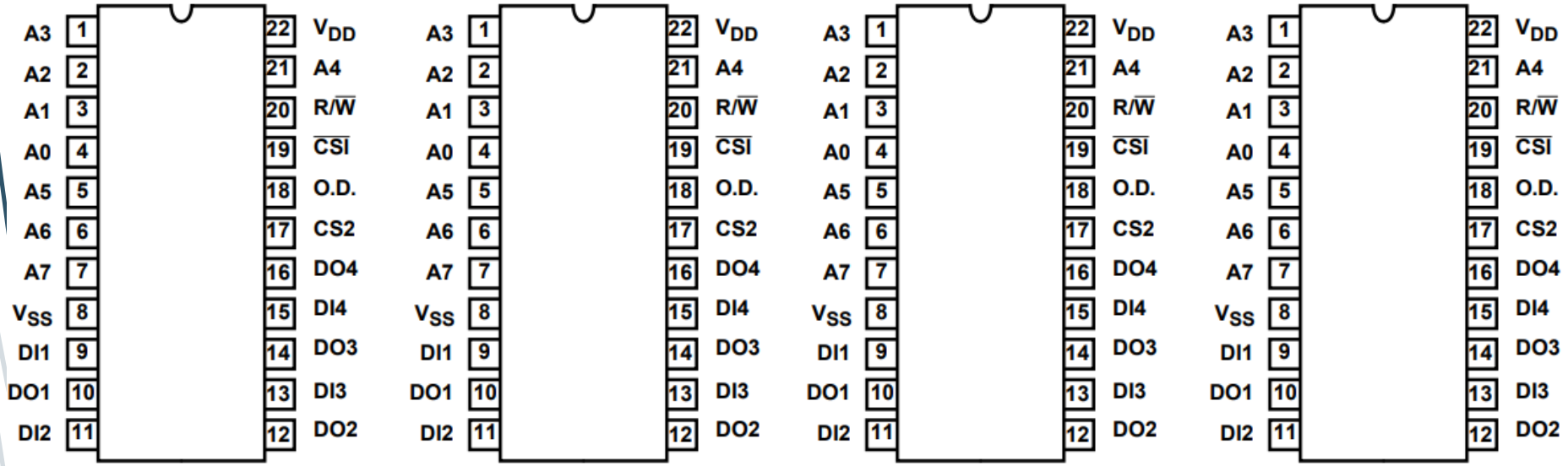
Circuitele de memorie - teoretic





Circuitele de memorie - practic: 256 x 4 MWS5101

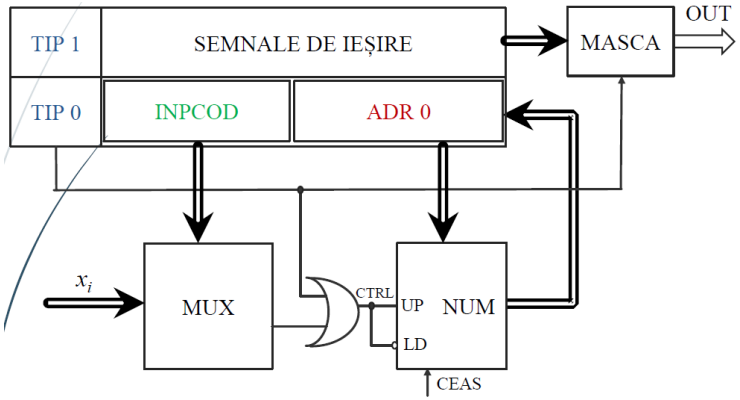




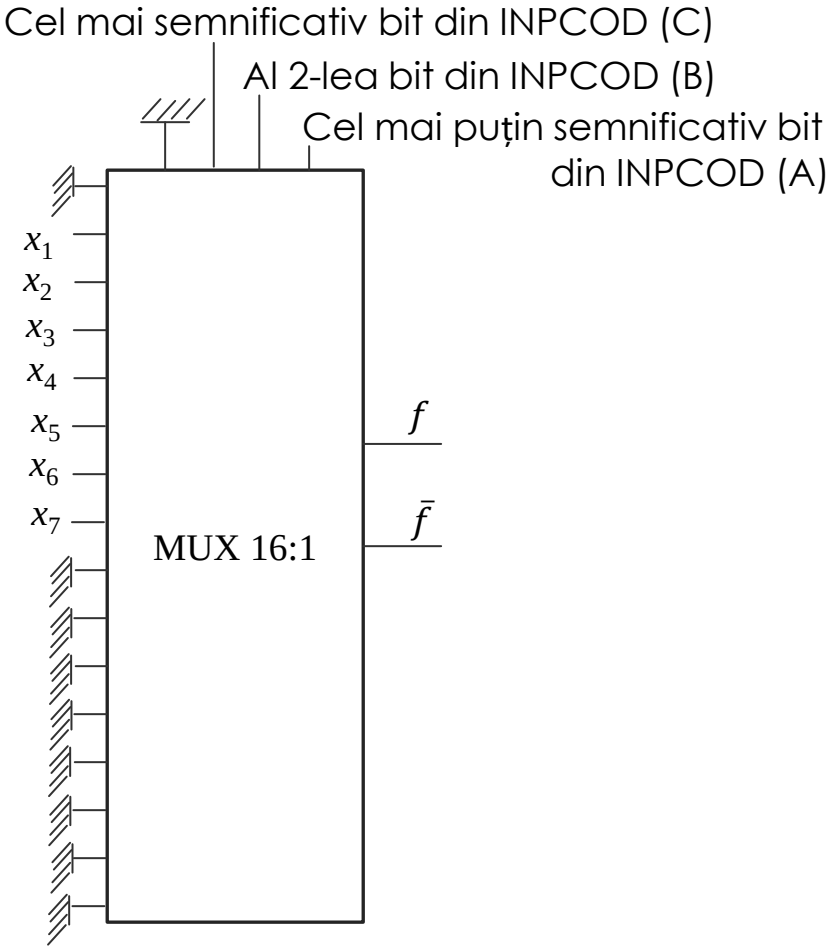
MODE	INPUTS				OUTPUT
	CHIP SELECT 1 ($\overline{CS_1}$)	CHIP SELECT 2 (CS_2)	OUTPUT DISABLE (OD)	READ/WRITE (R/$\overline{W}$)	
Read	0	1	0	1	Read

- A_7, A_6, ... A_0 – pinii pentru adresare
- DO_4, DO_3, DO_2, DO_1 – pinii pentru date (de ieșire)
- Citire → $!CS_1 = OD = 0$, $CS_2 = R/!W = 1$
- $DI_4 = DI_3 = DI_2 = DI_1 = 0$ – pinii pentru scriere (de intrare)

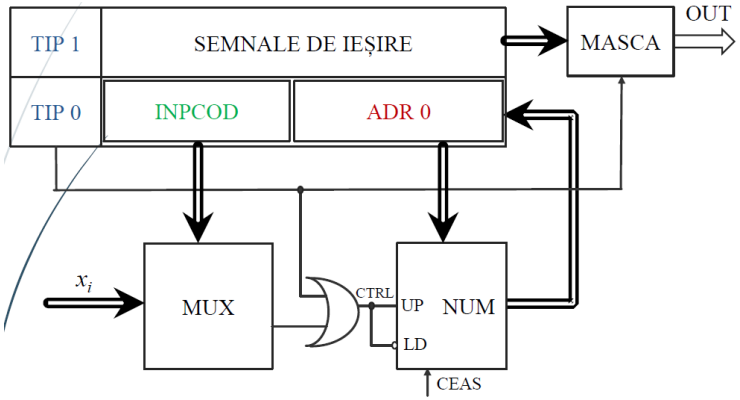
Multiplexorul 16:1- teoretic



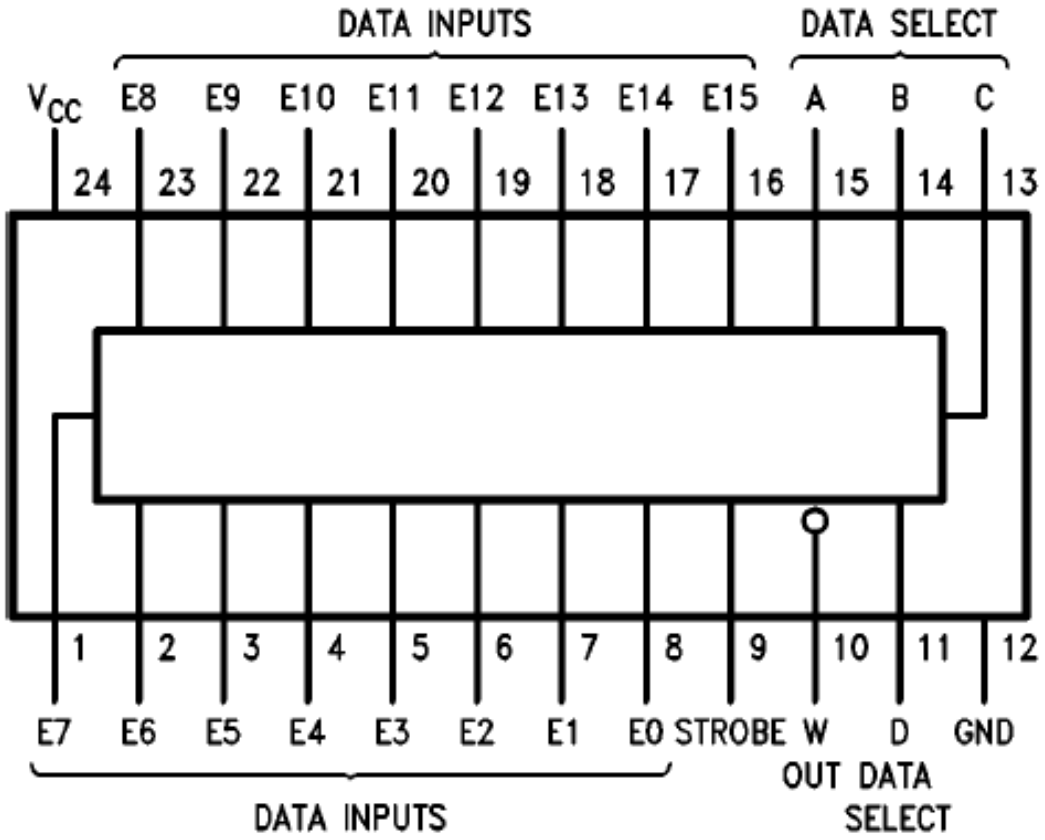
- Automat cu:
  - 20 stări
  - 7 intrări: x_1, x_2, x_3, x_4, x_5, x_6, x_7
  - 12 ieșiri: $y_1, y_2, \ldots, y_{12}$



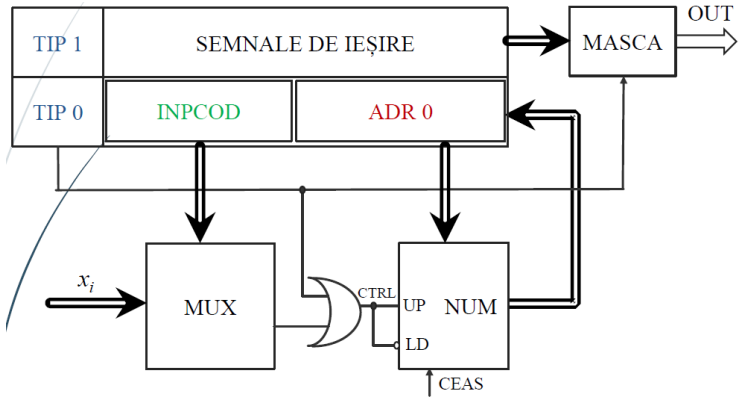
Multiplexorul 16:1 - practic: 54150



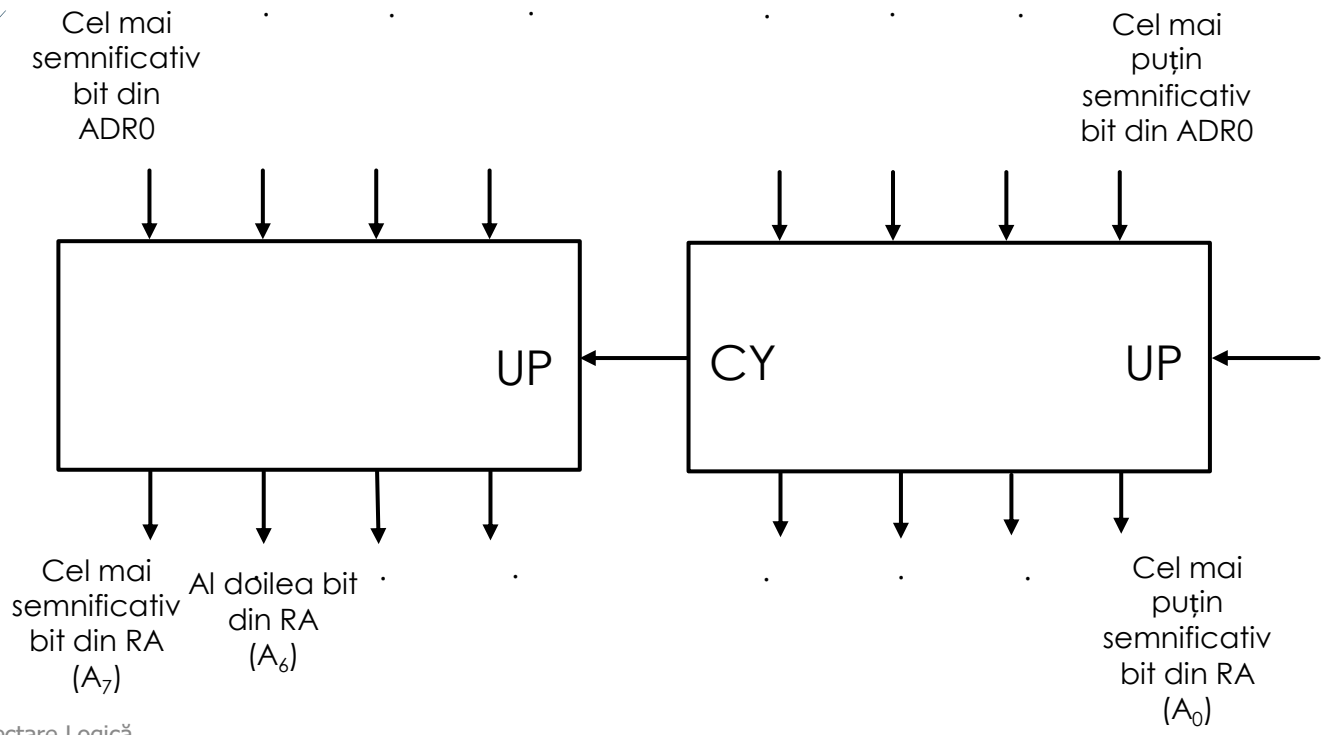
- E_0, E_1, ... E_{16} – pinii pentru intrări (vom folosi numai 7, restul vor fi conectați la masă)
- D, C, B, A – pinii pentru variabilele de selecție (vom folosi numai 3, cea mai semnificativă se leagă la masă)
- W – pinul pentru ieșirea negată a multiplexorului
- Strobe = 0 (Enable)



Registrul de adrese – teoretic

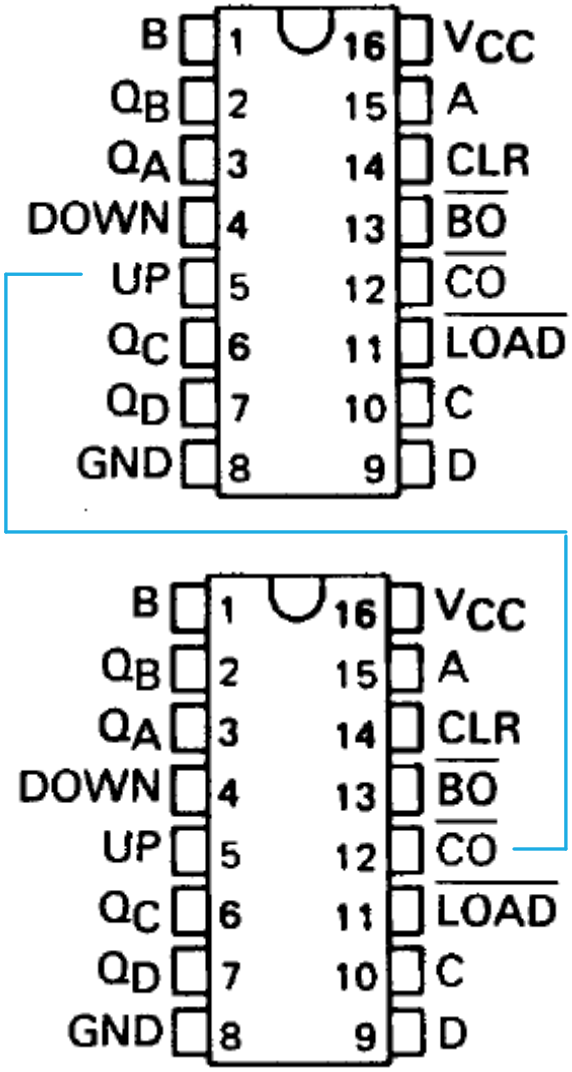


- RA – pe 8 biți
- Realizat printr-un numărător pe 8 biți
- Avem numărător pe 4 biți → am nevoie de 2 numărătoare



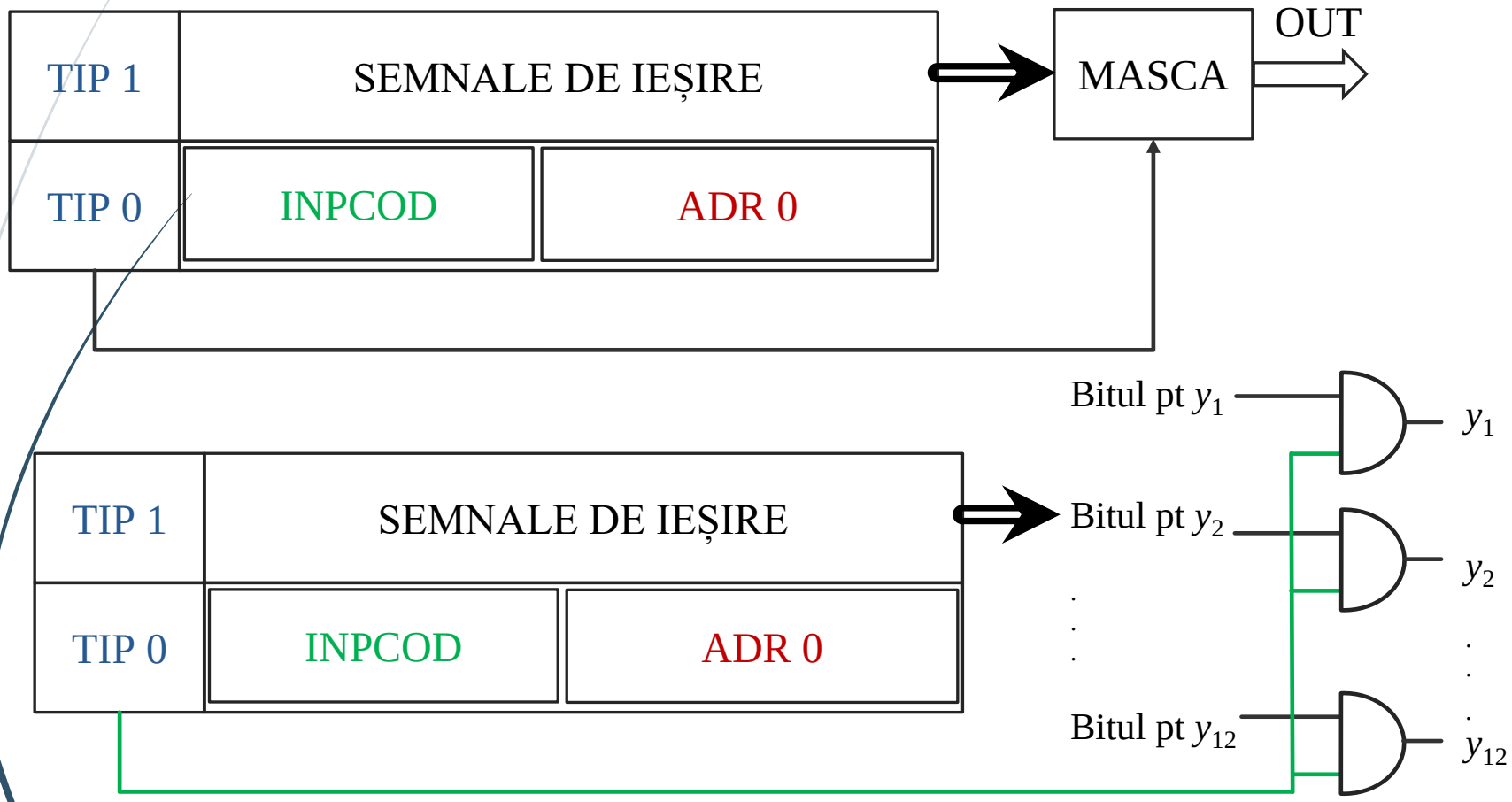
Registrul de adrese – practic: SN74193

- D, C, B, A – pinii pentru intrare (în cazul în care LOAD este activ)
- Q_D, Q_C, Q_B, Q_A – pinii pentru ieșirea numărătorului
- UP / DOWN – pinii pentru semnalul de ceas pentru numărare în sens crescător / descrescător
- CO / BO – pinii de ieșire pentru carry / barrow pentru numărare în sens crescător / descrescător
- Numărare în sens crescător pe 8 biți: pinul de CO de la un numărător (cel care numără biții mai puțin semnificativi) e conectat la pinul de UP de la al doilea numărător (cel care numără biții mai semnificativi)



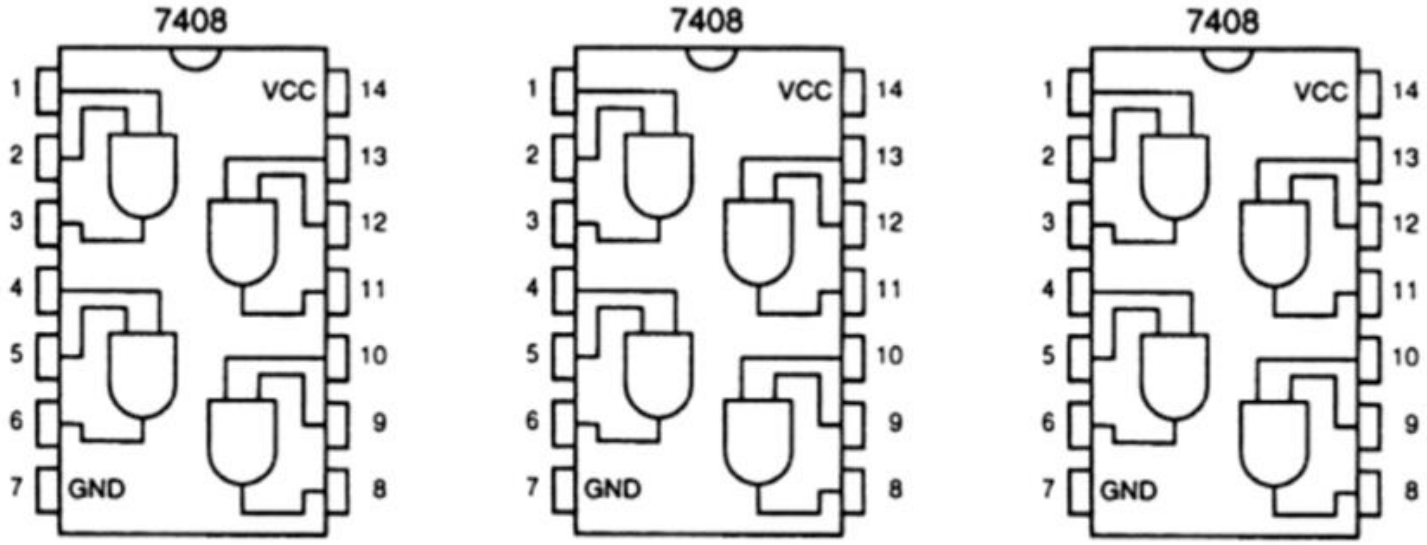
Masca pentru ieșiri - teoretic

- Tipul instrucțiunii = 1 => se generează ieșiri
- Tipul instrucțiunii = 0 => nu se generează ieșiri



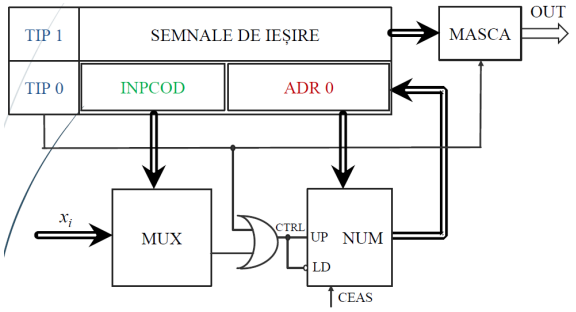
Masca pentru ieșiri – practic 7408

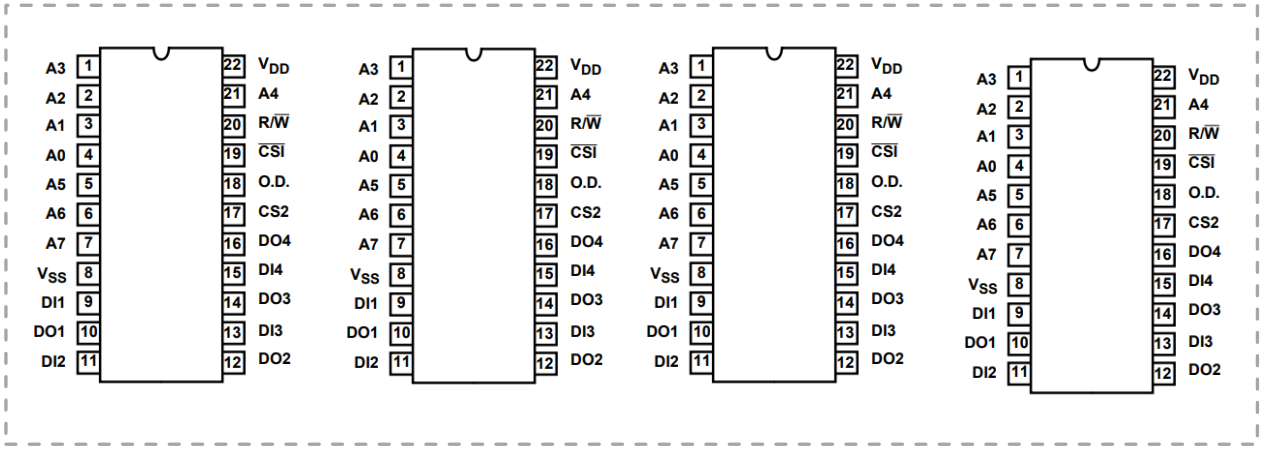
- 12 porți AND
- Chip-uri cu câte 4 porți AND → 3 chip-uri 7408



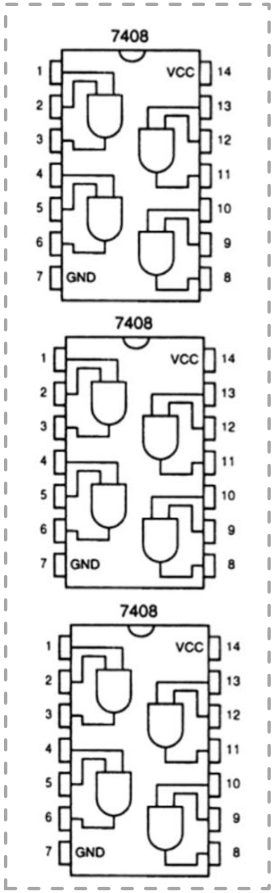
- 1,2 – pini de intrare pentru prima poartă
- 3 – pin de ieșire pentru prima poartă

Circuitele din structura de comandă

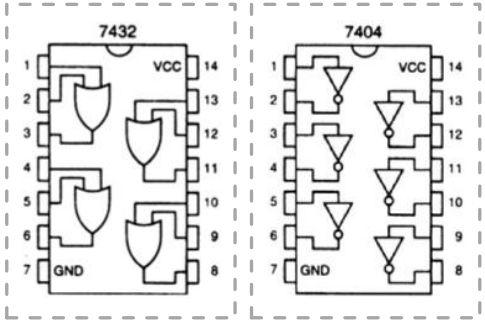




MEMORIA

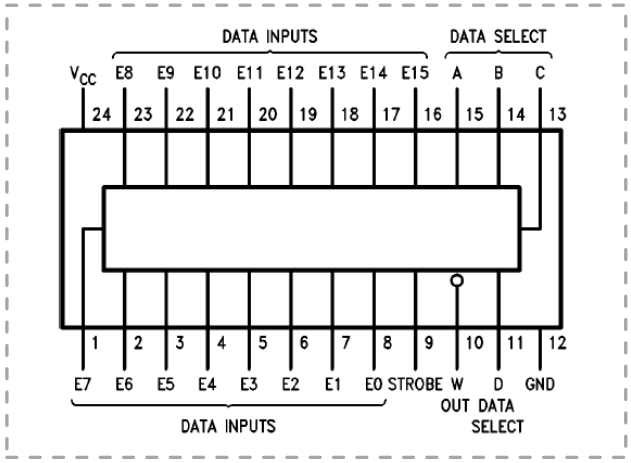


MASCA

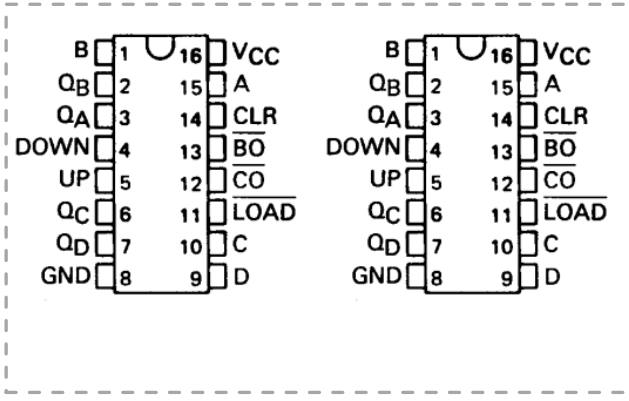


SAU

NOT

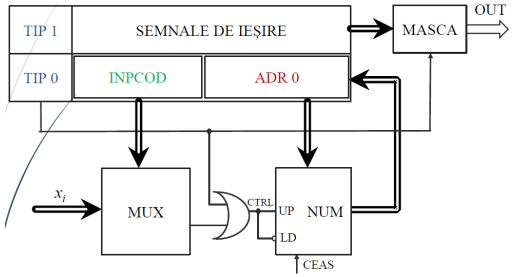


MUX

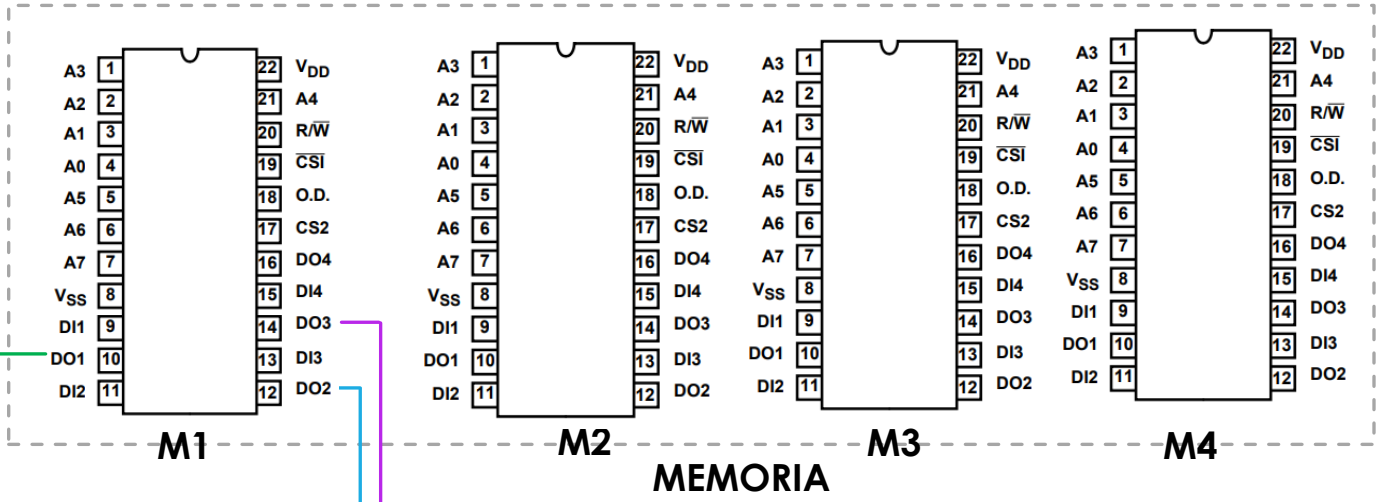


NUMĂRĂTOR

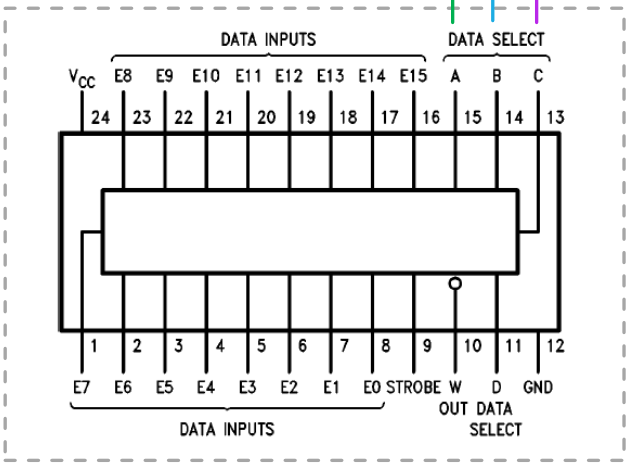
Conexiunile dintre pini: Memoria și multiplexorul



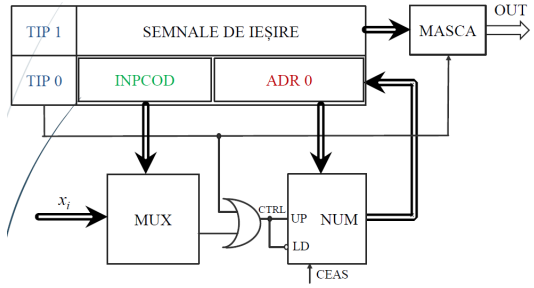
- M1 – Tip + INPCOD / y_{1-3}
- M2 – $ADR0_{7-4}$ / y_{4-7}
- M3 – $ADR0_{3-0}$ / y_{8-11}
- M4 – y_{12} + XXX



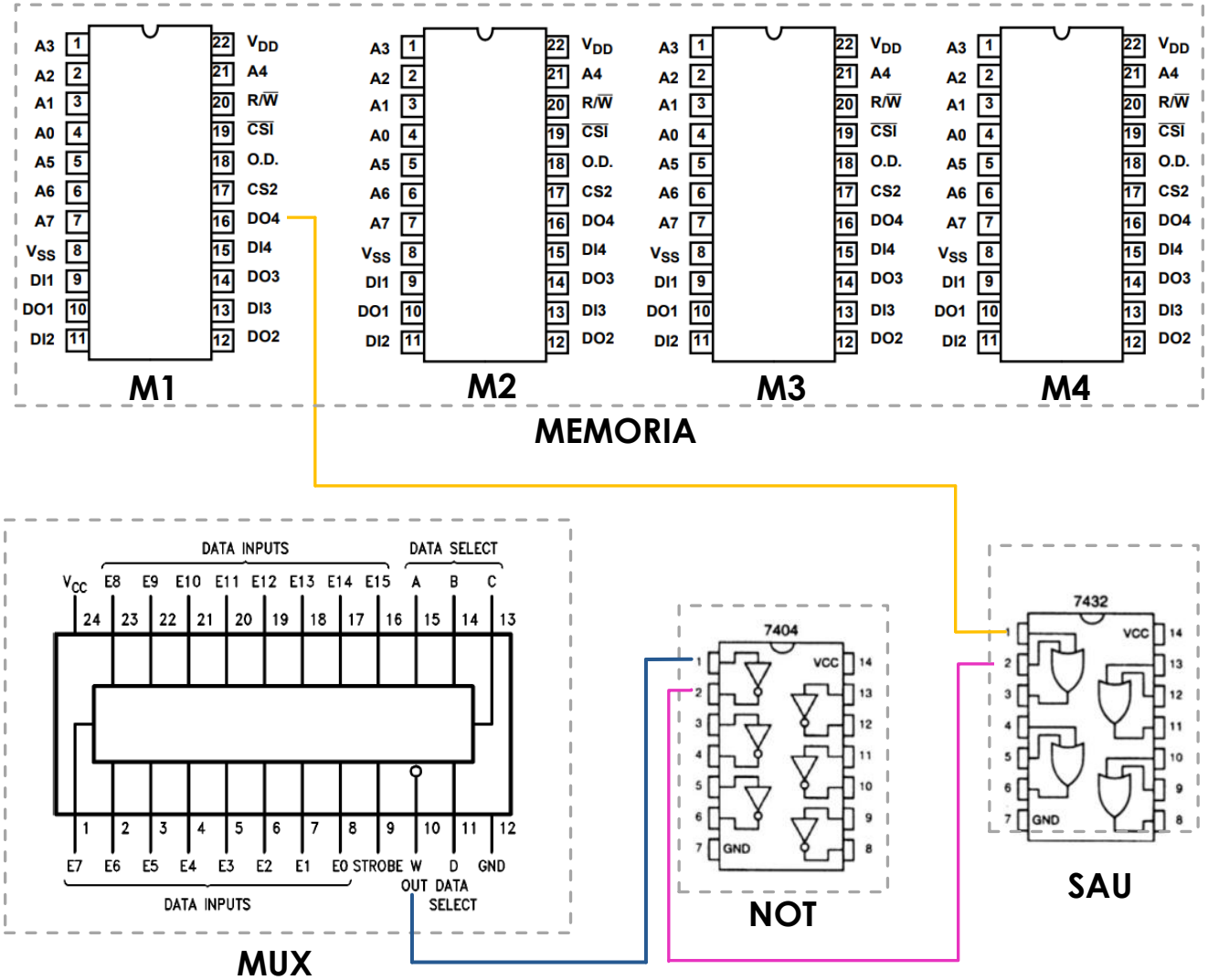
- DO_3 (M1) – C (MUX)
- DO_2 (M1) – B (MUX)
- DO_1 (M1) – A (MUX)
- E_1 – E_7 (MUX): conectați la intrările x_1, ... x_7
- D, E_0, E_8 – E_{15}, STROBE, GND (MUX), se conectează la masă



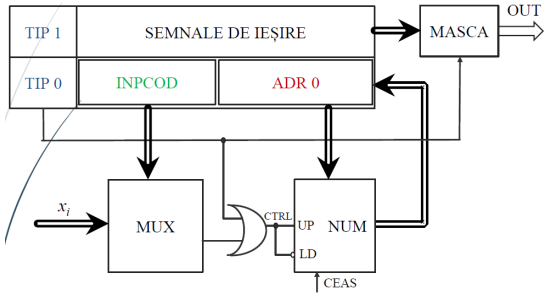
Conexiunile dintre pini: Memorie, MUX, poarta NOT, poarta SAU



- DO_4 (M1) – conectat cu pinul 1 din chip-ul SAU
- W din MUX – conectat cu pinul 1 din chip-ul NOT
- Pinul 2 din chip-ul NOT conectat cu pinul 2 din chip-ul SAU



Conexiunile dintre pini: Memorie, R_ADR, poarta SAU

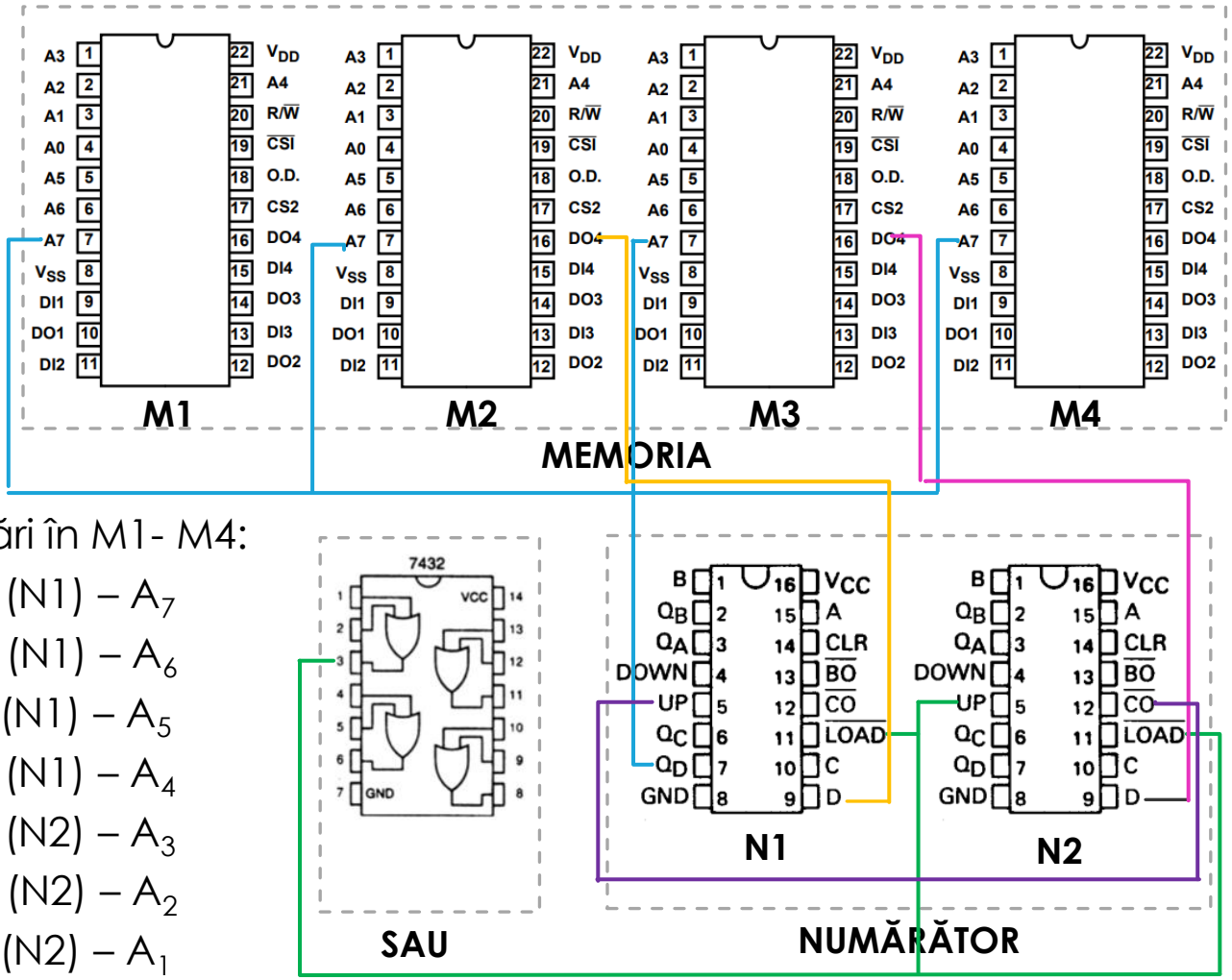


Intrări în numărător

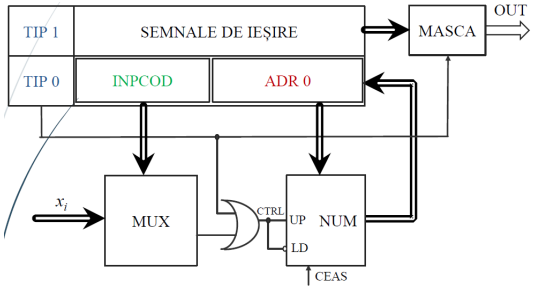
- Pinul 3 din chip-ul SAU – conectat cu pinul LOAD (N1-N2) și cu pinul UP (N2)
- CO (N2) – UP (N1)
- DO_4(M2) – D (N1)
- DO_3(M2) – C (N1)
- DO_2(M2) – B (N1)
- DO_1(M2) – A (N1)
- DO_4(M3) – D (N2)
- DO_3(M3) – C (N2)
- DO_2(M3) – B (N2)
- DO_1(M3) – A (N2)

Intrări în M1- M4:

- Q_D (N1) – A_7
- Q_C (N1) – A_6
- Q_B (N1) – A_5
- Q_A (N1) – A_4
- Q_D (N2) – A_3
- Q_C (N2) – A_2
- Q_B (N2) – A_1
- Q_A (N2) – A_0



Conexiunile dintre pini: Memoria și masca



- DO4 (M1) se duce în toate porțile AND (pinii 1, 4, 10, 13 de pe fiecare chip)
- DO3 (M1) – pinul 2 (AND1)
- DO2 (M1) – pinul 5 (AND1)
- ...
- Pinul 3 (AND1) – reprezintă ieșirea y_1
- Pinul 6 (AND1) – reprezintă ieșirea y_2
- ...

